

weft

A Modern Literate Programming Tool

Cuauhtemoc Pacheco
Derived from nuweb 1.64 by Preston Briggs¹
with contributions by Keith Harwood, Simon Wright,
Marc W. Mengel, John D. Ramsdell, and Gregor Goldbach

August 29, 2026

¹Original nuweb work supported by ARPA, through ONR grant N00014-91-J-1989.

Contents

1	Introduction	1
1.1	weft	1
1.2	Writing weft	3
1.2.1	Source File Extension: <code>.weft</code>	3
1.2.2	The Major Commands	4
1.2.3	The Minor Commands	7
1.3	Sectioning commands	8
1.4	Running weft	9
1.5	Markdown Output	10
1.6	Section Markers and Language Auto-Detection	11
1.6.1	Overriding the Language with <code>@l</code>	12
1.6.2	Mid-Line Inline Expansion	12
1.6.3	The <code>@L</code> Command: Defining New Languages	13
1.6.4	The Language Table	13
1.6.5	Syntax Highlighting in Markdown Output	14
1.6.6	Output Format of Section Markers	14
1.6.7	Debugging Tangled Code	14
1.7	Restrictions	18
1.8	Project Introspection	19
1.8.1	Map (<code>-m</code>)	19
1.8.2	Extract (<code>-e</code>)	19
1.8.3	Reverse Map (<code>-R</code>)	19
1.9	Acknowledgements	20
2	The Overall Structure	21
2.1	Files	21
2.1.1	The Main Files	23
2.1.2	Support Files	25
2.2	The Main Routine	25
2.2.1	Command-Line Arguments	26
2.2.2	Help Message	29
2.2.3	Short Flags	33
2.2.4	File Names	36
2.3	Pass One	41
2.3.1	Accumulating Definitions	44
2.3.2	Block Comments	46
2.3.3	Fixing the Cross References	50
2.3.4	Dealing with fragment parameters	50
2.4	Writing the Latex File	53
2.4.1	Formatting Definitions	59
2.4.2	Generating the Indices	75
2.5	Writing the Markdown File	81

2.5.1	Formatting Definitions in Markdown	84
2.5.2	Generating the Indices in Markdown	93
2.5.3	Identifier Cross-References in Markdown	98
2.6	Map and Extract Output	100
2.6.1	JSON Map Output	100
2.6.2	Fragment Extraction	102
2.6.3	Callers (Upward Call Chain)	104
2.6.4	Lint (Static Analysis)	106
2.6.5	Reverse Map Output	108
2.6.6	Error Annotator	112
2.7	Writing the Output Files	114
3	The Support Routines	118
3.1	Source Files	118
3.1.1	Global Declarations	118
3.1.2	Local Declarations	118
3.1.3	Reading a File	119
3.1.4	Opening a File	122
3.2	Scraps	122
3.2.1	Collecting Page Numbers	144
3.3	Names	145
3.3.1	Language Auto-Detection	151
3.3.2	User-Defined Languages	158
3.3.3	Weave Format Declaration	160
3.3.4	Collecting File and Fragment Names	161
3.3.5	Comment Styles	163
3.4	Searching for Index Entries	171
3.4.1	Retrieving scrap uses	172
3.4.2	Building the Automata	174
3.4.3	Searching the Scraps	176
3.5	Labels	182
3.6	Memory Management	184
3.6.1	Allocating Memory	184
3.6.2	Freeing Memory	185
4	Man page	187
5	Indices	192
5.1	Files	192
5.2	Fragments	192
5.3	Identifiers	196

Chapter 1

Introduction

In 1984, Knuth introduced the idea of *literate programming* and described a pair of tools to support the practise [3]. His approach was to combine Pascal code with T_EX documentation to produce a new language, WEB, that offered programmers a superior approach to programming. He wrote several programs in WEB, including `weave` and `tangle`, the programs used to support literate programming. The idea was that a programmer wrote one document, the web file, that combined documentation (written in T_EX [4]) with code (written in Pascal).

Running `tangle` on the web file would produce a complete Pascal program, ready for compilation by an ordinary Pascal compiler. The primary function of `tangle` is to allow the programmer to present elements of the program in any desired order, regardless of the restrictions imposed by the programming language. Thus, the programmer is free to present his program in a top-down fashion, bottom-up fashion, or whatever seems best in terms of promoting understanding and maintenance.

Running `weave` on the web file would produce a T_EX file, ready to be processed by T_EX. The resulting document included a variety of automatically generated indices and cross-references that made it much easier to navigate the code. Additionally, all of the code sections were automatically prettyprinted, resulting in a quite impressive document.

Knuth also wrote the programs for T_EX and METAFONT entirely in WEB, eventually publishing them in book form [5, 6]. These are probably the largest programs ever published in a readable form.

Inspired by Knuth's example, many people have experimented with WEB. Some people have even built web-like tools for their own favorite combinations of programming language and typesetting language. For example, CWEB, Knuth's current system of choice, works with a combination of C (or C++) and T_EX [8]. Another system, FunnelWeb, is independent of any programming language and only mildly dependent on T_EX [10]. Inspired by the versatility of FunnelWeb and by the daunting size of its documentation, Preston Briggs decided to write his own, very simple, tool for literate programming.¹

`weft` is a fork of `nuweb`. The original `nuweb` code, written by Preston Briggs and maintained over the years by several contributors, provided an excellent foundation: a simple, language-independent literate programming tool. `weft` builds on that foundation with a modernized codebase, a modular source structure organized by concept, the removal of the unmaintained HTML output path, and the addition of a Markdown weave output format. The goal is to continue evolving the tool while preserving the simplicity and philosophy that made `nuweb` valuable in the first place.

1.1 `weft`

⟨ *version 1* ⟩ ≡

1.1.1

¹There is another system similar to `nuweb`, written by Norman Ramsey, called *noweb* [9]. It perhaps suffers from being overly Unix-dependent and requiring several programs to use. On the other hand, its command syntax is very nice. In any case, `nuweb` certainly owes its name and a number of features to his inspiration.

weft works with any programming language and produces documentation in L^AT_EX [7] or Markdown format. L^AT_EX was the original choice because it supports a multi-level sectioning scheme and has facilities for drawing figures. Markdown support was added to allow literate programs to produce lightweight documentation that renders well on platforms like GitHub, GitLab, and any Markdown viewer, without requiring a T_EX installation.

Support for arbitrary programming languages was a design goal because programmers work in many languages (and sometimes combinations of several languages), *e.g.*, C, Fortran, C++, yacc, lex, Scheme, assembly, Postscript, and so forth. The need to support arbitrary programming languages has many consequences:

No prettyprinting Both WEB and CWEB are able to prettyprint the code sections of their documents because they understand the language well enough to parse it. Since weft aims to support *any* language, this feature must be abandoned. However, weft does allow particular individual formulas or fragments of L^AT_EX code to be formatted and still be parts of output files. Also, keywords in scraps can be surrounded by @_ to have them be bold in the output.

No index of identifiers Because WEB knows about Pascal, it is able to construct an index of all the identifiers occurring in the code sections (filtering out keywords and the standard type identifiers). Unfortunately, this isn't as easy for a language-independent tool. weft doesn't know what an identifier looks like in each language and certainly doesn't know all the keywords. (On the other hand, see the end of Section 1.2.3)

Of course, there must be some compensation for these losses or the whole idea would be a waste. Here are the advantages:

Simplicity The majority of the commands in WEB are concerned with control of the automatic prettyprinting. Since weft doesn't prettyprint, many commands are eliminated. A further set of commands is subsumed by L^AT_EX and may also be eliminated. As a result, the set of commands is reduced to only four members (explained in the next section). This simplicity is also reflected in the size of this tool, which is quite a bit smaller than the tools used with other approaches.

No prettyprinting Everyone disagrees about how their code should look, so automatic formatting annoys many people. One approach is to provide ways to control the formatting. weft's approach is simpler—it performs no automatic formatting and therefore allows the programmer complete control of code layout. weft does allow individual scraps to be presented in either verbatim, math, or paragraph mode in the T_EX output.

Control weft also offers the programmer complete control of the layout of his output files (the files generated during tangling). Of course, this is essential for languages that are sensitive to layout; but it is also important in many practical situations, *e.g.*, debugging.

Speed Since weft doesn't do too much, the weft tool runs quickly. By default it performs only tangling; weaving is activated with -w. When weaving is requested, both operations execute in a single pass.

Page numbers Inspired by the example of noweb, weft refers to all scraps by page number to simplify navigation. If there are multiple scraps on a page (say, page 17), they are distinguished by lower-case letters (*e.g.*, 17a, 17b, and so forth).

Multiple file output The programmer may specify more than one output file in a single weft file. This is required when constructing programs in a combination of languages (say, Fortran and C). It's also an advantage when constructing very large programs that would require a lot of compile time.

This last point is very important. By allowing the creation of multiple output files, the need for monolithic programs is avoided. This supports the creation of very large programs by groups of people.

A further reduction in compilation time is achieved by first writing each output file to a temporary location, then comparing the temporary file with the old version of the file. If there is no difference, the

temporary file can be deleted. If the files differ, the old version is deleted and the temporary file renamed. This approach works well in combination with `make` (or similar tools), since `make` will avoid recompiling untouched output files.

1.2 Writing weft

The bulk of a weft file will be ordinary L^AT_EX (or Markdown, when using the `-w md` flag). In fact, any L^AT_EX file can serve as input to weft and will be simply copied through, unchanged, to the documentation file—unless a weft command is discovered. Similarly, any Markdown file can be processed with `-w md` and will be copied through to the `.md` output. All weft commands begin with an “at-sign” (`@`). Therefore, a file without at-signs will be copied unchanged. weft commands are used to specify *output files*, define *fragments*, and delimit *scraps*. These are the basic features of interest to the weft tool—all else is simply text to be copied to the documentation file.

1.2.1 Source File Extension: `.weft`

weft source files use the `.weft` extension. This single, format-agnostic extension replaces the earlier `.w` and `.wmd` conventions. The prose surrounding scraps may be written in L^AT_EX or Markdown; the at-sign commands (`@o`, `@d`, `@<`, etc.) are the same in both cases.

The weave output format is specified in one of two ways:

In the source file The `@W` directive declares the intended format: `@W tex` for L^AT_EX, `@W md` for Markdown. This should appear in the preamble, before any `@o` or `@d` directives.

On the command line `-w md` or `-w tex` overrides the `@W` directive. Using `-w` alone (without a format token) reads the format from `@W`.

If no extension is given on the command line, weft appends `.weft` automatically.

Default Behavior: Tangle Only

By default, `weft myfile.weft` performs only tangling (producing the output files). No documentation is generated unless `-w` is given:

<code>weft programa.weft</code>	tangle only (default)
<code>weft -w programa.weft</code>	weave per <code>@W</code> (error if missing)
<code>weft -w md programa.weft</code>	weave Markdown (override)
<code>weft -w tex programa.weft</code>	weave LaTeX (override)

The `-t` flag suppresses weaving even if `-w` is given. The `-o` flag suppresses tangling.

Cosmetic Fences

When writing `.weft` files with Markdown prose, it is often convenient to surround scrap definitions with Markdown fenced code blocks (triple backticks) so that the source file renders nicely in Markdown previewers such as GitHub or VS Code. For example:

Some prose in Markdown.

```
'''  
@d My fragment  
@{code here  
@}  
'''
```

More prose.

weft silently strips these fence lines (lines that consist of three or more backticks, optionally followed by a language identifier) during Markdown weaving. They do not appear in the generated .md file and do not affect tangling. This is purely a cosmetic convenience for making the source file readable in any Markdown viewer.

Note that fences are only stripped in Markdown mode (`-w md`). In $\text{\LaTeX}$ mode, backtick lines are treated as ordinary text.

1.2.2 The Major Commands

Files and fragments are defined with the following commands:

@o *file-name flags scrap* Output a file. The file name is terminated by whitespace.

@d *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap.

@q *fragment-name scrap* Define a fragment. The fragment name is terminated by a return or the beginning of a scrap. This a quoted fragment.

A specific file may be specified several times, with each definition being written out, one after the other, in the order they appear. The definitions of fragments may be similarly specified piecemeal.

Fragment Names with Embedded Parameters

Fragment names can carry *parameters*—placeholder values embedded directly in the name using the `@'value@'` syntax. Inside the fragment body, `@1`, `@2`, ...`@9` refer to the first, second, ...ninth parameter in order of appearance.

For example, a reusable validation fragment:

```
@d Validate field @'name@' type @'string@'
@if (!data.@1) throw new Error("Missing @1");
if (typeof data.@1 !== '@2')
  throw new TypeError("@1 must be @2");
@}
```

Each invocation supplies its own arguments:

```
@<Validate field @'email@' type @'string@'@>
@<Validate field @'age@' type @'number@'@>
```

The first invocation expands to:

```
if (!data.email) throw new Error("Missing email");
if (typeof data.email !== 'string')
  throw new TypeError("email must be string");
```

Note three key points:

- The values in the *definition* (`@'name@'`, `@'string@'`) serve as default arguments and as documentation—they make the fragment name readable.
- At each *invocation*, different values can be substituted (`@'email@'` replaces `@1`, `@'string@'` replaces `@2`).
- If a fragment name is abbreviated with ..., any missing arguments fall back to the defaults given in the definition.

The difference between a quoted fragment (`@q`) and an ordinary one (`@d`) is that inside a quoted fragment fragments are not expanded on output. Rather, they are formatted as uses of fragments so that the output file can itself be `weft` source. This allows you to create files containing fragments which can undergo further processing before the fragments are expanded, while also describing and documenting them in one place.

You can have both quoted and unquoted fragments with the same name. They are written out in order as usual, with those introduced by `@q` being quoted and those with `@d` expanded as normal.

In quoted fragments, the `@f` filename is quoted as well, so that when it is expanded it refers to the finally produced file, not any of the intermediate ones.

Scraps

Scraps have specific begin markers and end markers to allow precise control over the contents and layout. Note that any amount of whitespace (including carriage returns) may appear between a name and the beginning of a scrap. Scraps may also appear in the running text where they are formatted (almost) identically to their use in definitions, but don't appear in any code output.

`@{anything@}` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in verbatim mode. Using the `-l` option will cause the program to typeset the scrap with the help of L^AT_EX's `listings` package.

`@[anything@ ]` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in paragraph mode, allowing sections of T_EX documents to be scraps, but still be prettyprinted in the document.

`@(anything@)` where the scrap body includes every character in *anything*—all the blanks, all the tabs, all the carriage returns. This scrap will be typeset in math mode. This allows this scrap to contain a formula which will be typeset nicely.

Inside a scrap, several special sequences are recognized.

Fragment invocation.

`@<fragment-name@>` Expands the named fragment inline. It is an error to specify recursive invocations. If the fragment name contains embedded parameters, the invocation supplies arguments at the same positions (see above).

When the reference appears *mid-line*—that is, after at least one visible character on the same line—section markers, `#line` directives, and fragment-name comments are automatically suppressed. The fragment content is spliced seamlessly into the surrounding text. At start-of-line positions (including indented lines), markers are emitted normally. See Section 1.6 for details.

Argument substitution.

`@1`, `@2`, ..., `@9` Inside a fragment body, substitutes the *n*th argument. If the argument was not supplied, the empty string is used.

At each argument position in an invocation, four forms are accepted:

`@'string@'` A literal string, copied verbatim (only `@` is interpreted, producing a single `@`). This is the most common form.

`@<fragment-name@>` The fragment is expanded and its output becomes the argument value.

`@{text@}` The text is expanded as an anonymous fragment (inheriting the caller's arguments) and the result is passed as the argument. Useful for combining text and arguments into a single value.

`@1`, `@2`, ..., `@9` Forwards an argument from the calling fragment to the called one.

Other in-scrap commands.

@xlabel@x Marks a place inside a scrap and associates it to the label (which can be any text not containing a @). Expands to the reference number of the scrap followed by a numeric value. Outside scraps it expands to the same value. It is used so that text outside scraps can refer to particular places within scraps.

@f Inside a scrap this is replaced by the name of the current output file.

@t Inside a scrap this is replaced by the title of the fragment as it is at the point it is used, with all parameters replaced by actual arguments.

@# At the beginning of a line in a scrap this will suppress the normal indentation for that line. Use this, for example, when you have a `#ifdef` inside a nested scrap. Writing `@##ifdef` will cause it to be lined up on the left rather than indented with the rest of its code.

@s Inside a scrap suppresses indentation for the following fragment expansion.

Note that fragment names may be abbreviated, either during invocation or definition. For example, it would be very tedious to have to type, repeatedly, the fragment name

```
@d Check for terminating at-sequence and return name if found
```

Therefore, `weft` provides a mechanism (borrowed from Knuth) of indicating abbreviated names.

```
@d Check for terminating...
```

Basically, the programmer need only type enough characters to identify the fragment name uniquely, followed by three periods. An abbreviation may even occur before the full version; `weft` simply preserves the longest version of a fragment name. Note also that blanks and tabs are insignificant within a fragment name; each string of them is replaced by a single blank.

Sometimes, for instance during program testing, it is convenient to comment out a few lines of code. In C or Fortran placing `/* ... */` around the relevant code is not a robust solution, as the code itself may contain comments. `weft` provides the command

```
@%
```

only to be used inside scraps. It behaves exactly the same as `%` in the normal $\text{\LaTeX}$ text body.

When scraps are written to a program file or a documentation file, tabs are expanded into spaces by default. Currently, `weft` assumes tab stops are set every eight characters. Furthermore, when a fragment is expanded in a scrap, the body of the fragment is indented to match the indentation of the fragment invocation. Therefore, care must be taken with languages (*e.g.*, Fortran) that are sensitive to indentation. These default behaviors may be changed for each output file (see below).

Flags

When defining an output file, the programmer has the option of using flags to control output of a particular file. The flags are intended to make life a little easier for programmers using certain languages. They introduce little language dependences; however, they do so only for a particular file. Thus it is still easy to mix languages within a single document. There are five “per-file” flags:

- d Forces the creation of `#line` directives in the output file. These are useful with C (and sometimes C++ and Fortran) on many Unix systems since they cause the compiler’s error messages to refer to the web file rather than to the output file. Similarly, they allow source debugging in terms of the web file.
- i Suppresses the indentation of fragments. That is, when a fragment is expanded within a scrap, it will *not* be indented to match the indentation of the fragment invocation. This flag would seem most useful for Fortran programmers.

- t Suppresses expansion of tabs in the output file. This feature seems important when generating `make` files.
 - cx Puts comments in generated code documenting the fragment that generated that code. The *x* selects the comment style for the language in use: `c` for C (`/* */`), `+` for C++ (`//`), and `p` for shell/Perl (`#`). If the global `-x` cross-reference flag is set the comment includes the page reference for the first scrap that generated the code.
 - s Suppresses section markers and `#line` directives in the tangled output for this file. The language is still detected (from the extension or a preceding `@l`), so the Markdown weaver can emit syntax-highlighting classes, but the tangled code is emitted without any tangle annotations. This is useful for bootstrap source files or generated output that should remain clean.
- These flags are still available for explicit control, but in most cases the automatic language detection described in Section 1.6 is more convenient.

1.2.3 The Minor Commands

There are some very low-level utility commands that may appear anywhere in the web file.

- @@ Causes a single “at sign” to be copied into the output.
- @_ Causes the text between it and the next `@_` to be made bold (for keywords, etc.)
- @i *file-name* Includes a file. Includes may be nested, though there is currently a limit of 10 levels. The file name should be complete (no extension will be appended) and should be terminated by a carriage return. Normally the current directory is searched for the file to be included, but this can be varied using the `-I` flag on the command line. Each such flag adds one directory to the search path and they are searched in the order given.
- @rx Changes the escape character ‘@’ to ‘*x*’. This must appear before any scrap definitions.
- @v Always replaced by the string established by the `-V` flag, or a default string if the flag isn’t given. This is intended to mark versions of the generated files.
- @l *tag* Sets the language for the next `@o` or `@d` directive. The tag is looked up in the language table (Section 1.6) to determine the comment style for section markers and the `class` attribute for Markdown syntax highlighting. Placed on the line before the directive it applies to.
- @L *name style* [+d] Defines a custom language mapping for the section marker system (see Section 1.6). This must appear in the document preamble, before any `@o` or `@d` directives that depend on it.
- @W *format* Declares the weave output format for this source file. Valid formats are `md` (Markdown) and `tex` (L^AT_EX). Used by `-w` when no explicit format is given on the command line. A CLI format (`-w md` or `-w tex`) overrides this directive. Should appear in the document preamble.

In the text of the document, that is outside scraps, you may include scrap-like material.

- @{*Anything*@} The included material, the *Anything*, is typeset as if it appeared inside a scrap. This is useful for referring to fragments in the text and for describing the literate programming process itself.
- @<*Fragment name*@> The fragment named is expanded in place in the text. The expansion is presented verbatim, it is not interpreted for typesetting, so any special environment must be set up before and after this is used.
- @c Outside of a scrap this is used to define a block comment using all the text until the next command. Inside a scrap the last block comment is expanded using the current indentation and breaking lines between words if 60 characters are exceeded.

Finally, there are three commands used to create indices to the fragment names, file definitions, and user-specified identifiers.

`@f` Create an index of file names.

`@m` Create an index of fragment names.

`@u` Create an index of user-specified identifiers.

These are usually placed in their own section in the $\text{\LaTeX}$ document; for example, see Chapter 5.

Identifiers must be explicitly specified for inclusion in the `@u` index. By convention, each identifier is marked at the point of its definition; all references to each identifier (inside scraps) will be discovered automatically. To “mark” an identifier for inclusion in the index, it must be mentioned at the end of a scrap. For example,

```
@d a scrap @{
  Suppose the variables FOO and BAR are being declared
  inside this scrap.
@| FOO BAR @}
```

Alphabetic identifiers have been used in this example, but any string of characters (not including whitespace or `@` characters) will do. Therefore, it’s possible to add index entries for things like `<=<` if desired. An identifier may be declared in more than one scrap.

In the generated index, each identifier appears with a list of all the scraps using and defining it, where the defining scraps are distinguished by underlining. Note that the identifier doesn’t actually have to appear in the defining scrap; it just has to be in the list of definitions at the end of a scrap.

1.3 Sectioning commands

For larger documents the indexes and usage lists get rather unwieldy and problems arise in naming things so that different things in different parts of the document don’t get confused. `weft` has a sectioning command which keeps the fragment names and user identifiers separate. Thus, you can give a fragment in one section the same name as a fragment in another and the two won’t be confused or connected in any way. Nor will user identifiers defined in one section be referenced in another. Except for the fact that scraps in successive sections can go into the same output file, this is the same as if the sections came from separate input files.

However, occasionally you may really want fragments from one section to be used in another. More often, you will want to identify a user identifier in one section with the same identifier in another (as, for example, a header file defined in one section is included in code in another). Extra commands allow a fragment defined in one section to be accessible from all other sections. Similarly, you can have scraps which define user identifiers and export them so that they can be used in other sections.

`@s` Start a new section.

`@S` Close the current section and don’t start another.

`@d+ fragment-name scrap` Define a fragment which is accessible in all sections, a global fragment.

`@D+` Likewise

`@q+` Likewise

`@Q+` Likewise

`@m+` Create an index of all such fragments.

`@u+` Create an index of globally accessible user identifiers.

There are two kinds of section, the base section which is where normally everything goes, and local sections which are introduced with the `@s` command. A local section comprises everything from the command which starts it to the one which ends it. A `@s` command will start a new local section. A `@S` command closes the current local section, but doesn’t open another, so what follows goes into the base section. Note that

fragments defined in the base section aren't global; they are accessible only in the base section, but they are accessible regardless of any local sections between their definition and their use.

Within a scrap:

`@<+fragment-name@>` Expand the globally accessible fragment with that name, rather than any local fragment.

`@+` Like `@|` except that the identifiers defined are exported to the global realm and are not directly referenced in any scrap in any section (not even the one where they are defined).

`@-` Like `@|` except that the identifiers are imported to the local realm. The cross-references show where the global variables are defined and defines the same names as locally accessible. Uses of the names within the section will point to this scrap.

Note that the `+` signs above are part of the commands. They are not part of the fragment names. If you want a fragment whose name begins with a plus sign, leave a space between the command and the name.

1.4 Running weft

weft is invoked using the following command:

```
weft flags file-name...
```

One or more files may be processed at a time. If a file name has no extension, `.weft` will be appended. While a file name may specify a file in another directory, the resulting documentation file will always be created in the current directory. For example,

```
weft /foo/bar/quux
```

will take as input the file `/foo/bar/quux.weft` and will produce the tangled output files only. To also generate documentation, use `-w` (see below).

By default, weft performs only tangling—it generates the output files defined by `@o` directives and nothing else. Documentation (weaving) is produced only when the `-w` flag is given. When `-w` is present, both tangling and weaving execute in a single pass. Three flags control what weft produces:

- `-t` Suppress weaving (useful to override `-w` in scripts).
- `-o` Suppress generation of the output files (weave only).
- `-c` Avoid testing output files for change before updating them.

Thus, the command

```
weft -to /foo/bar/quux
```

would simply scan the input and produce no output at all.

There are several additional command-line flags:

- `-v` For “verbose”, causes weft to write information about its progress to `stderr`.
- `-n` Forces scraps to be numbered sequentially from 1 (instead of using page numbers). This form is perhaps more desirable for small webs.
- `-s` Doesn't print list of scraps making up each file following each scrap.
- `-d` Print “dangling” identifiers – user identifiers which are never referenced, in indices, etc.
- `-p path` Prepend *path* to the filenames for all the output files.
- `-w [format]` Activate weaving (documentation generation). Without `-w`, no documentation file is produced. The optional *format* argument selects the output format:

- `-w md` — force Markdown output (`.md` file).
- `-w tex` — force L^AT_EX output (`.tex` file).
- `-w` alone — read the format from the `@W` directive in the source file. If `@W` is absent, `weft` reports an error.

A format given on the command line overrides any `@W` directive. When Markdown output is selected, sequential numbering (`-n`) is forced automatically, since Markdown has no concept of page numbers. The Markdown output uses HTML anchors (`<a id="...">`) for cross-references and `<pre><code>` blocks for scraps, allowing clickable navigation between definitions and references. Cosmetic fence lines (triple backticks) in the source are silently stripped.

- `-x` Include cross-reference numbers in scrap comments.
- `-h options` Provide options for the `hyperref` package.
- `-r` Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- `-l` Use the `listings` package for formatting scraps. Use this if you want to have a pretty-printer for your scraps. In order to e.g. have pretty Perl scraps, include the following L^AT_EX commands in your document:


```
\usepackage{listings}
...
\lstset{extendedchars=true,keepspaces=true,language=perl}
```

 See the `listings` documentation for a list of formatting options. Be sure to include a `\usepackage{listings}` in your document.
- `-V string` Provide *string* as the replacement for the `@v` operation.
- `-m` Emit a JSON map of the project to stdout and exit. The map describes every output file and every fragment, their definition locations, usage relationships, and transitive dependencies. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-e name` Extract a single fragment by name and emit its code and dependency information as Markdown to stdout. The match uses the same prefix logic as `@<name...@>`. Neither weaving nor tangling is performed. See Section 2.6 for details.
- `-R file[:line]` Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions in the file. No `.weft` source files are needed—the reverse map operates entirely on the tangled output. See Section 2.6 for details.

1.5 Markdown Output

When invoked with `-w md`, `weft` generates a Markdown file (`.md`) instead of a L^AT_EX file (`.tex`). This allows literate programs to be read on any platform that renders Markdown, including GitHub, GitLab, and local editors like VS Code.

The Markdown output preserves all the essential features of the L^AT_EX output—scrap definitions, cross-references, and indices—but uses a different set of formatting conventions:

Scrap headers File definitions appear as `**"filename"** **N** =` and fragment definitions as `**<Name* N>** =`, where *N* is the sequential scrap number.

Scrap bodies Code is wrapped in `<pre><code>` HTML blocks (not fenced code blocks). This allows fragment references within scraps to be rendered as clickable HTML links with italic and bold formatting, which would not be possible inside Markdown fenced blocks.

HTML escaping Characters `<`, `>`, and `&` inside scraps are escaped as `<`, `>`, and `&` respectively.

Cross-references Each scrap has an HTML anchor (`<a id="weftN">`) and cross-reference links use `<a href="#weftN">`. Footer notes (“defined by”, “referenced in”) appear as Markdown blockquotes.

Indices The `@f`, `@m`, and `@u` commands produce Markdown lists with links. In the user identifier index, defining scraps appear in bold.

Sequential numbering The `-n` flag is forced automatically, since Markdown has no page numbers. Scraps are numbered 1, 2, 3, ... without letter suffixes.

Prose Text outside scraps passes through unmodified. The author writes Markdown prose directly in the `.weft` source file; weft does not interpret or transform it.

Syntax highlighting When the language of a scrap can be determined—from a preceding `@l` tag, the `@L` command, or the file extension of an `@o` name—weft emits `<code class="language-X">` on the opening tag. This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically.

Tangling (output file generation) works identically regardless of the weave format. When `-w` is given, a single invocation produces both the documentation and the tangled output files in one pass.

1.6 Section Markers and Language Auto-Detection

When debugging tangled code, it is useful to correlate lines in the generated file back to the original web source. weft can auto-detect the programming language and generate *section markers*—comments that delimit the opening and closing of each scrap in the tangled output, including the source file name and line number. For C and C++, it also generates `#line` directives automatically.

The key idea is simple:

file extension → language → comment style → section markers in tangled output

For example, `@o server.js` has the extension `.js`. weft looks up `js` in its language table, finds that JavaScript uses `//` comments, and wraps each scrap in the tangled `server.js` with `//` markers:

```
// {1: web.weft:12}
console.log("hello");
// {1: }
```

Similarly, `@o main.c` matches `c`, which uses `/* */` comments and also enables `#line` directives:

```
#line 5 "core.weft"
/* {1: core.weft:5} */
#include <stdio.h>
/* {1: } */
```

And `@o config.py` matches `py`, using `#` comments:

```
# {3: settings.weft:40}
DATABASE_URL = "postgres://localhost/app"
# {3: }
```

This happens automatically—no flags are needed. The extension alone is enough.

1.6.1 Overriding the Language with @l

Sometimes the file extension is not enough, or you want a different behaviour. The @l command (lowercase) placed on the line before an @o or @d directive overrides the extension:

```
@l sql
@o queries.txt
@{SELECT * FROM users;
@}
```

Here the file is queries.txt, whose extension .txt has no comment style. But @l sql tells weft to use -- comments, so the tangled output gets:

```
-- {1: db.weft:5}
SELECT * FROM users;
-- {:1}
```

@l also works on @d (fragment) definitions. This is useful for fragments that will be included in files of a specific language:

```
@l python
@d Validate email
@{import re
def validate(email):
    return bool(re.match(r'^[^\@]+\@[^\@]+$', email))
@}
```

The language tag is stored on the fragment and used both for section markers in the tangled output and for syntax highlighting in the Markdown weave output.

@l is format-agnostic: it works identically in L^AT_EX and Markdown source files. Unlike the earlier approach of encoding the language in Markdown fences (‘‘python) or L^AT_EX markers (%%python), @l uses the same at-sign syntax as every other weft command.

Note that cosmetic fences (‘‘‘) may still be used around @o/@d definitions for readability in Markdown previewers—they are silently stripped during Markdown weaving (see Section 1.2.1). But fences no longer set the language; use @l for that.

Priority rules. When multiple sources of language information are available, weft resolves them in this order:

1. **Explicit per-file flags** (-d, -s, -cx, etc.) on the @o line always win for tangle behaviour.
2. **@l tag** immediately before the directive—overrides the extension.
3. **File extension** of the @o name—the default when no @l is present.

1.6.2 Mid-Line Inline Expansion

When a fragment reference @<name> appears in the middle of a line—after at least one visible character—weft automatically suppresses all tangle annotations for that expansion: section markers, #line directives, and fragment-name comments. The fragment content is spliced directly into the surrounding text, producing clean inline output.

This makes small constant fragments practical:

```
@d port @{4000@}
@d host @{"localhost"@}

@l c
```

```
@o config.c -s
@{const char *host = @<host@>;
int port = @<port@>;
@}
```

Tangling produces:

```
const char *host = "localhost";
int port = 4000;
```

If the same reference appears at the *start* of a line (even if indented), it is treated as a block expansion and markers are emitted normally. No special flags or directives are needed—the position on the line is the only thing that determines the behaviour.

This works with any language and any fragment size. When a multi-line fragment is expanded mid-line, its content is still spliced without markers—the rule is based on the *reference site*, not the fragment body.

1.6.3 The @L Command: Defining New Languages

The built-in table covers over 250 entries, but it cannot anticipate every language or every project's preferences. The @L command defines new language mappings—or overrides existing ones—directly in the document preamble.

@L *name style* [+d]

- *name* is the tag that will be recognised in @l commands and file extensions (@o *file.name*).
- *style* is the comment syntax: //, #, --, /*, or <!--.
- +d (optional) enables #line directive generation.

Examples:

```
@L wgs1 //          % WebGPU shading language: // comments
@L cedar //         % Cedar policy language
@L c //             % override built-in: use // instead of /* */
```

How @L interacts with the rest of the system. User-defined entries are searched *before* the built-in table, so @L c // effectively changes the comment style for all subsequent @o *main.c* directives (and for any @l c tag) from /* */ to //. Up to 32 user-defined languages may be registered.

@L should appear in the document preamble, before any @o or @d that depends on it. It is processed during pass 1 and silently skipped during weaving.

1.6.4 The Language Table

weft includes a built-in table of over 250 entries (languages, file extensions, and common aliases), organised by comment style. Each entry maps a tag to a comment style and optionally enables #line directives. The tag is matched against @l tags and file extensions alike—there is no distinction between them.

Comment style	Tags (representative)
<code>/* */ + #line</code>	c, h, cpp, cc, cxx, hpp, hxx, objc, objcpp, m, mm, ino, pde, cu, cuh, cilk
<code>/* */</code>	css, scss, sass, less, stylus, cl (OpenCL)
<code>//</code>	js, ts, jsx, tsx, mjs, cjs, mts, svelte, vue, java, scala, kotlin, kt, groovy, gradle, clj, cs, fs, fsx, vb, go, rust, rs, zig, swift, dart, odin, mojo, carbon, php, graphql, prisma, glsl, hlsl, wgsl, metal, vert, frag, proto, thrift, fbs, capnp, sol, reason, res, ml, sml, fut, apex, bicep, jsonnet, qml, p4
<code>#</code>	python, py, pyx, pyi, sh, bash, zsh, fish, ksh, nu, perl, pl, ruby, rb, r, tcl, awk, sed, coffee, makefile, mk, cmake, just, meson, bazel, bzl, julia, jl, nim, crystal, cr, elixir, ex, yaml, yml, toml, dockerfile, terraform, tf, hcl, nix, conf, ini, env, editorconfig, gitignore, powershell, ps1, xdc, sdc
<code>--</code>	lua, luau, sql, psql, plpgsql, plpgsql, mysql, sqlite, hive, haskell, hs, elm, purescript, purs, ada, adb, ads, vhdl, vhd, lean, lean4, agda, idris, idr, erlang
<code><!-- --></code>	html, htm, xml, svg, xhtml, xsl, xsd, dtd, rss, xaml, csproj, jsp, aspx, cshtml, erb, ejs, hbs, njk, twig, blade, pug, liquid, mustache, md, mdx, rst, tex, latex
(none)	json, csv, tsv, txt, diff, patch

For the full list, see the `lang_table` array in Section 3.3.1. If a tag is not found in either the user-defined (`@L`) or built-in table, weft defaults to `//` comments with no `#line` directives. Tags mapped to style 0 (e.g. `json`, `txt`) produce no markers at all.

1.6.5 Syntax Highlighting in Markdown Output

When weaving to Markdown (`-w md`), weft emits the language tag on the `<code>` element:

```
<pre><code class="language-python">def hello():
    print("hello")
</code></pre>
```

This allows client-side highlighters (highlight.js, Prism) and platforms like GitHub to apply syntax colouring automatically. The language tag comes from the same chain described above: `@L` tag or file extension.

For `@d` fragments, the tag comes from a preceding `@L` command (there is no file extension to infer from). If no language is known, weft emits a plain `<pre><code>` without a class.

1.6.6 Output Format of Section Markers

Each scrap in the tangled file is wrapped with an opening marker at the top and a closing marker at the bottom, using the comment syntax of the detected language. The opening marker includes the scrap number and the source location (file name and line number). When fragments are nested, each expansion gets its own pair of markers, producing a clear hierarchy in the output.

1.6.7 Debugging Tangled Code

When you work with literate programming, the code you execute is *tangled output*—generated files assembled from scraps scattered across `.w` files. An error at `server.js:47` does not correspond to line 47 of any file you wrote; it is line 47 of a file that weft assembled from fragments defined in `server.w`, `routes.w`, and `database.w`. Without markers, debugging means mentally reconstructing which scrap produced which line. With markers, the tangled file tells you directly.

Reading the markers. Each marker encodes three pieces of information:

Part	Meaning
{ or {:	Opening or closing delimiter
Number	Scrap ID—unique across the entire document
file:line	Source .w file and line number (opening only)

Examples by language. JavaScript (// comments):

```
// {3: routes.w:42}
app.get("/api/users", async (req, res) => {
  // {7: database.w:18}
  const users = await db.query("SELECT * FROM users");
  // {:7}
  res.json(users);
});
// {:3}
```

Python (# comments):

```
# {5: models.w:31}
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
# {:5}
```

C (/* */ comments with #line directives):

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) {
    void *ptr = malloc(size);
    if (!ptr) {
        /* {4: errors.w:8} */
        fprintf(stderr, "out of memory\n");
        exit(1);
        /* {:4} */
    }
    return ptr;
}
/* {:2} */
```

Lua (-- comments):

```
-- {9: config.w:55}
local defaults = {
    port = 8080,
    host = "localhost",
}
-- {:9}
```

HTML (<!-- --> comments):

```
<!-- {6: layout.w:20} -->
<nav class="sidebar">
  <!-- {11: navigation.w:7} -->
```

```

    <ul>
      <li><a href="/">Home</a></li>
    </ul>
    <!-- {11} -->
  </nav>
  <!-- {6} -->

```

Debugging workflow. The following four-step procedure traces any error in tangled output back to the literate source.

Step 1: Locate the error in tangled output. You receive an error such as:

```

TypeError: Cannot read property 'id' of undefined
    at Object.<anonymous> (server.js:47)

```

Open `server.js` and go to line 47.

Step 2: Read the nearest opening marker. Scan *upward* from the error line until you find an opening marker:

```
// {3: routes.w:42}
```

This tells you: scrap 3 generated this code, defined in `routes.w` starting at line 42.

Step 3: Go to the literate source. Open `routes.w` at line 42. You will find the scrap with the bug, surrounded by prose that explains its intent:

```

We fetch the user by ID from the database. If the user
doesn't exist, we should return a 404, but currently we
assume the user always exists --- this is the bug.

```

```

@d Handle user fetch
@{const user = await getUser(req.params.id);
  res.json(user.id);
@}

```

The prose makes the bug obvious: there is no null check. Fix it in the `.w` file, re-tangle, and the error is gone.

Step 4: Nested markers = nested fragments. When fragments invoke other fragments, markers nest:

```

// {3: routes.w:42}
app.get("/api/users/:id", async (req, res) => {
  // {7: database.w:18}
  const user = await db.query(
    "SELECT * FROM users WHERE id=$1",
    [req.params.id]);
  // {7}
  if (!user) return res.status(404).json({error: "not found"});
  res.json(user);
});
// {3}

```

If the bug is inside the `{7: ... {7}}` block, go to `database.w:18`, not `routes.w:42`. The *innermost enclosing markers* point to the most specific source location. Always look at the tightest pair around the error line.

Interpreting nested hierarchies. Complex tangled output can have several nesting levels. Read them like parentheses—each opening matches its corresponding close:

```
/* {1: main.w:10} */          outermost: main file structure
#include <stdio.h>
/* {2: io.w:25} */           middle: I/O subsystem
void print_report() {
    /* {5: format.w:8} */     innermost: formatting detail
    printf("Report v%d\n", VERSION);
    /* {5} */
}
/* {2} */
int main() {
    /* {3: init.w:14} */
    init_subsystems();
    /* {3} */
    print_report();
    return 0;
}
/* {1} */
```

Scrap numbers are *document-global*—scrap 1 is the first `@{...@}` in the entire woven document, scrap 5 is the fifth. They are stable as long as you do not reorder scraps.

C/C++: #line directives. For C-family languages, weft also generates **#line** directives *above* the opening marker. These make the C compiler itself report errors against the `.w` source file:

```
#line 12 "memory.w"
/* {2: memory.w:12} */
void *safe_malloc(size_t size) { ... }
/* {2} */
```

Now gcc will say:

```
memory.w:14: error: implicit declaration of function 'malloc'
```

instead of:

```
memory.c:47: error: implicit declaration of function 'malloc'
```

The **#line** directive and the section marker are complementary: **#line** makes the compiler report errors against the `.w` file directly; the `{N: file:line}` markers are human-readable when browsing the tangled output.

Common debugging scenarios. Runtime error with stack trace.

```
Error: ENOENT: no such file or directory, open '/data/config.json'
    at readConfig (server.js:123)
```

Open `server.js:123`, find the nearest marker `// {12: config.w:35}`, open `config.w:35`—the scrap that reads the config—and fix the path or add an existence check in the `.w` file.

Compiler error in C. With **#line** directives active, the compiler already reports:

```
parser.w:84: error: 'token_type' undeclared
```

Go directly to `parser.w:84`. The prose around the scrap explains what `token_type` should be and where it is defined.

Wrong output, no error. The program runs but produces wrong results. Open the tangled file and search for the relevant code. The markers tell you which `.w` file and scrap to examine:

```
# {15: transform.w:67}
result = [x * 2 for x in data]    # should be x * 3?
# {:15}
```

Go to `transform.w:67`. The prose says “triple each value” but the code says `* 2`. Bug found.

Identifying which fragment owns a line. Sometimes you know the line of code but not which `@d` definition produced it (multiple fragments with the same name accumulate). The scrap number is unique:

```
// {14: handlers.w:89}
validateInput(data);
// {:14}
// {22: handlers.w:112}
validatePermissions(user);
// {:22}
```

Scraps 14 and 22 are two different definitions (or continuations) of the same fragment. The line numbers point to each one independently.

Tips.

- *Never edit tangled files.* The markers remind you these are generated. Always trace back to the `.w` source and edit there.
- *Scrap numbers are stable* as long as you do not add or remove `@{...@}` blocks before them. If numbering shifts after edits, re-tangle and the markers update automatically.
- *Use `grep` on markers* to find all scraps from a specific file:

```
grep -n '{[0-9]*: database.w:' server.js
```
- *Closing markers as verification.* If you see `{:7}` but no `{7:` above it in your viewport, scroll up—the scrap is larger than your screen. If you see `{7:` but no `{:7}`, the scrap extends below.
- *Markers in test output.* If your test framework shows code snippets on failure, the markers point straight to the literate source.
- *Disable markers selectively.* Use explicit flags on `@o` (e.g. `@o file -t`) with no comment flags to suppress markers for a specific output file when comments would interfere—for example, binary formats, data files, or files where comments are not valid syntax.

1.7 Restrictions

Because `weft` is intended to be a simple tool, a few restrictions have been established. Over time, some of these may be eliminated; others seem fundamental.

- The handling of errors is not completely ideal. In some cases, `weft` simply warns of a problem and continues; in other cases it halts immediately. This behavior should be regularized.
- `weft` warns about references to fragments that haven’t been defined, but doesn’t halt. The name of the fragment is included in the output file surrounded by `<>` signs. This seems most convenient for development, but may change in the future.
- File names and index entries should not contain any `@` signs.
- Fragment names may be (almost) any well-formed `TeX` string. It makes sense to change fonts or use math mode; however, care should be taken to ensure matching braces, brackets, and dollar signs.
- Anything is allowed in the body of a scrap; however, very long scraps (horizontally or vertically) may not typeset well.

- Temporary files (created for comparison to the eventual output files) are placed in the current directory. Since they may be renamed to an output file name, all the output files should be on the same file system as the current directory. Alternatively, you can use the `-p` flag to specify where the files go.
- Because page numbers cannot be determined until the document has been typeset, `weft` must be rerun after `LATEX` to obtain a clean version of the document (very similar to the way it is sometimes necessary to rerun `LATEX` to obtain an up-to-date table of contents after significant edits). `weft` will warn (in most cases) when this needs to be done; in the remaining cases, `LATEX` will warn that labels may have changed. This restriction does not apply to Markdown output (`-w md`), which uses sequential numbering and requires no additional processing passes.

Very long scraps may be allowed to break across a page if declared with `@O` or `@D` (instead of `@o` and `@d`). This doesn't work very well as a default, since far too many short scraps will be broken across pages; however, as a user-controlled option, it seems very useful.

1.8 Project Introspection

The `-m` and `-e` flags support machine-readable introspection of a `weft` project. They run after pass 1 has built the name trees and do *not* invoke the weave or tangle phases.

1.8.1 Map (`-m`)

The command

```
weft -m project.weft
```

writes a JSON object to stdout with two arrays, `"files"` and `"fragments"`. For each entry the map records the name, language tag, definition scraps (with source file and line), and dependency relationships. File entries list the fragments they use; fragment entries list the scraps that invoke them and the fragments they reference in turn.

This gives an AI assistant or any external tool a complete dependency graph of the project in a single invocation, without having to parse the `weft` source files.

1.8.2 Extract (`-e`)

The command

```
weft -e "fragment name" project.weft
```

finds the named fragment (using prefix matching), then emits its scrap code and dependency information as Markdown to stdout. Referenced fragments are followed transitively, so the output contains the full closure of the requested fragment.

This allows an AI assistant to retrieve exactly the code it needs to understand or modify a specific fragment, together with all its dependencies, in one step.

1.8.3 Reverse Map (`-R`)

The `-m` and `-e` flags require the `.weft` source files. The reverse map provides the opposite direction: given a tangled output file, it parses the section markers already embedded in that file and reports which `.weft` source and line produced each region. No source files are needed and pass 1 is never invoked.

Single-line query.

```
weft -R server.js:47
```

emits a JSON object identifying the innermost scrap covering line 47:

```
{"scrap": 3, "file": "routes.weft", "line": 42}
```

If the line is outside any scrap region, an empty object `{}` is returned.

Full-file mapping.

```
weft -R server.js
```

emits a JSON object listing every scrap region in the file:

```
{ "source": "server.js", "regions": [
  { "start": 2, "end": 15, "scrap": 3,
    "file": "routes.weft", "line": 42 },
  { "start": 5, "end": 8, "scrap": 7,
    "file": "database.weft", "line": 18 }
]}
```

Regions may overlap when fragments are nested.

This enables CI/CD pipelines to translate compiler errors back to the literate source automatically, and lets AI agents resolve tangled-output locations without loading the `.weft` files at all.

1.9 Acknowledgements

Several people have contributed their time, ideas, and debugging skills. In particular, the contributions of the following people are gratefully acknowledged: Osman Buyukisik, Manuel Carriba, Adrian Clarke, Tim Harvey, Michael Lewis, Walter Ravenek, Rob Shillingsburg, Kayvan Sylvan, Dominique de Waleffe, and Scott Warren. Of course, most of these people would never have heard of nuweb (the original tool) without the efforts of George Greenwade.

Since maintenance has been taken over by Marc Mengel, Simon Wright and Keith Harwood online contributions have been made by:

- Walter Brown <wb@fnal.gov>
- Nicky van Foreest <n.d.vanforeest@math.utwente.nl>
- Javier Goizueta <jgoizueta@jazzfree.com>
- Alan Karp <karp@hp.com>

Chapter 2

The Overall Structure

Processing a web requires three major steps:

1. Read the source, accumulating file names, fragment names, scraps, and lists of cross-references.
2. Reread the source, copying out to the documentation file, with protection and cross-reference information for all the scraps.
3. Traverse the list of files names. For each file name:
 - (a) Dump all the defining scraps into a temporary file.
 - (b) If the file already exists and is unchanged, delete the temporary file; otherwise, rename the temporary file.

2.1 Files

I have divided the program into several files for quicker recompilation during development.

```
<global.h 21a> ≡  
<Include files 21b>  
<Type declarations 23a, ...>  
<Limits 23b>  
<Global variable declarations 27a, ...>  
<Function prototypes 41a, ...>  
<Operating System Dependencies 26>
```

21a

We'll need at least five of the standard system include files. We also require `_POSIX_C_SOURCE` at level 200809L so that POSIX functions like `strdup` are visible when compiling under strict C11 mode (`-std=c11`).

```
<Include files 21b> ≡  
  
#define _POSIX_C_SOURCE 200809L  
  
#include <stdlib.h>  
#include <stdio.h>  
#include <string.h>  
#include <ctype.h>  
#include <signal.h>  
#include <locale.h>
```

21b · used in 21a · defines `exit` 25f, 29d, 36bd, 39b, 45d, 46a, 66b, 104ab, 109, 114a, 119ab, 120a, 121a, 126bc, 128a, 133d, 134d, 139d, 159a, 162ab, 163d, 164a, 165, 166, 170b, 175b, 184, `fclose` 54, 74a, 81c, 109, 111, 113e, 114a, 115ab, 120b, 131, 142b, `FILE` 30, 53b, 54, 61b, 65c, 67a, 68cd, 69, 74ab, 75b, 77b, 79d, 81bc, 85b, 87d, 88a, 91ab, 93b, 94b, 96a, 97a, 98b, 99, 100, 102b, 109, 111, 113e, 115ab, 117de, 122b, 123bc, 124a, 130c, 131, 134b, 141b, 142b, 176ef, 177d, 178a, 179cd, 184, `fopen` 54, 81c, 109, 111, 114a, 115ab, 119b, 121a, 142b, `fprintf` 33, 36cd, 39b, 41c, 43d, 45d, 46a, 48e, 52b, 54, 56a, 58c, 60a, 62ae, 64g, 65b, 66b, 70d, 72efg, 74b, 76a, 79e, 81c, 84b, 86ae, 87c, 90bcd, 91b, 93c, 96b, 97c, 98d, 99, 100, 102b, 104ab, 106, 107a, 109, 113e, 114a, 116, 119ab, 120a, 121a, 123b, 124g, 126bc, 128a, 130c, 131, 133d, 134d, 136bc, 137a, 139cd, 141ab, 146c, 156, 158, 159ab, 160a, 162ab, 163d, 164a, 165, 166, 170b, 175b, 177b, 178c, 180abef, `fputs` 49a, 50c, 52b, 55ab, 56b, 57a, 58bc, 60ab, 61b, 62abcde, 64aefg, 65abc, 67a, 68cd, 69, 70e, 71b, 72g, 73, 74ab, 75a, 76abc, 78abc, 79ce, 83b, 84bc, 85b, 86ae, 87bcd, 88ab, 89b, 90abd, 91ab, 93acde, 94c, 95abc, 96b, 97bc, 98acd, 99, 100, 101, 102b, 103c, 109, 112, 115a, 123b, 131, 135, 139c, 140a, 177abc, 178bc, `getc` 115b, 118a, 119ab, 120ab, 121a, `getenv` 36c, `isgraph` 120a, 159a, 165, `islower` 147c, `isspace` 64a, 86b, 128a, 159ab, 162a, 164a, `isupper` never used, `malloc` 77b, 94b, 170b, 182d, `putc` 49ab, 57bc, 58c, 67a, 69, 70c, 72g, 74ab, 76ab, 78abc, 79e, 82abc, 88ab, 89ab, 90d, 91b, 93c, 94c, 95a, 123b, 135, 137ce, 138a, 139d, 140abc, `remove` 114b, 115ab, `setlocale` 36c, `size_t` 74a, 107c, 112, 181d, 182bd, `stderr` 33, 36cd, 39b, 41c, 43d, 45d, 46a, 54, 58c, 64g, 65b, 66b, 70d, 72g, 74b, 81c, 86e, 87c, 90d, 91b, 104ab, 106, 107a, 109, 113e, 114a, 116, 119ab, 120a, 121a, 124g, 126bc, 128a, 133d, 134d, 139d, 146c, 156, 158, 159ab, 160a, 162ab, 163d, 164a, 165, 166, 170b, 175b, 180bef, `strlen` 49a, 105a, 109, 142c, 145c, 146a, 171d, 179a, `toupper` 147c

I like to use `TRUE` and `FALSE` in my code. I'd use an `enum` here, except that some systems seem to provide definitions of `TRUE` and `FALSE` by default. The following code seems to work on all the local systems.

< Type declarations 23a > ≡

```
#ifndef FALSE
#define FALSE 0
#endif
#ifndef TRUE
#define TRUE 1
#endif

#if defined(__GNUC__) && __GNUC__ >= 7
#define FALLTHROUGH __attribute__((fallthrough))
#elif defined(__clang__)
#define FALLTHROUGH __attribute__((fallthrough))
#else
#define FALLTHROUGH ((void)0)
#endif
```

23a · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines `FALLTHROUGH` 43d, 57c, 70e, 82c, 118a, 146b, `FALSE` 27ab, 33, 35abcde, 36a, 39b, 57bc, 62d, 64d, 65c, 68d, 69, 74a, 78b, 79be, 82abc, 84a, 86c, 87d, 91b, 95a, 96b, 97c, 98d, 103a, 119a, 121a, 139d, 142b, 147a, 149ab, 156, 159b, 160b, 176b, 177b, 178cd, 179a, `TRUE` 27ab, 29d, 33, 35d, 39b, 52b, 57c, 62be, 65c, 70d, 74a, 78b, 79be, 82abc, 84a, 86ab, 87d, 91b, 95a, 96b, 97c, 98d, 102b, 103c, 119a, 123c, 124g, 139d, 147a, 149ab, 156, 159b, 176b, 177b, 178cd, 179a

The `FALLTHROUGH` macro marks intentional switch case fall-throughs as a statement rather than a comment. GCC 7+ and Clang recognise `__attribute__((fallthrough))` and suppress `-Wimplicit-fallthrough` warnings; other compilers see a harmless `((void)0)`. The prose surrounding each switch already explains why the fall-through is intended, so no inline comment is needed.

Here we define the maximum length for names (of fragments etc), and the version string reported by `--version`. The version is a hard-coded literal — the single source of truth for weft's own version — so that *any* build from a fresh checkout carries the correct version, whether it is compiled with `make bootstrap`, re-tangled with plain `make`, or released with `make dist`. Cutting a release is therefore a one-line edit here. (The `-V` flag and the `@v` directive remain a separate, user-facing feature for stamping a version into *other* documents woven by weft; they no longer drive weft's own version.)

< Limits 23b > ≡

```
#ifndef MAX_NAME_LEN
#define MAX_NAME_LEN 1024
#endif

#define WEFT_VERSION "< version 1 >"
```

23b · used in 21a · defines `MAX_NAME_LEN` 128a, 130c, 133d, 159a, 162a, 165, 167a, 175b, 179b, `WEFT_VERSION` 27b, 29d, 30

2.1.1 The Main Files

The code is divided into four main files (introduced here) and five support files (introduced in the next section). The file `main.c` will contain the driver for the whole program (see Section 2.2).

< main.c 23c > ≡

```
#include "global.h"
```

23c · defined at 23c, 25f

The first pass over the source file is contained in `pass1.c`. It handles collection of all the file names, fragments names, and scraps (see Section 2.3).

```
<pass1.c 23d> ≡  
#include "global.h"  
23d · defined at 23d, 41bc
```

The `.tex` file is created during a second pass over the source file. The file `latex.c` contains the code controlling the construction of the `.tex` file (see Section 2.4).

```
<latex.c 24a> ≡  
#include "global.h"  
static int scraps = 1;  
24a · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · uses scraps 122a
```

The Markdown output is created by the code in `markdown.c` (see Section 2.5).

```
<markdown.c 24b> ≡  
#include "global.h"  
static int scraps = 1;  
24b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · uses scraps 122a
```

The code controlling the creation of the output files is in `output.c` (see Section 2.7).

```
<output.c 24c> ≡  
#include "global.h"  
24c · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c
```

2.1.2 Support Files

The support files contain a variety of support routines used to define and manipulate the major data abstractions. The file `input.c` holds all the routines used for referring to source files (see Section 3.1).

```
< input.c 25a > ≡  
#include "global.h"  
25a · defined at 25a, 117de, 118ac, 121a
```

Creation and lookup of scraps is handled by routines in `scraps.c` (see Section 3.2).

```
< scraps.c 25b > ≡  
#include "global.h"  
25b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde,  
170abc, 171c, 172cd, 176bcf, 178ad, 179ac
```

The handling of file names and fragment names is detailed in `names.c` (see Section 3.3).

```
< names.c 25c > ≡  
#include "global.h"  
25c · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc
```

Memory allocation and deallocation is handled by routines in `arena.c` (see Section 3.6).

```
< arena.c 25d > ≡  
#include "global.h"  
25d · defined at 25d, 181e, 182ab, 183
```

Finally, for best portability, I seem to need a file containing (useless!) definitions of all the global variables.

```
< global.c 25e > ≡  
#include "global.h"  
< Operating System Dependencies 26 >  
< Global variable definitions 27b, ... >  
25e · defined at 25e, 181a
```

2.2 The Main Routine

The main routine is quite simple in structure. It wades through the optional command-line arguments, then handles any files listed on the command line.

```
< main.c 25f > + ≡  
  
#include <stdlib.h>  
int main(int argc, char **argv)  
{  
    int arg = 1;  
    < Interpret command-line arguments 29b, ... >  
    < Set locale information 36c >  
    < Handle reverse map mode 36b >  
    initialise_delimit_scrap_array();  
    < Process the remaining arguments (file names) 36d >  
    exit(0);  
}
```

We only have two major operating system dependencies; the separators for file names, and how to set environment variables. For now we assume the latter can be accomplished via `putenv()` in `stdlib.h`.

⟨ *Operating System Dependencies* 26 ⟩ ≡

```
#if defined(VMS)
#define PATH_SEP(c) (c=='|' || c==':')
#define PATH_SEP_CHAR ""
#define DEFAULT_PATH ""
#elif defined(MSDOS)
#define PATH_SEP(c) (c=='\\')
#define PATH_SEP_CHAR "\\"
#define DEFAULT_PATH "."
#else
#define PATH_SEP(c) (c=='/')
#define PATH_SEP_CHAR "/"
#define DEFAULT_PATH "."
#endif
```

26 · used in 21a, 25e

2.2.1 Command-Line Arguments

There are numerous possible command-line arguments:

- t Suppresses generation of the `.tex` file.
- o Suppresses generation of the output files.
- d list dangling identifier references in indexes.
- c Forces output files to overwrite old files of the same name without comparing for equality first.
- v The verbose flag. Forces output of progress reports.
- n Forces sequential numbering of scraps (instead of page numbers).
- s Doesn't print list of scraps making up file at end of each scrap.
- x Include cross-reference numbers in scrap comments.
- p *path* Prepend *path* to the filenames for all the output files.
- h *options* Provide options for the `hyperref` package.
- r Turn on hyperlinks. You must include the `—usepackage—` options in the text.
- l Deprecated no-op. Code scraps are always formatted with the `listings` package now, so this flag has no effect and is kept only for backward compatibility. Remember to `\usepackage{listings}` in your document preamble.

There are two ways to get hyper-links into your documentation. With one, the `-r` flag simply arranges for fragment cross-references to be hyperlinks. You have to provide all the rest of the `hyperref` material yourself. The `-h` flag does more work, but requires you provide the `hyperref` options on the command line. You would use the first if you know you will use the same tools all the time. You would use the second if you have to distribute the weft file and use the build system to provide the `hyperref` options.

The `-h` option is used as follows. If you want hyperlinks in your document give a string to this option which is a valid option for the `hyperref` package. For example, `"dvips,colorlinks=true"` sets it to use

the `dvips` driver and mark the links in colour. For more information about these options, see the `hyperref` documentation.

Then in your document you put the command `\WEFTuseHyperlinks` immediately after your `usepackage` commands. If you do both of these you will get hyperlinks. If you miss out either then nothing interesting happens.

Global flags are declared for each of the arguments.

< Global variable declarations 27a > ≡

```
extern int tex_flag;      /* if FALSE, don't emit the documentation file */
extern int output_flag;  /* if FALSE, don't emit the output files */
extern int compare_flag; /* if FALSE, overwrite without comparison */
extern int verbose_flag; /* if TRUE, write progress information */
extern int number_flag;  /* if TRUE, use a sequential numbering scheme */
extern int scrap_flag;   /* if FALSE, don't print list of scraps */
extern int dangling_flag; /* if FALSE, don't print dangling identifiers */
extern int xref_flag;    /* If TRUE, print cross-references in scrap comments */
extern int prepend_flag; /* If TRUE, prepend a path to the output file names */
extern char * dirpath;   /* The prepended directory path */
extern char * path_sep;  /* How to join path to filename */
extern int listings_flag; /* if TRUE, use listings package for scrap formatting */
extern int version_info_flag; /* If TRUE, set up version string */
extern char * version_string; /* What to print for @v */
extern int hyperref_flag; /* Are we preparing for hyperref
                           package. */
extern int hyperopt_flag; /* Are we preparing for hyperref options */
extern char * hyperoptions; /* The options to pass to the
                             hyperref package */
extern int includepath_flag; /* Do we have an include path? */
extern struct incl{char * name; struct incl * next;} * include_list;
                           /* The list of include paths */
extern int markdown_flag; /* if TRUE, generate Markdown instead of LaTeX */
extern int weave_flag;    /* if TRUE, weave was requested via -w */
extern int weave_format;  /* 0=none, 1=tex, 2=md */
extern int weave_format_from_cli; /* if TRUE, CLI override of @W */
extern int map_flag;      /* if TRUE, emit JSON map to stdout */
extern int extract_flag;  /* if TRUE, next arg is extract name */
extern char *extract_name; /* fragment name to extract */
extern int reverse_map_flag; /* if TRUE, reverse map mode */
extern char *reverse_map_arg; /* file[:line] argument for -R */
extern int bodies_flag;    /* if TRUE, include scrap bodies in -m JSON */
extern int prose_flag;     /* if TRUE, include prose context in -m JSON */
extern int diff_flag;      /* if TRUE, show diff instead of writing files */
extern int callers_flag;   /* if TRUE, show callers with -e */
extern int lint_flag;      /* if TRUE, run static analysis */
extern int errors_flag;    /* if TRUE, annotate compiler errors from stdin */
```

27a · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `bodies_flag` 27b, 29d, 100, `callers_flag` 27b, 29d, 39b, `compare_flag` 27b, 33, 114b, `dangling_flag` 27b, 33, 79e, 96b, `diff_flag` 27b, 29d, 114b, `errors_flag` 27b, 29d, 36b, `extract_flag` 27b, 33, 35e, `extract_name` 27b, 35e, 39b, `hyperoptions` 27b, 35d, 55a, 56a, `hyperopt_flag` 27b, 33, 35d, `hyperref_flag` 27b, 33, 35d, 55a, `incl` 27b, 35b, 39a, 119b, `includepath_flag` 27b, 33, 35b, `lint_flag` 27b, 29d, 39b, `map_flag` 27b, 33, 39b, `markdown_flag` 27b, 33, 39b, `number_flag` 27b, 33, 39b, 142b, `output_flag` 27b, 33, 39b, `prose_flag` 27b, 29d, 41d, 42b, 100, `reverse_map_arg` 27b, 36ab, `reverse_map_flag` 27b, 33, 36a, `scrap_flag` 27b, 33, 60a, 86d, `tex_flag` 27b, 33, 39b, 41c, 139d, `verbose_flag` 27b, 33, 41c, 54, 81c, 113e, `version_info_flag` 27b, 33, 35c, `version_string` 27b, 35c, 58b, 71b, 83b, `weave_flag` 27b, 33, 39b, `weave_format` 27b, 33, 39b, 158, `weave_format_from_cli` 27b, 33, 39b, 158, `xref_flag` 27b, 33, 140ac · uses `FALSE` 23a, `next` 121b, `prose` 121c, `reverse` 168c, `scraps` 122a, `TRUE` 23a

The flags are all initialized for correct default behavior.

⟨ *Global variable definitions* 27b ⟩ ≡

```
int tex_flag = TRUE;
int output_flag = TRUE;
int compare_flag = TRUE;
int verbose_flag = FALSE;
int number_flag = FALSE;
int scrap_flag = TRUE;
int dangling_flag = FALSE;
int xref_flag = FALSE;
int prepend_flag = FALSE;
char * dirpath = DEFAULT_PATH; /* Default directory path */
char * path_sep = PATH_SEP_CHAR;
int listings_flag = FALSE;
int version_info_flag = FALSE;
char default_version_string[] = WEFT_VERSION;
char * version_string = default_version_string;
int hyperref_flag = FALSE;
int hyperopt_flag = FALSE;
char * hyperoptions = "";
int includepath_flag = FALSE; /* Do we have an include path? */
struct incl * include_list = NULL;
/* The list of include paths */
int markdown_flag = FALSE;
int weave_flag = FALSE;
int weave_format = 0;
int weave_format_from_cli = FALSE;
int map_flag = FALSE;
int extract_flag = FALSE;
char *extract_name = NULL;
int reverse_map_flag = FALSE;
char *reverse_map_arg = NULL;
int bodies_flag = FALSE;
int prose_flag = FALSE;
int diff_flag = FALSE;
int callers_flag = FALSE;
int lint_flag = FALSE;
int errors_flag = FALSE;
```

27b · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `bodies_flag` 27a, `callers_flag` 27a, `compare_flag` 27a, `dangling_flag` 27a, `diff_flag` 27a, `errors_flag` 27a, `extract_flag` 27a, `extract_name` 27a, `FALSE` 23a, `hyperoptions` 27a, `hyperopt_flag` 27a, `hyperref_flag` 27a, `incl` 27a, `includepath_flag` 27a, `lint_flag` 27a, `map_flag` 27a, `markdown_flag` 27a, `number_flag` 27a, `output_flag` 27a, `prose_flag` 27a, `reverse_map_arg` 27a, `reverse_map_flag` 27a, `scrap_flag` 27a, `tex_flag` 27a, `TRUE` 23a, `verbose_flag` 27a, `version_info_flag` 27a, `version_string` 27a, `weave_flag` 27a, `weave_format` 27a, `weave_format_from_cli` 27a, `WEFT_VERSION` 23b, `xref_flag` 27a

A global variable `nw_char` will be used for the weft meta-character, which by default will be @.

⟨ *Global variable declarations* 28a ⟩ + ≡

```
extern int nw_char;
```

28a · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `nw_char` 28b, 30, 39b, 41d, 43d, 45d, 46a, 47a, 48bf, 49d, 51, 52ab, 57bc, 60a, 66a, 68c, 69, 70ae, 72ce, 76a, 79e, 82ac, 88a, 89b, 90b, 118a, 119a, 121a, 126bc, 127a, 128a, 129bc, 130c, 133d, 135, 137d, 138a, 139cd, 141abd, 146c, 156, 158, 159a, 162ab, 163a, 164a, 165, 166, 167a, 175ab, 177b, 178cd, 179b

⟨ *Global variable definitions* 28b ⟩ + ≡

```
int nw_char='@';
```

28b · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · defines `nw_char` 28a, 30, 39b, 41d, 43d, 45d, 46a,

47a, 48bf, 49d, 51, 52ab, 57bc, 60a, 66a, 68c, 69, 70ae, 72ce, 76a, 79e, 82ac, 88a, 89b, 90b, 118a, 119a, 121a, 126bc, 127a, 128a, 129bc, 130c, 133d, 135, 137d, 138a, 139cd, 141abd, 146c, 156, 158, 159a, 162ab, 163a, 164a, 165, 166, 167a, 175ab, 177b, 178cd, 179b

We save the invocation name of the command in a global variable `command_name` for use in error messages.

⟨ *Global variable declarations* 28c ⟩ + ≡

```
extern char *command_name;
```

28c · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `command_name` 29ab, 30, 33, 36d, 39b, 43d, 45d, 46a, 54, 58c, 65b, 70d, 72g, 74b, 81c, 87c, 90d, 91b, 104ab, 114a, 116, 119ab, 120a, 121a, 124g, 126bc, 128a, 133d, 134d, 139d, 146c, 156, 158, 159ab, 160a, 162ab, 163d, 164a, 165, 166, 175b

⟨ *Global variable definitions* 29a ⟩ + ≡

```
char *command_name = NULL;
```

29a · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `command_name` 28c

The invocation name is conventionally passed in `argv[0]`.

⟨ *Interpret command-line arguments* 29b ⟩ ≡

```
command_name = argv[0];
```

29b · defined at 29bc · used in 25f · uses `command_name` 28c

We need to examine the remaining entries in `argv`, looking for command-line arguments.

⟨ *Interpret command-line arguments* 29c ⟩ + ≡

```
while (arg < argc) {
    char *s = argv[arg];
    if (*s++ == '-') {
        ⟨ Check for long options 29d ⟩
        ⟨ Interpret the argument string s 33 ⟩
        arg++;
        ⟨ Perhaps get the prepend path 35a ⟩
        ⟨ Perhaps get the version info string 35c ⟩
        ⟨ Perhaps get the hyperref options 35d ⟩
        ⟨ Perhaps add an include path 35b ⟩
        ⟨ Perhaps get the extract name 35e ⟩
        ⟨ Perhaps get the reverse map argument 36a ⟩
    }
    else break;
}
```

29c · defined at 29bc · used in 25f

2.2.2 Help Message

When the user passes `--help`, we display a comprehensive reference that serves both human users and AI assistants. The help text is structured in sections—usage, options grouped by purpose, examples, and an AI workflow summary—so that a language model can parse it and use `weft` correctly without additional context.

We intercept long options before the character-by-character flag parsing. Currently `--help` and `--version` are the only long options.

⟨ *Check for long options* 29d ⟩ ≡

```
if (*s == '-') {
    if (strcmp(s + 1, "help") == 0) {
        ⟨ Print help message 30 ⟩
    }
}
```

```

    exit(0);
}
if (strcmp(s + 1, "version") == 0) {
    printf("weft %s\n", WEFT_VERSION);
    exit(0);
}
if (strcmp(s + 1, "bodies") == 0) {
    bodies_flag = TRUE;
    arg++;
    continue;
}
if (strcmp(s + 1, "prose") == 0) {
    prose_flag = TRUE;
    arg++;
    continue;
}
if (strcmp(s + 1, "diff") == 0) {
    diff_flag = TRUE;
    arg++;
    continue;
}
if (strcmp(s + 1, "callers") == 0) {
    callers_flag = TRUE;
    arg++;
    continue;
}
if (strcmp(s + 1, "lint") == 0) {
    lint_flag = TRUE;
    arg++;
    continue;
}
if (strcmp(s + 1, "errors") == 0) {
    errors_flag = TRUE;
    arg++;
    continue;
}
}

```

29d · used in 29c · uses [bodies_flag 27a](#), [callers_flag 27a](#), [diff_flag 27a](#), [errors_flag 27a](#), [exit 21b](#), [lint_flag 27a](#), [prose 121c](#), [prose_flag 27a](#), [TRUE 23a](#), [WEFT_VERSION 23b](#)

The help message itself. We write to `stdout` (not `stderr`) because `--help` is a deliberate request, not an error condition. The output is structured so that both humans and AI assistants can use `weft` effectively from `--help` alone: options grouped by function, per-file output flags, section marker format, a complete directives reference, and a concise AI workflow section.

⟨ *Print help message 30* ⟩ ≡

```

printf("weft %s -- A Literate Programming Tool\n\n", WEFT_VERSION);
printf("USAGE\n");
printf("  %s [options] file...\n\n", command_name);
printf("WHAT IS WEFT?\n");
printf("  weft is a literate programming tool: write .weft files where\n");
printf("  documentation and code live as one narrative.  weft TANGLES them\n");
printf("  into executable output files and optionally WEAVES formatted\n");
printf("  documentation (LaTeX or Markdown).  .weft files are the ONLY\n");
printf("  source of truth; tangled outputs are GENERATED -- never edit them.\n\n");
printf("HOW IT WORKS\n");
printf("  A .weft file contains prose with embedded code chunks:\n");
printf("    @o file.c @{ ... @}      Output file: tangle writes this to file.c\n");

```

```

printf("    @d Name    @{ ... @}    Fragment: reusable code block, invoked by name\n");
printf("    @<Name@>      Invocation: inserts the named fragment here\n");
printf("    @@              Literal @: escapes the meta-character\n");
printf("    @i other.weft    Include: pulls in another .weft file\n\n");
printf("Fragments compose hierarchically. An @o chunk may invoke @d fragments,\n");
printf("which invoke other fragments, forming a tree that weft flattens into\n");
printf("the final output file.\n\n");
printf("OPTIONS\n");
printf("Tangle control:\n");
printf("  -o              Suppress output file generation (parse only)\n");
printf("  -c              Overwrite files without comparing first\n");
printf("  -p path         Prepend path to all output file names\n");
printf("  -I path         Add directory to include search path (repeatable)\n");
printf("  -x              Include cross-reference numbers in section markers\n\n");
printf("Weave control:\n");
printf("  -w [format]     Weave documentation (format: md or tex)\n");
printf("                  Without format, uses @W directive from source\n");
printf("  -t              Suppress weave output (even if -w given)\n");
printf("  -n              Sequential scrap numbering (auto for Markdown)\n");
printf("  -s              Omit scrap composition lists\n\n");
printf("Analysis (AI-friendly tools):\n");
printf("  -m              Emit JSON structure map to stdout (all fragments,\n");
printf("                  files, definitions, and dependency graph)\n");
printf("  -e name         Extract named fragment and all its transitive\n");
printf("                  dependencies as self-contained Markdown to stdout\n");
printf("  -R file[:line] Reverse map: given a tangled output file (and\n");
printf("                  optional line number), prints the .weft source\n");
printf("                  location that generated it. Needs NO .weft files\n");
printf("                  -- reads section markers embedded in the output\n");
printf("  --bodies        Include raw scrap body text in -m JSON output\n");
printf("  --prose         Include preceding documentation in -m JSON output\n");
printf("  --diff          Show unified diff of what tangle would change\n");
printf("                  (no files written)\n");
printf("  --lint          Static analysis: unused fragments, undefined refs,\n");
printf("                  and fuzzy-match suggestions for likely typos\n");
printf("  --callers       With -e, show call chain from %co roots to fragment\n",
    nw_char);
printf("  --errors        Annotate compiler errors from stdin with .weft\n");
printf("                  source locations (reads section markers, no\n");
printf("                  .weft files needed)\n");
printf("                  Example: make 2>&1 | weft --errors\n\n");
printf("LaTeX:\n");
printf("  -r              Enable hyperref cross-reference links\n");
printf("  -h options      Pass options to hyperref package\n");
printf("  -l              Deprecated no-op (listings is always used)\n");
printf("  -d              List dangling identifier references\n\n");
printf("General:\n");
printf("  -V string       Set version string (expands @v in source)\n");
printf("  -v              Verbose progress output\n");
printf("  --help          Show this help message\n");
printf("  --version       Show version number\n\n");
printf("PER-FILE FLAGS (on @o)\n");
printf("  @o file [-d] [-i] [-t] [-s] [-cc|-c+|-cp] @{ ... @} \n");
printf("  -d              Emit #line N \"file\" directives (C/C++ debuggers)\n");
printf("  -i              Suppress auto-indentation of expanded fragments\n");
printf("  -t              Suppress tab expansion (write tabs literally)\n");
printf("  -cc             C block-comment markers: /* {N: file:line} */\n");
printf("  -c+            C++ line-comment markers: // {N: file:line}\n");
printf("  -cp            Hash-comment markers: # {N: file:line}\n");

```

```

printf("    -s                Suppress all section markers in output\n");
printf(" Without explicit flags, weft auto-detects from file extension\n");
printf(" (250+ extensions recognized). Use @l tag before @o to override.\n\n");
printf("SECTION MARKERS\n");
printf(" When comment style is known, tangled output includes markers:\n");
printf("    /* {5: source.weft:42} */    opening -- scrap 5, from line 42\n");
printf("    /* {5} */                    closing\n");
printf(" Comment adapts to language: // for C++/Go, # for Python/shell, etc.\n");
printf(" Use weft -R to parse these markers and reverse-map to source.\n\n");
printf("DIRECTIVES REFERENCE\n");
printf(" Core structure:\n");
printf("    @o file [flags] @{ code @}    Output file (tangled to disk)\n");
printf("    @d Name @{ code @}            Define reusable named fragment\n");
printf("    @d+ Name @{ code @}           Global fragment (visible across sectors)\n");
printf("    @q Name @{ code @}            Quoted fragment (no expansion on include)\n");
printf("    @Q / @D / @Q                 Uppercase = big-format scrap in LaTeX\n");
printf("    @<Name@>                      Invoke (expand) a named fragment\n");
printf("    @<+Name@>                     Invoke the global version of a fragment\n");
printf("    @i file                       Include another .weft file\n");
printf("    @@                           Literal @ character\n\n");
printf(" Fragment parameters:\n");
printf("    @'text@'                      Pass literal text argument to a fragment\n");
printf("    @l .. @g                     Substitute nth argument inside fragment\n");
printf("    @{ code @} as argument        Pass anonymous inline scrap as argument\n\n");
printf(" Inside scraps:\n");
printf("    @f                            Current output file name\n");
printf("    @t                            Current fragment title (as invoked)\n");
printf("    @v                            Version string (defaults to weft's version; -V\n");
printf(" overrides)\n");
printf("    @c                            Expand block comment at current indent\n");
printf("    @#                            Suppress auto-indent for this line\n");
printf("    @%%                           Comment to end of line (omitted)\n");
printf("    @| name                       Mark identifier(s) as defined here\n");
printf("    @+ @- @*                      Global-export / import / used markers\n");
printf("    @_ text @_                   Bold (LaTeX weave); skipped in tangle\n");
printf("    @x label @x                  Cross-reference label (anchor position)\n\n");
printf(" Scrap delimiters (LaTeX weave):\n");
printf("    @{ ... @}                    Verbatim code (default)\n");
printf("    @[ ... @]                    Paragraph mode\n");
printf("    @( ... @)                    Math mode\n\n");
printf(" Sections and structure:\n");
printf("    @s                            Start a new sector (section boundary)\n");
printf("    @S                            Close current sector (return to base)\n");
printf("    @W fmt                       Declare weave format: md or tex\n\n");
printf(" Language and formatting:\n");
printf("    @l tag                        Set language for next @o or @d\n");
printf("    @L name style [+d]           Define custom language (// # -- /* <!--)\n");
printf("    @r x                         Change escape character from @ to x\n\n");
printf(" Indexes (weave output only):\n");
printf("    @f @m @u                     File / fragment / identifier indexes\n");
printf("    @m+ @u+                      Global fragment / identifier indexes\n\n");
printf("EXAMPLES\n");
printf(" %s project.weft                Tangle (default)\n", command_name);
printf(" %s -w md project.weft          Weave to Markdown\n", command_name);
printf(" %s -w tex project.weft         Weave to LaTeX\n", command_name);
printf(" %s -m project.weft | jq '.files' JSON structure map\n", command_name);
printf(" %s -e \"Validate\" project.weft   Extract fragment\n", command_name);
printf(" %s -R server.js:47             Reverse map line\n", command_name);
printf(" %s -R server.js               All regions in file\n", command_name);

```

```

printf(" %s -v -p build/ project.weft          Verbose + path prefix\n", command_name);
printf(" %s -o project.weft                    Parse only, no output\n\n", command_name);
);
printf("AI ASSISTANT WORKFLOW\n");
printf(" GOLDEN RULE: .weft files are the ONLY source of truth.\n");
printf(" Tangled outputs (*.c, *.go, etc.) are GENERATED -- never edit them.\n");
printf(" If you edit a tangled file, your changes WILL be lost on next tangle.\n\n");
printf(" Step 1 -- Understand the project structure:\n");
printf(" %s -m project.weft > map.json\n", command_name);
printf(" The JSON contains: files (all @o output files), fragments (all @d\n");
printf(" named blocks), and a dependency graph (which fragments invoke which).\n");
printf(" Useful jq queries:\n");
printf(" jq '.files'                                List output files\n");
printf(" jq '.fragments'                            List all fragments\n");
printf(" jq '.fragments[] | select(.name | test(\"Auth\"))' Search\n\n");
printf(" Step 2 -- Read specific code without opening .weft files:\n");
printf(" %s -e \"fragment name\" project.weft\n", command_name);
printf(" Prints the fragment AND all its transitive dependencies as\n");
printf(" self-contained Markdown to stdout. Ideal for understanding\n");
printf(" one piece of the system without reading the whole source.\n\n");
printf(" Step 3 -- Edit the .weft source (NEVER tangled output):\n");
printf(" Locate the right .weft file (use -m to find it), then edit\n");
printf(" the @d or @o scrap that contains the code you need to change.\n");
printf(" Use standard text editing -- the prose around code is LaTeX or\n");
printf(" plain text, and code lives between @{ and @}.\n\n");
printf(" Step 4 -- Regenerate and verify:\n");
printf(" %s project.weft          Tangle (regenerate all output files)\n",
command_name);
printf(" Then build and test as usual.\n\n");
printf(" Step 5 -- Debug errors with reverse map:\n");
printf(" %s -R output.c:42        Find .weft source for line 42\n", command_name);
printf(" %s -R output.c           Show ALL .weft regions in file\n", command_name);
printf(" Reads section markers in the tangled output -- needs NO .weft files.\n");
printf(" Fix errors in the .weft source, then re-tangle (step 4).\n");

```

30 · used in 29d · uses [command_name 28c](#), [FILE 21b](#), [first 182a](#), [Name 144b](#), [next 121b](#), [nw_char 28ab](#), [prose 121c](#), [reverse 168c](#), [scraps 122a](#), [search 172c](#), [sector 121c](#), [uses 121c](#), [WEFT_VERSION 23b](#)

2.2.3 Short Flags

Several flags can be stacked behind a single minus sign; therefore, we've got to loop through the string, handling them all. If this flag requires an argument we skip to getting its argument straight away. This allows arguments to butt up to their flags and also avoids ambiguity about which value goes with which flag.

(Interpret the argument string s 33) ≡

```

{
    char c = *s++;
    while (c) {
        switch (c) {
            case 'c': compare_flag = FALSE;
                       break;
            case 'd': dangling_flag = TRUE;
                       break;
            case 'h': hyperopt_flag = TRUE;
                       goto HasValue;
            case 'I': includepath_flag = TRUE;
                       goto HasValue;
            case 'l': listings_flag = TRUE;
                       break;

```

```

    case 'n': number_flag = TRUE;
                break;
    case 'o': output_flag = FALSE;
                break;
    case 'p': prepend_flag = TRUE;
                goto HasValue;
    case 'r': hyperref_flag = TRUE;
                break;
    case 's': scrap_flag = FALSE;
                break;
    case 't': tex_flag = FALSE;
                break;
    case 'e': extract_flag = TRUE;
                goto HasValue;
    case 'R': reverse_map_flag = TRUE;
                goto HasValue;
    case 'm': map_flag = TRUE;
                break;
    case 'v': verbose_flag = TRUE;
                break;
    case 'V': version_info_flag = TRUE;
                goto HasValue;
    case 'w': goto HasWeaveFormat;
    case 'x': xref_flag = TRUE;
                break;
    default: fprintf(stderr, "%s: unexpected argument ignored. "
                    "Try '%s --help' for usage information.\n",
                    command_name, command_name);
                break;
}
c = *s++;
}
goto HasValue;
HasWeaveFormat:
{
    char *fmt = NULL;
    weave_flag = TRUE;
    if (*s != '\0')
        fmt = s;
    else if (arg + 1 < argc && argv[arg + 1][0] != '-') {
        /* Peek: only consume next arg if it looks like a format token */
        char *candidate = argv[arg + 1];
        if (strcmp(candidate, "md") == 0 || strcmp(candidate, "tex") == 0) {
            fmt = candidate;
            ++arg;
        }
    }
    if (fmt != NULL) {
        if (strcmp(fmt, "md") == 0) {
            weave_format = 2;
            markdown_flag = TRUE;
            weave_format_from_cli = TRUE;
        } else if (strcmp(fmt, "tex") == 0) {
            weave_format = 1;
            weave_format_from_cli = TRUE;
        } else {
            fprintf(stderr, "%s: unknown weave format '%s'. Use 'md' or 'tex'.\n",
                    command_name, fmt);
        }
    }
}

```

```

    }
    /* If fmt is NULL, -w alone: format will come from @W */
    s = "";
}
HasValue;;
}

33 · used in 29c · uses command_name 28c, compare_flag 27a, dangling_flag 27a, extract_flag 27a, FALSE 23a, fprintf 21b,
hyperopt_flag 27a, hyperref_flag 27a, includepath_flag 27a, map_flag 27a, markdown_flag 27a, next 121b,
number_flag 27a, output_flag 27a, reverse_map_flag 27a, scrap_flag 27a, stderr 21b, tex_flag 27a, TRUE 23a,
verbose_flag 27a, version_info_flag 27a, weave_flag 27a, weave_format 27a, weave_format_from_cli 27a, xref_flag 27a

```

⟨ *Perhaps get the prepend path* 35a ⟩ ≡

```

if (prepend_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    dirpath = s;
    prepend_flag = FALSE;
}

```

35a · used in 29c · uses FALSE 23a

⟨ *Perhaps add an include path* 35b ⟩ ≡

```

if (includepath_flag)
{
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;

    if (*s == '\0')
        s = argv[arg++];
    le->name = save_string(s);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    includepath_flag = FALSE;
}

```

35b · used in 29c · uses arena_getmem 182b, FALSE 23a, incl 27a, includepath_flag 27a, next 121b, save_string 146a

⟨ *Perhaps get the version info string* 35c ⟩ ≡

```

if (version_info_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    version_string = s;
    version_info_flag = FALSE;
}

```

35c · used in 29c · uses FALSE 23a, version_info_flag 27a, version_string 27a

⟨ *Perhaps get the hyperref options* 35d ⟩ ≡

```

if (hyperopt_flag)
{
    if (*s == '\0')
        s = argv[arg++];
    hyperoptions = s;
    hyperopt_flag = FALSE;
}

```

```
hyperref_flag = TRUE;
}
```

35d · used in 29c · uses FALSE 23a, hyperoptions 27a, hyperopt_flag 27a, hyperref_flag 27a, TRUE 23a

⟨ *Perhaps get the extract name* 35e ⟩ ≡

```
if (extract_flag) {
    extract_name = (*s != '\0') ? s : argv[arg++];
    extract_flag = FALSE;
}
```

35e · used in 29c · uses extract_flag 27a, extract_name 27a, FALSE 23a

⟨ *Perhaps get the reverse map argument* 36a ⟩ ≡

```
if (reverse_map_flag) {
    reverse_map_arg = (*s != '\0') ? s : argv[arg++];
    reverse_map_flag = FALSE;
}
```

36a · used in 29c · uses FALSE 23a, reverse_map_arg 27a, reverse_map_flag 27a

If the user gave `-R file[:line]`, we handle it immediately and exit. No `.weft` source files are required—only the tangled output file.

⟨ *Handle reverse map mode* 36b ⟩ ≡

```
if (errors_flag) {
    write_error_annotator();
    exit(0);
}
if (reverse_map_arg) {
    write_reverse_map(reverse_map_arg);
    exit(0);
}
```

36b · used in 25f · uses errors_flag 27a, exit 21b, reverse_map_arg 27a, write_error_annotator 112, write_reverse_map 109

In order to be able to process files in foreign languages we set the locale information. `isgraph()` and friends need this (see Section 3.3). Try to read `LC_CTYPE` from the environment. Use `LC_ALL` if that fails. Don't set the locale if reading `LC_ALL` fails, too. Print a warning if setting the program's locale fails.

⟨ *Set locale information* 36c ⟩ ≡

```
{
    /* try to get locale information */
    char *s=getenv("LC_CTYPE");
    if (s==NULL) s=getenv("LC_ALL");

    /* set it */
    if (s!=NULL)
        if(setlocale(LC_CTYPE, s)==NULL)
            fprintf(stderr, "Setting locale failed\n");
}
```

36c · used in 25f · uses fprintf 21b, getenv 21b, setlocale 21b, stderr 21b

2.2.4 File Names

We expect at least one file name. While a missing file name might be ignored without causing any problems, we take the opportunity to report the usage convention.

```

< Process the remaining arguments (file names) 36d > ≡
{
    if (arg >= argc) {
        fprintf(stderr, "%s: expected a file name.\n"
            "Usage: %s [options] file...\n"
            "Try '%s --help' for more information.\n",
            command_name, command_name, command_name);
        exit(-1);
    }
    do {
        < Handle the file name in argv[arg] 38a >
        arg++;
    } while (arg < argc);
}

36d · used in 25f · uses command_name 28c, exit 21b, fprintf 21b, stderr 21b

```

The code to handle a particular file name is rather more tedious than the actual processing of the file. A file name may be an arbitrarily complicated path name, with an optional extension. If no extension is present, we add `.w` as a default. The extended path name will be kept in a local variable `source_name`. The resulting documentation file will be written in the current directory; its name will be kept in the variable `tex_name`.

```
< Handle the file name in argv[arg] 38a > ≡
{
    char source_name[FILENAME_MAX];
    char tex_name[FILENAME_MAX];
    char aux_name[FILENAME_MAX];
    char *trim;
    char *dot;
    < Build source_name and tex_name 38b >
    < Process a file 39b >
}
```

38a · used in 36d · uses `source_name` 117b

I bump the pointer `p` through all the characters in `argv[arg]`, copying all the characters into `source_name` (via the pointer `q`).

At each slash, I update `trim` to point just past the slash in `source_name`. The effect is that `trim` will point at the file name without any leading directory specifications.

The pointer `dot` is made to point at the file name extension, if present. If there is no extension, we add `.weft` to the source name. The `tex_name` (weave output name) is computed later in “Process a file” once the weave format is known.

```
< Build source_name and tex_name 38b > ≡
{
    char *p = argv[arg];
    char *q = source_name;
    char c;
    trim = q;
    dot = NULL;
    c = *p++;
    while (c) {
        *q++ = c;
        if (PATH_SEP(c)) {
            trim = q;
            dot = NULL;
        }
        else if (c == '.')
            dot = q - 1;
        c = *p++;
    }
    < Add the source path to the include path list 39a >
    *q = '\0';
    if (!dot) {
        *q++ = '.';
        *q++ = 'w';
        *q++ = 'e';
        *q++ = 'f';
        *q++ = 't';
        *q = '\0';
    }
    /* Find trim again for tex_name computation later */
    {
        char *t = source_name;
        trim = t;
```

```

    dot = NULL;
    while (*t) {
        if (PATH_SEP(*t)) { trim = t + 1; dot = NULL; }
        else if (*t == '.') dot = t;
        t++;
    }
}
}

```

38b · used in 38a · uses [source_name 117b](#)

If the source file has a directory part we add that directory to the end of the search path so that the path of last resort is the same as the source path, not, as one person put it, something irrelevant like the directory weft was invoked from.

⟨ *Add the source path to the include path list* 39a ⟩ ≡

```

if (trim != source_name) {
    struct incl * le
        = (struct incl *)arena_getmem(sizeof(struct incl));
    struct incl ** p = &include_list;
    char sv = *trim;

    *trim = '\0';
    le->name = save_string(source_name);
    le->next = NULL;
    while (*p != NULL)
        p = &((*p)->next);
    *p = le;
    *trim = sv;
}

```

39a · used in 38b · uses [arena_getmem 182b](#), [incl 27a](#), [next 121b](#), [save_string 146a](#), [source_name 117b](#)

Now that we're finally ready to process a file, it's not really too complex. We bundle most of the work into three routines `pass1` (see Section 2.3), `write_tex` (see Section 2.4), and `write_files` (see Section 2.7). After we're finished with a particular file, we must remember to release its storage (see Section 3.6).

The default behavior is to tangle only. Weaving happens only when `-w` is given. The weave format is determined by (in priority order): a CLI argument (`-w md` or `-w tex`), or the `@W` directive in the source file. If `-w` is given without a format and no `@W` is present, weft reports an error and exits. The `-t` flag suppresses weaving even if `-w` was given.

⟨ *Process a file* 39b ⟩ ≡

```

{
    int do_weave = FALSE;
    int effective_format = 0;

    /* Reset weave_format for this file unless CLI locked it */
    if (!weave_format_from_cli)
        weave_format = 0;

    pass1(source_name);
    current_sector = 1;
    prev_sector = 1;

    if (lint_flag) {
        write_lint(file_names, macro_names);
        /* skip weave and tangle */
    } else if (map_flag) {
        write_map(file_names, macro_names);
    }
}

```

```

    /* skip weave and tangle */
} else if (extract_name) {
    if (callers_flag)
        write_callers(extract_name, file_names, macro_names);
    else
        write_extract(extract_name, file_names, macro_names);
    /* skip weave and tangle */
} else {
    /* Determine whether to weave and with which format */
    if (weave_flag && !tex_flag) {
        /* -t suppresses weave: tex_flag==FALSE means user passed -t */
    } else if (weave_flag) {
        effective_format = weave_format;
        if (effective_format == 0) {
            fprintf(stderr, "%s: -w requires a weave format.\n"
                "    Use -w md, -w tex, or add %cW md/%cW tex to the source file.\n",
                command_name, nw_char, nw_char);
            exit(-1);
        }
        do_weave = TRUE;
    }

    if (do_weave) {
        /* Compute weave output name now that format is known */
        const char *weave_ext = (effective_format == 2) ? ".md" : ".tex";
        if (dot) {
            *dot = '\\0';
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
            *dot = '.';
        } else {
            sprintf(tex_name, "%s%s%s%s", dirpath, path_sep, trim, weave_ext);
            sprintf(aux_name, "%s%s%s.aux", dirpath, path_sep, trim);
        }

        if (effective_format == 2) {
            markdown_flag = TRUE;
            number_flag = TRUE;
            collect_numbers(aux_name);
            write_md(source_name, tex_name, 0);
        } else {
            collect_numbers(aux_name);
            write_tex(source_name, tex_name, 0);
        }
    }

    if (output_flag)
        write_files(file_names);
}
arena_free();
}

```

39b · used in 38a · uses arena_free 183, callers_flag 27a, collect_numbers 142b, command_name 28c, current_sector 44c, exit 21b, extract_name 27a, FALSE 23a, file_names 144c, fprintf 21b, lint_flag 27a, macro_names 144c, map_flag 27a, markdown_flag 27a, number_flag 27a, nw_char 28ab, output_flag 27a, pass1 41c, prev_sector 44c, source_name 117b, stderr 21b, tex_flag 27a, TRUE 23a, weave_flag 27a, weave_format 27a, weave_format_from_cli 27a, write_callers 104a, write_extract 104b, write_files 113c, write_lint 107a, write_map 101, write_md 81c, write_tex 54

2.3 Pass One

During the first pass, we scan the file, recording the definitions of each fragment and file and accumulating all the scraps.

< Function prototypes 41a > ≡

```
extern void pass1(char *file_name);
```

41a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses file_name 121c, pass1 41c

The routine `pass1` takes a single argument, the name of the source file. It opens the file, then initializes the scrap structures (see Section 3.2) and the roots of the file-name tree, the fragment-name tree, and the tree of user-specified index entries (see Section 3.3). After completing all the necessary preparation, we make a pass over the file, filling in all our data structures. Next, we search all the scraps for references to the user-specified index entries. Finally, we must reverse all the cross-reference lists accumulated while scanning the scraps.

< pass1.c 41b > + ≡

```
static char prose_buf[4096];
static int prose_len = 0;
static char *pending_prose = NULL;
```

41b · defined at 23d, 41bc · defines pending_prose 41c, 42b, 45ab, prose_buf 42ab, prose_len 41c, 42ab

< pass1.c 41c > + ≡

```
void pass1(char *file_name)
{
    if (verbose_flag)
        fprintf(stderr, "reading %s\n", file_name);
    source_open(file_name);
    init_scraps();
    macro_names = NULL;
    file_names = NULL;
    user_names = NULL;
    prose_len = 0;
    pending_prose = NULL;
    < Scan the source file, looking for at-sequences 41d >
    if (tex_flag)
        search();
    < Reverse cross-reference lists 50a >
}
```

41c · defined at 23d, 41bc · defines pass1 39b, 41a, 70e, 89b · uses file_name 121c, file_names 144c, fprintf 21b, init_scraps 123a, macro_names 144c, pending_prose 41b, prose_len 41b, search 172c, source_open 121a, stderr 21b, tex_flag 27a, user_names 144c, verbose_flag 27a

The only thing we look for in the first pass are the command sequences. All ordinary text is skipped entirely.

< Scan the source file, looking for at-sequences 41d > ≡

```
{
    int c = source_get();
    while (c != EOF) {
        if (c == nw_char)
            < Scan at-sequence 43d >
        else if (prose_flag) {
            < Accumulate prose character 42a >
        }
    }
}
```

```

    c = source_get();
}
}
41d · used in 41c · uses nw_char 28ab, prose_flag 27a, source_get 118a

```

```

< Accumulate prose character 42a > ≡
if (prose_len < (int)sizeof(prose_buf) - 1)
    prose_buf[prose_len++] = c;

```

42a · used in 41d · uses prose_buf 41b, prose_len 41b

```

< Save pending prose 42b > ≡
if (prose_flag && prose_len > 0) {
    /* trim trailing whitespace */
    while (prose_len > 0 && (prose_buf[prose_len-1] == ' '
        || prose_buf[prose_len-1] == '\n'
        || prose_buf[prose_len-1] == '\t'
        || prose_buf[prose_len-1] == '\r'))
        prose_len--;
    prose_buf[prose_len] = '\0';
    pending_prose = save_string(prose_buf);
    prose_len = 0;
} else {
    pending_prose = NULL;
    prose_len = 0;
}

```

42b · used in 45ab · uses pending_prose 41b, prose_buf 41b, prose_flag 27a, prose_len 41b, save_string 146a

The @l command (lowercase) sets the language for the next @o or @d directive by storing the tag in pending_lang. It replaces the earlier mechanism of detecting Markdown fences (‘‘‘lang) and L^AT_EX markers (%%lang) during pass1. A typical usage is:

```

@l python
@d Validate input
@{import re
...
@}

```

```

< Collect language apply in pass1 42c > ≡
{
    char lang[64];
    char *lp = lang;
    int lc = source_get();
    /* skip optional whitespace */
    while (lc == ' ' || lc == '\t')
        lc = source_get();
    /* read language tag */
    while (lc != EOF && lc != '\n' && lc != ' ' && lc != '\t'
        && (lp - lang) < 62) {
        *lp++ = lc;
        lc = source_get();
    }
    *lp = '\0';
    if (lang[0] != '\0') {
        strncpy(pending_lang, lang, 63);
        pending_lang[63] = '\0';
    }
    /* consume rest of line */
}

```

```

while (lc != EOF && lc != '\n')
    lc = source_get();
}

```

42c · used in 43d · uses [pending_lang 144c](#), [source_get 118a](#)

The @L command (Section 3.3.2) lets users register custom language mappings. During pass1, the actual parsing and registration is handled by `collect_lang_def()` in `names.c`. During the weave passes (L^AT_EX and Markdown), we simply consume the rest of the line so that the arguments do not leak into the typeset output.

⟨ Collect user language definition in pass1 43a ⟩ ≡
`collect_lang_def();`

43a · used in 43d · uses [collect_lang_def 156](#)

⟨ Collect weave format declaration in pass1 43b ⟩ ≡
`collect_weave_format();`

43b · used in 43d · uses [collect_weave_format 158](#)

⟨ Skip at-L language definition line 43c ⟩ ≡

```

{ int lc = source_get();
  while (lc != '\n' && lc != EOF)
    lc = source_get();
}
c = source_get();

```

43c · used in 57c, 82c · uses [source_get 118a](#)

Six at-sequences are interesting during the first pass: @o and @O (file definitions), @d, @D, and @q (fragment definitions), @c (block comments), @s/@S (sectors), @l (language apply), and @L (language definitions). We skip past all others immediately, warning if unexpected sequences are discovered.

⟨ Scan at-sequence 43d ⟩ ≡

```

{
    char quoted = 0;

    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case 'O':
        case 'o': ⟨ Build output file definition 45a ⟩
            break;
        case 'Q':
        case 'q': quoted = 1;
            FALLTHROUGH;
        case 'D':
        case 'd': ⟨ Build fragment definition 45b ⟩
            break;
        case 's':
            ⟨ Step to next sector 44a ⟩
            break;
        case 'S':
            ⟨ Close the current sector 44b ⟩
            break;
    }
}

```

```

case '<':
case '(':
case '[':
case '{': < Skip over an in-text scrap 45d >
        break;
case 'c': < Collect a block comment 46b >
        break;
case 'l': < Collect language apply in pass1 42c >
        break;
case 'L': < Collect user language definition in pass1 43a >
        break;
case 'W': < Collect weave format declaration in pass1 43b >
        break;
case 'x':
case 'v':
case 'u':
case 'm':
case 'f': /* ignore during this pass */
        break;
default: if (c==nw_char) /* ignore during this pass */
        break;
        fprintf(stderr,
                "%s: unexpected %c sequence ignored (%s, line %d)\n",
                command_name, nw_char, source_name, source_line);
        break;
}
}

```

43d · used in 41d · uses `command_name` 28c, `FALLTHROUGH` 23a, `fprintf` 21b, `nw_char` 28ab, `source_get` 118a, `source_line` 117b, `source_name` 117b, `stderr` 21b, `update_delimit_scrap` 70a

< Step to next sector 44a > ≡

```

prev_sector += 1;
current_sector = prev_sector;
c = source_get();

```

44a · used in 43d, 57c, 82c · uses `current_sector` 44c, `prev_sector` 44c, `source_get` 118a

< Close the current sector 44b > ≡

```

current_sector = 1;
c = source_get();

```

44b · used in 43d, 57c, 82c · uses `current_sector` 44c, `source_get` 118a

< Global variable declarations 44c > + ≡

```

extern unsigned char current_sector;
extern unsigned char prev_sector;

```

44c · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `current_sector` 39b, 44abd, 76c, 79c, 93e, 95c, 126a, 128a, 162a, 165, `prev_sector` 39b, 44ad

< Global variable definitions 44d > + ≡

```

unsigned char current_sector = 1;
unsigned char prev_sector = 1;

```

44d · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `current_sector` 44c, `prev_sector` 44c

2.3.1 Accumulating Definitions

There are three steps required to handle a definition:

1. Build an entry for the name so we can look it up later.
2. Collect the scrap and save it in the table of scraps.
3. Attach the scrap to the name.

We go through the same steps for both file names and fragment names.

⟨ Build output file definition 45a ⟩ ≡

```
{
    Name *name;
    int scrap;
    ⟨ Save pending prose 42b ⟩
    name = collect_file_name(); /* returns a pointer to the name entry */
    scrap = collect_scrap();    /* returns an index to the scrap */
    if (pending_prose) set_scrap_prose(scrap, pending_prose);
    pending_prose = NULL;
    ⟨ Add scrap to name's definition list 45c ⟩
}
```

45a · used in 43d · uses `collect_file_name` 159a, `collect_scrap` 125f, `Name` 144b, `pending_prose` 41b, `set_scrap_prose` 132a

⟨ Build fragment definition 45b ⟩ ≡

```
{
    Name *name;
    int scrap;
    ⟨ Save pending prose 42b ⟩
    name = collect_macro_name();
    scrap = collect_scrap();
    if (pending_prose) set_scrap_prose(scrap, pending_prose);
    pending_prose = NULL;
    ⟨ Add scrap to name's definition list 45c ⟩
}
```

45b · used in 43d · uses `collect_macro_name` 162a, `collect_scrap` 125f, `Name` 144b, `pending_prose` 41b, `set_scrap_prose` 132a

Since a file or fragment may be defined by many scraps, we maintain them in a simple linked list. The list is actually built in reverse order, with each new definition being added to the head of the list.

⟨ Add scrap to name's definition list 45c ⟩ ≡

```
{
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = scrap;
    def->quoted = quoted;
    def->next = name->defs;
    name->defs = def;
    set_scrap_owner(scrap, name);
}
```

45c · used in 45ab · uses `arena_getmem` 182b, `defs` 121c, `next` 121b, `Scrap_Node` 144a, `set_scrap_owner` 132b

⟨ Skip over an in-text scrap 45d ⟩ ≡

```
{
    int c;
    int depth = 1;
    while ((c = source_get()) != EOF) {
        if (c == nw_char)
            ⟨ Skip over at-sign or go to skipped 46a ⟩
    }
}
```

```

    fprintf(stderr, "%s: unexpected EOF in text at (%s, %d)\n",
               command_name, source_name, source_line);
    exit(-1);

skipped: ;
}

45d · used in 43d · uses command_name 28c, exit 21b, fprintf 21b, nw_char 28ab, source_get 118a, source_line 117b,
source_name 117b, stderr 21b

⟨ Skip over at-sign or go to skipped 46a ⟩ ≡

{
    c = source_get();
    switch (c) {
        case '{': case '[': case '(': case '<':
            depth += 1;
            break;
        case '}': case ']': case ')': case '>':
            if (--depth == 0)
                goto skipped;
        case 'x': case '|': case ',':
        case '%': case '1': case '2':
        case '3': case '4': case '5': case '6':
        case '7': case '8': case '9': case '_':
        case 'f': case '#': case '+': case '-':
        case 'v': case '*': case 'c': case '\\':
        case 's':
            break;
        default:
            if (c != nw_char) {
                fprintf(stderr, "%s: unexpected %c%c in text at (%s, %d)\n",
                           command_name, nw_char, c, source_name, source_line);
                exit(-1);
            }
            break;
    }
}

46a · used in 45d · uses command_name 28c, exit 21b, fprintf 21b, nw_char 28ab, source_get 118a, source_line 117b,
source_name 117b, stderr 21b

```

2.3.2 Block Comments

```

⟨ Collect a block comment 46b ⟩ ≡

{
    char * p = blockBuff;
    char * e = blockBuff + (sizeof(blockBuff)/sizeof(blockBuff[0])) - 1;

    ⟨ Skip whitespace 46c ⟩
    while (p < e)
    {
        ⟨ Add one char to the block buffer 47a ⟩
    }
    if (p == e)
    {
        ⟨ Skip to the next nw-char 48b ⟩
    }
    *p = '\000';
}

```

```
}
```

46b · used in 43d · uses `blockBuff` 48c

⟨ *Skip whitespace* 46c ⟩ ≡

```
while (source_peek == ' '
      || source_peek == '\t'
      || source_peek == '\n')
    (void)source_get();
```

46c · used in 46b · uses `source_get` 118a, `source_peek` 118a

⟨ *Add one char to the block buffer* 47a ⟩ ≡

```
int c = source_get();

if (c == nw_char)
{
    ⟨ Add an at character to the block or break 47b ⟩
}
else if (c == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    ⟨ Add any other character to the block 47c ⟩
}
```

47a · used in 46b · uses `nw_char` 28ab, `source_get` 118a, `source_ungetc` 118c

⟨ *Add an at character to the block or break* 47b ⟩ ≡

```
int cc = source_peek;

if (cc == 'c')
{
    do
        c = source_get();
    while (c <= ' ');

    break;
}
else if (cc == 'd'
        || cc == 'D'
        || cc == 'q'
        || cc == 'Q'
        || cc == 'o'
        || cc == 'O'
        || cc == EOF)
{
    source_ungetc(&c);
    break;
}
else
{
    *p++ = c;
    *p++ = source_get();
}
```

47b · used in 47a · uses `source_get` 118a, `source_peek` 118a, `source_ungetc` 118c

⟨ *Add any other character to the block* 47c ⟩ ≡

```
⟨ Perhaps skip white-space 48a ⟩
*p++ = c;
```

47c · used in 47a

⟨ *Perhaps skip white-space* 48a ⟩ ≡

```
if (c == ' ')
{
    while (source_peek == ' ')
        c = source_get();
}
if (c == '\n')
{
    if (source_peek == '\n')
    {
        do
            c = source_get();
        while (source_peek == '\n');
    }
    else
        c = ' ';
}
```

48a · used in 47c · uses `source_get` 118a, `source_peek` 118a

⟨ *Skip to the next nw-char* 48b ⟩ ≡

```
int c;
```

```
while ((c = source_get()), c != nw_char && c != EOF)/* Skip */
    source_ungetc(&c);
```

48b · used in 46b · uses `nw_char` 28ab, `source_get` 118a, `source_ungetc` 118c

⟨ *Global variable declarations* 48c ⟩ + ≡

```
extern char blockBuff[6400];
```

48c · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `blockBuff` 46b, 48df

⟨ *Global variable definitions* 48d ⟩ + ≡

```
char blockBuff[6400];
```

48d · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `blockBuff` 48c

We don't show block comments inside scraps in the T_EX file, but we do show where they go.

⟨ *Show presence of a block comment* 48e ⟩ ≡

```
{
    begin_tex_escape(file);
    fprintf(file, "\\hbox{\\sffamily\\slshape (Comment)}");
    end_tex_escape(file);
}
```

48e · used in 70e · uses `begin_tex_escape` 68d, `end_tex_escape` 68d, `fprintf` 21b

The block comment is presently in the block buffer. Here we copy it into the scrap whence we shall copy it into the output file.

⟨ *Include block comment in a scrap* 48f ⟩ ≡

```

{
    char * p = blockBuff;

    push(nw_char, &writer);
    do
    {
        push(c, &writer);
        c = *p++;
    } while (c != '\0');
}

```

48f · used in 126c · uses `blockBuff` 48c, `nw_char` 28ab, `push` 125d

⟨ *Copy block comment from scrap* 49a ⟩ ≡

```

{
    int bgn = indent + global_indent;
    int posn = bgn + strlen(comment_begin[comment_flag]);
    int i;

    ⟨ Perhaps put a delayed indent 140b ⟩
    c = pop(&reader);
    fputs(comment_begin[comment_flag], file);
    while (c != '\0')
    {
        ⟨ Move a word to the file 49b ⟩
        ⟨ If we break the line at this word 49c ⟩
        {
            putc('\n', file);
            for (i = 0; i < bgn ; i++)
                putc(' ', file);
            c = pop(&reader);
            if (c != '\0')
            {
                posn = bgn + strlen(comment_mid[comment_flag]);
                fputs(comment_mid[comment_flag], file);
            }
        }
    }
    fputs(comment_end[comment_flag], file);
}

```

49a · used in 138a · uses `comment_begin` 161b, `comment_end` 161b, `comment_mid` 161b, `fputs` 21b, `pop` 130b, `putc` 21b, `strlen` 21b

⟨ *Move a word to the file* 49b ⟩ ≡

```

do
{
    putc(c, file);
    posn += 1;
    c = pop(&reader);
} while (c > ' ');

```

49b · used in 49a · uses `pop` 130b, `putc` 21b

⟨ *If we break the line at this word* 49c ⟩ ≡

```

if (c == '\n' || (c == ' ' && posn > 60))

```

49c · used in 49a

⟨ *Skip over a block comment* 49d ⟩ ≡

```

if (last == nw_char && c == 'c')
    while ((c = pop(reader.m)) != '\0')
        /* Skip */;

```

49d · used in 174 · uses [nw_char 28ab](#), [pop 130b](#)

⟨ *Begin or end a block comment* 49e ⟩ ≡

```

if (inBlock)
{
    ⟨ End block 62d ⟩
}
else
{
    ⟨ Start block 62b ⟩
}

```

49e · used in 57c

2.3.3 Fixing the Cross References

Since the definition and reference lists for each name are accumulated in reverse order, we take the time at the end of `pass1` to reverse them all so they'll be simpler to print out prettily. The code for `reverse_lists` appears in Section 3.3.

⟨ *Reverse cross-reference lists* 50a ⟩ ≡

```

{
    reverse_lists(file_names);
    reverse_lists(macro_names);
    reverse_lists(user_names);
}

```

50a · used in 41c · uses [file_names 144c](#), [macro_names 144c](#), [reverse_lists 168b](#), [user_names 144c](#)

2.3.4 Dealing with fragment parameters

Fragment parameters were added on later in nuweb's development. There still is not, for example, an index of fragment parameters. We need a data type to keep track of fragment parameters.

⟨ *Type declarations* 50b ⟩ + ≡

```

typedef int *Parameters;

```

50b · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines [Parameters 52a](#), [84a](#), [122b](#), [133d](#), [134b](#), [135](#)

When we are copying a scrap to the output, we check for an embedded parameter. If we find it we copy it. Otherwise it's an old-style parameter and we can then pull the *n*th string from the `Parameters` list when we see an `@1 @2`, etc.

⟨ *Handle macro parameter substitution* 50c ⟩ ≡

```

case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
{
    Arglist * args;
    Name * name;

    lookup(c - '1', inArgs, inParams, &name, &args);

```

```

if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)args;
    indent = write_scraps(file, spelling, q->defs,
                          global_indent + indent,
                          indent_chars, debug_flag,
                          tab_flag, indent_flag,
                          0, 0,
                          q->args, inParams,
                          local_parameters, "");
}
else if (name != NULL) {
    int i, nargs;
    char * p = name->spelling;
    Arglist *q = args;

    < Perhaps comment this macro 140a >
    indent = write_scraps(file, spelling, name->defs,
                          global_indent + indent,
                          indent_chars, debug_flag,
                          tab_flag, indent_flag,
                          comment_flag, location_flag,
                          args, name->arg,
                          local_parameters, p);
}
else if (args != NULL) {
    if (delayed_indent) {
        < Put out the indent 137c >
    }
    fputs((char *)args, file);
}
else if ( parameters && parameters[c - '1'] ) {
    Scrap_Node param_defs;
    param_defs.scrap = parameters[c - '1'];
    param_defs.next = 0;
    write_scraps(file, spelling, &param_defs,
                  global_indent + indent,
                  indent_chars, debug_flag,
                  tab_flag, indent_flag,
                  comment_flag, location_flag,
                  NULL, NULL, 0, "");
} else if (delayed_indent) {
    < Put out the indent 137c >
}
}

```

50c · used in 138a · uses Arglist 164d, defs 121c, delayed_indent 134c, Embed_Node 167d, fputs 21b, lookup 132d, Name 144b, next 121b, Scrap_Node 144a, write_scraps 134b

Now onto actually parsing fragment parameters from a call. We start off checking for fragment parameters, an @ (sequence followed by parameters separated by @, sequences, and terminated by a @) sequence.

We collect separate scraps for each parameter, and write the scrap numbers down in the text. For example, if the file has:

```
@<foo @( param1 @, param2 @)@>
```

we actually make new scraps, say 10 and 11, for param1 and param2, and write in the collected scrap:

```
@<foo @(10@,11@)@>
```

< Save macro parameters 51 > ≡

```

{
    int param_scrap;
    char param_buf[10];

    push(nw_char, &writer);
    push('(', &writer);
    do {
        param_scrap = collect_scrap();
        sprintf(param_buf, "%d", param_scrap);
        pushs(param_buf, &writer);
        push(nw_char, &writer);
        push(scrap_ended_with, &writer);
        add_to_use(name, current_scrap);
    } while( scrap_ended_with == ',' );
    do
        c = source_get();
    while( ' ' == c );
    if (c == nw_char) {
        c = source_get();
    }
    if (c != '>') {
        /* ZZZ print error */;
    }
}

```

51 · used in 129b · uses [add_to_use 129d](#), [collect_scrap 125f](#), [nw_char 28ab](#), [push 125d](#), [pushs 125e](#), [source_get 118a](#)

If we get inside, we have at least one parameter, which will be at the beginning of the parms buffer, and we prime the pump with the first character.

⟨ Check for macro parameters 52a ⟩ ≡

```

if (c == '(') {
    Parameters res = arena_getmem(10 * sizeof(int));
    int *p2 = res;
    int count = 0;
    int scrapnum;

    while( c && c != ')' ) {
        scrapnum = 0;
        c = pop(manager);
        while( '0' <= c && c <= '9' ) {
            scrapnum = scrapnum * 10 + c - '0';
            c = pop(manager);
        }
        if ( c == nw_char ) {
            c = pop(manager);
        }
        *p2++ = scrapnum;
    }
    while (count < 10) {
        *p2++ = 0;
        count++;
    }
    while( c && c != nw_char ) {
        c = pop(manager);
    }
    if ( c == nw_char ) {
        c = pop(manager);
    }
}

```

```

    }
    *parameters = res;
}

```

52a · used in 134a · uses [arena_getmem](#) 182b, [nw_char](#) 28ab, [Parameters](#) 50b, [pop](#) 130b

These are used in `write_tex` to output the argument list for a fragment.

⟨ *Format macro parameters* 52b ⟩ ≡

```

char sep;

sep = '(';
do {
    fputc(sep,file);

    fputs("{\\footnotesize ", file);
    write_single_scrap_ref(file, scraps + 1);
    fprintf(file, "\\label{scrap%d}", scraps + 1);
    fputs(" }", file);

    source_last = '{';
    {
        /* The parameter is rendered inside a fragment reference, i.e.
           inside a listings escape, so it must be single-line LaTeX with
           the special characters neutralised: switch copy_scrap into its
           inline mode for the duration. */
        int saved_inline = code_inline;
        code_inline = TRUE;
        copy_scrap(file, TRUE, NULL);
        code_inline = saved_inline;
    }

    ++scraps;

    sep = ',';
} while ( source_last != ')' && source_last != EOF );
fputs(" ) ",file);
do
    c = source_get();
while(c != nw_char && c != EOF);
if (c == nw_char) {
    c = source_get();
}

```

52b · used in 72g, 74b · uses [code_inline](#) 68d, [copy_scrap](#) 69, [fprintf](#) 21b, [fputs](#) 21b, [nw_char](#) 28ab, [scraps](#) 122a, [source_get](#) 118a, [source_last](#) 118a, [TRUE](#) 23a, [write_single_scrap_ref](#) 123c

2.4 Writing the Latex File

The second pass (invoked via a call to `write_tex`) copies most of the text from the source file straight into a `.tex` file. Definitions are formatted slightly and cross-reference information is printed out.

Note that all the formatting is handled in this section. If you don't like the format of definitions or indices or whatever, it'll be in this section somewhere. Similarly, if someone wanted to modify `weft` to work with a different typesetting system, this would be the place to look.

⟨ *Function prototypes* 53a ⟩ + ≡

```
extern void write_tex(char *file_name, char *tex_name, unsigned char sector);
```

53a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `file_name` 121c, `sector` 121c, `write_tex` 54

We need a few local function declarations before we get into the body of `write_tex`.

```
<latex.c 53b> + ≡
static void copy_scrap(FILE *file, int prefix, Name *name);           /* formats the
    body of a scrap */
static void latex_code_putc(FILE *file, int c);                     /* escapes one
    code character for LaTeX */
static const char *latex_language(const char *lang);               /* maps a weft
    language tag to a listings name */
static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps); /* formats a
    list of scrap numbers */
static void format_entry(Name *name, FILE *tex_file, unsigned char sector);
    /* formats an index entry */
static void format_file_entry(Name *name, FILE *tex_file);          /* formats a file index
    entry */
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector);
static void write_arg(FILE *tex_file, char *p);
static void write_literal(FILE *tex_file, char *p, int mode);
static void write_ArglistElement(FILE *file, Arglist *args, char **params);

53b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · uses Arglist 164d, copy_scrap 69,
FILE 21b, format_entry 77b, format_file_entry 75b, format_user_entry 79d, latex_code_putc 67a, latex_language 67b,
Name 144b, print_scrap_numbers 65c, scraps 122a, Scrap_Node 144a, sector 121c, write_arg 61b,
write_ArglistElement 74b, write_literal 68c
```

The routine `write_tex` takes two file names as parameters: the name of the web source file and the name of the `.tex` output file.

```
<latex.c 54> + ≡
void write_tex(char *file_name, char *tex_name, unsigned char sector)
{
    (void)sector;
    FILE *tex_file = fopen(tex_name, "w");
    if (tex_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", tex_name);
        source_open(file_name);
        < Write LaTeX limbo definitions 55a >
        < Copy source_file into tex_file 57b >
        fclose(tex_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, tex_name);
}
```

54 · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `write_tex` 39b, 53a · uses `command_name` 28c, `fclose` 21b, `FILE` 21b, `file_name` 121c, `fopen` 21b, `fprintf` 21b, `sector` 121c, `source_open` 121a, `stderr` 21b, `verbose_flag` 27a

Weft's woven output relies on a family of `\WEFT...` macros. We write default definitions for all of them into the document's limbo so that source files need not define anything new. If a user wants to change any of the macros (to use `hyperref`, to write in some language other than english, or to tune the code layout) he or she can redefine the commands *after* the point where weft's definitions are read, or (better) redefine them in the preamble with `\renewcommand`.

Three groups of macros are written. The first group are the textual and cross-reference macros (`\WEFTtarget`, `\WEFTlink`, the `\WEFTtxt...` labels, and so on). The second group, the *modern decoration macros* of §2.4, give a scrap its visual identity (fragment name, scrap number, the $\equiv/+ \equiv$ marker, the one-line footer) and route all colour through the degradable `\WEFTtint` indirection. The third group, written by the scrap-formatting fragment below, are the layout macros that frame a scrap.

Code bodies are typeset with the `listings` package: every scrap becomes an `lstlisting` environment. This gives syntax highlighting (driven by the scrap’s language, see `latex_language`), automatic line wrapping (`breaklines`), and page breaking for free. The document *must* therefore load `\usepackage{listings}` in its preamble — `weft` cannot inject it, because the `limbo` is written before `\documentclass` (a colour package, `color` or `xcolor`, is likewise the user’s to load, and enables the accent and syntax colours; without it the output still compiles in black and white). Fragment references that appear in the middle of code are typeset by briefly escaping out of the listing back to L^AT_EX; see §2.4 and the `escapeinside` option.

```
< Write LaTeX limbo definitions 55a > ≡
if (hyperref_flag) {
  fputs("\\newcommand{\\WEFTtarget}[2]{\\hypertarget{#1}{#2}}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{\\hyperlink{#1}{#2}}\\n", tex_file);
} else {
  fputs("\\newcommand{\\WEFTtarget}[2]{#2}\\n", tex_file);
  fputs("\\newcommand{\\WEFTlink}[2]{#2}\\n", tex_file);
}
fputs("\\newcommand{\\WEFTtxtMacroDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroRefIn}{used in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtMacroNoRef}{unreferenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtRefIn}{used in}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtNoRef}{unreferenced}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtFileDefBy}{defined at}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsUsed}{uses}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsNotUsed}{never used}\\n", tex_file);
fputs("\\newcommand{\\WEFTtxtIdsentsDefed}{defines}\\n", tex_file);
fputs("\\newcommand{\\WEFTsep}{}\\n", tex_file);
fputs("\\newcommand{\\WEFTnotglobal}{(not defined globally)}\\n", tex_file);
< Write the modern decoration macros 56b >
< Write the scrap formatting macros 55b >
fputs("\\newcommand{\\WEFTuseHyperlinks}{", tex_file);
if (hyperoptions[0] != '\\0')
{
  < Write the hyperlink usage macro 56a >
}
fputs("}\\n", tex_file);
< Write the AtBeginDocument setup 57a >
```

55a · used in 54 · uses `fputs` 21b, `hyperoptions` 27a, `hyperref_flag` 27a, `uses` 121c

The layout macros. `\WEFTbegin`/`\WEFTend` open and close the vertical block that holds a scrap’s header line, its `lstlisting` body, and its one-line metadata footer. `\WEFTbegin` sets the block ragged-right at the document’s ordinary size (the old `\small` is gone: the header now reads at full size, and the code carries its own, tighter, listings size). A fragment reference appearing in the middle of code is assembled inline and wrapped in an `\hbox` so that `listings`’ line-breaker never splits it while it sits inside an escape. (We use `\hbox` rather than `\mbox` because a fragment name may legitimately contain `\verb`, which is illegal in a macro argument such as `\mbox`’s but fine inside a primitive `\hbox`.) `\WEFTlstsetup` now carries `weft`’s *modern* `\lstset` defaults: a compact monospace with tightened leading, coloured keywords/strings/comments (through `\WEFTtint`, so colour is optional), line breaking, and *no* frame, background, or left indent. The user need not call it — the `\AtBeginDocument` hook of §2.4 applies it automatically once `listings` is loaded — but an explicit call, or a `\renewcommand`, still lets the user override the house style.

< *Write the scrap formatting macros* 55b > ≡

```
fputs("\\newcommand{\\WEFTbegin}{\\par\\addvspace{2.3ex plus .6ex}"
      "\\begingroup\\raggedright}\\n", tex_file);
fputs("\\newcommand{\\WEFTend}{\\par\\endgroup"
      "\\addvspace{2.3ex plus .6ex}}\\n", tex_file);
fputs("\\providecommand{\\WEFTlstsetup}{\\lstset{basicstyle=\\ttfamily"
      "\\fontsize{10pt}{10.8pt}\\selectfont,"
      "keywordstyle={\\WEFTtint{weftkw}\\bfseries},"
      "commentstyle={\\WEFTtint{weftcom}\\itshape},"
      "stringstyle={\\WEFTtint{weftstr}},"
      "breaklines=true,breakatwhitespace=false,"
      "columns=fullflexible,keepspaces=true,showstringspaces=false,"
      "tabsize=8,frame=none,xleftmargin=0pt,"
      "aboveskip=0pt,belowskip=0pt}}\\n", tex_file);
```

55b · used in 55a · uses `fputs` 21b

< *Write the hyperlink usage macro* 56a > ≡

```
fprintf(tex_file, "\\usepackage[%s]{hyperref}", hyperoptions);
```

56a · used in 55a · uses `fprintf` 21b, `hyperoptions` 27a

The modern decoration macros

The visual identity of a woven scrap — the way a fragment name reads, the little scrap number beside it, the definition/continuation marker, the one-line metadata footer — is factored into a handful of small macros written here. Emitting them once, as named commands, keeps the per-scrap output that follows compact and lets a reader (or the user) re-style the whole document by redefining a single command.

The design is *sober and self-contained*: a fragment name is set in the document's ordinary italic tinted with one accent colour, its scrap number in a small grey, the enclosing `< >` in the same grey, and the modern layout carries *no* rules, boxes, or shaded backgrounds — highlighting alone distinguishes the code.

Colour is applied through one level of indirection, `\\WEFTtint`. By default it is a no-op: `\\WEFTtint{weftaccent}` expands to nothing, so a document that loads *no* colour package still compiles, merely in black and white. When a colour package *is* present, the `\\AtBeginDocument` hook below swaps in `\\WEFTtintOn`, and the same call becomes `\\color{weftaccent}`. This is why every colour site is written `{\\WEFTtint{name}...}` rather than `\\color` directly: it degrades gracefully.

`\\WEFTlangle`/`\\WEFTrangle` are the grey angle brackets; `\\WEFTnumsep` is the space set between a fragment name and its scrap number (wide enough that `name 41a` never reads as one token); `\\WEFTeq` and `\\WEFTpluseq` are the definition (`≡`) and additive-continuation (`+ ≡`) markers; `\\WEFTdot` is the mid-dot that separates footer items; and `\\WEFTmetabegin`/`\\WEFTmetaend` open and close the small grey metadata line, which opens with the scrap's own number.

< *Write the modern decoration macros* 56b > ≡

```
fputs("\\newcommand{\\WEFTtint}[1]{\\n", tex_file);
fputs("\\newcommand{\\WEFTtintOn}[1]{\\color{#1}}\\n", tex_file);
fputs("\\newcommand{\\WEFTlangle}{\\{\\WEFTtint{weftcom}$\\langle$\\},\\n",
      tex_file);
fputs("\\newcommand{\\WEFTrangle}{\\},\\{\\WEFTtint{weftcom}$\\rangle$\\}\\n",
      tex_file);
fputs("\\newcommand{\\WEFTeq}{\\{\\{\\WEFTtint{weftcom}$\\equiv$\\}\\}\\n",
      tex_file);
fputs("\\newcommand{\\WEFTpluseq}{\\{\\{\\WEFTtint{weftcom}"
      "$\\mathord{+}\\equiv$\\}\\}\\n", tex_file);
fputs("\\newcommand{\\WEFTdot}{\\{\\{\\WEFTtint{weftcom}"
      "\\textperiodcentered\\}\\}\\n", tex_file);
fputs("\\newcommand{\\WEFTnumsep}{\\}\\n", tex_file);
fputs("\\newcommand{\\WEFTmetabegin}[1]{\\par\\vspace{3pt}\\begingroup"
      "\\footnotesize\\WEFTtint{weftcom}#1\\n", tex_file);
```

```
fputs("\\newcommand{\\WEFTmetaend}{\\endgroup\\par}\\n", tex_file);
```

56b · used in 55a · uses [fputs 21b](#)

The colours themselves, and the automatic application of `\WEFTlstsetup`, are deferred to `\AtBeginDocument`. This is essential: the limbo we are writing precedes `\documentclass`, where `\definecolor` and `\lstset` cannot run (no colour or listings package is loaded yet, and loading one there fails outright). The kernel hook `\AtBeginDocument` runs its body at `\begin{document}`, by which time the user's preamble has loaded its packages. We guard each half with ε -TeX's `\ifdefined` so the output still compiles when a colour package or `listings` is absent. The palette uses the `rgb` model, understood by both `color` and `xcolor`, so no specific colour package is mandated.

Write the AtBeginDocument setup 57a $\equiv$

```
fputs("\\AtBeginDocument{%\\n", tex_file);
fputs("\\ifdefined\\definecolor\\n", tex_file);
fputs("\\definecolor{weftaccent}{rgb}{0.184,0.435,0.922}%\\n", tex_file);
fputs("\\definecolor{weftkw}{rgb}{0.020,0.314,0.682}%\\n", tex_file);
fputs("\\definecolor{weftstr}{rgb}{0.067,0.388,0.161}%\\n", tex_file);
fputs("\\definecolor{weftcom}{rgb}{0.431,0.467,0.506}%\\n", tex_file);
fputs("\\let\\WEFTtint\\WEFTtintOn\\n", tex_file);
fputs("\\fi\\n", tex_file);
fputs("\\ifdefined\\lstset\\WEFTlstsetup\\fi\\n", tex_file);
fputs("}\\n", tex_file);
```

57a · used in 55a · uses [fputs 21b](#)

We make our second (and final) pass through the source web, this time copying characters straight into the `.tex` file. However, we keep an eye peeled for `@` characters, which signal a command sequence.

Copy source_file into tex_file 57b $\equiv$

```
{
  int inBlock = FALSE;
  int c = source_get();
  while (c != EOF) {
    if (c == nw_char)
    {
      Interpret at-sequence 57c
    }
    else {
      putc(c, tex_file);
      c = source_get();
    }
  }
}
```

57b · used in 54 · uses [FALSE 23a](#), [nw_char 28ab](#), [putc 21b](#), [source_get 118a](#)

Interpret at-sequence 57c $\equiv$

```
{
  int big_definition = FALSE;
  c = source_get();
  switch (c) {
    case 'r':
      c = source_get();
      nw_char = c;
      update_delimit_scrap();
      break;
    case '0': big_definition = TRUE;
              FALLTHROUGH;
```

```

case 'o': < Write output file definition 60a >
           break;
case 'Q':
case 'D': big_definition = TRUE;
           FALLTHROUGH;
case 'q':
case 'd': < Write macro definition 60b >
           break;
case 's':
           < Step to next sector 44a >
           break;
case 'S':
           < Close the current sector 44b >
           break;
case '{':
case '[':
case '(': < Write in-text scrap 64d >
           break;
case '<': < Expand macro into tex_file 58c >
           break;
case 'x': < Copy label from source into(58a tex_file ) 72a >
           c = source_get();
           break;
case 'c': < Begin or end a block comment 49e >
           c = source_get();
           break;
case 'f': < Write index of file names 75a >
           break;
case 'm': < Write index of macro names 76c >
           break;
case 'u': < Write index of user-specified names 79c >
           break;
case 'v': < Copy version info into tex file 58b >
           break;
case 'l':
case 'L': < Skip at-L language definition line 43c >
           break;
case 'W': < Skip at-L language definition line 43c >
           break;
default:
           if (c==nw_char)
               putc(c, tex_file);
           c = source_get();
           break;
    }
}

```

58a · used in 57b

```

< Copy version info into tex file 58b > ≡
fputs(version_string, tex_file);
c = source_get();

```

58b · used in 57c · uses fputs 21b, source_get 118a, version_string 27a

A fragment reference `@<name@>` written in the *documentation* text (outside any scrap) is a *cross-reference*, not an inclusion. `weft` renders it exactly like a reference in the middle of code — the grey `< >` brackets, the accent-tinted name, and a link to the fragment's first defining scrap — only without the `listings` escape, since prose is already ordinary L^AT_EX text. This is a deliberate change from the old `nuweb` behaviour, which *expanded* the fragment's whole body inline into the prose: a chunk name mentioned in a sentence would

dump code into the middle of it. Referencing, not expanding, is what a literate narrative wants (“as we saw in @<Say hello@>”). To show a fragment’s *code* in the prose, define and reference it as a scrap in the usual way.

```
< Expand macro into file 58c > ≡
{
  Arglist *a = collect_scrap_name(0);
  Name *name = a->name;
  char *p = name->spelling;
  int i = 0;
  fputs("\\WEFTlangle{\\WEFTtint{\\weftaccent}\\itshape ", file);
  while (*p != '\\000') {
    if (*p == ARG_CHR) {
      write_arg(file, name->arg[i++]);
      p++;
    }
    else
      fputc(*p++, file);
  }
  fputs("}\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{\\weftcom}", file);
  if (name->defs) {
    Scrap_Node *q = name->defs;
    fputs("\\WEFTlink{\\weft", file);
    write_single_scrap_ref(file, q->scrap);
    fputs("}{", file);
    write_single_scrap_ref(file, q->scrap);
    fputs("}", file);
    if (q->next)
      fputs(", \\ldots\\ ", file);
  }
  else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\\n",
            command_name, name->spelling);
  }
  fputs("}\\WEFTrangle", file);
  c = source_get();
}
```

58c · used in 57c · uses Arglist 164d, ARG_CHR 163b, collect_scrap_name 165, command_name 28c, defs 121c, fprintf 21b, fputs 21b, Name 144b, next 121b, putc 21b, Scrap_Node 144a, source_get 118a, stderr 21b, write_arg 61b, write_single_scrap_ref 123c

2.4.1 Formatting Definitions

We go through a fair amount of effort to format a fragment definition. The L^AT_EX for the previous fragment definition should look like this (perhaps modulo the scrap references). Note the shape of a modern header: the grey \\WEFTlangle/\\WEFTrangle brackets, the accent-tinted italic name, the small grey scrap number, and the \\WEFTeq (≡) kind marker; then a hair of space before the listing; and, after it, the one-line \\WEFTmetabegin...\\WEFTmetaend footer instead of the old bulleted list.

```
\\WEFTbegin
\\label{scrap70}
\\WEFTtarget{\\weft70}{\\WEFTlangle{\\WEFTtint{\\weftaccent}\\itshape %
  Interpret at-sequence}\\nobreak\\,{\\footnotesize\\WEFTtint{\\weftcom}}70}%
  \\WEFTrangle\\WEFTeq\\par\\nobreak\\vspace{0.6ex}\\nobreak
\\begin{lstlisting}[escapeinside={(*<}{>*)},language=C]
{
  int big_definition = FALSE;
```

```

c = source_get();
switch (c) {
  case 'o': (*<\hbox{\normalfont \WEFTlangle{\WEFTtint{weftaccent}}%
\itshape Write output file definition}\nobreak\,{\footnotesize%
\WEFTtint{weftcom}\WEFTlink{weft19a}{19a}}\WEFTrangle>*)
}
\end{lstlisting}
\WEFTmetabegin{70}\WEFTdot \WEFTtxtMacroRefIn\ \WEFTlink{weft17b}{17b}\WEFTmetaend
\WEFTend

```

The code body is now an `lstlisting`. Because `listings` reads the body verbatim, the special L^AT_EX characters need *no* escaping — they are printed as typed. The `escapeinside={(*<}{>*)}` option carves out the one place we *do* need L^AT_EX: a fragment reference in the middle of code, which is emitted between `(*<` and `>*)` and wrapped in an `\hbox` so `breaklines` cannot split it. The delimiter pair `(*<...>*)` is deliberately chosen to be a sequence that never occurs in real source (weft warns if a scrap body contains it). `listings` supplies page breaking, line wrapping, and — via the `language=` option, taken from the scrap's language tag — syntax highlighting.

Fragment and file definitions are formatted nearly identically. I've factored the common parts out into separate scraps.

Both headers now carry a *kind marker* after the closing angle bracket: `\WEFTeq` ($\equiv$) for a definition, or `\WEFTpluseq` ($+\equiv$) for an *additive continuation* — a scrap that appends to a fragment (or file) already begun elsewhere. We know which is which because `name->defs` is, after pass one's `reverse_lists`, in source order: its head is the *first* scrap that defines the name. So the current scrap is a continuation exactly when `name->defs->scrap` differs from the scrap we are emitting. The file header sets the file name in monospace (it *is* a filename); the fragment header sets its name in the document's italic. Both are tinted with the accent colour and followed, on the next line after a hair of vertical space, by the code.

```

< Write output file definition 60a > ≡
{
  Name *name = collect_file_name();
  int additive = (name->defs && name->defs->scrap != scraps);
  < Begin the scrap environment 62a >
  fputs("\WEFTtarget{weft", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("{}]", tex_file);
  fprintf(tex_file, "\\WEFTlangle{\\WEFTtint{weftaccent}}\\verb%c%s%c\\nobreak"
          "\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}}", nw_char, name->spelling,
          nw_char);
  write_single_scrap_ref(tex_file, scraps);
  fputs("{}\\WEFTrangle", tex_file);
  fputs(additive ? "\\WEFTpluseq" : "\\WEFTeq", tex_file);
  fputs("\\par\\nobreak\\vspace{0.6ex}\\nobreak\\n", tex_file);
  < Fill in the middle of the scrap environment 62e >
  < Begin the cross-reference environment 64e >
  if ( scrap_flag ) {
    < Write file defs 64g >
  }
  format_defs_refs(tex_file, scraps);
  format_uses_refs(tex_file, scraps++);
  < Finish the cross-reference environment 64f >
  < Finish the scrap environment 64a >
}

```

60a · used in 57c · uses `collect_file_name` 159a, `defs` 121c, `format_defs_refs` 178a, `format_uses_refs` 176f, `fprintf` 21b, `fputs` 21b, `Name` 144b, `nw_char` 28ab, `scraps` 122a, `scrap_flag` 27a, `write_single_scrap_ref` 123c

The fragment name's *content* is still emitted verbatim by the *Write the macro's name* fragment, so a programmer may put italics or bold face in the midst of the name; the surrounding `\itshape` and accent

tint are only the default dress.

< Write macro definition 60b > ≡

```
{
  Name *name = collect_macro_name();
  int additive = (name->defs && name->defs->scrap != scraps);

  < Begin the scrap environment 62a >
  fputs("\\WEFTtarget{weft", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("{}\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", tex_file);
  < Write the macro's name 61a >
  fputs("\\\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("\\\\WEFTrangle", tex_file);
  fputs(additive ? "\\WEFTpluseq" : "\\WEFTeq", tex_file);
  fputs("\\\\par\\nobreak\\vspace{0.6ex}\\nobreak\\n", tex_file);
  < Fill in the middle of the scrap environment 62e >
  < Begin the cross-reference environment 64e >
  < Write macro defs 65a >
  < Write macro refs 65b >
  format_defs_refs(tex_file, scraps);
  format_uses_refs(tex_file, scraps++);
  < Finish the cross-reference environment 64f >
  < Finish the scrap environment 64a >
}
```

60b · used in 57c · uses [collect_macro_name 162a](#), [defs 121c](#), [format_defs_refs 178a](#), [format_uses_refs 176f](#), [fputs 21b](#), [Name 144b](#), [scraps 122a](#), [write_single_scrap_ref 123c](#)

< Write the macro's name 61a > ≡

```
{
  char * p = name->spelling;
  int i = 0;

  while (*p != '\\000') {
    if (*p == ARG_CHR) {
      write_arg(tex_file, name->arg[i++]);
      p++;
    }
    else
      fputc(*p++, tex_file);
  }
}
```

61a · used in 60b, 78a · uses [ARG_CHR 163b](#), [write_arg 61b](#)

< latex.c 61b > + ≡

```
static void write_arg(FILE * tex_file, char * p)
{
  fputs("\\\\hbox{\\slshape\\sffamily ", tex_file);
  while (*p)
  {
    switch (*p)
    {
      case '$':
      case '_':
      case '~':
      case '#':
```

```

        fputc('\\', tex_file);
        break;
    default:
        break;
    }
    fputc(*p, tex_file);
    p++;
}

fputs("\\}/", tex_file);
}

```

61b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `write_arg` 53b, 58c, 61a, 70e, 74b · uses `FILE` 21b, `fputs` 21b

A scrap is opened with `\WEFTbegin` and closed with `\WEFTend` (both defined among the scrap-formatting macros above). Both big (`@0`, `@D`) and small (`@o`, `@d`) definitions use the same breakable block; the `inBlock` bookkeeping lets a run of small definitions share one open group, separated by a little vertical space.

```

⟨ Begin the scrap environment 62a ⟩ ≡
{
    if (big_definition)
    {
        if (inBlock)
        {
            ⟨ End block 62d ⟩
        }
        fputs("\\WEFTbegin", tex_file);
    }
    else
    {
        if (inBlock)
        {
            ⟨ Switch block 62c ⟩
        }
        else
        {
            ⟨ Start block 62b ⟩
        }
    }
    fprintf(tex_file, "\\label{scrap%d}\\n", scraps);
}

```

62a · used in 60ab · uses `fprintf` 21b, `fputs` 21b, `scraps` 122a

```

⟨ Start block 62b ⟩ ≡
fputs("\\WEFTbegin\\n", tex_file);
inBlock = TRUE;

```

62b · used in 49e, 62a · uses `fputs` 21b, `TRUE` 23a

```

⟨ Switch block 62c ⟩ ≡
fputs("\\par\\vspace{\\baselineskip}\\n", tex_file);

```

62c · used in 62a · uses `fputs` 21b

```

⟨ End block 62d ⟩ ≡
fputs("\\WEFTend\\n", tex_file);
inBlock = FALSE;

```

62d · used in 49e, 62a, 64a · uses `FALSE` 23a, `fputs` 21b

The body of a code scrap (`@{...@}`) is emitted as an `lstlisting`. We tag it with our fixed `escapeinside` pair and, when the scrap has a language listings understands, with a `language=` option so the code is highlighted.

Both ends of the listing must sit on their own lines. We emit a newline right after the closing `]` of the options, so the first line of code starts fresh; without it a scrap beginning with `#` (a C `#ifndef`, say) would land immediately after `]`, and `listings` reads that `#` as a macro parameter and aborts with *Illegal parameter number*. Symmetrically, `\end{lstlisting}` must start its own line, so we emit a newline before it even when the scrap body did not end in one. The modern layout draws no end-of-scrap glyph: the old diamond (`\WEFTsep`) is retired — its macro is still defined, but empty, for backward compatibility — and the metadata footer that follows delimits the scrap on its own. A non-code block scrap (the unusual `@d name @[...@]` or `@(...@)` forms) is emitted as before, without a listing.

< Fill in the middle of the scrap environment 62e > ≡

```
{
  int is_code = (source_last == '{');
  if (is_code) {
    const char *lstlang = latex_language(name->lang);
    /* the delimiter halves are split so this source line does not itself
       contain the escape sequence when weft typesets its own source */
    fputs("\\begin{lstlisting}[escapeinside={(*) "<>" "*)}", tex_file);
    if (lstlang)
      fprintf(tex_file, ",language=%s", lstlang);
    /* newline after ']' so the first code line starts fresh; a scrap
       beginning with '#' would otherwise break listings (see prose above) */
    fputs("]\n", tex_file);
  }
  extra_scrap = 0;
  copy_scrap(tex_file, TRUE, name);
  if (is_code)
    /* the closing tag is split across two literals so that this very
       line does not end a listing when weft typesets its own source
       (the same reason this comment says "the closing tag" rather than
       spelling it out); the leading newline keeps it on its own line */
    fputs("\n\\end{ls" "tlisting}\n", tex_file);
}
```

62e · used in 60ab · uses `copy_scrap` 69, `extra_scrap` 64b, `first` 182a, `fprintf` 21b, `fputs` 21b, `latex_language` 67b, `prose` 121c, `source_last` 118a, `TRUE` 23a

We've got one last spacing command, controlling the amount of white space after a scrap.

Note also the whitespace eater. I use it to remove any blank lines that appear after a scrap in the source file. This way, text following a scrap will not be indented. Again, this is a matter of personal taste.

```

< Finish the scrap environment 64a > ≡
{
  scraps += extra_scraps;
  if (big_definition)
    fputs("\\WEFTend\n", tex_file);
  else
  {
    < End block 62d >
  }
  do
    c = source_get();
  while (isspace(c));
}

```

64a · used in 60ab · uses `extra_scraps` 64b, `fputs` 21b, `isspace` 21b, `scraps` 122a, `source_get` 118a

```

< Global variable declarations 64b > + ≡
extern int extra_scraps;

```

64b · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `extra_scraps` 62e, 64ac, 74a, 86ab, 91b

```

< Global variable definitions 64c > + ≡
int extra_scraps = 0;

```

64c · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `extra_scraps` 64b

```

< Write in-text scrap 64d > ≡
copy_scrap(tex_file, FALSE, NULL);
c = source_get();

```

64d · used in 57c · uses `copy_scrap` 69, `FALSE` 23a, `source_get` 118a

Formatting Cross References

```

< Begin the cross-reference environment 64e > ≡
{
  fputs("\\WEFTmetabegin{", tex_file);
  write_single_scrap_ref(tex_file, scraps);
  fputs("}", tex_file);}

```

64e · used in 60ab · uses `fputs` 21b, `scraps` 122a, `write_single_scrap_ref` 123c

The footer is one horizontal line, opened above with the scrap's own number. Every subsequent item is introduced by `\WEFTdot`, the grey separator, so there is never a leading or trailing dot to special-case; an item that has nothing to say simply emits nothing.

```

< Finish the cross-reference environment 64f > ≡
{
  fputs("\\WEFTmetaend\n", tex_file);
}

```

64f · used in 60ab · uses `fputs` 21b

```

< Write file defs 64g > ≡

```

```

{
  if (name->defs) {
    if (name->defs->next) {
      fputs("\\WEFTdot \\WEFTtxtFileDefBy\\ ", tex_file);
      print_scrap_numbers(tex_file, name->defs);
    }
  } else {
    fprintf(stderr,
            "would have crashed in 'Write file defs' for '%s'\n",
            name->spelling);
  }
}

```

64g · used in 60a · uses [defs 121c](#), [fprintf 21b](#), [fputs 21b](#), [next 121b](#), [print_scrap_numbers 65c](#), [stderr 21b](#)

⟨ *Write macro defs* 65a ⟩ ≡

```

{
  if (name->defs->next) {
    fputs("\\WEFTdot \\WEFTtxtMacroDefBy\\ ", tex_file);
    print_scrap_numbers(tex_file, name->defs);
  }
}

```

65a · used in 60b · uses [defs 121c](#), [fputs 21b](#), [next 121b](#), [print_scrap_numbers 65c](#)

⟨ *Write macro refs* 65b ⟩ ≡

```

{
  if (name->uses) {
    if (name->uses->next) {
      fputs("\\WEFTdot \\WEFTtxtMacroRefIn\\ ", tex_file);
      print_scrap_numbers(tex_file, name->uses);
    }
    else {
      fputs("\\WEFTdot \\WEFTtxtMacroRefIn\\ ", tex_file);
      fputs("\\WEFTlink{weft", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}{", tex_file);
      write_single_scrap_ref(tex_file, name->uses->scrap);
      fputs("}", tex_file);
    }
  }
  else {
    fputs("\\WEFTdot {\\WEFTtxtMacroNoRef}", tex_file);
    fprintf(stderr, "%s: <%s> never referenced.\n",
            command_name, name->spelling);
  }
}

```

65b · used in 60b · uses [command_name 28c](#), [fprintf 21b](#), [fputs 21b](#), [next 121b](#), [print_scrap_numbers 65c](#), [stderr 21b](#),
uses [121c](#), [write_single_scrap_ref 123c](#)

⟨ *latex.c* 65c ⟩ + ≡

```

static void print_scrap_numbers(FILE *tex_file, Scrap_Node *scraps)
{
  int page;
  fputs("\\WEFTlink{weft", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, -1, &page);
  fputs("}{", tex_file);
  write_scrap_ref(tex_file, scraps->scrap, TRUE, &page);
  fputs("}", tex_file);
}

```

```

scrap = scraps->next;
while (scraps) {
    fputs("\\WEFTlink{weft", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, -1, &page);
    fputs("}{", tex_file);
    write_scrap_ref(tex_file, scraps->scrap, FALSE, &page);
    scraps = scraps->next;
    fputs("}", tex_file);
}
}

```

65c · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `print_scrap_numbers` 53b, 64g, 65ab, 76b, 78c · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `next` 121b, `page` 121c, `scraps` 122a, `Scrap_Node` 144a, `TRUE` 23a, `write_scrap_ref` 123b

Formatting a Scrap

This is the only place we really care whether a scrap is delimited with `@{...@}`, `@[...@]`, or `@(...@)`, and we base our output sequences on that. The code (`@{...@}`) mode is now handled by `listings`, so its per-line strings are all empty: `copy_scrap` just copies the body out verbatim, one raw line per source line, and the surrounding `lstlisting` does the formatting. The `@[...@]` (verbatim `TEX`) and `@(...@)` (math) modes keep their old inline treatment.

We still keep the array `delimit_scrap`: it carries the small *begin/end* strings for the math mode (a pair of \$) and, for every mode, the single character that stands in for the weft character. Upon initialisation we copy the strings from `orig_delimit_scrap` into a writeable array; we might change the *insert nw_char* entry later, when the weft character changes.

```

<latex.c 66a> + ≡
static char *orig_delimit_scrap[3][5] = {
    /* {} mode (code): begin, end, insert nw_char, prefix, suffix.
       All empty: listings copies the body verbatim, one raw line per
       source line, so there is no per-line wrapper. */
    { "", "", "@", "", "" },
    /* [] mode: begin, end, insert nw_char, prefix, suffix */
    { "", "", "@", "", "" },
    /* () mode: begin, end, insert nw_char, prefix, suffix */
    { "$", "$", "@", "", "" },
};

```

```
static char *delimit_scrap[3][5];
```

66a · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · uses `nw_char` 28ab

The function `initialise_delimit_scrap_array` just copies the originals into a writeable array.

```

<latex.c 66b> + ≡
void initialise_delimit_scrap_array(void) {
    int i,j;
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 5; j++) {
            if((delimit_scrap[i][j] = strdup(orig_delimit_scrap[i][j])) == NULL) {
                fprintf(stderr, "Not enough memory for string allocation\n");
                exit(EXIT_FAILURE);
            }
        }
    }
}

```

66b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · uses `exit` 21b, `fprintf` 21b, `stderr` 21b

Copying one code character to the `.tex` file. Because the code is no longer wrapped in `\verb`, every character that is special to L^AT_EX has to be turned into something harmless in `\ttfamily` text. A literal space becomes `\WEFTsp` (a breakable, fixed-width the code body itself needs none of this — `listings` copies it verbatim. `latex_code_putc` is used only for the little bits of text that are emitted *inside* an escape (a fragment reference's name and its substituted arguments), which land in ordinary L^AT_EX text mode and so must have the L^AT_EX-special characters neutralised.

```
<latex.c 67a> + ≡
static void latex_code_putc(FILE *file, int c)
{
    switch (c) {
        case '\\': fputs("\\textbackslash{}", file); break;
        case '{': fputs("\\{", file); break;
        case '}': fputs("\\}", file); break;
        case '$': fputs("\\$", file); break;
        case '&': fputs("\\&", file); break;
        case '#': fputs("\\#", file); break;
        case '%': fputs("\\%", file); break;
        case '_': fputs("\\_", file); break;
        case '^': fputs("\\textasciicircum{}", file); break;
        case '~': fputs("\\textasciitilde{}", file); break;
        default: putc(c, file); break;
    }
}
```

67a · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `latex_code_putc` 53b, 68c, 69, 74a · uses `FILE` 21b, `fputs` 21b, `putc` 21b

The language mapper turns a scrap's weft language tag ("`c`", "`python`", ...) into the exact name `listings` expects, or `NULL` when we have no safe mapping. Returning `NULL` (rather than guessing) matters: `listings` aborts the run on an unknown language=, so we only ever name languages it is known to ship.

```
<latex.c 67b> + ≡
static const char *latex_language(const char *lang)
{
    static const struct { const char *tag, *lst; } map[] = {
        {"c", "C"}, {"h", "C"},
        {"cpp", "C++"}, {"c++", "C++"}, {"cc", "C++"}, {"cxx", "C++"},
        {"hpp", "C++"}, {"hxx", "C++"},
        {"java", "Java"},
        {"python", "Python"}, {"py", "Python"},
        {"sql", "SQL"},
        {"html", "HTML"}, {"htm", "HTML"}, {"xml", "XML"},
        {"php", "PHP"},
        {"perl", "Perl"}, {"pl", "Perl"},
        {"ruby", "Ruby"}, {"rb", "Ruby"},
        {"pascal", "Pascal"}, {"pas", "Pascal"},
        {"lisp", "Lisp"}, {"el", "Lisp"}, {"scheme", "Lisp"},
        {"matlab", "Matlab"},
        {"tex", "[LaTeX]TeX"}, {"latex", "[LaTeX]TeX"},
        {"make", "make"}, {"makefile", "make"},
        {"awk", "Awk"},
        {"fortran", "Fortran"}, {"f", "Fortran"}, {"f90", "Fortran"},
        {"r", "R"}, {"lua", "Lua"}, {"ada", "Ada"},
        {"vhdl", "VHDL"}, {"verilog", "Verilog"},
    }
```

```

    {"erlang","erlang"}, {"cobol","COBOL"}, {"eiffel","Eiffel"},
    {"ocaml","Caml"}, {"caml","Caml"},
    {NULL,NULL}
};
int i;
if (lang == NULL || lang[0] == '\0')
    return NULL;
for (i = 0; map[i].tag; i++) {
    const char *a = lang, *b = map[i].tag;
    while (*a && *b && tolower((unsigned char)*a) == (unsigned char)*b) { a++; b++; }
    if (*a == '\0' && *b == '\0')
        return map[i].lst;
}
return NULL;
}

```

67b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `latex_language` 53b, 62e

⟨ *Function prototypes* 68a ⟩ + ≡

```
void initialise_delimit_scrap_array(void);
```

68a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a

⟨ `latex.c` 68b ⟩ + ≡

```
int scrap_type = 0;
```

68b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `scrap_type` 68d, 69, 70de, 72g

⟨ `latex.c` 68c ⟩ + ≡

```
static void write_literal(FILE * tex_file, char * p, int mode)
{
    fputs(delimit_scrap[mode][0], tex_file);
    while (*p != '\000') {
        if (*p == nw_char)
            fputs(delimit_scrap[mode][2], tex_file);
        else if (mode == 0)
            latex_code_putc(tex_file, (unsigned char)*p);
        else
            fputc(*p, tex_file);
        p++;
    }
    fputs(delimit_scrap[mode][1], tex_file);
}

```

68c · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `write_literal` 53b, 72g, 74b · uses `FILE` 21b, `fputs` 21b, `latex_code_putc` 67a, `nw_char` 28ab

Two little helpers bracket every place where we must step out of the verbatim body to run L^AT_EX (a fragment reference, a bold keyword, an italic title, a substituted parameter). In code mode this opens and closes a `listings` escape; in math mode it closes and reopens the math run; in the verbatim-T_EX (`@[...@]`) mode both are empty.

The escape delimiters are written with the two halves as separate string literals ("`(*`" "`<`"). The C compiler concatenates them, so the emitted text is exactly `(*<`; but the *source* never contains the delimiter as one contiguous token, which matters because weft typesets its own source — otherwise these very lines would trip the `listings` tokeniser when they appear in a scrap.

When `code_inline` is set we are already rendering inside an escape (a parameter of a fragment reference), so we must *not* open a nested escape — both helpers do nothing in code mode.

<latex.c 68d> + ≡

```
int code_inline = FALSE;
```

```
static void begin_tex_escape(FILE *file) /* leave the verbatim body */
{
    if (scrap_type == 0) { if (!code_inline) fputs("(\"<", file); }
    else fputs(delimit_scrap[scrap_type][1], file);
}
static void end_tex_escape(FILE *file) /* re-enter the verbatim body */
{
    if (scrap_type == 0) { if (!code_inline) fputs(">\" \")", file); }
    else fputs(delimit_scrap[scrap_type][0], file);
}
```

68d · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `begin_tex_escape` 48e, 70e, 72efg, `code_inline` 52b, 69, `end_tex_escape` 48e, 70e, 72efg · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `scrap_type` 68b, 69

<latex.c 69> + ≡

```
static void copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    int coll_h1 = 0, coll_h2 = 0, coll_warned = FALSE;
    char ** params = name->arg;
    if (source_last == '{') scrap_type = 0;
    if (source_last == '[') scrap_type = 1;
    if (source_last == '(') scrap_type = 2;
    c = source_get();
    if (prefix) fputs(delimit_scrap[scrap_type][3], file);
    fputs(delimit_scrap[scrap_type][0], file);
    while (1) {
        switch (c) {
            case '\n': if (code_inline) {
                putc(' ', file); /* an inline parameter stays on one line */
                indent = 0;
                break;
            }
            fputs(delimit_scrap[scrap_type][1], file);
            if (prefix) fputs(delimit_scrap[scrap_type][4], file);
            fputs("\n", file);
            if (prefix) fputs(delimit_scrap[scrap_type][3], file);
            fputs(delimit_scrap[scrap_type][0], file);
            indent = 0;
            break;
            case '\t': <Expand tab into spaces 70c>
                break;
            default:
                if (c==nw_char)
                {
                    <Check at-sequence for end-of-scrap 70e>
                    break;
                }
                if (code_inline)
                    latex_code_putc(file, c); /* inside an escape: neutralise LaTeX */
                else {
                    <Note a delimiter collision 70d>
                    putc(c, file);
                }
                indent++;
        }
    }
}
```

```

        break;
    }
    c = source_get();
}
}

```

69 · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `copy_scrap` 52b, 53b, 62e, 64d, `scrap_type` 68bd, 70de, 72g · uses `code_inline` 68d, `FALSE` 23a, `FILE` 21b, `fputs` 21b, `latex_code_putc` 67a, `Name` 144b, `nw_char` 28ab, `putc` 21b, `source_get` 118a, `source_last` 118a

When we encounter the command to change the ‘weft character’ we call this function. It updates the scrap formatting directives accordingly.

```

⟨ latex.c 70a ⟩ + ≡
void update_delimit_scrap(void)
{
    /* Every mode's "insert nw_char" string is the single character that
       stands in for the weft character inside a scrap; keep them in sync
       with the (possibly changed) weft character. */
    delimit_scrap[0][2][0] = nw_char;
    delimit_scrap[1][2][0] = nw_char;
    delimit_scrap[2][2][0] = nw_char;
}

```

70a · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `update_delimit_scrap` 43d, 57c, 70b, 82c · uses `nw_char` 28ab

```

⟨ Function prototypes 70b ⟩ + ≡
void update_delimit_scrap(void);

```

70b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `update_delimit_scrap` 70a

```

⟨ Expand tab into spaces 70c ⟩ ≡
{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}

```

70c · used in 69, 137e · uses `putc` 21b

Since the code body is copied verbatim, a scrap that literally contains our escape delimiter ((*< or >*)) would desynchronise listings’ tokeniser. This never happens in ordinary source, but we watch a two-character rolling window and warn once per scrap if it does, so a corrupted listing is never silent.

```

⟨ Note a delimiter collision 70d ⟩ ≡
if (scrap_type == 0) {
    if ((coll_h2 == '(' && coll_h1 == '*' && c == '<') ||
        (coll_h2 == '>' && coll_h1 == '*' && c == ')')) {
        if (!coll_warned) {
            fprintf(stderr,
                "%s: warning: a scrap body contains the listings escape "
                "delimiter; its typeset code may be corrupted.\n", command_name);
            coll_warned = TRUE;
        }
    }
}

```

```

coll_h2 = coll_h1;
coll_h1 = c;
}
70d · used in 69 · uses command_name 28c, fprintf 21b, scrap_type 68b, 69, stderr 21b, TRUE 23a

< Check at-sequence for end-of-scrap 70e > ≡
{
c = source_get();
switch (c) {
case 'c': < Show presence of a block comment 48e >
break;
case 'x': < Copy label from source into(71a file ) 72a >
break;
case 'v': < Copy version info into file 71b >
case 's':
break;
case '+':
case '-':
case '*':
case '|': < Skip over index entries 72c >
FALLTHROUGH;
case ',':
case ')':
case ']':
case '}': fputs(delimit_scrap[scrap_type][1], file);
return;
case '<': < Format macro name 72g >
break;
case '%': < Skip commented-out code 72d >
break;
case '_': < Bold Keyword 72e >
break;
case 't': < Italic "fragment title" 72f >
break;
case 'f': < Italic "file name" 72f >
break;
case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
if (name == NULL
|| name->arg[c - '1'] == NULL) {
fputs(delimit_scrap[scrap_type][2], file);
fputc(c, file);
}
else {
begin_tex_escape(file);
write_arg(file, name->arg[c - '1']);
end_tex_escape(file);
}
break;
default:
if (c==nw_char)
{
fputs(delimit_scrap[scrap_type][2], file);
break;
}
/* ignore these since pass1 will have warned about them */
break;
}

```

```
}
}
```

71a · used in 69

⟨ *Copy version info into file* 71b ⟩ ≡
 fputs(version_string, file);

71b · used in 70e, 89b, 138a · uses fputs 21b, version_string 27a

⟨ *Copy label from source into* 72a ⟩ ≡
 {
 ⟨ *Get label from*(72b source_get()) 179b ⟩
 write_label(label_name, 01);
 }

72b · used in 57c, 70e, 82c, 89b · uses source_get 118a

There's no need to check for errors here, since we will have already pointed out any during the first pass.

⟨ *Skip over index entries* 72c ⟩ ≡
 {
 do {
 do
 c = source_get();
 while (c != nw_char);
 c = source_get();
 } while (c != '}' && c != ']' && c != ')') ;
 }

72c · used in 70e, 89b · uses nw_char 28ab, source_get 118a

⟨ *Skip commented-out code* 72d ⟩ ≡
 {
 do
 c = source_get();
 while (c != '\n');
 }

72d · used in 70e, 89b, 126c · uses source_get 118a

This scrap helps deal with bold keywords:

⟨ *Bold Keyword* 72e ⟩ ≡
 {
 begin_tex_escape(file);
 fprintf(file, "\\hbox{\\sffamily\\bfseries ");
 c = source_get();
 do {
 fputc(c, file);
 c = source_get();
 } while (c != nw_char);
 c = source_get();
 fprintf(file, "}");
 end_tex_escape(file);
 }

72e · used in 70e · uses begin_tex_escape 68d, end_tex_escape 68d, fprintf 21b, nw_char 28ab, source_get 118a

⟨ *Italic "whatever"* 72f ⟩ ≡

```

{
  begin_tex_escape(file);
  fprintf(file, "\\hbox{\\sffamily\\slshape whatever}");
  end_tex_escape(file);
}

```

72f · used in 70e · uses [begin_tex_escape 68d](#), [end_tex_escape 68d](#), [fprintf 21b](#)

⟨ *Format macro name* 72g ⟩ ≡

```

{
  Arglist *args = collect_scrap_name(-1);
  Name *name = args->name;
  char *p = name->spelling;
  Arglist *q = args->args;
  int narg = 0;

  begin_tex_escape(file);
  if (prefix)
    /* \normalfont resets the listing's \ttfamily basicstyle, so the
       fragment name is set in the ordinary document font (roman italic)
       rather than as italic typewriter. */
    fputs("\\hbox{\\normalfont ", file);
  fputs("\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", file);
  while (*p != '\\000') {
    if (*p == ARG_CHR) {
      if (q == NULL) {
        write_literal(file, name->arg[narg], scrap_type);
      }
      else {
        write_ArglistElement(file, q, params);
        q = q->next;
      }
      p++;
      narg++;
    }
    else
      fputc(*p++, file);
  }
  fputs("}\\nobreak", file);
  if (scrap_name_has_parameters) {
    ⟨ Format macro parameters 52b ⟩
  }
  fprintf(file, "\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}");
  if (name->defs)
    ⟨ Write abbreviated definition list 73 ⟩
  else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\\n",
            command_name, name->spelling);
  }
  fputs("}\\WEFTrangle", file);
  if (prefix)
    fputs("}", file);
  end_tex_escape(file);
}

```

72g · used in 70e · uses [Arglist 164d](#), [ARG_CHR 163b](#), [begin_tex_escape 68d](#), [collect_scrap_name 165](#), [command_name 28c](#), [defs 121c](#), [end_tex_escape 68d](#), [fprintf 21b](#), [fputs 21b](#), [Name 144b](#), [next 121b](#), [putc 21b](#), [scrap_type 68b](#), [69](#), [stderr 21b](#), [write_ArglistElement 74b](#), [write_literal 68c](#)

⟨ *Write abbreviated definition list 73* ⟩ ≡

```
{
  Scrap_Node *p = name->defs;
  fputs("\\WEFTlink{weft", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("{", file);
  write_single_scrap_ref(file, p->scrap);
  fputs("}", file);
  p = p->next;
  if (p)
    fputs(", \\ldots\\ ", file);
}
```

73 · used in 72g, 74b · uses [defs 121c](#), [fputs 21b](#), [next 121b](#), [Scrap_Node 144a](#), [write_single_scrap_ref 123c](#)

A quoted scrap (`@{...@}`) can appear as the argument of a parameterised fragment reference, e.g. `@<Name @{@code@} eman@>`. In the old `\verb`-based output that code was simply set in a verbatim run in the middle of the reference. Under `listings` we cannot leave the escape here — the reference is being built inside an `\hbox`, and closing the escape mid-box would leave the braces straddling the escape boundary (a ! Extra } error). Instead we capture the scrap’s expansion into a memory buffer and re-emit it *inside* the escape, as L^AT_EX-escaped typewriter text on a single line, so the box stays balanced.

⟨ *Emit a quoted scrap argument inline 74a* ⟩ ≡

```
{
  Scrap_Node * qq = (Scrap_Node *)q;
  char *buf = NULL;
  size_t sz = 0;
  FILE *mem = open_memstream(&buf, &sz);
  qq->quoted = TRUE;
  if (mem) {
    char *s;
    write_scraps(mem, "", qq, -1, "", 0, 0, 0, 0, 0, NULL, params, 0, "");
    fclose(mem);
    fputs("\\hbox{\\ttfamily ", file);
    for (s = buf; *s != '\\0'; s++) {
      if (*s == '\\n')
        putc(' ', file);
      else
        latex_code_putc(file, (unsigned char)*s);
    }
    fputs("}", file);
    free(buf);
  }
  extra_scraps++;
  qq->quoted = FALSE;
}
```

74a · used in 74b · uses [extra_scraps 64b](#), [FALSE 23a](#), [fclose 21b](#), [FILE 21b](#), [fputs 21b](#), [latex_code_putc 67a](#), [putc 21b](#), [Scrap_Node 144a](#), [size_t 21b](#), [TRUE 23a](#), [write_scraps 134b](#)

⟨ *latex.c 74b* ⟩ + ≡

```
static void
write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
  Name *name = args->name;
  Arglist *q = args->args;

  if (name == NULL) {
    char * p = (char*)q;
```

```

    if (p[0] == ARG_CHR) {
        write_arg(file, params[p[1] - '1']);
    } else {
        write_literal(file, (char *)q, 0);
    }
} else if (name == (Name *)1) {
    < Emit a quoted scrap argument inline 74a >
} else {
    char * p = name->spelling;
    fputs("$\\langle\\,${\\itshape ", file);
    while (*p != '\\000') {
        if (*p == ARG_CHR) {
            write_ArglistElement(file, q, params);
            q = q->next;
            p++;
        }
        else
            fputc(*p++, file);
    }
    fputs("\\nobreak\\ ", file);
    if (scrap_name_has_parameters) {
        int c;

        < Format macro parameters 52b >
    }
    fprintf(file, "{\\footnotesize ");
    if (name->defs)
        < Write abbreviated definition list 73 >
    else {
        putc('?', file);
        fprintf(stderr, "%s: never defined <%s>\\n",
            command_name, name->spelling);
    }
    fputs("}$\\,\\rangle$", file);
}
}

```

74b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines `write_ArglistElement` 53b, 72g · uses `Arglist` 164d, `ARG_CHR` 163b, `command_name` 28c, `defs` 121c, `FILE` 21b, `fprintf` 21b, `fputs` 21b, `Name` 144b, `next` 121b, `putc` 21b, `stderr` 21b, `write_arg` 61b, `write_literal` 68c

2.4.2 Generating the Indices

The three back-of-book indices — of file names, fragment names, and user-specified identifiers — are each a compact vertical list, one entry per line. Their entries share the scrap presentation’s visual language: a fragment name is set in the accent-tinted italic inside the grey `\WEFTlangle/\WEFTrangle` brackets, a file name and an identifier in accent and keyword-coloured monospace respectively, and each entry’s `\WEFTtxtDefBy/\WEFTtxtRefIn` metadata in the small grey used everywhere else. Colour flows through the same `\WEFTtint` indirection, so the indices too degrade to black and white when no colour package is loaded.

```

< Write index of file names 75a > ≡
{
    if (file_names) {
        fputs("\\n{\\small\\begin{list}{}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_file_entry(file_names, tex_file);
    }
}

```

```

    fputs("\\end{list}}", tex_file);
}
c = source_get();
}

```

75a · used in 57c · uses [file_names 144c](#), [format_file_entry 75b](#), [fputs 21b](#), [source_get 118a](#)

[⟨ latex.c 75b ⟩](#) + ≡

```

static void format_file_entry(Name *name, FILE *tex_file)
{
    while (name) {
        format_file_entry(name->llink, tex_file);
        ⟨ Format a file index entry 76a ⟩
        name = name->rlink;
    }
}

```

75b · defined at [24a](#), [53b](#), [54](#), [61b](#), [65c](#), [66ab](#), [67ab](#), [68bcd](#), [69](#), [70a](#), [74b](#), [75b](#), [77ab](#), [79bd](#) · defines [format_file_entry 53b](#), [75a](#)
· uses [FILE 21b](#), [Name 144b](#)

[⟨ Format a file index entry 76a ⟩](#) ≡

```

fputs("\\item ", tex_file);
fprintf(tex_file, "{\\WEFTtint{weftaccent}\\verb%c%s%c}\\ ",
        nw_char, name->spelling, nw_char);
⟨ Write file's defining scrap numbers 76b ⟩
putc('\\n', tex_file);

```

76a · used in [75b](#) · uses [fprintf 21b](#), [fputs 21b](#), [nw_char 28ab](#), [putc 21b](#)

[⟨ Write file's defining scrap numbers 76b ⟩](#) ≡

```

{
    Scrap_Node *p = name->defs;
    fputs("{\\footnotesize\\WEFTtint{weftcom} {\\WEFTtxtDefBy}", tex_file);
    putc(' ', tex_file);
    if (p->next) {
        print_scrap_numbers(tex_file, p);
    }
    else {
        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("{", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("}", tex_file);
    }
    putc('}', tex_file);
}

```

76b · used in [76a](#) · uses [defs 121c](#), [fputs 21b](#), [next 121b](#), [print_scrap_numbers 65c](#), [putc 21b](#), [Scrap_Node 144a](#), [write_single_scrap_ref 123c](#)

[⟨ Write index of macro names 76c ⟩](#) ≡

```

{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\\n{\\small\\begin{list}}{\\setlength{\\itemsep}{-\\parsep}",

```

```

        tex_file);
    fputs("\\setlength{\\itemindent}{-\\leftmargin}\\n", tex_file);
    format_entry(macro_names, tex_file, sector);
    fputs("\\end{list}\\n", tex_file);
} else {
    fputs("None.\\n", tex_file);
}
}

```

```
c = source_get();
```

76c · used in 57c · uses [current_sector 44c](#), [format_entry 77b](#), [fputs 21b](#), [has_sector 79b](#), [macro_names 144c](#), [sector 121c](#), [source_get 118a](#), [source_ungetc 118c](#)

[⟨ latex.c 77a ⟩](#) + ≡

```

static int load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}

```

77a · defined at [24a](#), [53b](#), [54](#), [61b](#), [65c](#), [66ab](#), [67ab](#), [68bcd](#), [69](#), [70a](#), [74b](#), [75b](#), [77ab](#), [79bd](#) · defines [load_entry 77b](#) · uses [Name 144b](#)

[⟨ latex.c 77b ⟩](#) + ≡

```

static void format_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    Name ** nms = malloc(num_scraps()*sizeof(Name *));
    int n = load_entry(name, nms, 0);
    int i;

    ⟨ Sort nms of size n for ⟨ Rob's ordering 77c ⟩ 77d ⟩
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        ⟨ Format an index entry 78a ⟩
    }
}

```

77b · defined at [24a](#), [53b](#), [54](#), [61b](#), [65c](#), [66ab](#), [67ab](#), [68bcd](#), [69](#), [70a](#), [74b](#), [75b](#), [77ab](#), [79bd](#) · defines [format_entry 53b](#), [76c](#) · uses [FILE 21b](#), [load_entry 77a](#), [malloc 21b](#), [Name 144b](#), [num_scraps 122a](#), [sector 121c](#)

[⟨ Rob's ordering 77c ⟩](#) ≡

```
robs_strcmp(ki->spelling, kj->spelling) < 0
```

77c · used in [77b](#), [94b](#) · uses [robs_strcmp 147c](#)

[⟨ Sort key of size n for ordering 77d ⟩](#) ≡

```

int j;
for (j = 1; j < n; j++)
{
    int i = j - 1;
    Name * kj = key[j];

    do
    {

```

```

    Name * ki = key[i];

    if (ordering)
        break;
    key[i + 1] = ki;
    i -= 1;
} while (i >= 0);
key[i + 1] = kj;
}

```

77d · used in 77b, 94b · uses Name 144b

⟨ *Format an index entry* 78a ⟩ ≡

```

if (name->sector == sector){
    fputs("\\item ", tex_file);
    fputs("\\WEFTlangle{\\WEFTtint{weftaccent}\\itshape ", tex_file);
    ⟨ Write the macro's name 61a ⟩
    fputs("\\nobreak\\WEFTnumsep{\\footnotesize\\WEFTtint{weftcom}", tex_file);
    ⟨ Write defining scrap numbers 78b ⟩
    fputs("\\WEFTrangle\\ ", tex_file);
    ⟨ Write referencing scrap numbers 78c ⟩
    putc('\\n', tex_file);
}

```

78a · used in 77b · uses fputs 21b, putc 21b, sector 121c

⟨ *Write defining scrap numbers* 78b ⟩ ≡

```

{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("\\WEFTlink{weft", tex_file);
        write_scrap_ref(tex_file, p->scrap, -1, &page);
        fputs("{", tex_file);
        write_scrap_ref(tex_file, p->scrap, TRUE, &page);
        fputs("}", tex_file);
        p = p->next;
        while (p) {
            fputs("\\WEFTlink{weft", tex_file);
            write_scrap_ref(tex_file, p->scrap, -1, &page);
            fputs("{", tex_file);
            write_scrap_ref(tex_file, p->scrap, FALSE, &page);
            fputs("}", tex_file);
            p = p->next;
        }
    }
    else
        putc('?', tex_file);
}

```

78b · used in 78a · uses defs 121c, FALSE 23a, fputs 21b, next 121b, page 121c, putc 21b, Scrap_Node 144a, TRUE 23a, write_scrap_ref 123b

⟨ *Write referencing scrap numbers* 78c ⟩ ≡

```

{
    Scrap_Node *p = name->uses;
    fputs("{\\footnotesize\\WEFTtint{weftcom} ", tex_file);
    if (p) {
        fputs("{\\WEFTtxtRefIn}", tex_file);
        putc(' ', tex_file);
    }
}

```

```

    if (p->next) {
        print_scrap_numbers(tex_file, p);
    }
    else {
        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("{}{", tex_file);
        write_single_scrap_ref(tex_file, p->scrap);
        fputs("}", tex_file);
    }
}
else
    fputs("{\\WEFTtxtNoRef}", tex_file);
putc('}', tex_file);
}

```

78c · used in 78a · uses fputs 21b, next 121b, print_scrap_numbers 65c, putc 21b, Scrap_Node 144a, uses 121c, write_single_scrap_ref 123c

⟨ *Function prototypes* 79a ⟩ + ≡

```
extern int has_sector(Name *, unsigned char);
```

79a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses has_sector 79b, Name 144b

⟨ *latex.c* 79b ⟩ + ≡

```

int has_sector(Name * name, unsigned char sector)
{
    while(name) {
        if (name->sector == sector)
            return TRUE;
        if (has_sector(name->llink, sector))
            return TRUE;
        name = name->rlink;
    }
    return FALSE;
}

```

79b · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines has_sector 76c, 79ac, 93e, 95c · uses FALSE 23a, Name 144b, sector 121c, TRUE 23a

⟨ *Write index of user-specified names* 79c ⟩ ≡

```

{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n{\\small\\begin{list}{\\setlength{\\itemsep}{-\\parsep}",
            tex_file);
        fputs("\\setlength{\\itemindent}{-\\leftmargin}}\\n", tex_file);
        format_user_entry(user_names, tex_file, sector);
        fputs("\\end{list}}", tex_file);
    }
}

```

79c · used in 57c · uses current_sector 44c, format_user_entry 79d, fputs 21b, has_sector 79b, sector 121c, source_get 118a, user_names 144c

```

< latex.c 79d > + ≡
static void format_user_entry(Name *name, FILE *tex_file, unsigned char sector)
{
    while (name) {
        format_user_entry(name->llink, tex_file, sector);
        < Format a user index entry 79e >
        name = name->rlink;
    }
}

79d · defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd · defines format_user_entry 53b, 79c
· uses FILE 21b, Name 144b, sector 121c

< Format a user index entry 79e > ≡
if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
        fprintf(tex_file, "\\item {\\WEFTtint{weftkw}\\verb%c%s%c}: ",
            nw_char, name->spelling, nw_char);
        if (!uses) {
            fputs("\\underline{", tex_file);
            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("{}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("})", tex_file);
            page = -2;
            defs = defs->next;
        }
        else
            if (!defs || uses->scrap < defs->scrap) {
                fputs("\\WEFTlink{weft", tex_file);
                write_scrap_ref(tex_file, uses->scrap, -1, &page);
                fputs("{}{", tex_file);
                write_scrap_ref(tex_file, uses->scrap, TRUE, &page);
                fputs("}", tex_file);
                uses = uses->next;
            }
        else {
            if (defs->scrap == uses->scrap)
                uses = uses->next;
            fputs("\\underline{", tex_file);

            fputs("\\WEFTlink{weft", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("{}{", tex_file);
            write_single_scrap_ref(tex_file, defs->scrap);
            fputs("})", tex_file);
            page = -2;
            defs = defs->next;
        }
        while (uses || defs) {
            if (uses && (!defs || uses->scrap < defs->scrap)) {
                fputs("\\WEFTlink{weft", tex_file);
                write_scrap_ref(tex_file, uses->scrap, -1, &page);
                fputs("{}{", tex_file);
                write_scrap_ref(tex_file, uses->scrap, FALSE, &page);
            }
        }
    }
}

```

```

        fputs("}", tex_file);
        uses = uses->next;
    }
    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(" ", tex_file);

        fputs("\\WEFTlink{weft", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}{", tex_file);
        write_single_scrap_ref(tex_file, defs->scrap);
        fputs("}", tex_file);

        putc('}', tex_file);
        page = -2;
        defs = defs->next;
    }
}
fputs(".\\n", tex_file);
}
}

```

79e · used in 79d · uses [dangling_flag](#) 27a, [defs](#) 121c, [FALSE](#) 23a, [fprintf](#) 21b, [fputs](#) 21b, [next](#) 121b, [nw_char](#) 28ab, [page](#) 121c, [putc](#) 21b, [Scrap_Node](#) 144a, [sector](#) 121c, [TRUE](#) 23a, [uses](#) 121c, [write_scrap_ref](#) 123b, [write_single_scrap_ref](#) 123c

2.5 Writing the Markdown File

This section implements the Markdown weave output, activated by the `-w md` command-line flag. It is structurally parallel to the L^AT_EX output in Section 2.4, but emits Markdown with embedded HTML for scraps (using `<pre><code>` blocks so that fragment references inside code can be clickable links).

(Function prototypes 81a) + ≡

```
extern void write_md(char *file_name, char *md_name, unsigned char sector);
```

81a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses [file_name](#) 121c, [sector](#) 121c, [write_md](#) 81c

We need a few local function declarations before we get into the body of `write_md`.

(markdown.c 81b) + ≡

```
static void md_copy_scrap(FILE *file, int prefix, Name *name);
static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps);
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_format_file_entry(Name *name, FILE *md_file);
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector);
static void md_write_arg(FILE *md_file, char *p);
static void md_write_literal(FILE *md_file, char *p);
static void md_write_ArglistElement(FILE *file, Arglist *args, char **params);
static void md_format_uses_refs(FILE *md_file, int scrap);
static void md_format_defs_refs(FILE *md_file, int scrap);

```

81b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · uses [Arglist](#) 164d, [FILE](#) 21b, [md_copy_scrap](#) 88a, [md_format_defs_refs](#) 98b, [md_format_entry](#) 94b, [md_format_file_entry](#) 93b, [md_format_user_entry](#) 96a, [md_format_uses_refs](#) 97a, [md_print_scrap_numbers](#) 87d, [md_write_arg](#) 85b, [md_write_ArglistElement](#) 91b, [md_write_literal](#) 91a, [Name](#) 144b, [scraps](#) 122a, [Scrap_Node](#) 144a, [sector](#) 121c

The routine `write_md` takes two file names as parameters: the name of the web source file and the name of the `.md` output file.

```
<markdown.c 81c> + ≡
void write_md(char *file_name, char *md_name, unsigned char sector)
{
    (void)sector;
    FILE *md_file = fopen(md_name, "w");
    if (md_file) {
        if (verbose_flag)
            fprintf(stderr, "writing %s\n", md_name);
        source_open(file_name);
        <Copy source_file into md_file 82a>
        fclose(md_file);
    }
    else
        fprintf(stderr, "%s: can't open %s\n", command_name, md_name);
}
```

81c · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `write_md` 39b, 81a · uses `command_name` 28c, `fclose` 21b, `FILE` 21b, `file_name` 121c, `fopen` 21b, `fprintf` 21b, `sector` 121c, `source_open` 121a, `stderr` 21b, `verbose_flag` 27a

We make our second pass through the source web, copying characters straight into the `.md` file. We watch for `@` characters which signal a command sequence. We also detect and discard cosmetic fence lines (lines that are exactly `'''` optionally followed by a language tag).

```
<Copy source_file into md_file 82a> ≡
{
    int c = source_get();
    int at_line_start = TRUE;
    while (c != EOF) {
        if (c == nw_char)
            {
                <Interpret md at-sequence 82c>
            }
        else if (c == ''' && at_line_start) {
            <Check for cosmetic fence line 82b>
        }
        else {
            if (c == '\n') at_line_start = TRUE;
            else at_line_start = FALSE;
            putc(c, md_file);
            c = source_get();
        }
    }
}
```

82a · used in 81c · uses `FALSE` 23a, `nw_char` 28ab, `putc` 21b, `source_get` 118a, `TRUE` 23a

Cosmetic fence lines are lines that start with `'''`. We peek ahead to see if the line matches; if so, we skip to end of line. Otherwise we output what we already consumed and continue.

```
<Check for cosmetic fence line 82b> ≡
{
    int c2 = source_get();
    int c3 = (c2 == ''' ? source_get() : 0;
    if (c2 == ''' && c3 == ''' ) {
        /* Skip rest of fence line */
    }
```

```

    int fc = source_get();
    while (fc != '\n' && fc != EOF)
        fc = source_get();
    c = source_get();
    at_line_start = TRUE;
} else {
    /* Not a fence, output what we consumed */
    putc(' ', md_file);
    if (c2 == ' ') {
        putc(' ', md_file);
        c = c3;
    } else {
        c = c2;
    }
    at_line_start = FALSE;
}
}
}
82b · used in 82a · uses FALSE 23a, putc 21b, source_get 118a, TRUE 23a

```

⟨ *Interpret md at-sequence 82c* ⟩ ≡

```

{
    int big_definition = FALSE;
    c = source_get();
    switch (c) {
        case 'r':
            c = source_get();
            nw_char = c;
            update_delimit_scrap();
            break;
        case 'O': big_definition = TRUE;
            FALLTHROUGH;
        case 'o': ⟨ Write md output file definition 84b ⟩
            break;
        case 'Q':
        case 'D': big_definition = TRUE;
            FALLTHROUGH;
        case 'q':
        case 'd': ⟨ Write md macro definition 84c ⟩
            break;
        case 's':
            ⟨ Step to next sector 44a ⟩
            break;
        case 'S':
            ⟨ Close the current sector 44b ⟩
            break;
        case '{':
        case '[':
        case '(': ⟨ Write md in-text scrap 86c ⟩
            break;
        case '<': ⟨ Expand md macro into md_file 84a ⟩
            break;
        case 'x': ⟨ Copy label from source into(83a md_file ) 72a ⟩
            c = source_get();
            break;
        case 'c': /* Block comments: no special handling in Markdown */
            c = source_get();
            break;
        case 'f': ⟨ Write md index of file names 93a ⟩
    }
}

```

```

        break;
    case 'm': < Write md index of macro names 93e >
        break;
    case 'u': < Write md index of user-specified names 95c >
        break;
    case 'v': < Copy version info into md file 83b >
        break;
    case 'l':
    case 'L': < Skip at-L language definition line 43c >
        break;
    case 'W': < Skip at-L language definition line 43c >
        break;
    default:
        if (c==nw_char)
            putc(c, md_file);
        c = source_get();
        break;
    }
}

```

83a · used in 82a

< Copy version info into md file 83b > ≡
fputs(version_string, md_file);
c = source_get();

83b · used in 82c · uses fputs 21b, source_get 118a, version_string 27a

< Expand md macro into file 84a > ≡

```

{
    Parameters local_parameters = 0;
    int changed;
    char indent_chars[MAX_INDENT];
    Arglist *a;
    Name *name;
    Arglist * args;
    a = collect_scrap_name(0);
    name = a->name;
    args = instance(a->args, NULL, NULL, &changed);
    name->mark = TRUE;
    write_scraps(file, md_name, name->defs, 0, indent_chars, 0, 0, 1, 0, 0,
        args, name->arg, local_parameters, md_name);
    name->mark = FALSE;
    c = source_get();
}

```

84a · used in 82c · uses Arglist 164d, collect_scrap_name 165, defs 121c, FALSE 23a, instance 132e, MAX_INDENT 113d, Name 144b, Parameters 50b, source_get 118a, TRUE 23a, write_scraps 134b

2.5.1 Formatting Definitions in Markdown

File and fragment definitions use HTML anchors and formatting. A file definition header looks like:

```

<a id="weft1"></a>
**"filename"** **1** =
<pre><code>...code...</code></pre>

```

A fragment definition header looks like:

```

<a id="weft1"></a>

```

```
**&lt;*Fragment Name* 1&gt;** =
<pre><code>...code...</code></pre>
```

```
< Write md output file definition 84b > ≡
{
    Name *name = collect_file_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("\n></a>\n", md_file);
    fprintf(md_file, "**\"%s\"**", name->spelling);
    fputs(" **", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("** =\n", md_file);
    < Fill in the md scrap 86a >
    < Write md file cross-refs 86d >
    < Finish md scrap 86b >
}
```

84b · used in 82c · uses [collect_file_name](#) 159a, [fprintf](#) 21b, [fputs](#) 21b, [Name](#) 144b, [scraps](#) 122a, [write_single_scrap_ref](#) 123c

```
< Write md macro definition 84c > ≡
{
    Name *name = collect_macro_name();
    (void)big_definition;
    fputs("\n<a id=\"weft\", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("\n></a>\n", md_file);
    fputs("**&lt;*", md_file);
    < Write the md macro's name 85a >
    fputs("* ", md_file);
    write_single_scrap_ref(md_file, scraps);
    fputs("&gt;** =\n", md_file);
    < Fill in the md scrap 86a >
    < Write md macro cross-refs 87a >
    < Finish md scrap 86b >
}
```

84c · used in 82c · uses [collect_macro_name](#) 162a, [fputs](#) 21b, [Name](#) 144b, [scraps](#) 122a, [write_single_scrap_ref](#) 123c

```
< Write the md macro's name 85a > ≡
{
    char * p = name->spelling;
    int i = 0;

    while (*p != '\000') {
        if (*p == ARG_CHR) {
            md_write_arg(md_file, name->arg[i++]);
            p++;
        }
        else
            fputc(*p++, md_file);
    }
}
```

85a · used in 84c, 94c · uses [ARG_CHR](#) 163b, [md_write_arg](#) 85b

```
< markdown.c 85b > + ≡
```

```

static void md_write_arg(FILE * md_file, char * p)
{
    fputs("<i>", md_file);
    while (*p)
    {
        switch (*p)
        {
            case '<':
                fputs("&lt;", md_file);
                break;
            case '>':
                fputs("&gt;", md_file);
                break;
            case '&':
                fputs("&amp;", md_file);
                break;
            default:
                fputc(*p, md_file);
                break;
        }
        p++;
    }
    fputs("</i>", md_file);
}

```

85b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `md_write_arg` 81b, 85a, 89b, 91b · uses `FILE` 21b, `fputs` 21b

```

< Fill in the md scrap 86a > ≡
{
    if (name->lang[0] != '\0')
        fprintf(md_file, "<pre><code class=\"language-%s\">", name->lang);
    else
        fputs("<pre><code>", md_file);
    extra_scrap = 0;
    md_copy_scrap(md_file, TRUE, name);
    fputs("</code></pre>\n", md_file);
}

```

86a · used in 84bc · uses `extra_scrap` 64b, `fprintf` 21b, `fputs` 21b, `md_copy_scrap` 88a, `TRUE` 23a

```

< Finish md scrap 86b > ≡
{
    scrap += extra_scrap;
    do
        c = source_get();
    while (isspace(c));
    at_line_start = TRUE;
}

```

86b · used in 84bc · uses `extra_scrap` 64b, `isspace` 21b, `scrap` 122a, `source_get` 118a, `TRUE` 23a

```

< Write md in-text scrap 86c > ≡
md_copy_scrap(md_file, FALSE, NULL);
c = source_get();

```

86c · used in 82c · uses `FALSE` 23a, `md_copy_scrap` 88a, `source_get` 118a

Cross References in Markdown

Cross-references appear as blockquotes after each scrap.

< Write md file cross-refs 86d > ≡

```
{
    if ( scrap_flag ) {
        < Write md file defs 86e >
    }
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}
```

86d · used in 84b · uses md_format_defs_refs 98b, md_format_uses_refs 97a, scraps 122a, scrap_flag 27a

< Write md file defs 86e > ≡

```
{
    if (name->defs) {
        if (name->defs->next) {
            fputs("> *File defined by* ", md_file);
            md_print_scrap_numbers(md_file, name->defs);
        }
    } else {
        fprintf(stderr,
            "would have crashed in 'Write md file defs' for '%s'\n",
            name->spelling);
    }
}
```

86e · used in 86d · uses defs 121c, fprintf 21b, fputs 21b, md_print_scrap_numbers 87d, next 121b, stderr 21b

< Write md macro cross-refs 87a > ≡

```
{
    < Write md macro defs 87b >
    < Write md macro refs 87c >
    md_format_defs_refs(md_file, scraps);
    md_format_uses_refs(md_file, scraps++);
}
```

87a · used in 84c · uses md_format_defs_refs 98b, md_format_uses_refs 97a, scraps 122a

< Write md macro defs 87b > ≡

```
{
    if (name->defs->next) {
        fputs("> *Fragment defined by* ", md_file);
        md_print_scrap_numbers(md_file, name->defs);
    }
}
```

87b · used in 87a · uses defs 121c, fputs 21b, md_print_scrap_numbers 87d, next 121b

< Write md macro refs 87c > ≡

```
{
    if (name->uses) {
        if (name->uses->next) {
            fputs("> *Fragment referenced in* ", md_file);
            md_print_scrap_numbers(md_file, name->uses);
        }
        else {
            fputs("> *Fragment referenced in* ", md_file);
            fputs("<a href=\"#weft\"", md_file);
            write_single_scrap_ref(md_file, name->uses->scrap);
            fputs("\>", md_file);
            write_single_scrap_ref(md_file, name->uses->scrap);
        }
    }
}
```

```

        fputs("</a>", md_file);
        fputs(".\n", md_file);
    }
}
else {
    fputs("> *Fragment never referenced*.\\n", md_file);
    fprintf(stderr, "%s: <%s> never referenced.\\n",
            command_name, name->spelling);
}
}

```

87c · used in 87a · uses `command_name` 28c, `fprintf` 21b, `fputs` 21b, `md_print_scrap_numbers` 87d, `next` 121b, `stderr` 21b, `uses` 121c, `write_single_scrap_ref` 123c

⟨`markdown.c` 87d⟩ + ≡

```

static void md_print_scrap_numbers(FILE *md_file, Scrap_Node *scraps)
{
    int page;
    fputs("<a href=\"#weft\"", md_file);
    write_scrap_ref(md_file, scraps->scrap, -1, &page);
    fputs(">", md_file);
    write_scrap_ref(md_file, scraps->scrap, TRUE, &page);
    fputs("</a>", md_file);
    scraps = scraps->next;
    while (scraps) {
        fputs("<a href=\"#weft\"", md_file);
        write_scrap_ref(md_file, scraps->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, scraps->scrap, FALSE, &page);
        fputs("</a>", md_file);
        scraps = scraps->next;
    }
    fputs(".\\n", md_file);
}

```

87d · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `md_print_scrap_numbers` 81b, 86e, 87bc, 93d, 95b · uses `FALSE` 23a, `FILE` 21b, `fputs` 21b, `next` 121b, `page` 121c, `scraps` 122a, `Scrap_Node` 144a, `TRUE` 23a, `write_scrap_ref` 123b

Formatting a Scrap in Markdown

Scraps are output inside `<pre><code>` blocks. Characters `<`, `>`, and `&` are HTML-escaped. At-sequences within scraps (macro references, keywords, parameters, etc.) emit HTML markup directly.

⟨`markdown.c` 88a⟩ + ≡

```

static int md_scrap_type = 0;

static void md_copy_scrap(FILE *file, int prefix, Name *name)
{
    int indent = 0;
    int c;
    char **params = name ? name->arg : NULL;
    if (source_last == '{') md_scrap_type = 0;
    if (source_last == '[') md_scrap_type = 1;
    if (source_last == '(') md_scrap_type = 2;
    c = source_get();
    while (1) {
        switch (c) {
            case '\\n': fputs("\\n", file);

```

```

        indent = 0;
        break;
case '\t': < Expand md tab into spaces 89a >
        break;
default:
    if (c==nw_char)
    {
        < Check md at-sequence for end-of-scrap 89b >
        break;
    }
    if (md_scrap_type == 0 && prefix) {
        < HTML-escape character c 88b >
    } else {
        putc(c, file);
    }

        indent++;
        break;
}
c = source_get();
}
}

```

88a · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines md_copy_scrap 81b, 86ac,
md_scrap_type 89b · uses FILE 21b, fputs 21b, Name 144b, nw_char 28ab, putc 21b, source_get 118a, source_last 118a

< HTML-escape character c 88b > ≡

```

switch (c) {
    case '<': fputs("<", file); break;
    case '>': fputs(">", file); break;
    case '&': fputs("&", file); break;
    default: putc(c, file); break;
}

```

88b · used in 88a · uses fputs 21b, putc 21b

< Expand md tab into spaces 89a > ≡

```

{
    int delta = 8 - (indent % 8);
    indent += delta;
    while (delta > 0) {
        putc(' ', file);
        delta--;
    }
}

```

89a · used in 88a · uses putc 21b

< Check md at-sequence for end-of-scrap 89b > ≡

```

{
    c = source_get();
    switch (c) {
        case 'c': < Show md presence of a block comment 90a >
            break;
        case 'x': < Copy label from source into(89c file ) 72a >
            break;
        case 'v': < Copy version info into file 71b >
        case 's':
            break;
        case '+':
        case '-':
    }
}

```

```

case '*':
case '|': < Skip over index entries 72c >
case ',':
case ')':
case ']':
case '}': return;
case '<': < Format md macro name 90d >
        break;
case '%': < Skip commented-out code 72d >
        break;
case '_': < Bold md Keyword 90b >
        break;
case 't': < Italic md "fragment title" 90c >
        break;
case 'f': < Italic md "file name" 90c >
        break;
case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
        if (name == NULL
            || name->arg[c - '1'] == NULL) {
            if (md_scrap_type == 0 && prefix)
                fputs("&", file);
            else
                putc(nw_char, file);
            fputc(c, file);
        }
        else {
            md_write_arg(file, name->arg[c - '1']);
        }
        break;
default:
        if (c==nw_char)
        {
            if (md_scrap_type == 0 && prefix)
                fputs("&", file);
            else
                putc(nw_char, file);
            break;
        }
        /* ignore these since pass1 will have warned about them */
        break;
    }
}

```

89c · used in [88a](#)

```

< Show md presence of a block comment 90a > ≡
{
    fputs("<i>(Comment)</i>", file);
}

```

90a · used in [89b](#) · uses [fputs 21b](#)

```

< Bold md Keyword 90b > ≡
{
    fprintf(file, "<b>");
    c = source_get();
    do {
        if (c == '<') fputs("&lt;", file);
    }
}

```

```

        else if (c == '>') fputs(">", file);
        else if (c == '&') fputs("&", file);
        else fputc(c, file);
        c = source_get();
    } while (c != nw_char);
    c = source_get();
    fprintf(file, "</b>");
}

```

90b · used in 89b · uses `fprintf 21b`, `fputs 21b`, `nw_char 28ab`, `source_get 118a`

⟨ *Italic md "whatever"* 90c ⟩ ≡

```

{
    fprintf(file, "<i>whatever</i>");
}

```

90c · used in 89b · uses `fprintf 21b`

⟨ *Format md macro name* 90d ⟩ ≡

```

{
    Arglist *args = collect_scrap_name(-1);
    Name *name = args->name;
    char *p = name->spelling;
    Arglist *q = args->args;
    int narg = 0;

    fputs("&lt;<a href=\"#weft\", file);
    if (name->defs) {
        write_single_scrap_ref(file, name->defs->scrap);
    } else {
        putc('?', file);
    }
    fputs("\><i>", file);
    while (*p != '\000') {
        if (*p == ARG_CHR) {
            if (q == NULL) {
                md_write_literal(file, name->arg[narg]);
            }
            else {
                md_write_ArglistElement(file, q, params);
                q = q->next;
            }
            p++;
            narg++;
        }
        else {
            if (*p == '<') fputs("&lt;", file);
            else if (*p == '>') fputs(">", file);
            else if (*p == '&') fputs("&", file);
            else fputc(*p, file);
            p++;
        }
    }
    fputs("</i> ", file);
    if (name->defs) {
        Scrap_Node *dp = name->defs;
        write_single_scrap_ref(file, dp->scrap);
        dp = dp->next;
        if (dp)
            fputs(", ...", file);
    }
}

```

```

}
else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
}
fputs("</a>&gt;", file);
}

```

90d · used in 89b · uses Arglist 164d, ARG_CHR 163b, collect_scrap_name 165, command_name 28c, defs 121c, fprintf 21b, fputs 21b, md_write_ArglistElement 91b, md_write_literal 91a, Name 144b, next 121b, putc 21b, Scrap_Node 144a, stderr 21b, write_single_scrap_ref 123c

⟨markdown.c 91a⟩ + ≡

```

static void md_write_literal(FILE * md_file, char * p)
{
    while (*p != '\000') {
        if (*p == '<') fputs("&lt;", md_file);
        else if (*p == '>') fputs("&gt;", md_file);
        else if (*p == '&') fputs("&amp;", md_file);
        else fputc(*p, md_file);
        p++;
    }
}

```

91a · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines md_write_literal 81b, 90d, 91b · uses FILE 21b, fputs 21b

⟨markdown.c 91b⟩ + ≡

```

static void
md_write_ArglistElement(FILE * file, Arglist * args, char ** params)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        char * p = (char*)q;

        if (p[0] == ARG_CHR) {
            md_write_arg(file, params[p[1] - '1']);
        } else {
            md_write_literal(file, (char *)q);
        }
    } else if (name == (Name *)1) {
        Scrap_Node * qq = (Scrap_Node *)q;
        qq->quoted = TRUE;
        write_scraps(file, "", qq,
                    -1, "", 0, 0, 0, 0, 0,
                    NULL, params, 0, "");
        extra_scraps++;
        qq->quoted = FALSE;
    } else {
        char * p = name->spelling;
        fputs("&lt;<i>", file);
        while (*p != '\000') {
            if (*p == ARG_CHR) {
                md_write_ArglistElement(file, q, params);
                q = q->next;
                p++;
            }
        }
    }
}

```

```

    }
    else {
        if (*p == '<') fputs("<lt;\"", file);
        else if (*p == '>') fputs(">\"", file);
        else if (*p == '&') fputs("&\"", file);
        else fputc(*p, file);
        p++;
    }
}
fputs("</i> ", file);
if (name->defs) {
    Scrap_Node *dp = name->defs;
    write_single_scrap_ref(file, dp->scrap);
    dp = dp->next;
    if (dp)
        fputs(", ...", file);
}
else {
    putc('?', file);
    fprintf(stderr, "%s: never defined <%s>\n",
            command_name, name->spelling);
}
fputs("&gt;", file);
}
}

```

91b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `md_write_ArglistElement` 81b, 90d · uses `Arglist` 164d, `ARG_CHR` 163b, `command_name` 28c, `defs` 121c, `extra_scraps` 64b, `FALSE` 23a, `FILE` 21b, `fprintf` 21b, `fputs` 21b, `md_write_arg` 85b, `md_write_literal` 91a, `Name` 144b, `next` 121b, `putc` 21b, `Scrap_Node` 144a, `stderr` 21b, `TRUE` 23a, `write_scraps` 134b, `write_single_scrap_ref` 123c

2.5.2 Generating the Indices in Markdown

⟨ *Write md index of file names* 93a ⟩ ≡

```

{
    if (file_names) {
        fputs("\n", md_file);
        md_format_file_entry(file_names, md_file);
        fputs("\n", md_file);
    }
    c = source_get();
}

```

93a · used in 82c · uses `file_names` 144c, `fputs` 21b, `md_format_file_entry` 93b, `source_get` 118a

⟨ *markdown.c* 93b ⟩ + ≡

```

static void md_format_file_entry(Name *name, FILE *md_file)
{
    while (name) {
        md_format_file_entry(name->llink, md_file);
        ⟨ Format a md file index entry 93c ⟩
        name = name->rlink;
    }
}

```

93b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `md_format_file_entry` 81b, 93a · uses `FILE` 21b, `Name` 144b

⟨ *Format a md file index entry* 93c ⟩ ≡

```
fputs("- ", md_file);
fprintf(md_file, "'\\'%s\\' " , name->spelling);
< Write md file's defining scrap numbers 93d >
putc('\\n', md_file);
```

93c · used in 93b · uses [fprintf 21b](#), [fputs 21b](#), [putc 21b](#)

< Write md file's defining scrap numbers 93d > ≡

```
{
    Scrap_Node *p = name->defs;
    fputs("*Defined by* ", md_file);
    if (p->next) {
        md_print_scrap_numbers(md_file, p);
    }
    else {
        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs("\\>", md_file);
        write_single_scrap_ref(md_file, p->scrap);
        fputs("</a>.", md_file);
    }
}
```

93d · used in 93c · uses [defs 121c](#), [fputs 21b](#), [md_print_scrap_numbers 87d](#), [next 121b](#), [Scrap_Node 144a](#), [write_single_scrap_ref 123c](#)

< Write md index of macro names 93e > ≡

```
{
    unsigned char sector = current_sector;
    int c = source_get();
    if (c == '+')
        sector = 0;
    else
        source_ungetc(&c);
    if (has_sector(macro_names, sector)) {
        fputs("\\n", md_file);
        md_format_entry(macro_names, md_file, sector);
        fputs("\\n", md_file);
    } else {
        fputs("None.\\n", md_file);
    }
}
c = source_get();
```

93e · used in 82c · uses [current_sector 44c](#), [fputs 21b](#), [has_sector 79b](#), [macro_names 144c](#), [md_format_entry 94b](#), [sector 121c](#), [source_get 118a](#), [source_ungetc 118c](#)

< markdown.c 94a > + ≡

```
static int md_load_entry(Name * name, Name ** nms, int n)
{
    while (name) {
        n = md_load_entry(name->llink, nms, n);
        nms[n++] = name;
        name = name->rlink;
    }
    return n;
}
```

94a · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines [md_load_entry 94b](#) · uses [Name 144b](#)

< markdown.c 94b > + ≡

```
static void md_format_entry(Name *name, FILE *md_file, unsigned char sector)
{
    Name ** nms = malloc(num_scrap()*sizeof(Name *));
    int n = md_load_entry(name, nms, 0);
    int i;

    < Sort nms of size n for < Rob's ordering 77c > 77d >
    for (i = 0; i < n; i++)
    {
        Name * name = nms[i];

        < Format a md index entry 94c >
    }
    free(nms);
}
```

94b · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines md_format_entry 81b, 93e · uses FILE 21b, malloc 21b, md_load_entry 94a, Name 144b, num_scrap 122a, sector 121c

< Format a md index entry 94c > ≡

```
if (name->sector == sector){
    fputs("- ", md_file);
    fputs("<*", md_file);
    < Write the md macro's name 85a >
    fputs("* ", md_file);
    < Write md defining scrap numbers 95a >
    fputs(">", md_file);
    < Write md referencing scrap numbers 95b >
    putc('\n', md_file);
}
```

94c · used in 94b · uses fputs 21b, putc 21b, sector 121c

< Write md defining scrap numbers 95a > ≡

```
{
    Scrap_Node *p = name->defs;
    if (p) {
        int page;
        fputs("<a href=\"#weft\", md_file);
        write_scrap_ref(md_file, p->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, p->scrap, TRUE, &page);
        fputs("</a>", md_file);
        p = p->next;
        while (p) {
            fputs("<a href=\"#weft\", md_file);
            write_scrap_ref(md_file, p->scrap, -1, &page);
            fputs(">", md_file);
            write_scrap_ref(md_file, p->scrap, FALSE, &page);
            fputs("</a>", md_file);
            p = p->next;
        }
    }
    else
        putc('?', md_file);
}
```

95a · used in 94c · uses defs 121c, FALSE 23a, fputs 21b, next 121b, page 121c, putc 21b, Scrap_Node 144a, TRUE 23a, write_scrap_ref 123b

< Write md referencing scrap numbers 95b > ≡

```
{
    Scrap_Node *p = name->uses;
    if (p) {
        fputs("*Referenced in* ", md_file);
        if (p->next) {
            md_print_scrap_numbers(md_file, p);
        }
        else {
            fputs("<a href=\"#weft\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs(\">\", md_file);
            write_single_scrap_ref(md_file, p->scrap);
            fputs("</a>.", md_file);
        }
    }
    else
        fputs("*Not referenced*.", md_file);
}
```

95b · used in 94c · uses fputs 21b, md_print_scrap_numbers 87d, next 121b, Scrap_Node 144a, uses 121c, write_single_scrap_ref 123c

< Write md index of user-specified names 95c > ≡

```
{
    unsigned char sector = current_sector;
    c = source_get();
    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    if (has_sector(user_names, sector)) {
        fputs("\n", md_file);
        md_format_user_entry(user_names, md_file, sector);
        fputs("\n", md_file);
    }
}
```

95c · used in 82c · uses current_sector 44c, fputs 21b, has_sector 79b, md_format_user_entry 96a, sector 121c, source_get 118a, user_names 144c

< markdown.c 96a > + ≡

```
static void md_format_user_entry(Name *name, FILE *md_file, unsigned char sector)
{
    while (name) {
        md_format_user_entry(name->llink, md_file, sector);
        < Format a md user index entry 96b >
        name = name->rlink;
    }
}
```

96a · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines md_format_user_entry 81b, 95c · uses FILE 21b, Name 144b, sector 121c

< Format a md user index entry 96b > ≡

```
if (name->sector == sector){
    Scrap_Node *uses = name->uses;
    if ( uses || dangling_flag ) {
        int page;
        Scrap_Node *defs = name->defs;
```

```

fprintf(md_file, "- '%s': ", name->spelling);
if (!uses) {
    fputs("**", md_file);
    fputs("<a href=\"#weft\"", md_file);
    write_single_scrap_ref(md_file, defs->scrap);
    fputs(">", md_file);
    write_single_scrap_ref(md_file, defs->scrap);
    fputs("</a>**", md_file);
    page = -2;
    defs = defs->next;
}
else
    if (!defs || uses->scrap < defs->scrap) {
        fputs("<a href=\"#weft\"", md_file);
        write_scrap_ref(md_file, uses->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, uses->scrap, TRUE, &page);
        fputs("</a>", md_file);
        uses = uses->next;
    }
else {
    if (defs->scrap == uses->scrap)
        uses = uses->next;
    fputs("**", md_file);

    fputs("<a href=\"#weft\"", md_file);
    write_single_scrap_ref(md_file, defs->scrap);
    fputs(">", md_file);
    write_single_scrap_ref(md_file, defs->scrap);
    fputs("</a>**", md_file);
    page = -2;
    defs = defs->next;
}
while (uses || defs) {
    if (uses && (!defs || uses->scrap < defs->scrap)) {
        fputs("<a href=\"#weft\"", md_file);
        write_scrap_ref(md_file, uses->scrap, -1, &page);
        fputs(">", md_file);
        write_scrap_ref(md_file, uses->scrap, FALSE, &page);
        fputs("</a>", md_file);
        uses = uses->next;
    }
    else {
        if (uses && defs->scrap == uses->scrap)
            uses = uses->next;
        fputs(", **", md_file);

        fputs("<a href=\"#weft\"", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs(">", md_file);
        write_single_scrap_ref(md_file, defs->scrap);
        fputs("</a>**", md_file);
        page = -2;
        defs = defs->next;
    }
}
fputs(".\n", md_file);
}
}

```

96b · used in 96a · uses `dangling_flag` 27a, `defs` 121c, `FALSE` 23a, `fprintf` 21b, `fputs` 21b, `next` 121b, `page` 121c, `Scrap_Node` 144a, `sector` 121c, `TRUE` 23a, `uses` 121c, `write_scrap_ref` 123b, `write_single_scrap_ref` 123c

2.5.3 Identifier Cross-References in Markdown

These functions are the Markdown equivalents of `format_uses_refs` and `format_defs_refs` from `scraps.c`, but emit Markdown blockquote format.

⟨ `markdown.c` 97a ⟩ + ≡

```
static void
md_format_uses_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_uses(scrap);
    if (p != NULL)
        ⟨ Write md uses references 97b ⟩
}
```

97a · defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b · defines `md_format_uses_refs` 81b, 86d, 87a · uses `FILE` 21b, `get_scrap_uses` 123d, `Uses` 176d

⟨ *Write md uses references* 97b ⟩ ≡

```
{
    char join = ' ';
    fputs("> *Uses:* ", md_file);
    do {
        ⟨ Write one md use reference 97c ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}
```

97b · used in 97a · uses `fputs` 21b, `next` 121b, `Uses` 176d

⟨ *Write one md use reference* 97c ⟩ ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs)
{
    do {
        ⟨ Write one md referenced scrap 98a ⟩
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else
{
    fputs("(not defined globally)", md_file);
}
```

97c · used in 97b · uses `defs` 121c, `FALSE` 23a, `first` 182a, `fprintf` 21b, `fputs` 21b, `Name` 144b, `next` 121b, `page` 121c, `Scrap_Node` 144a, `TRUE` 23a

⟨ *Write one md referenced scrap* 98a ⟩ ≡

```
fputs("<a href=\"#weft\", md_file);
write_scrap_ref(md_file, defs->scrap, -1, &page);
fputs(">", md_file);
write_scrap_ref(md_file, defs->scrap, first, &page);
fputs("</a>", md_file);
```

98a · used in 97c, 98d · uses [defs 121c](#), [first 182a](#), [fputs 21b](#), [page 121c](#), [write_scrap_ref 123b](#)

[〈 markdown.c 98b 〉](#) + ≡

```
static void
md_format_defs_refs(FILE * md_file, int scrap)
{
    Uses * p = get_scrap_defs(scrap);
    if (p != NULL)
        〈 Write md defs references 98c 〉
}
```

98b · defined at [24b](#), [81bc](#), [85b](#), [87d](#), [88a](#), [91ab](#), [93b](#), [94ab](#), [96a](#), [97a](#), [98b](#) · defines [md_format_defs_refs 81b](#), [86d](#), [87a](#) · uses [FILE 21b](#), [get_scrap_defs 123e](#), [Uses 176d](#)

[〈 Write md defs references 98c 〉](#) ≡

```
{
    char join = ' ';
    fputs("> *Defines:*", md_file);
    do {
        〈 Write one md def reference 98d 〉
        join = ',';
        p = p->next;
    }while (p != NULL);
    fputs(".\n", md_file);
}
```

98c · used in [98b](#) · uses [fputs 21b](#), [next 121b](#)

[〈 Write one md def reference 98d 〉](#) ≡

```
Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(md_file,
        "%c '%s' ",
        join, name->spelling);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("*Never used*", md_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            〈 Write one md referenced scrap 98a 〉
            first = FALSE;
        }
        defs = defs->next;
    }while (defs != NULL);
}
```

98d · used in [98c](#) · uses [defs 121c](#), [FALSE 23a](#), [first 182a](#), [fprintf 21b](#), [fputs 21b](#), [Name 144b](#), [next 121b](#), [page 121c](#), [Scrap_Node 144a](#), [TRUE 23a](#), uses [121c](#)

2.6 Map and Extract Output

Two modes support AI-assisted navigation of weft projects:

- m (map) emits a JSON object to stdout describing every output file and every fragment, their definition locations, usage relationships, and transitive dependencies.
- e *name* (extract) emits the scrap code and dependency information for a single fragment (matched by prefix) as Markdown to stdout.

Both modes run after `pass1` has built the name trees and do *not* invoke the weave or tangle phases.

2.6.1 JSON Map Output

The `write_map` function performs an in-order traversal of the `file_names` and `macro_names` binary search trees, emitting a JSON object with two arrays: "files" and "fragments".

For each `Name` node, it records:

- The spelling (name of the file or fragment).
- The sector number.
- The language tag (if any).
- Each definition scrap with its source file and line.
- Each usage scrap (scraps that reference this name).
- For definition scraps, the list of fragments referenced inside (via `get_scrap_uses`).

A helper function escapes strings for JSON output.

```
<output.c 99> + ≡
static void json_puts(FILE *f, const char *s)
{
    fputc('\"', f);
    while (*s) {
        switch (*s) {
            case '\"': fputs("\\\"", f); break;
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if ((unsigned char)*s < 0x20)
                    fprintf(f, "\\u%04x", (unsigned char)*s);
                else
                    fputc(*s, f);
        }
        s++;
    }
    fputc('\"', f);
}
```

99 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c · defines `json_puts` 100,
109 · uses `FILE` 21b, `fprintf` 21b, `fputs` 21b

The `emit_name_json` helper emits a single name entry as a JSON object. The `is_file` flag distinguishes file names (which use "uses_fragments" for fragments referenced inside their definition scraps) from macro fragments (which use "used_by" for the scraps that invoke them, and "uses" for fragments they invoke).

```

<output.c 100> + ≡
static void emit_name_json(FILE *f, Name *name, int is_file, int *first)
{
    Scrap_Node *sn;
    int first_inner;

    if (!name) return;
    emit_name_json(f, name->llink, is_file, first);

    if (!*first) fputs("\n", f);
    *first = 0;

    fputs("    {\n", f);
    fputs("        \"name\": ", f); json_puts(f, name->spelling);
    if (name->lang[0]) {
        fputs(",\n        \"lang\": ", f); json_puts(f, name->lang);
    }
    fprintf(f, ",\n        \"sector\": %d", name->sector);

    /* defs array */
    fputs(",\n        \"defs\": [", f);
    first_inner = 1;
    for (sn = name->defs; sn; sn = sn->next) {
        if (!first_inner) fputs(", ", f);
        first_inner = 0;
        fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
        json_puts(f, scrap_file_name(sn->scrap));
        fprintf(f, ", \"line\": %d, \"end_line\": %d",
            scrap_file_line(sn->scrap), scrap_end_line(sn->scrap));
        if (bodies_flag) {
            fputs(", \"body\": ", f);
            dump_scrap_json(f, sn->scrap);
        }
        if (prose_flag && scrap_prose(sn->scrap)) {
            fputs(", \"prose\": ", f);
            json_puts(f, scrap_prose(sn->scrap));
        }
        fputc('}', f);
    }
    fputs("]", f);

    if (is_file) {
        /* uses_fragments: fragments referenced inside this file's scraps */
        fputs(",\n        \"uses_fragments\": [", f);
        first_inner = 1;
        for (sn = name->defs; sn; sn = sn->next) {
            Uses *u;
            for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
                if (!first_inner) fputs(", ", f);
                first_inner = 0;
                json_puts(f, u->defn->spelling);
            }
        }
        fputs("]", f);
    } else {
        /* used_by: scraps that invoke this fragment */
        fputs(",\n        \"used_by\": [", f);
        first_inner = 1;
        for (sn = name->uses; sn; sn = sn->next) {

```

```

    if (!first_inner) fputs(" ", f);
    first_inner = 0;
    fprintf(f, "{\"scrap\": %d, \"file\": ", sn->scrap);
    json_puts(f, scrap_file_name(sn->scrap));
    fprintf(f, ", \"line\": %d}", scrap_file_line(sn->scrap));
}
fputs("]", f);

/* uses: fragments referenced inside this fragment's def scraps */
fputs("\n    \"uses\": [", f);
first_inner = 1;
for (sn = name->defs; sn; sn = sn->next) {
    Uses *u;
    for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
        if (!first_inner) fputs(" ", f);
        first_inner = 0;
        json_puts(f, u->defn->spelling);
    }
}
fputs("]", f);

fputs("\n    ]", f);

emit_name_json(f, name->rlink, is_file, first);
}

100 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines emit_name_json 101 · uses bodies_flag 27a, defs 121c, dump_scrap_json 131, end_line 121c, FILE 21b, first 182a,
fprintf 21b, fputs 21b, get_scrap_uses 123d, json_puts 99, Name 144b, next 121b, prose 121c, prose_flag 27a,
scraps 122a, scrap_end_line 124d, scrap_file_line 124c, scrap_file_name 124b, Scrap_Node 144a, scrap_prose 124e,
sector 121c, Uses 176d, uses 121c

⟨output.c 101⟩ + ≡
void write_map(Name *files, Name *macros)
{
    int first;

    fputs("{\n    \"files\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, files, 1, &first);
    fputs("\n ],\n    \"fragments\": [\n", stdout);
    first = 1;
    emit_name_json(stdout, macros, 0, &first);
    fputs("\n ]\n}\n", stdout);
}

101 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c · defines write_map 39b,
113b · uses emit_name_json 100, first 182a, fputs 21b, Name 144b

```

2.6.2 Fragment Extraction

The `write_extract` function finds a fragment by name (using prefix matching, like `@<name...@>` in the source), then emits the scrap code and dependency information as Markdown. It recurses into referenced fragments to provide the full transitive closure of the requested fragment.

To find a name in the BST without creating a new entry, we use a read-only search function:

⟨output.c 102a⟩ + ≡

```

static Name *find_name(Name *node, const char *spelling)
{
    Name *found;
    if (!node) return NULL;
    if (robs_strcmp(node->spelling, (char *)spelling) == 0)
        return node;
    found = find_name(node->llink, spelling);
    if (found) return found;
    return find_name(node->rlink, spelling);
}

```

102a · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines find_name 104ab · uses Name 144b, robs_strcmp 147c

A recursive helper emits one fragment and all its transitive dependencies. A visited array (indexed by Name pointer cast to a simple mark) prevents infinite recursion on cyclic references.

<output.c 102b> + ≡

```

static void extract_fragment(FILE *f, Name *name, Name *files, Name *macros)
{
    Scrap_Node *sn;

    if (!name || name->mark) return;
    name->mark = TRUE;

    for (sn = name->defs; sn; sn = sn->next) {
        fprintf(f, "\n## \"s\" --- s:%d\n\n",
            name->spelling,
            scrap_file_name(sn->scrap),
            scrap_file_line(sn->scrap));
        if (name->lang[0])
            fprintf(f, "'s\n", name->lang);
        else
            fputs("'\n", f);
        dump_scrap_text(f, sn->scrap);
        fputs("\n'\n", f);

        /* Show dependencies */
        {
            Uses *u = get_scrap_uses(sn->scrap);
            if (u) {
                fputs("\n### Uses\n", f);
                for (; u; u = u->next)
                    fprintf(f, "- s\n", u->defn->spelling);
            }
        }

        /* Show used_by */
        if (name->uses) {
            Scrap_Node *usn;
            fputs("\n### Used by\n", f);
            for (usn = name->uses; usn; usn = usn->next) {
                fprintf(f, "- scrap %d (s:%d)\n",
                    usn->scrap,
                    scrap_file_name(usn->scrap),
                    scrap_file_line(usn->scrap));
            }
        }
    }
}

```

```

/* Recurse into dependencies */
for (sn = name->defs; sn; sn = sn->next) {
    Uses *u;
    for (u = get_scrap_uses(sn->scrap); u; u = u->next) {
        extract_fragment(f, u->defn, files, macros);
    }
}
}

```

102b · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `extract_fragment` 104b · uses `defs` 121c, `dump_scrap_text` 130c, `FILE` 21b, `fprintf` 21b, `fputs` 21b,
`get_scrap_uses` 123d, `Name` 144b, `next` 121b, `scrap_file_line` 124c, `scrap_file_name` 124b, `Scrap_Node` 144a, `TRUE` 23a,
`Uses` 176d, `uses` 121c

Clear marks after extraction so the name trees remain usable:

```

<output.c 103a> + ≡
static void clear_marks(Name *node)
{
    if (!node) return;
    clear_marks(node->llink);
    node->mark = FALSE;
    clear_marks(node->rlink);
}

```

103a · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `clear_marks` 104ab · uses `FALSE` 23a, `Name` 144b

2.6.3 Callers (Upward Call Chain)

The `--callers` feature, used with `-e`, traces the upward call chain from a given fragment back to its @o file roots. Each scrap records its owning `Name` (set in `pass1` when the scrap is attached to a definition list), so we can walk upward through the `uses` list to find all callers.

```

<output.c 103b> + ≡
static int is_file_name(Name *root, Name *target)
{
    if (!root) return 0;
    if (root == target) return 1;
    if (is_file_name(root->llink, target)) return 1;
    return is_file_name(root->rlink, target);
}

```

103b · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `is_file_name` 103c · uses `Name` 144b, `root` 169c

```

<output.c 103c> + ≡
static void print_callers(Name *name, Name *file_names, int depth)
{
    Scrap_Node *usn;
    int i;

    if (!name || name->mark) return;
    name->mark = TRUE;

    for (i = 0; i < depth; i++) fputs(" ", stdout);
    if (is_file_name(file_names, name))
        printf("[file] %s\n", name->spelling);
}

```

```

else
    printf("%s\n", name->spelling);

for (usn = name->uses; usn; usn = usn->next) {
    Name *owner = scrap_owner(usn->scrap);
    if (owner && !owner->mark)
        print_callers(owner, file_names, depth + 1);
}
}
103c · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines print_callers 104a · uses file_names 144c, fputs 21b, is_file_name 103b, Name 144b, next 121b, owner 121c,
Scrap_Node 144a, scrap_owner 124f, TRUE 23a, uses 121c

```

```

⟨output.c 104a⟩ + ≡
void write_callers(char *name, Name *files, Name *macros)
{
    Name *found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);
    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }
    printf("Callers of \"%s\":\n\n", found->spelling);
    printf("%s\n", found->spelling);
    {
        Scrap_Node *usn;
        for (usn = found->uses; usn; usn = usn->next) {
            Name *owner = scrap_owner(usn->scrap);
            if (owner)
                print_callers(owner, files, 1);
        }
    }
    clear_marks(files);
    clear_marks(macros);
}

```

104a · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines write_callers 39b, 113b · uses clear_marks 103a, command_name 28c, exit 21b, find_name 102a, fprintf 21b,
Name 144b, next 121b, owner 121c, print_callers 103c, Scrap_Node 144a, scrap_owner 124f, stderr 21b, uses 121c

```

⟨output.c 104b⟩ + ≡
void write_extract(char *name, Name *files, Name *macros)
{
    Name *found = NULL;

    /* Search in macro_names first, then file_names */
    found = find_name(macros, name);
    if (!found)
        found = find_name(files, name);

    if (!found) {
        fprintf(stderr, "%s: fragment \"%s\" not found\n", command_name, name);
        exit(1);
    }

    fprintf(stdout, "# Extract: \"%s\"\n", found->spelling);
    extract_fragment(stdout, found, files, macros);
}

```

```

    /* Clean up marks */
    clear_marks(files);
    clear_marks(macros);
}

```

104b · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
 defines `write_extract` 39b, 113b · uses `clear_marks` 103a, `command_name` 28c, `exit` 21b, `extract_fragment` 102b,
`file_names` 144c, `find_name` 102a, `first` 182a, `fprintf` 21b, `macro_names` 144c, `Name` 144b, `stderr` 21b

2.6.4 Lint (Static Analysis)

The `--lint` flag performs a post-pass1 static analysis of the fragment namespace. It reports three kinds of issues:

1. **Undefined references:** fragments that are used (`@<Name@>`) but never defined (`@d Name`).
2. **Unused fragments:** fragments defined but never referenced by any other scrap or output file.
3. **Fuzzy-match suggestions:** for undefined references, the closest defined name within Levenshtein distance 3.

```

<output.c 105a> + ≡
static int levenshtein(const char *a, const char *b)
{
    int la = (int)strlen(a), lb = (int)strlen(b);
    int prev[256], curr[256];
    int i, j;
    if (la > 255 || lb > 255) return 999;
    for (j = 0; j <= lb; j++) prev[j] = j;
    for (i = 1; i <= la; i++) {
        curr[0] = i;
        for (j = 1; j <= lb; j++) {
            int cost = (a[i-1] != b[j-1]) ? 1 : 0;
            int del = prev[j] + 1;
            int ins = curr[j-1] + 1;
            int sub = prev[j-1] + cost;
            curr[j] = del < ins ? (del < sub ? del : sub) : (ins < sub ? ins : sub);
        }
        for (j = 0; j <= lb; j++) prev[j] = curr[j];
    }
    return prev[lb];
}

```

105a · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
 defines `levenshtein` 105b · uses `strlen` 21b

```

<output.c 105b> + ≡

#define MAX_LINT_NAMES 4096
static Name *lint_names[MAX_LINT_NAMES];
static int lint_name_count;

static void collect_lint_names(Name *node)
{
    if (!node) return;
    collect_lint_names(node->llink);
    if (node->defs && lint_name_count < MAX_LINT_NAMES)
        lint_names[lint_name_count++] = node;
    collect_lint_names(node->rlink);
}

```

```
}
```

```
static const char *find_closest(const char *target)
{
    int i, best_dist = 4;
    const char *best = NULL;
    for (i = 0; i < lint_name_count; i++) {
        int d = levenshtein(target, lint_names[i]->spelling);
        if (d < best_dist) {
            best_dist = d;
            best = lint_names[i]->spelling;
        }
    }
    return best;
}
```

105b · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·

defines collect_lint_names 107a, find_closest 106, lint_names never used, lint_name_count 107a, MAX_LINT_NAMES never used · uses defs 121c, levenshtein 105a, Name 144b

⟨output.c 106⟩ + ≡

```
static int lint_name(Name *node, int check_unused, int *issues)
{
    if (!node) return 0;
    lint_name(node->llink, check_unused, issues);

    if (node->defs == NULL && node->uses != NULL) {
        /* undefined reference */
        const char *suggestion = find_closest(node->spelling);
        fprintf(stderr, "lint: undefined fragment \"%s\"", node->spelling);
        if (node->uses) {
            fprintf(stderr, " (used at %s:%d)",
                    scrap_file_name(node->uses->scrap),
                    scrap_file_line(node->uses->scrap));
        }
        if (suggestion)
            fprintf(stderr, " -- did you mean \"%s\"?", suggestion);
        fputc('\n', stderr);
        (*issues)++;
    }
    else if (node->defs != NULL && node->uses == NULL && check_unused) {
        fprintf(stderr, "lint: unused fragment \"%s\" (defined at %s:%d)\n",
                node->spelling,
                scrap_file_name(node->defs->scrap),
                scrap_file_line(node->defs->scrap));
        (*issues)++;
    }

    lint_name(node->rlink, check_unused, issues);
    return 0;
}
```

106 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c · defines lint_name 107a · uses defs 121c, find_closest 105b, fprintf 21b, Name 144b, scrap_file_line 124c, scrap_file_name 124b, stderr 21b, uses 121c

⟨output.c 107a⟩ + ≡

```
void write_lint(Name *file_names, Name *macro_names)
{
```

```

int issues = 0;
lint_name_count = 0;
collect_lint_names(file_names);
collect_lint_names(macro_names);

/* check files for undefined refs (not unused -- files are roots) */
lint_name(file_names, 0, &issues);
/* check macros for both undefined and unused */
lint_name(macro_names, 1, &issues);

if (issues)
    fprintf(stderr, "lint: %d issue(s) found\n", issues);
else
    fprintf(stderr, "lint: no issues found\n");
}

```

107a · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `write_lint` 39b, 113b · uses `collect_lint_names` 105b, `file_names` 144c, `fprintf` 21b, `lint_name` 106,
`lint_name_count` 105b, `macro_names` 144c, `Name` 144b, `stderr` 21b

2.6.5 Reverse Map Output

The reverse map feature provides programmatic navigation from tangled output back to the `.weft` source. Given a tangled output file (and optionally a line number), it parses the section markers embedded in that file and emits JSON mapping each region to its originating scrap, source file, and source line.

This operates entirely on the tangled output—no `.weft` files are needed and `pass1` is never invoked. The section markers have the form:

```

COMMENT_BEGIN{N: file:line}COMMENT_END      (opening)
COMMENT_BEGIN{:N}COMMENT_END                (closing)

```

where `COMMENT_BEGIN` and `COMMENT_END` vary by language (`/* ... */`, `// ...`, `# ...`, etc.).

A `MarkerFrame` records the data extracted from one opening marker. We maintain a stack of these to handle nested fragment expansions.

```

<output.c 107b> + ≡
typedef struct {
    int scrap;
    char file[FILENAME_MAX];
    int line;
    int start_line; /* line in the tangled file where this marker opened */
} MarkerFrame;

```

107b · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `MarkerFrame` 107c, 109, 111, 112

The `parse_open_marker` function scans a line for the pattern `{N: file:line}`. It returns 1 on success, filling in the supplied `MarkerFrame`. The scan is intentionally loose about what precedes the opening brace so that any comment style works.

```

<output.c 107c> + ≡
static int parse_open_marker(const char *line, MarkerFrame *mf)
{
    const char *p = line;
    while (*p) {
        if (*p == '{') {
            const char *start = p + 1;
            /* scrap number */

```

```

if (*start < '0' || *start > '9') { p++; continue; }
int scrap = 0;
while (*start >= '0' && *start <= '9')
    scrap = scrap * 10 + (*start++ - '0');
if (*start != ':' || *(start+1) != ')') { p++; continue; }
start += 2; /* skip ": " */
/* file:line -- find the closing brace, then work backwards for :line */
const char *end = strchr(start, '}');
if (!end) { p++; continue; }
/* find last ':' between start and end */
const char *colon = NULL;
for (const char *q = end - 1; q >= start; q--) {
    if (*q == ':') { colon = q; break; }
}
if (!colon) { p++; continue; }
/* digits after colon? */
int src_line = 0;
const char *d = colon + 1;
if (d == end) { p++; continue; }
while (d < end) {
    if (*d < '0' || *d > '9') break;
    src_line = src_line * 10 + (*d++ - '0');
}
if (d != end) { p++; continue; }
/* success */
mf->scrap = scrap;
mf->line = src_line;
{
    size_t len = (size_t)(colon - start);
    if (len >= FILENAME_MAX) len = FILENAME_MAX - 1;
    memcpy(mf->file, start, len);
    mf->file[len] = '\0';
}
return 1;
}
p++;
}
return 0;
}

```

107c · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `parse_open_marker` 109, 111 · uses `MarkerFrame` 107b, `size_t` 21b

The `parse_close_marker` function scans for `{:N}` and returns the scrap number, or `-1` on failure.

⟨output.c 108⟩ + ≡

```

static int parse_close_marker(const char *line)
{
    const char *p = line;
    while (*p) {
        if (*p == '{' && *(p+1) == ':') {
            const char *start = p + 2;
            if (*start < '0' || *start > '9') { p++; continue; }
            int scrap = 0;
            while (*start >= '0' && *start <= '9')
                scrap = scrap * 10 + (*start++ - '0');
            if (*start == ',')
                return scrap;
        }
    }
}

```

```

    p++;
}
return -1;
}

```

108 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `parse_close_marker` 109, 111

The main function `write_reverse_map` parses its argument to split an optional line number from the filename, opens the tangled file, and reads it line by line. A stack of `MarkerFrame` entries tracks nested scrap regions.

In *line mode* (a specific line was requested), as soon as `current_line` reaches the target the innermost stack frame is emitted and the function returns.

In *full mode* (no line number), every time a close marker is encountered the region from the matching open to the current line is emitted.

```

<output.c 109> + ≡
void write_reverse_map(const char *arg)
{
    char filename[FILENAME_MAX];
    int target_line = -1;
    FILE *fp;
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0;
    int current_line = 0;
    int first_region = 1;

    /* Parse arg: find last ':' where suffix is all digits */
    {
        const char *colon = NULL;
        const char *p;
        for (p = arg + strlen(arg) - 1; p > arg; p--) {
            if (*p == ':') { colon = p; break; }
        }
        if (colon) {
            const char *d = colon + 1;
            int all_digits = (*d != '\0');
            while (*d) {
                if (*d < '0' || *d > '9') { all_digits = 0; break; }
                d++;
            }
            if (all_digits) {
                target_line = atoi(colon + 1);
                snprintf(filename, FILENAME_MAX, "%.s", (int)(colon - arg), arg);
            } else {
                snprintf(filename, FILENAME_MAX, "%s", arg);
            }
        } else {
            snprintf(filename, FILENAME_MAX, "%s", arg);
        }
    }

    fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "weft: cannot open %s: ", filename);
        perror(NULL);
        exit(1);
    }
}

```

```

if (target_line < 0) {
    /* Full mode header */
    fputs("{\"source\": ", stdout);
    json_puts(stdout, filename);
    fputs(", \"regions\": [\n", stdout);
}

while (fgets(buf, sizeof(buf), fp)) {
    MarkerFrame mf;
    int close_scrap;
    current_line++;

    if (parse_open_marker(buf, &mf)) {
        mf.start_line = current_line;
        if (depth < 256)
            stack[depth++] = mf;
        continue;
    }

    close_scrap = parse_close_marker(buf);
    if (close_scrap >= 0) {
        /* Find matching frame on stack */
        int i;
        for (i = depth - 1; i >= 0; i--) {
            if (stack[i].scrap == close_scrap) {
                if (target_line < 0) {
                    /* Full mode: emit region */
                    if (!first_region) fputs(",\n", stdout);
                    first_region = 0;
                    fprintf(stdout,
                        "  {\"start\": %d, \"end\": %d, \"scrap\": %d, \"file\": ",
                        stack[i].start_line + 1, current_line - 1, stack[i].scrap);
                    json_puts(stdout, stack[i].file);
                    fprintf(stdout, ", \"line\": %d}\n", stack[i].line);
                }
                /* Remove frame by shifting */
                depth--;
                for (; i < depth; i++)
                    stack[i] = stack[i + 1];
                break;
            }
        }
        continue;
    }

    /* Regular line -- check for line mode match */
    if (target_line > 0 && current_line == target_line) {
        if (depth > 0) {
            MarkerFrame *inner = &stack[depth - 1];
            fprintf(stdout, "{ \"scrap\": %d, \"file\": ", inner->scrap);
            json_puts(stdout, inner->file);
            fprintf(stdout, ", \"line\": %d}\n", inner->line);
        } else {
            fputs("{}\n", stdout);
        }
        fclose(fp);
        return;
    }
}

```

```

}

fclose(fp);

if (target_line > 0) {
    /* Line was beyond end of file or not inside a scrap */
    fputs("{}\n", stdout);
} else {
    fputs("\n[]\n", stdout);
}
}

```

109 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines `write_reverse_map` 36b, 113b · uses `exit` 21b, `fclose` 21b, `FILE` 21b, `fopen` 21b, `fprintf` 21b, `fputs` 21b,
`json_puts` 99, `MarkerFrame` 107b, `parse_close_marker` 108, `parse_open_marker` 107c, `stack` 117e, `stderr` 21b, `strlen` 21b

2.6.6 Error Annotator

The `--errors` flag reads compiler output from stdin and annotates lines matching `file:line:` patterns with the corresponding `.weft` source location. It works by opening each referenced file, parsing its section markers, and finding which scrap region contains the error line. Lines that don't match the pattern are passed through unchanged.

```

<output.c 111> + ≡
static int reverse_lookup_in_file(const char *filename, int target_line,
                                MarkerFrame *result)
{
    FILE *fp = fopen(filename, "r");
    char buf[8192];
    MarkerFrame stack[256];
    int depth = 0, current_line = 0;

    if (!fp) return 0;

    while (fgets(buf, sizeof(buf), fp)) {
        MarkerFrame mf;
        int close_scrap;
        current_line++;

        if (parse_open_marker(buf, &mf)) {
            mf.start_line = current_line;
            if (depth < 256)
                stack[depth++] = mf;
            continue;
        }

        close_scrap = parse_close_marker(buf);
        if (close_scrap >= 0) {
            int i;
            for (i = depth - 1; i >= 0; i--) {
                if (stack[i].scrap == close_scrap) {
                    depth--;
                    for (; i < depth; i++)
                        stack[i] = stack[i + 1];
                    break;
                }
            }
            continue;
        }
    }
}

```

```

        if (current_line == target_line && depth > 0) {
            *result = stack[depth - 1];
            fclose(fp);
            return 1;
        }
    }
    fclose(fp);
    return 0;
}

111 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·
defines reverse_lookup_in_file 112 · uses fclose 21b, FILE 21b, fopen 21b, MarkerFrame 107b, parse_close_marker 108,
parse_open_marker 107c, stack 117e

⟨output.c 112⟩ + ≡
void write_error_annotator(void)
{
    char line[8192];

    while (fgets(line, sizeof(line), stdin)) {
        const char *p = line;
        char err_file[FILENAME_MAX];
        int err_line = -1;
        int parsed = 0;

        /* Try to parse file:digits: pattern */
        /* Skip optional drive letter on Windows (e.g. C:) */
        if (p[0] && p[1] == ':' && ((p[0] >= 'A' && p[0] <= 'Z') ||
            (p[0] >= 'a' && p[0] <= 'z'))) {
            p += 2;
        }
        {
            const char *colon1 = strchr(p, ':');
            if (colon1 && colon1 > p) {
                const char *d = colon1 + 1;
                int num = 0;
                int has_digits = 0;
                while (*d >= '0' && *d <= '9') {
                    num = num * 10 + (*d - '0');
                    has_digits = 1;
                    d++;
                }
                if (has_digits && *d == ':') {
                    size_t flen = (size_t)(colon1 - line);
                    if (flen < FILENAME_MAX) {
                        memcpy(err_file, line, flen);
                        err_file[flen] = '\0';
                        err_line = num;
                        parsed = 1;
                    }
                }
            }
        }

        fputs(line, stdout);

        if (parsed) {
            MarkerFrame mf;

```

```

    if (reverse_lookup_in_file(err_file, err_line, &mf)) {
        int offset = err_line - mf.start_line;
        printf("  [weft: %s:%d, scrap %d",
            mf.file, mf.line + offset, mf.scrap);
        printf("]\n");
    }
}
}
}
}

```

112 · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·

defines [write_error_annotator](#) 36b, 113b · uses [fputs](#) 21b, [letter](#) 121c, [MarkerFrame](#) 107b, [reverse_lookup_in_file](#) 111, [size_t](#) 21b

2.7 Writing the Output Files

⟨ *Function prototypes* 113a ⟩ + ≡

```
extern void write_files(Name *files);
```

113a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses [Name](#) 144b, [write_files](#) 113c

⟨ *Function prototypes* 113b ⟩ + ≡

```

extern void write_map(Name *file_names, Name *macro_names);
extern void write_extract(char *name, Name *file_names, Name *macro_names);
extern void write_reverse_map(const char *arg);
extern void write_callers(char *name, Name *file_names, Name *macro_names);
extern void write_lint(Name *file_names, Name *macro_names);
extern void write_error_annotator(void);

```

113b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses [file_names](#) 144c, [macro_names](#) 144c, [Name](#) 144b, [write_callers](#) 104a, [write_error_annotator](#) 112, [write_extract](#) 104b, [write_lint](#) 107a, [write_map](#) 101, [write_reverse_map](#) 109

⟨ *output.c* 113c ⟩ + ≡

```

void write_files(Name *files)
{
    while (files) {
        write_files(files->llink);
        ⟨ Write out files->spelling 113e ⟩
        files = files->rlink;
    }
}

```

113c · defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c ·

defines [write_files](#) 39b, 113a · uses [Name](#) 144b

`MAX_INDENT` defines the maximum number of characters tracked per line for indentation purposes. When a fragment is expanded mid-line, the characters preceding it are recorded so that subsequent lines of the fragment can reproduce the same indentation. Languages like HTML, XML, and SVG routinely contain lines longer than 500 characters, so we set this to 8192. If a line exceeds this limit, indentation beyond the recorded portion falls back to spaces instead of causing a fatal error.

⟨ *Type declarations* 113d ⟩ + ≡

```
#define MAX_INDENT 8192
```

113d · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines MAX_INDENT 84a, 113e, 136a, 137c

```
< Write out files->spelling 113e > ≡
{
    static char temp_name[FILENAME_MAX];
    static char real_name[FILENAME_MAX];
    static int temp_name_count = 0;
    char indent_chars[MAX_INDENT];
    FILE *temp_file;

    < Find a free temporary file 114a >

    sprintf(real_name, "%s%s%s", dirpath, path_sep, files->spelling);
    if (verbose_flag)
        fprintf(stderr, "writing %s [%s]\n", files->spelling, temp_name);
    write_scraps(temp_file, files->spelling, files->defs, 0, indent_chars,
        files->debug_flag, files->tab_flag, files->indent_flag,
        files->comment_flag, files->location_flag,
        NULL, NULL, 0, files->spelling);
    fclose(temp_file);

    < Move the temporary file to the target, if required 114b >
}
```

113e · used in 113c · uses defs 121c, fclose 21b, FILE 21b, fprintf 21b, MAX_INDENT 113d, stderr 21b, verbose_flag 27a, write_scraps 134b

```
< Find a free temporary file 114a > ≡
for( temp_name_count = 0; temp_name_count < 10000; temp_name_count++) {
    sprintf(temp_name, "%s%snw%06d", dirpath, path_sep, temp_name_count);
#ifdef O_EXCL
    if (-1 != (temp_file_fd = open(temp_name, O_CREAT|O_WRONLY|O_EXCL))) {
        temp_file = fdopen(temp_file_fd, "w");
        break;
    }
#else
    if (0 != (temp_file = fopen(temp_name, "a"))) {
        if ( 0L == ftell(temp_file)) {
            break;
        } else {
            fclose(temp_file);
            temp_file = 0;
        }
    }
#endif
}
if (!temp_file) {
    fprintf(stderr, "%s: can't create %s for a temporary file\n",
        command_name, temp_name);
    exit(-1);
}
```

114a · used in 113e · uses command_name 28c, exit 21b, fclose 21b, fopen 21b, fprintf 21b, stderr 21b

Note the call to **remove** before **rename** – the ANSI/ISO C standard does *not* guarantee that renaming a file to an existing filename will overwrite the file.

```
< Move the temporary file to the target, if required 114b > ≡
if (diff_flag) {
```

```

    < Show diff of temp file vs real file 115a >
} else if (compare_flag)
    < Compare the temp file and the old file 115b >
else {
    remove(real_name);
    < Rename the temporary file to the target 116 >
}

114b · used in 113e · uses compare_flag 27a, diff_flag 27a, remove 21b

```

```

< Show diff of temp file vs real file 115a > ≡
{
    FILE *old_file = fopen(real_name, "r");
    if (!old_file) {
        /* New file: show all lines with + prefix */
        FILE *tf = fopen(temp_name, "r");
        if (tf) {
            char dbuf[8192];
            printf("new file: %s\n", real_name);
            while (fgets(dbuf, sizeof(dbuf), tf))
                printf("+%s", dbuf);
            fclose(tf);
        }
    } else {
        char cmd[FILENAME_MAX * 2 + 32];
        FILE *dp;
        fclose(old_file);
        sprintf(cmd, "diff -u \"%s\" \"%s\"", real_name, temp_name);
        dp = popen(cmd, "r");
        if (dp) {
            char dbuf[8192];
            while (fgets(dbuf, sizeof(dbuf), dp))
                fputs(dbuf, stdout);
            pclose(dp);
        }
    }
    remove(temp_name);
}

```

115a · used in 114b · uses fclose 21b, FILE 21b, fopen 21b, fputs 21b, remove 21b

Again, we use a call to `remove` before `rename`.

```

< Compare the temp file and the old file 115b > ≡
{
    FILE *old_file = fopen(real_name, "r");
    if (old_file) {
        int x, y;
        temp_file = fopen(temp_name, "r");
        do {
            x = getc(old_file);
            y = getc(temp_file);
        } while (x == y && x != EOF);
        fclose(old_file);
        fclose(temp_file);
        if (x == y)
            remove(temp_name);
        else {
            remove(real_name);
            < Rename the temporary file to the target 116 >
        }
    }
}

```

```

    }
}
else
    ⟨ Rename the temporary file to the target 116 ⟩
}

115b · used in 114b · uses fclose 21b, FILE 21b, fopen 21b, getc 21b, remove 21b

⟨ Rename the temporary file to the target 116 ⟩ ≡
if (0 != rename(temp_name, real_name)) {
    fprintf(stderr, "%s: can't rename output file to %s\n",
               command_name, real_name);
}

116 · used in 114b, 115b · uses command_name 28c, fprintf 21b, stderr 21b

```

Chapter 3

The Support Routines

3.1 Source Files

3.1.1 Global Declarations

We need two routines to handle reading the source files.

⟨ Function prototypes 117a ⟩ + ≡

```
extern void source_open(char *name); /* pass in the name of the source file */
extern int source_get(void); /* no args; returns the next char or EOF */
extern int source_last; /* what last source_get() returned. */
extern int source_peek; /* The next character to get */
```

117a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses next 121b, source_get 118a, source_last 118a, source_open 121a, source_peek 118a

There are also two global variables maintained for use in error messages and such.

⟨ Global variable declarations 117b ⟩ + ≡

```
extern char *source_name; /* name of the current file */
extern int source_line; /* current line in the source file */
```

117b · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines source_line 43d, 45d, 46a, 117c, 118ac, 119ab, 120ab, 121a, 126ac, 128a, 146c, 156, 158, 159ab, 160a, 162a, 163d, 165, 166, source_name 38ab, 39ab, 43d, 45d, 46a, 117c, 119ab, 120ab, 121a, 126ac, 128a, 146c, 156, 158, 159ab, 160a, 162ab, 163d, 164a, 165, 166

⟨ Global variable definitions 117c ⟩ + ≡

```
char *source_name = NULL;
int source_line = 0;
```

117c · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses source_line 117b, source_name 117b

3.1.2 Local Declarations

⟨ input.c 117d ⟩ + ≡

```
static FILE *source_file; /* the current input file */
static int double_at;
static int include_depth;
```

117d · defined at 25a, 117de, 118ac, 121a · defines double_at 119a, 121a, include_depth 119b, 120b, 121a, source_file 118ac, 119ab, 120ab, 121a · uses FILE 21b

⟨ input.c 117e ⟩ + ≡

```
static struct {
    FILE *file;
    char *name;
    int line;
} stack[10];
```

117e · defined at 25a, 117de, 118ac, 121a · defines `stack` 109, 111, 119b, 120b · uses `FILE` 21b

3.1.3 Reading a File

The routine `source_get` returns the next character from the current source file. It notices newlines and keeps the line counter `source_line` up to date. It also catches EOF and watches for @ characters. All other characters are immediately returned. We define `source_last` to let us tell which type of scrap we are defining.

⟨ `input.c` 118a ⟩ + ≡

```
int source_peek;
int source_last;
int source_get(void)
{
    int c;
    source_last = c = source_peek;
    switch (c) {
        case EOF:  ⟨ Handle EOF 120b ⟩
                    return c;
        case '\n': source_line++;
                    FALLTHROUGH;
        default:
            if (c==nw_char)
            {
                ⟨ Handle an "at" character 119a ⟩
                return c;
            }
            source_peek = getc(source_file);
            return c;
    }
}
```

118a · defined at 25a, 117de, 118ac, 121a · defines `source_get` 41d, 42c, 43cd, 44ab, 45d, 46ac, 47ab, 48ab, 51, 52b, 57bc, 58bc, 64ad, 69, 70e, 72bcde, 75a, 76c, 79c, 82abc, 83b, 84a, 86bc, 88a, 89b, 90b, 93ae, 95c, 117a, 119b, 120b, 126bc, 127b, 128a, 129b, 156, 158, 159ab, 160a, 162ab, 163a, 164a, 165, 166, 167a, `source_last` 52b, 62e, 69, 88a, 117a, `source_peek` 46c, 47b, 48a, 117a, 118c, 119ab, 120b, 121a · uses `FALLTHROUGH` 23a, `getc` 21b, `nw_char` 28ab, `source_file` 117d, `source_line` 117b

`source_ungetc` pushes a read character back to the `source_file`.

⟨ *Function prototypes* 118b ⟩ + ≡

```
extern void source_ungetc(int*);
```

118b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `source_ungetc` 118c

⟨ `input.c` 118c ⟩ + ≡

```
void source_ungetc(int *c)
{
    ungetc(source_peek, source_file);
    if(*c == '\n')
        source_line--;
    source_peek=*c;
}
```

```
}
```

118c · defined at 25a, 117de, 118ac, 121a · defines `source_ungetc` 47ab, 48b, 76c, 93e, 118b · uses `source_file` 117d, `source_line` 117b, `source_peek` 118a

This whole @ character handling mess is pretty annoying. I want to recognize @i so I can handle include files correctly. At the same time, it makes sense to recognize illegal @ sequences and complain; this avoids ever having to check anywhere else. Unfortunately, I need to avoid tripping over the @@ sequence; hence this whole unsatisfactory double_at business.

⟨ *Handle an “at” character* 119a ⟩ ≡

```
{
  c = getc(source_file);
  if (double_at) {
    source_peek = c;
    double_at = FALSE;
    c = nw_char;
  }
  else
    switch (c) {
      case 'i': ⟨ Open an include file 119b ⟩
        break;
      case '#': case 'f': case 'm': case 'u': case 'v':
      case 'd': case 'o': case 'D': case 'O': case 's':
      case 'q': case 'Q': case 'S': case 'l': case 'L': case 't':
      case 'W':
      case '+':
      case '-':
      case '*':
      case '\\':
      case '{': case '}': case '<': case '>': case '|':
      case '(': case ')': case '[': case ']':
      case '%': case '_':
      case ':': case ',': case 'x': case 'c':
      case '1': case '2': case '3': case '4': case '5':
      case '6': case '7': case '8': case '9':
      case 'r':
        source_peek = c;
        c = nw_char;
        break;
      default:
        if (c==nw_char)
        {
          source_peek = c;
          double_at = TRUE;
          break;
        }
        fprintf(stderr, "%s: bad %c sequence %c[%d] (%s, line %d)\n",
                  command_name, nw_char, c, c, source_name, source_line);
        exit(-1);
    }
}
```

119a · used in 118a · uses `command_name` 28c, `double_at` 117d, `exit` 21b, `FALSE` 23a, `fprintf` 21b, `getc` 21b, `nw_char` 28ab, `source_file` 117d, `source_line` 117b, `source_name` 117b, `source_peek` 118a, `stderr` 21b, `TRUE` 23a

⟨ *Open an include file* 119b ⟩ ≡

```
{
  char name[FILENAME_MAX];
```

```

char fullname[FILENAME_MAX];
struct incl * p = include_list;

if (include_depth >= 10) {
    fprintf(stderr, "%s: include nesting too deep (%s, %d)\n",
        command_name, source_name, source_line);
    exit(-1);
}
< Collect include-file name 120a >
stack[include_depth].file = source_file;
fullname[0] = '\0';
for (;;) {
    strcat(fullname, name);
    source_file = fopen(fullname, "r");
    if (source_file || !p)
        break;
    strcpy(fullname, p->name);
    strcat(fullname, "/");
    p = p->next;
}
if (!source_file) {
    fprintf(stderr, "%s: can't open include file %s\n",
        command_name, name);
    source_file = stack[include_depth].file;
}
else
{
    stack[include_depth].name = source_name;
    stack[include_depth].line = source_line + 1;
    include_depth++;
    source_line = 1;
    source_name = save_string(fullname);
}
source_peek = getc(source_file);
c = source_get();
}

119b · used in 119a · uses command_name 28c, exit 21b, fopen 21b, fprintf 21b, getc 21b, incl 27a, include_depth 117d,
next 121b, save_string 146a, source_file 117d, source_get 118a, source_line 117b, source_name 117b, source_peek 118a,
stack 117e, stderr 21b

```

```

< Collect include-file name 120a > ≡
{
    char *p = name;
    do
        c = getc(source_file);
    while (c == ' ' || c == '\t');
    while (isgraph(c)) {
        *p++ = c;
        c = getc(source_file);
    }
    *p = '\0';
    if (c != '\n') {
        fprintf(stderr, "%s: unexpected characters after file name (%s, %d)\n",
            command_name, source_name, source_line);
        exit(-1);
    }
}

```

120a · used in 119b · uses command_name 28c, exit 21b, fprintf 21b, getc 21b, isgraph 21b, source_file 117d,

source_line 117b, source_name 117b, stderr 21b

If an EOF is discovered, the current file must be closed and input from the next stacked file must be resumed. If no more files are on the stack, the EOF is returned.

⟨ *Handle EOF* 120b ⟩ ≡

```
{
    fclose(source_file);
    if (include_depth) {
        include_depth--;
        source_file = stack[include_depth].file;
        source_line = stack[include_depth].line;
        source_name = stack[include_depth].name;
        source_peek = getc(source_file);
        c = source_get();
    }
}
```

120b · used in 118a · uses `fclose` 21b, `getc` 21b, `include_depth` 117d, `source_file` 117d, `source_get` 118a, `source_line` 117b, `source_name` 117b, `source_peek` 118a, `stack` 117e

3.1.4 Opening a File

The routine `source_open` takes a file name and tries to open the file. If unsuccessful, it complains and halts. Otherwise, it sets `source_name`, `source_line`, and `double_at`.

⟨ *input.c* 121a ⟩ + ≡

```
void source_open(char *name)
{
    source_file = fopen(name, "r");
    if (!source_file) {
        fprintf(stderr, "%s: couldn't open %s\n", command_name, name);
        exit(-1);
    }
    nw_char = '@';
    source_name = name;
    source_line = 1;
    source_peek = getc(source_file);
    double_at = FALSE;
    include_depth = 0;
}
```

121a · defined at 25a, 117de, 118ac, 121a · defines `source_open` 41c, 54, 81c, 117a · uses `command_name` 28c, `double_at` 117d, `exit` 21b, `FALSE` 23a, `fopen` 21b, `fprintf` 21b, `getc` 21b, `include_depth` 117d, `nw_char` 28ab, `source_file` 117d, `source_line` 117b, `source_name` 117b, `source_peek` 118a, `stderr` 21b

3.2 Scraps

⟨ *scraps.c* 121b ⟩ + ≡

```
#define SLAB_SIZE 1024

typedef struct slab {
    struct slab *next;
    char chars[SLAB_SIZE];
} Slab;
```

121b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines next 27a, 30, 33, 35b, 39a, 45c, 50c, 58c, 64g, 65abc, 72g, 73, 74b, 76b, 78bc, 79e, 86e, 87bcd, 90d, 91b, 93d, 95ab, 96b, 97bc, 98cd, 100, 102b, 103c, 104a, 117a, 119b, 125d, 128b, 129ad, 130b, 132de, 133bc, 134b, 141ad, 144a, 164de, 167ce, 168acd, 169abd, 171a, 173ab, 174, 176bcd, 177ab, 178bc, 181e, 182cd, 184, Slab 121c, 125cd, 126a, 130b, SLAB_SIZE 125d, 130b, 132c, 172a · uses slab 121c

⟨ scraps.c 121c ⟩ + ≡

```
typedef struct {
    char *file_name;
    Slab *slab;
    struct uses *uses;
    struct uses *defs;
    int file_line;
    int end_line;
    int page;
    char letter;
    unsigned char sector;
    char *prose;
    struct name *owner;
} ScrapEntry;
```

121c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines defs 45c, 50c, 58c, 60ab, 64g, 65a, 72g, 73, 74b, 76b, 78b, 79e, 84a, 86e, 87b, 90d, 91b, 93d, 95a, 96b, 97c, 98ad, 100, 102b, 105b, 106, 113e, 122b, 123e, 126a, 129a, 133b, 134b, 135, 136bc, 137a, 139cd, 140ac, 141ac, 144b, 149a, 167d, 168b, 174, 177bc, 178ac, end_line 100, 124d, 126ac, file_line 124c, 126ab, 135, file_name 41ac, 53a, 54, 81ac, 124b, 126ab, 136bc, letter 112, 123b, 126a, 143d, owner 103c, 104a, 124f, 126a, 132b, page 65c, 78b, 79e, 87d, 95a, 96b, 97c, 98ad, 122b, 123bc, 126a, 142bc, 143bd, 177bc, 178c, 184, prose 27a, 29d, 30, 62e, 100, 124e, 126a, 132a, 184, ScrapEntry 122a, 123a, 126a, sector 30, 53ab, 54, 76c, 77b, 78a, 79bcde, 81abc, 93e, 94bc, 95c, 96ab, 100, 126a, 128ab, 129ac, 130c, 139d, 141d, 144b, 145b, 146bc, 147a, 148b, 149a, 162a, 163d, 165, 174, slab 121b, 126a, 130c, 135, 174, uses 30, 55a, 65b, 78c, 79e, 87c, 95b, 96b, 98d, 100, 102b, 103c, 104a, 106, 123d, 126a, 128b, 129d, 144b, 149ab, 168b, 174, 176df, 178c · uses Slab 121b

There's some issue with reading the auxiliary file if it has more than 255 lines, so (as a workaround only) increase the size of the array of ScrapEntrys held in SCRAP[n].

⟨ scraps.c 122a ⟩ + ≡

```
#define SCRAP_BITS 10
#define SCRAP_SIZE (1<<SCRAP_BITS)
#define SCRAP_MASK (SCRAP_SIZE - 1)
#define SCRAP_SHIFT SCRAP_BITS
static ScrapEntry *SCRAP[SCRAP_SIZE];

#define scrap_array(i) SCRAP[(i) >> SCRAP_SHIFT][(i) & SCRAP_MASK]

static int scraps;
int num_scraps(void)
{
    return scraps;
}
⟨ Forward declarations for scraps.c 134c, ... ⟩
```

122a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines num_scraps 77b, 94b, 122b, SCRAP 123a, 126a, scraps 24ab, 27a, 30, 52b, 53b, 60ab, 62a, 64ae, 65c, 81b, 84bc, 86bd, 87ad, 100, 123a, 126a, 142bc, 143bc, 174, 184, scrap_array 123bde, 124bcdef, 126abc, 128b, 129a, 130c, 132ab, 135, 136bc, 142bc, 143bd, 174, 176f, 178a, SCRAP_BITS never used, SCRAP_MASK 126a, SCRAP_SHIFT 126a, SCRAP_SIZE 123a, 126a · uses ScrapEntry 121c

⟨ *Function prototypes* 122b ⟩ + ≡

```
extern void init_scraps(void);
extern int collect_scrap(void);
extern int write_scraps(FILE *file, char *spelling, Scrap_Node *defs, int global_indent,
    char *indent_chars, char debug_flag, char tab_flag, char indent_flag, unsigned char
    comment_flag, char location_flag, Arglist *inArgs, char *inParams[9], Parameters
    parameters, char *title);
extern void write_scrap_ref(FILE *file, int num, int first, int *page);
extern void write_single_scrap_ref(FILE *file, int num);
extern int num_scraps(void);
extern Uses * get_scrap_uses(int scrap);
extern Uses * get_scrap_defs(int scrap);
```

122b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses Arglist 164d, collect_scrap 125f, defs 121c, FILE 21b, first 182a, get_scrap_defs 123e, get_scrap_uses 123d, init_scraps 123a, num_scraps 122a, page 121c, Parameters 50b, Scrap_Node 144a, Uses 176d, write_scraps 134b, write_scrap_ref 123b, write_single_scrap_ref 123c

⟨ *scraps.c* 123a ⟩ + ≡

```
void init_scraps(void)
{
    scraps = 1;
    SCRAP[0] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(ScrapEntry));
}
```

123a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines init_scraps 41c, 122b · uses arena_getmem 182b, SCRAP 122a, ScrapEntry 121c, scraps 122a, SCRAP_SIZE 122a

⟨ *scraps.c* 123b ⟩ + ≡

```
void write_scrap_ref(FILE *file, int num, int first, int *page)
{
    if (scrap_array(num).page >= 0) {
        if (first!=0)
            fprintf(file, "%d", scrap_array(num).page);
        else if (scrap_array(num).page != *page)
            fprintf(file, ", %d", scrap_array(num).page);
        if (scrap_array(num).letter > 0)
            fputc(scrap_array(num).letter, file);
    }
    else {
        if (first!=0)
            putc('?', file);
        else
            fputs("?", file);
        ⟨ Warn (only once) about needing to rerun after Latex 124g ⟩
    }
    if (first>=0)
        *page = scrap_array(num).page;
}
```

123b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines write_scrap_ref 65c, 78b, 79e, 87d, 95a, 96b, 98a, 122b, 123c, 177c · uses FILE 21b, first 182a, fprintf 21b, fputs 21b, letter 121c, page 121c, putc 21b, scrap_array 122a

⟨ *scraps.c* 123c ⟩ + ≡

```
void write_single_scrap_ref(FILE *file, int num)
{
    int page;
```

```

    write_scrap_ref(file, num, TRUE, &page);
}

123c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d,
169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines write_single_scrap_ref 52b, 58c, 60ab, 64e, 65b, 73, 76b, 78c,
79e, 84bc, 87c, 90d, 91b, 93d, 95b, 96b, 122b, 140ac, 180a · uses FILE 21b, page 121c, TRUE 23a, write_scrap_ref 123b

⟨ scraps.c 123d ⟩ + ≡
Uses * get_scrap_uses(int scrap)
{
    return scrap_array(scrap).uses;
}

123d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d,
169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines get_scrap_uses 97a, 100, 102b, 122b · uses scrap_array 122a,
Uses 176d, uses 121c

⟨ scraps.c 123e ⟩ + ≡
Uses * get_scrap_defs(int scrap)
{
    return scrap_array(scrap).defs;
}

123e · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde,
170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines get_scrap_defs 98b, 122b · uses defs 121c, scrap_array 122a, Uses 176d

These accessors expose scrap metadata that is needed by the --map and --extract modes (Section 2.6)
without leaking the internal ScrapEntry or Slab types.

⟨ Function prototypes 124a ⟩ + ≡
extern const char *scrap_file_name(int i);
extern int scrap_file_line(int i);
extern int scrap_end_line(int i);
extern const char *scrap_prose(int i);
extern Name *scrap_owner(int i);
extern void dump_scrap_text(FILE *file, int scrap_idx);
extern void dump_scrap_json(FILE *f, int scrap_idx);
extern void set_scrap_prose(int i, char *p);
extern void set_scrap_owner(int i, Name *n);

124a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e,
177d, 179d, 181d · used in 21a · uses dump_scrap_json 131, dump_scrap_text 130c, FILE 21b, Name 144b,
scrap_end_line 124d, scrap_file_line 124c, scrap_file_name 124b, scrap_owner 124f, scrap_prose 124e,
set_scrap_owner 132b, set_scrap_prose 132a

⟨ scraps.c 124b ⟩ + ≡
const char *scrap_file_name(int i) { return scrap_array(i).file_name; }

124b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d,
169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines scrap_file_name 100, 102b, 106, 124a · uses file_name 121c,
scrap_array 122a

⟨ scraps.c 124c ⟩ + ≡
int scrap_file_line(int i) { return scrap_array(i).file_line; }

124c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d,
169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines scrap_file_line 100, 102b, 106, 124a · uses file_line 121c,
scrap_array 122a

```

`< scraps.c 124d > + ≡`
`int scrap_end_line(int i) { return scrap_array(i).end_line; }`
124d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `scrap_end_line` 100, 124a · uses `end_line` 121c, `scrap_array` 122a

`< scraps.c 124e > + ≡`
`const char *scrap_prose(int i) { return scrap_array(i).prose; }`
124e · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `scrap_prose` 100, 124a · uses `prose` 121c, `scrap_array` 122a

`< scraps.c 124f > + ≡`
`Name *scrap_owner(int i) { return scrap_array(i).owner; }`
124f · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `scrap_owner` 103c, 104a, 124a · uses `Name` 144b, `owner` 121c, `scrap_array` 122a

`< Warn (only once) about needing to rerun after Latex 124g > ≡`

```

{
    if (!already_warned) {
        fprintf(stderr, "%s: you'll need to rerun weft after running latex\n",
            command_name);
        already_warned = TRUE;
    }
}

```

124g · used in 123b, 142c · uses `already_warned` 125a, `command_name` 28c, `fprintf` 21b, `stderr` 21b, `TRUE` 23a

`< Global variable declarations 125a > + ≡`
`extern int already_warned;`
125a · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `already_warned` 124g, 125b, 142b

`< Global variable definitions 125b > + ≡`
`int already_warned = 0;`
125b · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses `already_warned` 125a

`< scraps.c 125c > + ≡`

```

typedef struct {
    Slab *scrap;
    Slab *prev;
    int index;
} Manager;

```

125c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `Manager` 125def, 130bc, 132c, 133d, 135, 170a, 171c, 174 · uses `Slab` 121b

`< scraps.c 125d > + ≡`

```

static void push(char c, Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    scrap->chars[index++] = c;
    if (index == SLAB_SIZE) {
        Slab *new_slab = (Slab *) arena_getmem(sizeof(Slab));
        scrap->next = new_slab;
        manager->scrap = new_slab;
    }
}

```

```

    index = 0;
}
manager->index = index;
}

```

125d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `push` 48f, 51, 125e, 126bc, 127a, 129bc · uses `arena_getmem` 182b, `Manager` 125c, `next` 121b, `Slab` 121b, `SLAB_SIZE` 121b

```

< scraps.c 125e > + ≡
static void pushes(char *s, Manager *manager)
{
    while (*s)
        push(*s++, manager);
}

```

125e · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `pushs` 51, 127a, 129c · uses `Manager` 125c, `push` 125d

```

< scraps.c 125f > + ≡
int collect_scrap(void)
{
    int current_scrap, lblseq = 0;
    int depth = 1;
    Manager writer;
    < Create new scrap, managed by writer 126a >
    < Accumulate scrap and return scraps++ 126b >
}

```

125f · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `collect_scrap` 45ab, 51, 122b, 167c · uses `Manager` 125c

```

< Create new scrap, managed by writer 126a > ≡
{
    Slab *scrap = (Slab *) arena_getmem(sizeof(Slab));
    if ((scraps & SCRAP_MASK) == 0)
        SCRAP[scraps >> SCRAP_SHIFT] = (ScrapEntry *) arena_getmem(SCRAP_SIZE * sizeof(
            ScrapEntry));
    scrap_array(scraps).slab = scrap;
    scrap_array(scraps).file_name = save_string(source_name);
    scrap_array(scraps).file_line = source_line;
    scrap_array(scraps).page = -1;
    scrap_array(scraps).letter = 0;
    scrap_array(scraps).uses = NULL;
    scrap_array(scraps).defs = NULL;
    scrap_array(scraps).sector = current_sector;
    scrap_array(scraps).end_line = source_line;
    scrap_array(scraps).prose = NULL;
    scrap_array(scraps).owner = NULL;
    writer.scrap = scrap;
    writer.index = 0;
    current_scrap = scraps++;
}

```

126a · used in 125f · uses `arena_getmem` 182b, `current_sector` 44c, `defs` 121c, `end_line` 121c, `file_line` 121c, `file_name` 121c, `letter` 121c, `owner` 121c, `page` 121c, `prose` 121c, `save_string` 146a, `SCRAP` 122a, `ScrapEntry` 121c, `scraps` 122a, `scrap_array` 122a, `SCRAP_MASK` 122a, `SCRAP_SHIFT` 122a, `SCRAP_SIZE` 122a, `sector` 121c, `Slab` 121b, `slab` 121c, `source_line` 117b, `source_name` 117b, uses 121c

< Accumulate scrap and return scraps++ 126b > ≡

```
{
  int c = source_get();
  while (1) {
    switch (c) {
      case EOF: fprintf(stderr, "%s: unexpect EOF in (%s, %d)\n",
                        command_name, scrap_array(current_scrap).file_name,
                        scrap_array(current_scrap).file_line);
                exit(-1);
      default:
        if (c==nw_char)
          {
            < Handle at-sign during scrap accumulation 126c >
            break;
          }
        push(c, &writer);
        c = source_get();
        break;
    }
  }
}
```

126b · used in 125f · uses [command_name 28c](#), [exit 21b](#), [file_line 121c](#), [file_name 121c](#), [fprintf 21b](#), [nw_char 28ab](#), [push 125d](#), [scrap_array 122a](#), [source_get 118a](#), [stderr 21b](#)

< Handle at-sign during scrap accumulation 126c > ≡

```
{
  c = source_get();
  switch (c) {
    case '(':
    case '[':
    case '{': depth++;
              break;

    case '+':
    case '-':
    case '*':
    case '|': < Collect user-specified index entries 128a >
              /* Fall through */

    case ')':
    case ']':
    case '}': if (--depth > 0)
              break;
              /* else fall through */

    case ',':
              push('\0', &writer);
              scrap_ended_with = c;
              scrap_array(current_scrap).end_line = source_line;
              return current_scrap;

    case '<': < Handle macro invocation in scrap 129b >
              break;

    case '%': < Skip commented-out code 72d >
              /* emit line break to the output file to keep #line in sync. */
              push('\n', &writer);
              c = source_get();
              break;

    case 'x': < Get label while collecting scrap 127a >
              break;

    case 'c': < Include block comment in a scrap 48f >
              break;
  }
```

```

case '1': case '2': case '3':
case '4': case '5': case '6':
case '7': case '8': case '9':
case 'f': case '#': case 'v':
case 't': case 's':
    push(nw_char, &writer);
    break;
case '_': c = source_get();
    break;
default :
    if (c==nw_char)
    {
        push(nw_char, &writer);
        push(nw_char, &writer);
        c = source_get();
        break;
    }
    fprintf(stderr, "%s: unexpected %c%c in scrap (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}

```

126c · used in 126b · uses [command_name 28c](#), [end_line 121c](#), [exit 21b](#), [fprintf 21b](#), [nw_char 28ab](#), [push 125d](#), [scrap_array 122a](#), [source_get 118a](#), [source_line 117b](#), [source_name 117b](#), [stderr 21b](#)

⟨ *Get label while collecting scrap* 127a ⟩ ≡

```

{
    ⟨ Get label from(127b source_get() ) 179b ⟩
    ⟨ Save label to label store 180c ⟩
    push(nw_char, &writer);
    push('x', &writer);
    pushs(label_name, &writer);
    push(nw_char, &writer);
}

```

127b · used in 126c · uses [source_get 118a](#)

⟨ *Collect user-specified index entries* 128a ⟩ ≡

```

{
    do {
        int type = c;
        do {
            char new_name[MAX_NAME_LEN];
            char *p = new_name;
            unsigned int sector = 0;
            do
                c = source_get();
            while (isspace(c));
            if (c != nw_char) {
                Name *name;
                do {
                    *p++ = c;
                    c = source_get();
                } while (c != nw_char && !isspace(c));
                *p = '\0';
                switch (type) {
                    case '*':
                        sector = current_sector;
                        ⟨ Add user identifier use 128b ⟩

```

```

        break;
    case '-':
        < Add user identifier use 128b >
        /* Fall through */
    case '|':
        sector = current_sector;
        /* Fall through */
    case '+':
        < Add user identifier definition 129a >
        break;
    }
}
} while (c != nw_char);
c = source_get();
}while (c == '|' || c == '*' || c == '-' || c == '+');
if (c != '}' && c != ']' && c != ')') {
    fprintf(stderr, "%s: unexpected %c%c in index entry (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}

```

128a · used in 126c · uses [command_name 28c](#), [current_sector 44c](#), [exit 21b](#), [fprintf 21b](#), [isspace 21b](#), [MAX_NAME_LEN 23b](#), [Name 144b](#), [nw_char 28ab](#), [sector 121c](#), [source_get 118a](#), [source_line 117b](#), [source_name 117b](#), [stderr 21b](#)

```

< Add user identifier use 128b > ≡
name = name_add(&user_names, new_name, sector);
if (!name->uses || name->uses->scrap != current_scrap) {
    Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    use->scrap = current_scrap;
    use->next = name->uses;
    name->uses = use;
    add_uses(&(scrap_array(current_scrap).uses), name);
}

```

128b · used in 128a · uses [add_uses 176c](#), [arena_getmem 182b](#), [name_add 148b](#), [next 121b](#), [scrap_array 122a](#), [Scrap_Node 144a](#), [sector 121c](#), [user_names 144c](#), uses 121c

```

< Add user identifier definition 129a > ≡
name = name_add(&user_names, new_name, sector);
if (!name->defs || name->defs->scrap != current_scrap) {
    Scrap_Node *def = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
    def->scrap = current_scrap;
    def->next = name->defs;
    name->defs = def;
    add_uses(&(scrap_array(current_scrap).defs), name);
}

```

129a · used in 128a · uses [add_uses 176c](#), [arena_getmem 182b](#), [defs 121c](#), [name_add 148b](#), [next 121b](#), [scrap_array 122a](#), [Scrap_Node 144a](#), [sector 121c](#), [user_names 144c](#)

```

< Handle macro invocation in scrap 129b > ≡
{
    Arglist * args = collect_scrap_name(current_scrap);
    Name *name = args->name;
    < Save macro name 129c >
    add_to_use(name, current_scrap);
    if (scrap_name_has_parameters) {
        < Save macro parameters 51 >
    }
}

```

```

    push(nw_char, &writer);
    push('>', &writer);
    c = source_get();
}

```

129b · used in 126c · uses [add_to_use](#) 129d, [Arglist](#) 164d, [collect_scrap_name](#) 165, [Name](#) 144b, [nw_char](#) 28ab, [push](#) 125d, [source_get](#) 118a

```

⟨ Save macro name 129c ⟩ ≡
{
    char buff[24];

    push(nw_char, &writer);
    push('<', &writer);
    push(name->sector, &writer);
    sprintf(buff, "%d", arglist_register(args));
    pushs(buff, &writer);
}

```

129c · used in 129b · uses [arglist_register](#) 134d, [nw_char](#) 28ab, [push](#) 125d, [pushs](#) 125e, [sector](#) 121c

```

⟨ scraps.c 129d ⟩ + ≡
void
add_to_use(Name * name, int current_scrap)
{
    if (!name->uses || name->uses->scrap != current_scrap) {
        Scrap_Node *use = (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
        use->scrap = current_scrap;
        use->next = name->uses;
        name->uses = use;
    }
}

```

129d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines [add_to_use](#) 51, 129b, 130a, 167e · uses [arena_getmem](#) 182b, [Name](#) 144b, [next](#) 121b, [Scrap_Node](#) 144a, uses 121c

```

⟨ Function prototypes 130a ⟩ + ≡
extern void add_to_use(Name * name, int current_scrap);

130a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses add\_to\_use 129d, Name 144b

```

```

⟨ scraps.c 130b ⟩ + ≡
static char pop(Manager *manager)
{
    Slab *scrap = manager->scrap;
    int index = manager->index;
    char c = scrap->chars[index++];
    if (index == SLAB_SIZE) {
        manager->prev = scrap;
        manager->scrap = scrap->next;
        index = 0;
    }
    manager->index = index;
    return c;
}

```

130b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines [pop](#) 49abd, 52a, 130c, 133d, 134a, 135, 137b, 138a, 139b, 170c ·

uses [Manager 125c](#), [next 121b](#), [Slab 121b](#), [SLAB_SIZE 121b](#)

The function `dump_scrap_text` walks the slab chain for a given scrap and writes every printable character (plus newlines and tabs) to the supplied file handle. The internal meta-character sequences that encode fragment references are decoded back to their `@<name@>` form; other control sequences are silently skipped.

([scraps.c 130c](#)) + ≡

```
void dump_scrap_text(FILE *file, int scrap_idx)
{
    Manager reader;
    char c;
    reader.scrap = scrap_array(scrap_idx).slab;
    reader.prev = NULL;
    reader.index = 0;
    c = pop(&reader);
    while (c) {
        if (c == nw_char) {
            /* handle internal control sequences */
            c = pop(&reader);
            if (c == '<') {
                /* fragment reference encoded as: sector_byte + decimal_index
                 followed by nw_char + '>' */
                char idx_buf[MAX_NAME_LEN];
                char *p = idx_buf;
                int idx;
                (void)pop(&reader); /* sector byte, discard */
                c = pop(&reader);
                while (c != nw_char && c != '\0') {
                    *p++ = c;
                    c = pop(&reader);
                }
                *p = '\0';
                if (sscanf(idx_buf, "%d", &idx) == 1) {
                    Arglist *a = arglist_lookup(idx);
                    if (a && a->name) {
                        fprintf(file, "%c<%s%c>", nw_char, a->name->spelling, nw_char);
                    }
                }
                /* skip past closing '>' marker */
            }
            if (c == nw_char)
                c = pop(&reader);
            /* skip optional parameter block */
            if (c == '(') {
                while (c && c != nw_char)
                    c = pop(&reader);
                if (c == nw_char)
                    c = pop(&reader);
            }
        }
        else {
            /* other control seq: skip */
        }
        c = pop(&reader);
        continue;
    }
    fputc(c, file);
    c = pop(&reader);
}
}
```

130c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `dump_scrap_text` 102b, 124a, 131 · uses `Arglist` 164d, `arglist_lookup` 134d, `FILE` 21b, `fprintf` 21b, `Manager` 125c, `MAX_NAME_LEN` 23b, `nw_char` 28ab, `pop` 130b, `scrap_array` 122a, `sector` 121c, `slab` 121c

The function `dump_scrap_json` is similar to `dump_scrap_text` but wraps the output in a JSON string, escaping special characters inline. It is used by `--bodies` to embed scrap code in the `-m` JSON output.

```
<scraps.c 131> +≡
void dump_scrap_json(FILE *f, int scrap_idx)
{
    FILE *tmp = tmpfile();
    int ch;
    if (!tmp) { fputs("\\""", f); return; }
    dump_scrap_text(tmp, scrap_idx);
    fflush(tmp);
    fseek(tmp, 0, SEEK_SET);
    fputc('"', f);
    while ((ch = fgetc(tmp)) != EOF) {
        switch (ch) {
            case '\\': fputs("\\\\", f); break;
            case '\n': fputs("\\n", f); break;
            case '\r': fputs("\\r", f); break;
            case '\t': fputs("\\t", f); break;
            default:
                if (ch < 0x20)
                    fprintf(f, "\\u%04x", ch);
                else
                    fputc(ch, f);
        }
    }
    fputc('"', f);
    fclose(tmp);
}
```

131 · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `dump_scrap_json` 100, 124a · uses `dump_scrap_text` 130c, `fclose` 21b, `FILE` 21b, `fprintf` 21b, `fputs` 21b

```
<scraps.c 132a> +≡
void set_scrap_prose(int i, char *p) { scrap_array(i).prose = p; }
```

132a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `set_scrap_prose` 45ab, 124a · uses `prose` 121c, `scrap_array` 122a

```
<scraps.c 132b> +≡
void set_scrap_owner(int i, Name *n) { scrap_array(i).owner = n; }
```

132b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `set_scrap_owner` 45c, 124a · uses `Name` 144b, `owner` 121c, `scrap_array` 122a

```
<scraps.c 132c> +≡
static void backup(int n, Manager *manager)
{
    int index = manager->index;
    if (n > index
        && manager->prev != NULL)
```

```

{
    manager->scrap = manager->prev;
    manager->prev = NULL;
    index += SLAB_SIZE;
}
manager->index = (n <= index ? index - n : 0);
}

```

132c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines backup 137b · uses Manager 125c, SLAB_SIZE 121b

⟨ scraps.c 132d ⟩ + ≡

```

void
lookup(int n, Arglist * par, char * arg[9], Name **name, Arglist ** args)
{
    int i;
    Arglist * p = par;

    for (i = 0; i < n && p != NULL; i++)
        p = p->next;
    if (p == NULL) {
        char * a = arg[n];

        *name = NULL;
        *args = (Arglist *)a;
    }
    else {
        *name = p->name;
        *args = p->args;
    }
}

```

132d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines lookup 50c, 133b · uses Arglist 164d, Name 144b, next 121b

⟨ scraps.c 132e ⟩ + ≡

```

Arglist * instance(Arglist * a, Arglist * par, char * arg[9], int * ch)
{
    if (a != NULL) {
        int changed = 0;
        Arglist *args, *next;
        Name* name;
        ⟨ Set up name, args and next 133b ⟩
        if (changed){
            ⟨ Build a new arglist 133c ⟩
            *ch = 1;
        }
    }

    return a;
}

```

132e · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines instance 84a, 133ab, 139d · uses Arglist 164d, Name 144b, next 121b

⟨ Function prototypes 133a ⟩ + ≡

```

Arglist *instance(Arglist *a, Arglist *par, char *arg[9], int *ch);

```

133a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses Arglist 164d, instance 132e

⟨ *Set up name, args and next* 133b ⟩ ≡

```
next = instance(a->next, par, arg, &changed);
name = a->name;
if (name == (Name *)1) {
    Embed_Node * q = (Embed_Node *)arena_getmem(sizeof(Embed_Node));
    q->defs = (Scrap_Node *)a->args;
    q->args = par;
    args = (Arglist *)q;
    changed = 1;
} else if (name != NULL)
    args = instance(a->args, par, arg, &changed);
else {
    char * p = (char *)a->args;
    if (p[0] == ARG_CHR) {
        lookup(p[1] - '1', par, arg, &name, &args);
        changed = 1;
    }
    else {
        args = a->args;
    }
}
```

133b · used in 132e · uses [arena_getmem](#) 182b, [Arglist](#) 164d, [ARG_CHR](#) 163b, [defs](#) 121c, [Embed_Node](#) 167d, [instance](#) 132e, [lookup](#) 132d, [Name](#) 144b, [next](#) 121b, [Scrap_Node](#) 144a

⟨ *Build a new arglist* 133c ⟩ ≡

```
a = (Arglist *)arena_getmem(sizeof(Arglist));
a->name = name;
a->args = args;
a->next = next;
```

133c · used in 132e · uses [arena_getmem](#) 182b, [Arglist](#) 164d, [next](#) 121b

⟨ *scrap.c* 133d ⟩ + ≡

```
static Arglist *pop_scrap_name(Manager *manager, Parameters *parameters)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    Arglist * args;
    int c;
    int idx;

    (void)pop(manager); /* not sure why we have to pop twice */
    c = pop(manager);

    while (c != nw_char) {
        *p++ = c;
        c = pop(manager);
    }
    *p = '\000';
    if (sscanf(name, "%d", &idx) != 1)
    {
        fprintf(stderr, "%s: found an internal problem (2)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    ⟨ Check for end of scrap name 134a ⟩
    return args;
}
```

133d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `pop_scrap_name` 139d · uses `Arglist` 164d, `arglist_lookup` 134d, `command_name` 28c, `exit` 21b, `fprintf` 21b, `Manager` 125c, `MAX_NAME_LEN` 23b, `nw_char` 28ab, `Parameters` 50b, `pop` 130b, `stderr` 21b

```
< Check for end of scrap name 134a > ≡
{
    c = pop(manager);
    < Check for macro parameters 52a >
}
```

134a · used in 133d · uses `pop` 130b

```
< scraps.c 134b > + ≡
int write_scraps(FILE *file, char *spelling, Scrap_Node *defs,
                 int global_indent, char *indent_chars,
                 char debug_flag, char tab_flag, char indent_flag,
                 unsigned char comment_flag, char location_flag,
                 Arglist *inArgs,
                 char *inParams[9], Parameters parameters, char *title)
{
    /* This is in file file name */
    int indent = 0;
    int newline = 1;
    while (defs) {
        < Copy defs->scrap to file 135 >
        defs = defs->next;
    }
    return indent + global_indent;
}
```

134b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `write_scraps` 50c, 74a, 84a, 91b, 113e, 122b, 139d, 141c · uses `Arglist` 164d, `defs` 121c, `FILE` 21b, `next` 121b, `Parameters` 50b, `Scrap_Node` 144a

```
< Forward declarations for scraps.c 134c > ≡
int delayed_indent = 0;
```

134c · defined at 134cd, 141b, 161b, 176a · used in 122a · defines `delayed_indent` 50c, 135, 137d, 139d, 140ab

```
< Forward declarations for scraps.c 134d > + ≡
```

```
#define MAX_ARGLISTS 4096
static Arglist *arglist_table[MAX_ARGLISTS];
static int arglist_count = 0;

void arglist_reset(void)
{
    arglist_count = 0;
}

static int arglist_register(Arglist *a)
{
    if (arglist_count >= MAX_ARGLISTS) {
        fprintf(stderr, "%s: too many macro argument lists (max %d)\n",
                command_name, MAX_ARGLISTS);
        exit(EXIT_FAILURE);
    }
    arglist_table[arglist_count] = a;
```

```

    return arglist_count++;
}

```

```

static Arglist *arglist_lookup(int idx)
{
    if (idx < 0 || idx >= arglist_count) {
        fprintf(stderr, "%s: invalid arglist index %d\n", command_name, idx);
        exit(EXIT_FAILURE);
    }
    return arglist_table[idx];
}

```

134d · defined at 134cd, 141b, 161b, 176a · used in 122a · defines `arglist_count` never used, `arglist_lookup` 130c, 133d, 175b, `arglist_register` 129c, `arglist_reset` 181d, 183, `arglist_table` never used, `MAX_ARGLISTS` never used · uses `Arglist` 164d, `command_name` 28c, `exit` 21b, `fprintf` 21b, `stderr` 21b

⟨ *Copy defs->scrap to file* 135 ⟩ ≡

```

{
    char c;
    Manager reader;
    Parameters local_parameters = 0;
    int line_number = scrap_array(defs->scrap).file_line;
    reader.scrap = scrap_array(defs->scrap).slab;
    reader.prev = NULL;
    reader.index = 0;
    ⟨ Insert debugging information if required 136b ⟩
    ⟨ Insert section opening marker 136c ⟩
    if (delayed_indent)
    {
        ⟨ Insert appropriate indentation 137b ⟩
    }
    c = pop(&reader);
    while (c) {
        switch (c) {
            case '\n':
                if (global_indent >= 0) {
                    putc(c, file);
                    line_number++;
                    newline = 1;
                    delayed_indent = 0;
                    ⟨ Insert appropriate indentation 137b ⟩
                    break;
                } else {
                    /* Don't show newlines in embedded fragments */
                    fputs(". . .", file);
                    return 0;
                }
            case '\t': ⟨ Handle tab characters on output 137e ⟩
                delayed_indent = 0;
                break;
            default:
                if (c==nw_char)
                {
                    ⟨ Check for macro invocation in scrap 138a ⟩
                    break;
                }
                putc(c, file);
                if (global_indent >= 0) {
                    ⟨ Add more indentation ' ' 136a ⟩

```

```

    }
    indent++;
    if (c > ' ') newline = 0;
    delayed_indent = 0;
    break;
}
c = pop(&reader);
}
< Insert section closing marker 137a >
}

```

135 · used in 134b · uses `defs 121c`, `delayed_indent 134c`, `file_line 121c`, `fputs 21b`, `Manager 125c`, `nw_char 28ab`, `Parameters 50b`, `pop 130b`, `putc 21b`, `scrap_array 122a`, `slab 121c`

We need to make sure that we don't overflow `indent_chars[]`. If the line is longer than `MAX_INDENT`, we simply stop recording; the Put out the indent scrap will use spaces for any positions beyond the recorded range.

```

< Add more indentation char 136a > ≡
{
    if (global_indent + indent < MAX_INDENT)
        indent_chars[global_indent + indent] = char;
}

```

136a · used in 135, 137e, 138a · uses `MAX_INDENT 113d`

```

< Insert debugging information if required 136b > ≡
if (debug_flag) {
    fprintf(file, "\n#line %d \"%s\"\n",
            line_number, scrap_array(defs->scrap).file_name);
    < Insert appropriate indentation 137b >
}

```

136b · used in 135, 138b · uses `defs 121c`, `file_name 121c`, `fprintf 21b`, `scrap_array 122a`

Section markers in the Knuth style delimit each scrap with opening and closing comments that include the scrap number and source location. For example, in C: `/* {5: weft.w:28} */` and `/* {:5} */`.

```

< Insert section opening marker 136c > ≡
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d}: %s{:d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            scrap_array(defs->scrap).file_name,
            line_number,
            comment_end[comment_flag]);
}

```

136c · used in 135 · uses `comment_begin 161b`, `comment_end 161b`, `defs 121c`, `file_name 121c`, `fprintf 21b`, `scrap_array 122a`

```

< Insert section closing marker 137a > ≡
if (location_flag && comment_flag) {
    fprintf(file, "%s{:d}%s\n",
            comment_begin[comment_flag],
            defs->scrap,
            comment_end[comment_flag]);
}

```

137a · used in 135 · uses `comment_begin 161b`, `comment_end 161b`, `defs 121c`, `fprintf 21b`

⟨ *Insert appropriate indentation* 137b ⟩ ≡

```
{
    char c1 = pop(&reader);
    char c2 = pop(&reader);

    if (indent_flag && !(⟨ Indent suppressed 137d ⟩)) {
        ⟨ Put out the indent 137c ⟩
    }
    indent = 0;
    backup(2, &reader);
}
```

137b · used in 135, 136b · uses backup 132c, pop 130b

⟨ *Put out the indent* 137c ⟩ ≡

```
if (tab_flag)
    for (indent=0; indent<global_indent; indent++)
        putc(' ', file);
else
    for (indent=0; indent<global_indent; indent++)
        putc(indent < MAX_INDENT ? indent_chars[indent] : ' ', file);
```

137c · used in 50c, 137b · uses MAX_INDENT 113d, putc 21b

Indent will be suppressed if the next character is a newline or if the next two characters are @#. If the next two characters are @<, we suppress the indent for now but mark that it may be needed when the next fragment is started.

⟨ *Indent suppressed* 137d ⟩ ≡

```
c1 == '\n'
|| (c1 == nw_char && (c2 == '#' || (delayed_indent != (c2 == '<'))))
```

137d · used in 137b · uses delayed_indent 134c, nw_char 28ab

⟨ *Handle tab characters on output* 137e ⟩ ≡

```
{
    if (tab_flag)
        ⟨ Expand tab into spaces 70c ⟩
    else {
        putc('\t', file);
        if (global_indent >= 0) {
            ⟨ Add more indentation ^t' 136a ⟩
        }
        indent++;
    }
}
```

137e · used in 135 · uses putc 21b

⟨ *Check for macro invocation in scrap* 138a ⟩ ≡

```
{
    int oldin = indent;
    char oldcf = comment_flag;
    c = pop(&reader);
    switch (c) {
        case 't': ⟨ Copy fragment title into file 140c ⟩
            break;
        case 'c': ⟨ Copy block comment from scrap 49a ⟩
            break;
        case 'f': ⟨ Copy file name into file 139c ⟩
```

```

        break;
case 'x': < Copy label from scrap into file 139a >
case '_': break;
case 'v': < Copy version info into file 71b >
        break;
case 's': indent = -global_indent;
        comment_flag = 0;
        break;
case '<': < Copy macro into file 139d >
        < Resync the line number after a fragment reference 138b >
        indent = oldin;
        comment_flag = oldcf;
        break;
< Handle macro parameter substitution 50c >
        indent = oldin;
        break;
default:
        if(c==nw_char)
        {
                putc(c, file);
                if (global_indent >= 0) {
                        < Add more indentation ' ' 136a >
                }
                indent++;
                break;
        }
        /* ignore, since we should already have a warning */
        break;
}
}

```

138a · used in 135 · uses `nw_char` 28ab, `pop` 130b, `putc` 21b

The `#line` marker that keeps the C preprocessor's line count in step is emitted *after* a fragment reference expands, to resync to the parent scrap's current line. That is right when the reference sat on a line of its own, but for a *mid-line* reference — `int port = @<port>;` or `#define V "@<version>"` — an unconditional resync would inject a stray `\n#line\n` into the middle of the line, splitting the expression or string literal across two lines (and breaking the C it produces). So we resync only when the reference *began* its line (`newline` is still set): a mid-line reference then splices its fragment's value in place and the surrounding text stays on one line, exactly as `@<port>` becoming 4000 should. The child expansion already suppresses its own markers mid-line (see `Copy macro into file`); this closes the remaining gap left by the trailing resync.

```

< Resync the line number after a fragment reference 138b > ≡
if (newline) {
    < Insert debugging information if required 136b >
}

```

138b · used in 138a

```

< Copy label from scrap into file 139a > ≡
{
    < Get label from(139b pop(&reader) ) 179b >
    write_label(label_name, file);
}

```

139b · used in 138a · uses `pop` 130b

```

< Copy file name into file 139c > ≡
if (defs->quoted)

```

```

    fprintf(file, "%cf", nw_char);
else
    fputs(spelling, file);
139c · used in 138a · uses defs 121c, fprintf 21b, fputs 21b, nw_char 28ab

< Copy macro into file 139d > ≡
{
    Arglist *a = pop_scrap_name(&reader, &local_parameters);
    Name *name = a->name;
    int changed;
    Arglist * args = instance(a->args, inArgs, inParams, &changed);
    int i, nargs;
    char * p = name->spelling;
    char * * inParams = name->arg;
    Arglist *q = args;

    if (name->mark) {
        fprintf(stderr, "%s: recursive macro discovered involving <%s>\n",
            command_name, name->spelling);
        exit(-1);
    }
    if (name->defs && !name->quoted) {
        unsigned char saved_comment = comment_flag;
        char saved_location = location_flag;
        char saved_debug = debug_flag;
        if (name->lang[0] != '\0') {
            comment_flag = name->comment_flag;
            location_flag = name->location_flag;
            debug_flag = name->debug_flag;
        }
        if (!newline) {
            comment_flag = 0;
            location_flag = FALSE;
            debug_flag = FALSE;
        }
        < Perhaps comment this macro 140a >
        name->mark = TRUE;
        indent = write_scraps(file, spelling, name->defs, global_indent + indent,
            indent_chars, debug_flag, tab_flag, indent_flag,
            comment_flag, location_flag, args, name->arg,
            local_parameters, name->spelling);
        comment_flag = saved_comment;
        location_flag = saved_location;
        debug_flag = saved_debug;
        indent -= global_indent;
        name->mark = FALSE;
    }
    else
    {
        if (delayed_indent)
        {
            for (i = indent + global_indent; --i >= 0; )
                putc(' ', file);
        }

        fprintf(file, "%c<", nw_char);
        if (name->sector == 0)
            fputc('+', file);
    }
}

```

```

    < Comment this macro use 141a >
    fprintf(file, "%c>", nw_char);
    if (!defs->quoted && !tex_flag)
        fprintf(stderr, "%s: macro never defined <%s>\n",
            command_name, name->spelling);
}
}

```

139d · used in 138a · uses Arglist 164d, command_name 28c, defs 121c, delayed_indent 134c, exit 21b, FALSE 23a, fprintf 21b, instance 132e, Name 144b, nw_char 28ab, pop_scrap_name 133d, putc 21b, sector 121c, stderr 21b, tex_flag 27a, TRUE 23a, write_scrap 134b

```

< Perhaps comment this macro 140a > ≡
if (comment_flag && newline) {
    < Perhaps put a delayed indent 140b >
    fputs(comment_begin[comment_flag], file);
    < Comment this macro use 141a >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, name->defs->scrap);
    }
    fputs(comment_end[comment_flag], file);
    putc('\n', file);
    if (!delayed_indent)
        for (i = indent + global_indent; --i >= 0; )
            putc(' ', file);
}

```

140a · used in 50c, 139d · uses comment_begin 161b, comment_end 161b, defs 121c, delayed_indent 134c, fputs 21b, putc 21b, write_single_scrap_ref 123c, xref_flag 27a

```

< Perhaps put a delayed indent 140b > ≡
if (delayed_indent)
    for (i = indent + global_indent; --i >= 0; )
        putc(' ', file);

```

140b · used in 49a, 140a · uses delayed_indent 134c, putc 21b

```

< Copy fragment title into file 140c > ≡
{
    char * p = title;
    Arglist *q = inArgs;
    int narg;

    < Comment this macro use 141a >
    if (xref_flag) {
        putc(' ', file);
        write_single_scrap_ref(file, defs->scrap);
    }
}

```

140c · used in 138a · uses Arglist 164d, defs 121c, putc 21b, write_single_scrap_ref 123c, xref_flag 27a

```

< Comment this macro use 141a > ≡
narg = 0;
while (*p != '\000') {
    if (*p == ARG_CHR) {
        if (q == NULL) {
            if (defs->quoted)
                fprintf(file, "%c'%s%c'", nw_char, inParams[narg], nw_char);

```

```

        else
            fprintf(file, "'%s'", inParams[narg]);
    }
    else {
        comment_ArglistElement(file, q, defs->quoted);
        q = q->next;
    }
    p++;
    narg++;
}
else
    fputc(*p++, file);
}

```

141a · used in 139d, 140ac · uses ARG_CHR 163b, comment_ArglistElement 141b, defs 121c, fprintf 21b, next 121b, nw_char 28ab

< Forward declarations for scraps.c 141b > + ≡

```

static void
comment_ArglistElement(FILE * file, Arglist * args, int quote)
{
    Name *name = args->name;
    Arglist *q = args->args;

    if (name == NULL) {
        if (quote)
            fprintf(file, "%c'%s%c'", nw_char, (char *)q, nw_char);
        else
            fprintf(file, "'%s'", (char *)q);
    } else if (name == (Name *)1) {
        < Include an embedded scrap in comment 141c >
    } else {
        < Include a fragment use in comment 141d >
    }
}

```

141b · defined at 134cd, 141b, 161b, 176a · used in 122a · defines comment_ArglistElement 141ad · uses Arglist 164d, FILE 21b, fprintf 21b, Name 144b, nw_char 28ab

< Include an embedded scrap in comment 141c > ≡

```

Embed_Node * e = (Embed_Node *)q;
fputc('{', file);
write_scraps(file, "", e->defs, -1, "", 0, 0, 0, 0, 0, e->args, NULL, NULL, "");
fputc('}', file);

```

141c · used in 141b · uses defs 121c, Embed_Node 167d, write_scraps 134b

< Include a fragment use in comment 141d > ≡

```

char * p = name->spelling;
if (quote)
    fputc(nw_char, file);
fputc('<', file);
if (quote && name->sector == 0)
    fputc('++', file);
while (*p != '\000') {
    if (*p == ARG_CHR) {
        comment_ArglistElement(file, q, quote);
        q = q->next;
        p++;
    }
}

```

```

    else
        fputc(*p++, file);
}
if (quote)
    fputc(nw_char, file);
fputc('>', file);

```

141d · used in 141b · uses ARG_CHR 163b, comment_ArglistElement 141b, next 121b, nw_char 28ab, sector 121c

3.2.1 Collecting Page Numbers

⟨ *Function prototypes* 142a ⟩ + ≡

```
extern void collect_numbers(char *aux_name);
```

142a · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses collect_numbers 142b

⟨ *scraps.c* 142b ⟩ + ≡

```

void collect_numbers(char *aux_name)
{
    if (number_flag) {
        int i;
        for (i=1; i<scraps; i++)
            scrap_array(i).page = i;
    }
    else {
        FILE *aux_file = fopen(aux_name, "r");
        already_warned = FALSE;
        if (aux_file) {
            char aux_line[500];
            while (fgets(aux_line, 500, aux_file)) {
                ⟨ Read line in .aux file 142c ⟩
            }
            fclose(aux_file);
            ⟨ Add letters to scraps with duplicate page numbers 143a ⟩
        }
    }
}

```

142b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines collect_numbers 39b, 142a · uses already_warned 125a, FALSE 23a, fclose 21b, FILE 21b, fopen 21b, number_flag 27a, page 121c, scraps 122a, scrap_array 122a

The *memoir* document class creates references in the .aux file with nested braces, so after each open brace we have to wait for the matching closing brace.

⟨ *Read line in .aux file* 142c ⟩ ≡

```

int scrap_number;
int page_number;
int i;
int dummy_idx;
int bracket_depth = 1;
if (1 == sscanf(aux_line,
                "\\newlabel{scrap%d}{{%n",
                &scrap_number,
                &dummy_idx)) {
    for (i = dummy_idx; i < (int)strlen(aux_line) && bracket_depth > 0; i++) {
        if (aux_line[i] == '{') bracket_depth++;
        else if (aux_line[i] == '}') bracket_depth--;
    }
}

```

```

}
if (i > dummy_idx
    && i < (int)strlen(aux_line)
    && 1 == sscanf(aux_line+i, "{%d}" ,&page_number)) {
    if (scrap_number < scraps)
        scrap_array(scrap_number).page = page_number;
    else
        < Warn (only once) about needing to rerun after Latex 124g >
}
}

```

142c · used in 142b · uses [page 121c](#), [scraps 122a](#), [scrap_array 122a](#), [strlen 21b](#)

< Add letters to scraps with duplicate page numbers 143a > ≡

```

{
    int i = 0;

    < Step i to the next valid scrap 143b >
    < For all remaining scraps 143c > {
        int j = i;
        < Step j to the next valid scrap 143b >
        < Perhaps add letters to the page numbers 143d >
        i = j;
    }
}

```

143a · used in 142b

< Step i to the next valid scrap 143b > ≡

```

do
    i++;
while (i < scraps && scrap_array(i).page == -1);

```

143b · used in 143a · uses [page 121c](#), [scraps 122a](#), [scrap_array 122a](#)

< For all remaining scraps 143c > ≡

```

while (i < scraps)

```

143c · used in 143a · uses [scraps 122a](#)

< Perhaps add letters to the page numbers 143d > ≡

```

if (scrap_array(i).page == scrap_array(j).page) {
    if (scrap_array(i).letter == 0)
        scrap_array(i).letter = 'a';
    scrap_array(j).letter = scrap_array(i).letter + 1;
}

```

143d · used in 143a · uses [letter 121c](#), [page 121c](#), [scrap_array 122a](#)

3.3 Names

Each scrap is linked through a simple node:

< Type declarations 144a > + ≡

```

typedef struct scrap_node {
    struct scrap_node *next;
    int scrap;
    char quoted;
} Scrap_Node;

```

144a · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines `Scrap_Node` 45c, 50c, 53b, 58c, 65c, 73, 74a, 76b, 78bc, 79e, 81b, 87d, 90d, 91b, 93d, 95ab, 96b, 97c, 98d, 100, 102b, 103c, 104a, 122b, 128b, 129ad, 133b, 134b, 144b, 167cd, 168bc, 174, 176ab, 177b, 178c · uses `next` 121b

A `Name` represents a file or fragment definition. Beyond the spelling and the binary tree links, several flags control how the tangled output is formatted:

- `debug_flag` — if TRUE, emit `#line` directives (set by `-d` or auto-detected for C/C++).
- `comment_flag` — selects the comment style for section markers and block comments: 0=none, 1=C, 2=C++, 3=shell, 4=Lua/SQL, 5=HTML/XML (see Section 3.3.5).
- `location_flag` — if TRUE, emit Knuth-style opening and closing section markers around each scrap in the tangled output. Activated automatically by `@l` auto-detection or the `@L` command.
- `lang` — the language tag (e.g. "c", "python") used for syntax highlighting in the Markdown weave output. Set from a preceding `@l` command or inferred from the file extension of an `@o` name. Empty if unknown.

⟨ *Type declarations* 144b ⟩ + ≡

```
typedef struct name {
    char *spelling;
    struct name *llink;
    struct name *rlink;
    Scrap_Node *defs;
    Scrap_Node *uses;
    char * arg[9];
    int mark;
    char tab_flag;
    char indent_flag;
    char debug_flag;
    char location_flag;
    char suppress_markers;
    unsigned char comment_flag;
    char lang[64];
    unsigned char sector;
} Name;
```

144b · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines `Name` 30, 45ab, 50c, 53b, 58c, 60ab, 69, 72g, 74b, 75b, 77abd, 79abd, 81b, 84abc, 88a, 90d, 91b, 93b, 94ab, 96a, 97c, 98d, 100, 101, 102ab, 103abc, 104ab, 105b, 106, 107a, 113abc, 124af, 128a, 129bd, 130a, 132bde, 133b, 139d, 141b, 144c, 145ab, 146b, 147a, 148b, 149a, 159a, 162a, 164bcde, 166, 167c, 168bd, 171a, 172cd, 174, 176acd, 177b, 178c, 179a · uses `defs` 121c, `Scrap_Node` 144a, `sector` 121c, `uses` 121c

The variable `pending_lang` acts as a one-shot channel between the `@l` command in `pass1` (which fills it with a language tag like "js" or "python") and `collect_file_name` or `collect_macro_name` (which consume it to set the flags and the `lang` field on the next `@o` or `@d` name).

⟨ *Global variable declarations* 144c ⟩ + ≡

```
extern Name *file_names;
extern Name *macro_names;
extern Name *user_names;
extern int scrap_name_has_parameters;
extern int scrap_ended_with;
extern char pending_lang[64];
```

144c · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · defines `file_names` 39b, 41c, 50a, 75a, 93a, 103c, 104b, 107a, 113b, 145a, 159a, `macro_names` 39b, 41c, 50a, 76c, 93e, 104b, 107a, 113b, 145a, 163d, `pending_lang` 42c, 145a, 160b, 161a, `user_names` 41c, 50a, 79c, 95c, 128b, 129a, 145a, 172c · uses `Name` 144b

⟨ *Global variable definitions* 145a ⟩ + ≡

```
Name *file_names = NULL;
Name *macro_names = NULL;
Name *user_names = NULL;
int scrap_name_has_parameters;
int scrap_ended_with;
char pending_lang[64] = "";
```

145a · defined at 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a · used in 25e · uses [file_names](#) 144c, [macro_names](#) 144c, [Name](#) 144b, [pending_lang](#) 144c, [user_names](#) 144c

⟨ *Function prototypes* 145b ⟩ + ≡

```
extern Name *collect_file_name(void);
extern Name *collect_macro_name(void);
extern Arglist *collect_scrap_name(int current_scrap);
extern Name *name_add(Name **rt, char *spelling, unsigned char sector);
extern Name *prefix_add(Name **rt, char *spelling, unsigned char sector);
extern char *save_string(char *s);
extern void reverse_lists(Name *names);
extern void collect_lang_def(void);
extern void collect_weave_format(void);
```

145b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses [Arglist](#) 164d, [collect_file_name](#) 159a, [collect_lang_def](#) 156, [collect_macro_name](#) 162a, [collect_scrap_name](#) 165, [collect_weave_format](#) 158, [Name](#) 144b, [name_add](#) 148b, [prefix_add](#) 146b, [reverse_lists](#) 168b, [save_string](#) 146a, [sector](#) 121c

⟨ *names.c* 145c ⟩ + ≡

```
enum { LESS, GREATER, EQUAL, PREFIX, EXTENSION };
```

```
static int compare(char *x, char *y)
{
    int len, result;
    int x1 = strlen(x);
    int y1 = strlen(y);
    int xp = x[x1 - 1] == ' ';
    int yp = y[y1 - 1] == ' ';
    if (xp) x1--;
    if (yp) y1--;
    len = x1 < y1 ? x1 : y1;
    result = strncmp(x, y, len);
    if (result < 0) return GREATER;
    else if (result > 0) return LESS;
    else if (x1 < y1) {
        if (xp) return EXTENSION;
        else return LESS;
    }
    else if (x1 > y1) {
        if (yp) return PREFIX;
        else return GREATER;
    }
    else return EQUAL;
}
```

145c · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines [compare](#) 146b, 147a, [EQUAL](#) 146b, 147a, [EXTENSION](#) 146b, 147a, [GREATER](#) 146b, 147a, [LESS](#) 146b, 147a, [PREFIX](#) 146b, 147a · uses [strlen](#) 21b

⟨ *names.c* 146a ⟩ + ≡

```
char *save_string(char *s)
```

```

{
    char *new_str = (char *) arena_getmem((strlen(s) + 1) * sizeof(char));
    strcpy(new_str, s);
    return new_str;
}

146a · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines save_string 35b, 39a,
42b, 119b, 126a, 145b, 146b, 149a, 164c, 168a · uses arena_getmem 182b, strlen 21b

⟨names.c 146b⟩ + ≡
static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector);

static char * found_name = NULL;

Name *prefix_add(Name **rt, char *spelling, unsigned char sector)
{
    Name *node = *rt;
    int cmp;

    while (node) {
        switch ((cmp = compare(node->spelling, spelling))) {
            case GREATER:    rt = &node->rlink;
                            break;
            case LESS:       rt = &node->llink;
                            break;
            case EQUAL:
                found_name = node->spelling;
                FALLTHROUGH;
            case EXTENSION: if (node->sector > sector) {
                            rt = &node->rlink;
                            break;
                        }
                            else if (node->sector < sector) {
                                rt = &node->llink;
                                break;
                            }
                            if (cmp == EXTENSION)
                                node->spelling = save_string(spelling);
                            return node;
            case PREFIX:    ⟨Check for ambiguous prefix 146c⟩
                            return node;
        }
        node = *rt;
    }
    ⟨Create new name entry 149a⟩
}

146b · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines prefix_add 145b, 163d
· uses compare 145c, EQUAL 145c, EXTENSION 145c, FALLTHROUGH 23a, GREATER 145c, LESS 145c, Name 144b, PREFIX 145c,
save_string 146a, sector 121c

```

Since a very short prefix might match more than one fragment name, I need to check for other matches to avoid mistakes. Basically, I simply continue the search down *both* branches of the tree.

```

⟨Check for ambiguous prefix 146c⟩ ≡
{
    if (ambiguous_prefix(node->llink, spelling, sector) ||
        ambiguous_prefix(node->rlink, spelling, sector))
        fprintf(stderr,

```

```

        "%s: ambiguous prefix %c<%s...%c> (%s, line %d)\n",
        command_name, nw_char, spelling, nw_char, source_name, source_line);
}
146c · used in 146b · uses command_name 28c, fprintf 21b, nw_char 28ab, sector 121c, source_line 117b, source_name 117b,
stderr 21b

```

⟨names.c 147a⟩ + ≡

```

static int ambiguous_prefix(Name *node, char *spelling, unsigned char sector)
{
    while (node) {
        switch (compare(node->spelling, spelling)) {
            case GREATER:    node = node->rlink;
                            break;
            case LESS:      node = node->llink;
                            break;
            case EXTENSION:
            case PREFIX:
            case EQUAL:     if (node->sector > sector) {
                            node = node->rlink;
                            break;
                        }
                            else if (node->sector < sector) {
                            node = node->llink;
                            break;
                        }
                            return TRUE;
        }
    }
    return FALSE;
}

```

147a · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · uses compare 145c, EQUAL 145c, EXTENSION 145c, FALSE 23a, GREATER 145c, LESS 145c, Name 144b, PREFIX 145c, sector 121c, TRUE 23a

Rob Shillingsburg suggested that I organize the index of user-specified identifiers more traditionally; that is, not relying on strict ASCII comparisons via `strcmp`. Ideally, we'd like to see the index ordered like this:

```

aardvark
Adam
atom
Atomic
atoms

```

The function `robs_strcmp` implements the desired predicate. It returns -2 for alphabetically less-than, -1 for less-than but only differing in case, zero for equal, 1 for greater-than but only in case and 2 for alphabetically greater-than.

⟨Function prototypes 147b⟩ + ≡

```

extern int robs_strcmp(char*, char*);

```

147b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses robs_strcmp 147c

⟨names.c 147c⟩ + ≡

```

int robs_strcmp(char* x, char* y)
{
    int cmp = 0;

    for (; *x && *y; x++, y++)

```

```

{
    < Skip invisibles on x 148a >
    < Skip invisibles on y 148a >
    if (*x == *y)
        continue;
    if (islower(*x) && toupper(*x) == *y)
    {
        if (!cmp) cmp = 1;
        continue;
    }
    if (islower(*y) && *x == toupper(*y))
    {
        if (!cmp) cmp = -1;
        continue;
    }
    return 2*(toupper(*x) - toupper(*y));
}
if (*x)
    return 2;
if (*y)
    return -2;
return cmp;
}

```

147c · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `robs_strcmp` 77c, 102a, 147b, 148b, 149a, 176c · uses `islower` 21b, `toupper` 21b

Certain character sequences are invisible when printed. We don't want them to be considered for the alphabetical ordering.

< Skip invisibles on p 148a > ≡

```

if (*p == ' | ')
    p++;

```

148a · used in 147c

< names.c 148b > + ≡

Name *name_add(Name **rt, char *spelling, unsigned char sector)

```

{
    Name *node = *rt;
    while (node) {
        int result = robs_strcmp(node->spelling, spelling);
        if (result > 0)
            rt = &node->llink;
        else if (result < 0)
            rt = &node->rlink;
        else
        {
            found_name = node->spelling;
            if (node->sector > sector)
                rt = &node->llink;
            else if (node->sector < sector)
                rt = &node->rlink;
            else
                return node;
        }
        node = *rt;
    }
}
< Create new name entry 149a >
}

```

148b · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `name_add` 128b, 129a, 145b, 159a · uses `Name` 144b, `robs_strcmp` 147c, `sector` 121c

```

⟨ Create new name entry 149a ⟩ ≡
{
    node = (Name *) arena_getmem(sizeof(Name));
    if (found_name && robs_strcmp(found_name, spelling) == 0)
        node->spelling = found_name;
    else
        node->spelling = save_string(spelling);
    node->mark = FALSE;
    node->llink = NULL;
    node->rlink = NULL;
    node->uses = NULL;
    node->defs = NULL;
    node->arg[0] =
    node->arg[1] =
    node->arg[2] =
    node->arg[3] =
    node->arg[4] =
    node->arg[5] =
    node->arg[6] =
    node->arg[7] =
    node->arg[8] = NULL;
    node->tab_flag = TRUE;
    node->indent_flag = TRUE;
    node->debug_flag = FALSE;
    node->location_flag = FALSE;
    node->suppress_markers = FALSE;
    node->comment_flag = 0;
    node->lang[0] = '\0';
    node->sector = sector;
    *rt = node;
    return node;
}

```

149a · used in 146b, 148b · uses `arena_getmem` 182b, `defs` 121c, `FALSE` 23a, `Name` 144b, `robs_strcmp` 147c, `save_string` 146a, `sector` 121c, `TRUE` 23a, uses 121c

3.3.1 Language Auto-Detection

When a Markdown fence (‘‘‘c, ‘‘‘python) or a L^AT_EX marker (%%c) precedes an @o directive, weft auto-detects the language’s comment style and, for C/C++, enables #line directives. This is driven by a data table that maps fence tags to one of the six comment styles defined in Section 3.3.5.

Each entry records:

- **name** — the fence tag (lowercase, e.g. "js", "python", "lua").
- **comment** — index into the `comment_begin`/`comment_end` arrays (1–5).
- **line_directive** — `TRUE` only for languages whose toolchains honour #line (C family).

The built-in table covers over 250 entries (languages, file extensions, and aliases) organised by comment style. For any language not listed, the @L command (Section 3.3.2) lets users add entries at the document level. User-defined entries are searched first, so they can also override built-in defaults. If a fence tag matches neither table, weft defaults to C++ line comments (//) with no #line.

⟨ names.c 149b ⟩ + ≡

```

typedef struct {
    const char *name;
    unsigned char comment;
    char line_directive;
} LangEntry;

static const LangEntry lang_table[] = {
    /* ---- Style 1: C block comments WITH #line ---- */
    {"c", 1, TRUE},
    {"h", 1, TRUE},
    {"hpp", 2, TRUE},
    {"hxx", 2, TRUE},
    {"hh", 2, TRUE},
    {"cpp", 2, TRUE},
    {"c++", 2, TRUE},
    {"cxx", 2, TRUE},
    {"cc", 2, TRUE},
    {"objc", 1, TRUE},
    {"objcpp", 2, TRUE},
    {"m", 1, TRUE},
    {"mm", 2, TRUE},
    {"ino", 2, TRUE}, /* Arduino */
    {"pde", 2, TRUE}, /* Processing (C-like) */
    /* ---- Style 1: C block comments WITHOUT #line ---- */
    {"css", 1, FALSE},
    {"scss", 1, FALSE},
    {"sass", 1, FALSE},
    {"less", 1, FALSE},
    {"stylus", 1, FALSE},
    {"styl", 1, FALSE},
    /* ---- Style 2: C++ line comments (//) ---- */
    /* --- JavaScript / TypeScript --- */
    {"js", 2, FALSE},
    {"javascript", 2, FALSE},
    {"jsx", 2, FALSE},
    {"mjs", 2, FALSE},
    {"cjs", 2, FALSE},
    {"ts", 2, FALSE},
    {"typescript", 2, FALSE},
    {"tsx", 2, FALSE},
    {"mts", 2, FALSE},
    {"cts", 2, FALSE},
    {"svelte", 2, FALSE},
    {"vue", 2, FALSE},
    /* --- JVM languages --- */
    {"java", 2, FALSE},
    {"scala", 2, FALSE},
    {"sc", 2, FALSE},
    {"kotlin", 2, FALSE},
    {"kt", 2, FALSE},
    {"kts", 2, FALSE},
    {"groovy", 2, FALSE},
    {"gradle", 2, FALSE},
    {"clj", 2, FALSE}, /* Clojure (;; but // is close enough) */
    {"cljs", 2, FALSE},
    {"cljc", 2, FALSE},
    /* --- .NET --- */
    {"cs", 2, FALSE},
    {"csharp", 2, FALSE},

```

```

{"fs",                2, FALSE},
{"fsharp",            2, FALSE},
{"fsx",               2, FALSE},
{"vb",                2, FALSE}, /* VB.NET uses ' but // is a safe fallback */
/* --- Systems languages --- */
{"go",                2, FALSE},
{"rust",              2, FALSE},
{"rs",                2, FALSE},
{"zig",               2, FALSE},
{"d",                 2, FALSE},
{"v",                 2, FALSE},
{"vlang",             2, FALSE},
{"swift",             2, FALSE},
{"dart",              2, FALSE},
{"odin",              2, FALSE},
{"jai",               2, FALSE},
{"carbon",            2, FALSE},
{"mojo",              2, FALSE},
/* --- Web / general --- */
{"php",               2, FALSE},
{"hack",              2, FALSE},
{"jsonc",             2, FALSE},
{"json5",             2, FALSE},
{"graphql",           2, FALSE},
{"gql",               2, FALSE},
{"prisma",            2, FALSE},
{"wgs1",              2, FALSE}, /* WebGPU shading language */
/* --- GPU / shaders --- */
{"glsl",              2, FALSE},
{"hlsl",              2, FALSE},
{"vert",              2, FALSE},
{"frag",              2, FALSE},
{"comp",              2, FALSE},
{"geom",              2, FALSE},
{"tesc",              2, FALSE},
{"tese",              2, FALSE},
{"metal",             2, FALSE},
{"cu",                2, TRUE}, /* CUDA: supports #line */
{"cuh",               2, TRUE},
{"cl",                1, FALSE}, /* OpenCL: C-style block comments */
/* --- Protocol / IDL / blockchain --- */
{"proto",             2, FALSE},
{"protobuf",          2, FALSE},
{"thrift",             2, FALSE},
{"flatbuffers",       2, FALSE},
{"fbs",               2, FALSE},
{"capnp",             2, FALSE},
{"avdl",              2, FALSE}, /* Avro IDL */
{"solidity",          2, FALSE},
{"sol",               2, FALSE},
{"vyper",             3, FALSE},
{"vy",                3, FALSE},
{"move",              2, FALSE},
/* --- Functional (// style) --- */
{"reason",            2, FALSE},
{"re",                2, FALSE},
{"res",               2, FALSE}, /* ReScript */
{"ml",                2, FALSE}, /* OCaml also uses (**) but // in comments */
{"mli",               2, FALSE},

```

```

{"sml",          2, FALSE},
{"sig",          2, FALSE},
{"fut",          2, FALSE}, /* Futhark */
/* --- Other // languages --- */
{"actionscript", 2, FALSE},
{"as",           2, FALSE},
{"apex",         2, FALSE},
{"bicep",        2, FALSE},
{"ceu",          2, FALSE},
{"chapel",       2, FALSE},
{"chpl",         2, FALSE},
{"cilk",         2, TRUE},
{"jsonnet",      2, FALSE},
{"libsonnet",    2, FALSE},
{"p4",           2, FALSE},
{"qml",          2, FALSE},
{"slang",        2, FALSE},
/* ---- Style 3: Shell / hash comments (#) ---- */
/* --- Python --- */
{"py",           3, FALSE},
{"python",       3, FALSE},
{"pyx",          3, FALSE}, /* Cython */
{"pxd",          3, FALSE},
{"pyi",          3, FALSE}, /* type stubs */
{"sage",         3, FALSE}, /* SageMath */
/* --- Shell --- */
{"sh",           3, FALSE},
{"bash",         3, FALSE},
{"zsh",          3, FALSE},
{"fish",         3, FALSE},
{"ksh",          3, FALSE},
{"csh",          3, FALSE},
{"tcsh",         3, FALSE},
{"dash",         3, FALSE},
{"nu",           3, FALSE}, /* Nushell */
/* --- Classic scripting --- */
{"perl",         3, FALSE},
{"pl",           3, FALSE},
{"pm",           3, FALSE},
{"ruby",         3, FALSE},
{"rb",           3, FALSE},
{"rake",         3, FALSE},
{"gemspec",      3, FALSE},
{"r",            3, FALSE},
{"rmd",          3, FALSE},
{"tcl",          3, FALSE},
{"awk",          3, FALSE},
{"sed",          3, FALSE},
/* --- Build / make --- */
{"makefile",     3, FALSE},
{"make",         3, FALSE},
{"mk",           3, FALSE},
{"cmake",        3, FALSE},
{"just",         3, FALSE},
{"justfile",     3, FALSE},
{"meson",        3, FALSE},
{"scons",        3, FALSE},
{"bazel",        3, FALSE},
{"bzl",          3, FALSE},

```

```

{"buck",          3, FALSE},
{"pants",         3, FALSE},
{"gnumakefile",   3, FALSE},
/* --- Scientific / modern --- */
{"julia",         3, FALSE},
{"jl",            3, FALSE},
{"nim",           3, FALSE},
{"nimble",        3, FALSE},
{"crystal",       3, FALSE},
{"cr",            3, FALSE},
{"elixir",        3, FALSE},
{"ex",            3, FALSE},
{"exs",           3, FALSE},
{"coco",          3, FALSE}, /* Coconut (Python superset) */
{"coffee",       3, FALSE}, /* CoffeeScript */
{"litcoffee",    3, FALSE},
/* --- Config / DevOps --- */
{"yaml",          3, FALSE},
{"yml",           3, FALSE},
{"toml",          3, FALSE},
{"dockerfile",    3, FALSE},
{"docker",        3, FALSE},
{"containerfile", 3, FALSE},
{"terraform",     3, FALSE},
{"tf",            3, FALSE},
{"tfvars",        3, FALSE},
{"hcl",           3, FALSE},
{"nix",           3, FALSE},
{"conf",          3, FALSE},
{"ini",           3, FALSE},
{"properties",    3, FALSE},
{"env",           3, FALSE},
{"editorconfig",  3, FALSE},
{"gitignore",     3, FALSE},
{"dockerignore",  3, FALSE},
{"flake8",        3, FALSE},
{"pylintrc",      3, FALSE},
{"powershell",    3, FALSE},
{"ps1",           3, FALSE},
{"psm1",          3, FALSE},
{"psd1",          3, FALSE},
/* --- Hardware description (# style) --- */
{"tcl",           3, FALSE},
{"xdc",           3, FALSE}, /* Xilinx constraints */
{"sdc",           3, FALSE}, /* Synopsys constraints */
/* --- Data / query (# style) --- */
{"gn",            3, FALSE}, /* GN build */
{"cfg",           3, FALSE},
{"pip",           3, FALSE},
{"cask",          3, FALSE}, /* Homebrew */
{"gemfile",       3, FALSE},
{"vagrantfile",   3, FALSE},
{"procfile",      3, FALSE},
/* ---- Style 4: double-dash comments (--) ---- */
{"lua",           4, FALSE},
{"luau",          4, FALSE}, /* Roblox Lua */
{"sql",           4, FALSE},
{"psql",          4, FALSE},
{"plsql",         4, FALSE},

```

```

{"plpgsql",      4, FALSE},
{"mysql",        4, FALSE},
{"sqlite",       4, FALSE},
{"hive",         4, FALSE},
{"presto",       4, FALSE},
{"sparksql",     4, FALSE},
{"hs",           4, FALSE},
{"haskell",      4, FALSE},
{"lhs",          4, FALSE},
{"elm",          4, FALSE},
{"purescript",   4, FALSE},
{"purs",         4, FALSE},
{"ada",          4, FALSE},
{"adb",          4, FALSE},
{"ads",          4, FALSE},
{"vhdl",         4, FALSE},
{"vhd",          4, FALSE},
{"lean",         4, FALSE},
{"lean4",        4, FALSE},
{"agda",         4, FALSE},
{"idr",          4, FALSE}, /* Idris */
{"idris",        4, FALSE},
{"erl",          4, FALSE}, /* Erlang uses % but -- is common in embedded SQL */
{"erlang",       4, FALSE},
/* ---- Style 5: HTML comments (<!-- -->) ---- */
{"html",         5, FALSE},
{"htm",          5, FALSE},
{"xml",          5, FALSE},
{"svg",          5, FALSE},
{"xhtml",        5, FALSE},
{"xsl",          5, FALSE},
{"xslt",         5, FALSE},
{"xsd",          5, FALSE},
{"dtd",          5, FALSE},
{"rss",          5, FALSE},
{"atom",         5, FALSE},
{"mathml",       5, FALSE},
{"plist",        5, FALSE},
{"xaml",         5, FALSE},
{"csproj",       5, FALSE},
{"fsproj",       5, FALSE},
{"vbproj",       5, FALSE},
{"vcxproj",      5, FALSE},
{"sln",          5, FALSE},
{"props",        5, FALSE},
{"targets",      5, FALSE},
{"nuspec",       5, FALSE},
{"resx",         5, FALSE},
{"wxml",         5, FALSE},
{"axml",         5, FALSE},
{"jspx",         5, FALSE},
{"jsp",          5, FALSE},
{"asp",          5, FALSE},
{"ascx",         5, FALSE},
{"aspx",         5, FALSE},
{"cshtml",       5, FALSE}, /* Razor */
{"vbhtml",       5, FALSE},
{"erb",          5, FALSE},
{"ejs",          5, FALSE},

```

```

{"njk",          5, FALSE}, /* Nunjucks */
{"hbs",          5, FALSE}, /* Handlebars */
{"mustache",     5, FALSE},
{"liquid",       5, FALSE},
{"twig",         5, FALSE},
{"blade",        5, FALSE}, /* Laravel Blade */
{"pug",          5, FALSE}, /* Pug/Jade */
{"slim",         5, FALSE},
{"haml",         5, FALSE},
{"md",           5, FALSE}, /* Markdown: HTML comments are valid */
{"markdown",     5, FALSE},
{"mdx",          5, FALSE},
{"rst",          5, FALSE}, /* reStructuredText (... is native but <!-- works) */
{"tex",          5, FALSE}, /* LaTeX: % is native but no style for it */
{"latex",        5, FALSE},
/* ---- Style 0: no comments (data formats) ---- */
{"json",         0, FALSE},
{"json5",        0, FALSE},
{"csv",          0, FALSE},
{"tsv",          0, FALSE},
{"txt",          0, FALSE},
{"text",         0, FALSE},
{"log",          0, FALSE},
{"diff",         0, FALSE},
{"patch",        0, FALSE},
/* Sentinel */
{NULL,           0, FALSE}
};

/* User-defined languages via @L command */
#define MAX_USER_LANGS 32
static LangEntry user_langs[MAX_USER_LANGS];
static char user_lang_names[MAX_USER_LANGS][64];
static int user_lang_count = 0;

static const LangEntry *find_lang(const char *name)
{
    int i;
    const LangEntry *p;
    /* User-defined languages override built-in */
    for (i = 0; i < user_lang_count; i++)
        if (strcmp(user_langs[i].name, name) == 0) return &user_langs[i];
    for (p = lang_table; p->name; p++)
        if (strcmp(p->name, name) == 0) return p;
    return NULL;
}

static void lang_to_flags(const char *lang,
                        unsigned char *comment_flag,
                        char *debug_flag,
                        char *location_flag)
{
    const LangEntry *e = find_lang(lang);
    if (e) {
        *comment_flag = e->comment;
        *debug_flag = e->line_directive;
        *location_flag = (e->comment != 0);
    } else {
        /* Unknown language: default to // comments, no #line */
    }
}

```

```

    *comment_flag = 2;
    *debug_flag = FALSE;
    *location_flag = TRUE;
}
}

```

149b · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `find_lang` 160b, `LangEntry` 160b, `lang_table` never used, `lang_to_flags` 160b, 161a, `user_langs` 156, `user_lang_count` 156, `user_lang_names` 156 · uses `FALSE` 23a, `TRUE` 23a, `uses` 121c

3.3.2 User-Defined Languages

The `@L` command lets users register new language mappings (or override built-in ones) directly in the document preamble. This is useful for languages that are not in the built-in table, or when the default comment style for a language is not what the user wants.

Syntax: `@L name style [+d]`

- *name* — the fence tag that will trigger this mapping (e.g. `nim`, `glsl`, `wgsl`).
- *style* — one of `//`, `#`, `--`, `/*`, or `<!--`, selecting the comment delimiters.
- `+d` — optional; if present, enables `#line` directive generation for this language.

Examples:

```

@L nim #           % Nim uses # comments
@L glsl // +d      % GLSL uses // and supports #line
@L c //           % override: use // instead of /* */ for C

```

Up to 32 user-defined languages may be registered. They are searched before the built-in table, so a definition like `@L c //` effectively overrides the built-in mapping for C.

The `@L` command is processed during `pass1` and silently skipped during weaving. It should appear in the document preamble (before any `@o` or `@d` directives that depend on it).

```

⟨names.c 156⟩ + ≡
void collect_lang_def(void)
{
    char name[64];
    char *p = name;
    char style[16];
    char *sp;
    int c;
    unsigned char comment = 0;
    char line_dir = FALSE;

    /* Skip whitespace after @L */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read language name */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - name) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';
}

```

```

if (name[0] == '\0') {
    fprintf(stderr, "%s: expected language name after %cL (%s, %d)\n",
        command_name, nw_char, source_name, source_line);
    while (c != EOF && c != '\n') c = source_get();
    return;
}

/* Skip whitespace */
while (c == ' ' || c == '\t')
    c = source_get();

/* Read comment style token */
sp = style;
while (c != EOF && c != ' ' && c != '\t' && c != '\n'
    && (sp - style) < 14) {
    *sp++ = c;
    c = source_get();
}
*sp = '\0';

/* Map style string to comment index */
if (strcmp(style, "//") == 0)    comment = 2;
else if (strcmp(style, "#") == 0)    comment = 3;
else if (strcmp(style, "--") == 0)    comment = 4;
else if (strcmp(style, "/*") == 0)    comment = 1;
else if (strcmp(style, "<!--") == 0)    comment = 5;
else {
    fprintf(stderr,
        "%s: unknown comment style '%s' for %cL (%s, %d)\n",
        command_name, style, nw_char, source_name, source_line);
    fprintf(stderr, " Valid styles: // # -- /* <!--\n");
    while (c != EOF && c != '\n') c = source_get();
    return;
}

/* Check for optional +d flag (#line directives) */
while (c == ' ' || c == '\t')
    c = source_get();
if (c == '+') {
    c = source_get();
    if (c == 'd') line_dir = TRUE;
}

/* Consume rest of line */
while (c != EOF && c != '\n') c = source_get();

/* Register the user language */
if (user_lang_count < MAX_USER_LANGS) {
    snprintf(user_lang_names[user_lang_count],
        sizeof(user_lang_names[user_lang_count]), "%s", name);
    user_langs[user_lang_count].name = user_lang_names[user_lang_count];
    user_langs[user_lang_count].comment = comment;
    user_langs[user_lang_count].line_directive = line_dir;
    user_lang_count++;
} else {
    fprintf(stderr, "%s: too many %cL definitions (max %d) (%s, %d)\n",
        command_name, nw_char, MAX_USER_LANGS,
        source_name, source_line);
}

```

```
}
```

156 · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `collect_lang_def` 43a, 145b · uses `command_name` 28c, `FALSE` 23a, `fprintf` 21b, `nw_char` 28ab, `source_get` 118a, `source_line` 117b, `source_name` 117b, `stderr` 21b, `TRUE` 23a, `user_langs` 149b, `user_lang_count` 149b, `user_lang_names` 149b

3.3.3 Weave Format Declaration

The `@W` command declares the weave output format in the source file. Valid tokens are `md` (Markdown) and `tex` (L^AT_EX). If the user has already specified a format on the command line (`-w md` or `-w tex`), the declaration is silently ignored.

```
<names.c 158> + ≡
void collect_weave_format(void)
{
    char token[64];
    char *p = token;
    int c;

    /* Skip whitespace after @W */
    c = source_get();
    while (c == ' ' || c == '\t')
        c = source_get();

    /* Read format token */
    while (c != EOF && c != ' ' && c != '\t' && c != '\n'
           && (p - token) < 62) {
        *p++ = c;
        c = source_get();
    }
    *p = '\0';

    /* Consume rest of line */
    while (c != EOF && c != '\n') c = source_get();

    if (token[0] == '\0') {
        fprintf(stderr, "%s: expected format after %cW (md or tex) (%s, %d)\n",
                command_name, nw_char, source_name, source_line);
        return;
    }

    /* If CLI already set the format, skip silently */
    if (weave_format_from_cli)
        return;

    if (strcmp(token, "md") == 0)
        weave_format = 2;
    else if (strcmp(token, "tex") == 0)
        weave_format = 1;
    else {
        fprintf(stderr, "%s: unknown weave format '%s' for %cW (%s, %d)\n",
                command_name, token, nw_char, source_name, source_line);
        fprintf(stderr, " Valid formats: md tex\n");
    }
}
```

158 · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `collect_weave_format` 43b, 145b · uses `command_name` 28c, `fprintf` 21b, `nw_char` 28ab, `source_get` 118a, `source_line` 117b, `source_name` 117b, `stderr` 21b, `weave_format` 27a, `weave_format_from_cli` 27a

3.3.4 Collecting File and Fragment Names

An output file name (@o) is terminated by whitespace. After reading the name we check for optional per-file flags (-d, -t, -i, -c) and then, if no explicit comment or debug flags were given, apply any pending language defaults from a preceding fence, %% marker, or @L definition.

<names.c 159a> + ≡

```
Name *collect_file_name(void)
{
    Name *new_name;
    char name[MAX_NAME_LEN];
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    while (isspace(c))
        c = source_get();
    while (isgraph(c)) {
        *p++ = c;
        c = source_get();
    }
    if (p == name) {
        fprintf(stderr, "%s: expected file name (%s, %d)\n",
            command_name, source_name, start_line);
        exit(-1);
    }
    *p = '\0';
    /* File names are always global. */
    new_name = name_add(&file_names, name, 0);
    <Handle optional per-file flags 159b>
    <Apply pending language if no explicit flags 160b>
    c2 = source_get();
    if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
        fprintf(stderr, "%s: expected %c{, %c[, or %c( after file name (%s, %d)\n",
            command_name, nw_char, nw_char, nw_char, source_name, start_line);
        exit(-1);
    }
    return new_name;
}
```

159a · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc ·

defines collect_file_name 45a, 60a, 84b, 145b · uses command_name 28c, exit 21b, file_names 144c, fprintf 21b, isgraph 21b, isspace 21b, MAX_NAME_LEN 23b, Name 144b, name_add 148b, nw_char 28ab, source_get 118a, source_line 117b, source_name 117b, stderr 21b

<Handle optional per-file flags 159b> ≡

```
{
    while (1) {
        while (isspace(c))
            c = source_get();
        if (c == '-') {
            c = source_get();
            do {
                switch (c) {
                    case 't': new_name->tab_flag = FALSE;
                        break;
                    case 'd': new_name->debug_flag = TRUE;
                        break;
                    case 'i': new_name->indent_flag = FALSE;
                        break;
```

```

        case 's': new_name->suppress_markers = TRUE;
                    break;
        case 'c': < Get comment delimiters 160a >
                    break;
        default : fprintf(stderr, "%s: unexpected per-file flag (%s, %d)\n",
                            command_name, source_name, source_line);
                    break;
    }
    c = source_get();
    } while (!isspace(c));
}
else break;
}
}

```

159b · used in 159a · uses [command_name 28c](#), [FALSE 23a](#), [fprintf 21b](#), [isspace 21b](#), [source_get 118a](#), [source_line 117b](#), [source_name 117b](#), [stderr 21b](#), [TRUE 23a](#)

The explicit per-file comment flags (`-cc`, `-c+`, `-cp`) select among the first three comment styles. These flags predate the auto-detection system; they still work and take precedence over any fence-based or `@L`-based setting.

The `-s` flag suppresses `#line` directives and section markers in the tangled output for that file, while still allowing language auto-detection to set the comment style and the `lang` tag (used by the Markdown weaver for syntax highlighting classes). This is useful for bootstrap source files or any output where tangle markers are unwanted.

< Get comment delimiters 160a > ≡

```

c = source_get();
if (c == 'c')
    new_name->comment_flag = 1;
else if (c == '+')
    new_name->comment_flag = 2;
else if (c == 'p')
    new_name->comment_flag = 3;
else
    fprintf(stderr, "%s: Unrecognised comment flag (%s, %d)\n",
            command_name, source_name, source_line);

```

160a · used in 159b · uses [command_name 28c](#), [fprintf 21b](#), [source_get 118a](#), [source_line 117b](#), [source_name 117b](#), [stderr 21b](#)

If there is a pending language tag from a fence or `%%%` marker, and the user has not specified explicit comment or debug flags, we apply the language defaults automatically.

< Apply pending language if no explicit flags 160b > ≡

```

if (pending_lang[0] != '\0') {
    strncpy(new_name->lang, pending_lang, 63);
    new_name->lang[63] = '\0';
    if (new_name->comment_flag == 0 && !new_name->debug_flag) {
        lang_to_flags(pending_lang,
                      &new_name->comment_flag,
                      &new_name->debug_flag,
                      &new_name->location_flag);
    }
    pending_lang[0] = '\0';
} else if (new_name->lang[0] == '\0') {
    const char *dot = strrchr(new_name->spelling, '.');
    if (dot && dot[1]) {
        const LangEntry *e = find_lang(dot + 1);
        if (e) {

```

```

    strncpy(new_name->lang, dot + 1, 63);
    new_name->lang[63] = '\0';
    if (new_name->comment_flag == 0 && !new_name->debug_flag) {
        lang_to_flags(new_name->lang,
                      &new_name->comment_flag,
                      &new_name->debug_flag,
                      &new_name->location_flag);
    }
}
}
}
if (new_name->suppress_markers) {
    new_name->debug_flag = FALSE;
    new_name->location_flag = FALSE;
}
}
160b · used in 159a · uses FALSE 23a, find_lang 149b, LangEntry 149b, lang_to_flags 149b, pending_lang 144c

```

For @d (macro) definitions, the language tag is stored and flags are inferred from @l, just as for output files. At expansion time, if a fragment carries an explicit @l tag its own flags override the parent’s—so @l txt before @d suppresses markers and newlines, enabling true inline expansion.

```

⟨ Apply pending language to macro 161a ⟩ ≡
if (pending_lang[0] != '\0') {
    strncpy(node->lang, pending_lang, 63);
    node->lang[63] = '\0';
    if (node->comment_flag == 0 && !node->debug_flag) {
        lang_to_flags(pending_lang,
                      &node->comment_flag,
                      &node->debug_flag,
                      &node->location_flag);
    }
    pending_lang[0] = '\0';
}
161a · used in 162b, 164a · uses lang_to_flags 149b, pending_lang 144c

```

3.3.5 Comment Styles

weft supports six comment styles, indexed 0–5. The arrays below provide the opening, continuation, and closing delimiters for each. Style 0 means “no comments”; the remaining five cover the vast majority of programming languages:

Index	Style	Begin	Mid	End
0	none	(empty)	(empty)	(empty)
1	C	/*	*	*/
2	C++	//	//	(empty)
3	Shell	#	#	(empty)
4	Lua/SQL	--	--	(empty)
5	HTML	<!--	(spaces)	-->

These are used both for block comments (@c) and for the Knuth-style section markers generated by the language auto-detection system.

```

⟨ Forward declarations for scraps.c 161b ⟩ + ≡
char * comment_begin[6] = { "", "/* ", "// ", "# ", "-- ", "<!-- "};
char * comment_mid[6] = { "", " * ", "// ", "# ", "-- ", " "};
char * comment_end[6] = { "", " */", "", "", "", " -->"};

```

161b · defined at 134cd, 141b, 161b, 176a · used in 122a · defines `comment_begin` 49a, 136c, 137a, 140a, `comment_end` 49a, 136c, 137a, 140a, `comment_mid` 49a

Name terminated by `\n` or `@{`; but keep skipping until `@{`

<names.c 162a> + ≡

```
Name *collect_macro_name(void)
{
    char name[MAX_NAME_LEN];
    char args[1000];
    char * arg[9];
    char * argp = args;
    int argc = 0;
    char *p = name;
    int start_line = source_line;
    int c = source_get(), c2;
    unsigned char sector = current_sector;

    if (c == '+') {
        sector = 0;
        c = source_get();
    }
    while (isspace(c))
        c = source_get();
    while (c != EOF) {
        Name * node;
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                        do
                            c = source_get();
                        while (c == ' ' || c == '\t');
                        break;
            case '\n': <Skip until scrap begins, then return name 164a>
            default:
                if (c==nw_char)
                {
                    <Check for terminating at-sequence and return name 162b>
                    break;
                }
                *p++ = c;
                c = source_get();
                break;
        }
    }
    fprintf(stderr, "%s: expected fragment name (%s, %d)\n",
            command_name, source_name, start_line);
    exit(-1);
    return NULL; /* unreachable return to avoid warnings on some compilers */
}
```

162a · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc ·

defines `collect_macro_name` 45b, 60b, 84c, 145b · uses `command_name` 28c, `current_sector` 44c, `exit` 21b, `fprintf` 21b, `isspace` 21b, `MAX_NAME_LEN` 23b, `Name` 144b, `nw_char` 28ab, `sector` 121c, `source_get` 118a, `source_line` 117b, `source_name` 117b, `stderr` 21b

<Check for terminating at-sequence and return name 162b> ≡

```
{
    c = source_get();
```

```

switch (c) {
  case '(':
  case '[':
  case '{': < Cleanup and install name 163d >
            < Apply pending language to macro 161a >
            return install_args(node, argc, arg);
  case '\\': < Enter the next argument 163a >
            break;
  default:
    if (c==nw_char)
    {
      *p++ = c;
      break;
    }
    fprintf(stderr,
             "%s: unexpected %c%c in fragment definition name (%s, %d)\n",
             command_name, nw_char, c, source_name, start_line);
    exit(-1);
}
}

```

162b · used in 162a · uses [command_name 28c](#), [exit 21b](#), [fprintf 21b](#), [nw_char 28ab](#), [source_get 118a](#), [source_name 117b](#), [stderr 21b](#)

< Enter the next argument 163a > ≡

```

arg[argc] = argp;
while ((c = source_get()) != EOF) {
  if (c==nw_char) {
    c2 = source_get();
    if (c2=='\\') {
      < Make this argument 163c >
      c = source_get();
      break;
    }
    else
      *argp++ = c2;
  }
  else
    *argp++ = c;
}

```

163a · used in 162b · uses [ARG_CHR 163b](#), [nw_char 28ab](#), [source_get 118a](#)

< Type declarations 163b > + ≡

```
#define ARG_CHR '\\001'
```

163b · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines [ARG_CHR 58c](#), 61a, 72g, 74b, 85a, 90d, 91b, 133b, 141ad, 163a, 166, 167b

< Make this argument 163c > ≡

```

if (argc < 9) {
  *argp++ = '\\000';
  argc += 1;
}

```

163c · used in 163a

< Cleanup and install name 163d > ≡

```
{
```

```

if (p > name && p[-1] == ' ')
    p--;
if (p - name > 3 && p[-1] == '.' && p[-2] == '.' && p[-3] == '.') {
    p[-3] = ' ';
    p -= 2;
}
if (p == name || name[0] == ' ') {
    fprintf(stderr, "%s: empty name (%s, %d)\n",
            command_name, source_name, source_line);
    exit(-1);
}
*p = '\0';
node = prefix_add(&macro_names, name, sector);
}

```

163d · used in 162b, 164a, 166 · uses [command_name 28c](#), [exit 21b](#), [fprintf 21b](#), [macro_names 144c](#), [prefix_add 146b](#), [sector 121c](#), [source_line 117b](#), [source_name 117b](#), [stderr 21b](#)

< Skip until scrap begins, then return name 164a > ≡

```

{
    do
        c = source_get();
    while (isspace(c));
    c2 = source_get();
    if (c != nw_char || (c2 != '{' && c2 != '(' && c2 != '[')) {
        fprintf(stderr, "%s: expected %c{ after fragment name (%s, %d)\n",
                command_name, nw_char, source_name, start_line);
        exit(-1);
    }
    < Cleanup and install name 163d >
    < Apply pending language to macro 161a >
    return install_args(node, argc, arg);
}

```

164a · used in 162a · uses [command_name 28c](#), [exit 21b](#), [fprintf 21b](#), [isspace 21b](#), [nw_char 28ab](#), [source_get 118a](#), [source_name 117b](#), [stderr 21b](#)

< Function prototypes 164b > + ≡

```
extern Name *install_args(Name *name, int argc, char *arg[9]);
```

164b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses [Name 144b](#)

< names.c 164c > + ≡

```

Name *install_args(Name * name, int argc, char *arg[9])
{
    int i;

    for (i = 0; i < argc; i++) {
        if (name->arg[i] == NULL)
            name->arg[i] = save_string(arg[i]);
    }
    return name;
}

```

164c · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · uses [Name 144b](#), [save_string 146a](#)

< Type declarations 164d > + ≡

```
typedef struct arglist
{Name * name;
struct arglist * args;
struct arglist * next;
} Arglist;
```

164d · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines Arglist 50c, 53b, 58c, 72g, 74b, 81b, 84a, 90d, 91b, 122b, 129b, 130c, 132de, 133abcd, 134bd, 139d, 140c, 141b, 145b, 164e, 165, 167cd, 168a, 169e, 170b, 171a, 175b · uses Name 144b, next 121b

⟨names.c 164e⟩ + ≡

```
Arglist * buildArglist(Name * name, Arglist * a)
{
    Arglist * args = (Arglist *)arena_getmem(sizeof(Arglist));

    args->args = a;
    args->next = NULL;
    args->name = name;
    return args;
}
```

164e · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines buildArglist 166, 167c, 168a · uses arena_getmem 182b, Arglist 164d, Name 144b, next 121b

Terminated by @>

⟨names.c 165⟩ + ≡

```
Arglist * collect_scrap_name(int current_scrap)
{
    char name[MAX_NAME_LEN];
    char *p = name;
    int c = source_get();
    unsigned char sector = current_sector;
    Arglist * head = NULL;
    Arglist ** tail = &head;

    if (c == '+')
    {
        sector = 0;
        c = source_get();
    }
    while (c == ' ' || c == '\t')
        c = source_get();
    while (c != EOF) {
        switch (c) {
            case '\t':
            case ' ': *p++ = ' ';
                    do
                        c = source_get();
                    while (c == ' ' || c == '\t');
                    break;
            default:
                if (c==nw_char)
                {
                    ⟨Look for end of scrap name and return 166⟩
                    break;
                }
                if (!isgraph(c)) {
                    fprintf(stderr,
```

```

        "%s: unexpected character in fragment name (%s, %d)\n",
        command_name, source_name, source_line);
    exit(-1);
}
*p++ = c;
c = source_get();
break;
}
}
fprintf(stderr, "%s: unexpected end of file (%s, %d)\n",
        command_name, source_name, source_line);
exit(-1);
return NULL; /* unreachable return to avoid warnings on some compilers */
}

165 · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc ·
defines collect_scrap_name 58c, 72g, 84a, 90d, 129b, 145b, 167e · uses Arglist 164d, command_name 28c, current_sector 44c,
exit 21b, fprintf 21b, isgraph 21b, MAX_NAME_LEN 23b, nw_char 28ab, sector 121c, source_get 118a, source_line 117b,
source_name 117b, stderr 21b

```

< Look for end of scrap name and return 166 > ≡

```

{
    Name * node;

    c = source_get();
    switch (c) {

        case '\\': {
            < Add plain string argument 167a >
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '1': case '2': case '3':
        case '4': case '5': case '6':
        case '7': case '8': case '9': {
            < Add a propagated argument 167b >
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '{': {
            < Add an inline scrap argument 167c >
        }
        *p++ = ARG_CHR;
        c = source_get();
        break;
        case '<':
            < Add macro call argument 167e >
            *p++ = ARG_CHR;
            c = source_get();
            break;
        case '(':
            scrap_name_has_parameters = 1;
            < Cleanup and install name 163d >
            return buildArglist(node, head);
        case '>':
            scrap_name_has_parameters = 0;
            < Cleanup and install name 163d >
    }
}

```

```

        return buildArglist(node, head);

default:
    if (c==nw_char)
    {
        *p++ = c;
        c = source_get();
        break;
    }
    fprintf(stderr,
        "%s: unexpected %c%c in fragment invocation name (%s, %d)\n",
        command_name, nw_char, c, source_name, source_line);
    exit(-1);
}
}

```

166 · used in 165 · uses ARG_CHR 163b, buildArglist 164e, command_name 28c, exit 21b, fprintf 21b, Name 144b, nw_char 28ab, source_get 118a, source_line 117b, source_name 117b, stderr 21b

A plain string argument has no name and a string as a value.

⟨ *Add plain string argument* 167a ⟩ ≡

```

char buff[MAX_NAME_LEN];
char * s = buff;
int c, c2;

while ((c = source_get()) != EOF) {
    if (c==nw_char) {
        c2 = source_get();
        if (c2=='\''')
            break;
        *s++ = c2;
    }
    else
        *s++ = c;
}
*s = '\000';

```

⟨ *Add buff to current arg list* 168a ⟩

167a · used in 166 · uses MAX_NAME_LEN 23b, nw_char 28ab, source_get 118a

A parameter argument propagated into an interior fragment has no name and a string of ARG_CHAR followed by a digit as value.

⟨ *Add a propagated argument* 167b ⟩ ≡

```

char buff[3];
buff[0] = ARG_CHR;
buff[1] = c;
buff[2] = '\000';

```

⟨ *Add buff to current arg list* 168a ⟩

167b · used in 166 · uses ARG_CHR 163b

⟨ *Add an inline scrap argument* 167c ⟩ ≡

```

int s = collect_scrap();
Scrap_Node * d = (Scrap_Node *)arena_getmem(sizeof(Scrap_Node));
d->scrap = s;
d->quoted = 0;
d->next = NULL;
*tail = buildArglist((Name *)1, (Arglist *)d);

```

```
tail = &(*tail)->next;
```

167c · used in 166 · uses [arena_getmem](#) 182b, [Arglist](#) 164d, [buildArglist](#) 164e, [collect_scrap](#) 125f, [Name](#) 144b, [next](#) 121b, [Scrap_Node](#) 144a

⟨ *Type declarations* 167d ⟩ + ≡

```
typedef struct embed {
    Scrap_Node * defs;
    Arglist * args;
} Embed_Node;
```

167d · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines [Embed_Node](#) 50c, 133b, 141c · uses [Arglist](#) 164d, [defs](#) 121c, [Scrap_Node](#) 144a

⟨ *Add macro call argument* 167e ⟩ ≡

```
*tail = collect_scrap_name(current_scrap);
if (current_scrap >= 0)
    add_to_use((*tail)->name, current_scrap);
tail = &(*tail)->next;
```

167e · used in 166 · uses [add_to_use](#) 129d, [collect_scrap_name](#) 165, [next](#) 121b

⟨ *Add buff to current arg list* 168a ⟩ ≡

```
*tail = buildArglist(NULL, (Arglist *)save_string(buff));
tail = &(*tail)->next;
```

168a · used in 167ab · uses [Arglist](#) 164d, [buildArglist](#) 164e, [next](#) 121b, [save_string](#) 146a

⟨ *names.c* 168b ⟩ + ≡

```
static Scrap_Node *reverse(Scrap_Node *a); /* a forward declaration */
```

```
void reverse_lists(Name *names)
{
    while (names) {
        reverse_lists(names->llink);
        names->defs = reverse(names->defs);
        names->uses = reverse(names->uses);
        names = names->rlink;
    }
}
```

168b · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines [reverse_lists](#) 50a, 145b · uses [defs](#) 121c, [Name](#) 144b, [reverse](#) 168c, [Scrap_Node](#) 144a, [uses](#) 121c

Just for fun, here's a non-recursive version of the traditional list reversal code. Note that it reverses the list in place; that is, it does no new allocations.

⟨ *names.c* 168c ⟩ + ≡

```
static Scrap_Node *reverse(Scrap_Node *a)
{
    if (a) {
        Scrap_Node *b = a->next;
        a->next = NULL;
        while (b) {
            Scrap_Node *c = b->next;
            b->next = a;
            a = b;
            b = c;
        }
    }
}
```

```

    return a;
}

```

168c · defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc · defines `reverse` 27a, 30, 168b · uses `next` 121b, `Scrap_Node` 144a

3.4 Searching for Index Entries

Given the array of scraps and a set of index entries, we need to search all the scraps for occurrences of each entry. The obvious approach to this problem would be quite expensive for large documents; however, there is an interesting paper describing an efficient solution [1].

```

< scraps.c 168d > + ≡
typedef struct name_node {
    struct name_node *next;
    Name *name;
} Name_Node;

```

168d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `Name_Node` 169a, 173ab, 174 · uses `Name` 144b, `next` 121b

```

< scraps.c 169a > + ≡
typedef struct goto_node {
    Name_Node *output;           /* list of words ending in this state */
    struct move_node *moves;     /* list of possible moves */
    struct goto_node *fail;      /* and where to go when no move fits */
    struct goto_node *next;      /* next goto node with same depth */
} Goto_Node;

```

169a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `Goto_Node` 169bcd, 172c, 173ab, 174 · uses `Name_Node` 168d, `next` 121b

```

< scraps.c 169b > + ≡
typedef struct move_node {
    struct move_node *next;
    Goto_Node *state;
    char c;
} Move_Node;

```

169b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `Move_Node` 169d, 173ab · uses `Goto_Node` 169a, `next` 121b

```

< scraps.c 169c > + ≡
static Goto_Node *root[256];
static int max_depth;
static Goto_Node **depths;

```

169c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `depths` 172c, 173ab, `max_depth` 172c, 173ab, `root` 103b, 172c, 173ab, 174, 176ac · uses `Goto_Node` 169a

```

< scraps.c 169d > + ≡
static Goto_Node *goto_lookup(char c, Goto_Node *g)
{
    Move_Node *m = g->moves;
    while (m && m->c != c)
        m = m->next;
    if (m)
        return m->state;
}

```

```

    else
        return NULL;
}

```

169d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines goto_lookup 173ab, 174 · uses Goto_Node 169a, Move_Node 169b, next 121b

3.4.1 Retrieving scrap uses

```

< scraps.c 169e > + ≡
typedef struct ArgMgr_s
{
    char * pv;
    char * bgn;
    Arglist * arg;
    struct ArgMgr_s * old;
} ArgMgr;

```

169e · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines ArgMgr 170abc, 171c · uses Arglist 164d

```

< scraps.c 170a > + ≡
typedef struct ArgManager_s
{
    Manager * m;
    ArgMgr * a;
} ArgManager;

```

170a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines ArgManager 170bc, 171c, 172c, 174, 179a · uses ArgMgr 169e, Manager 125c

```

< scraps.c 170b > + ≡
static void
pushArglist(ArgManager * mgr, Arglist * a)
{
    ArgMgr * b = malloc(sizeof(ArgMgr));

    if (b == NULL)
    {
        fprintf(stderr, "Can't allocate space for an argument manager\n");
        exit(EXIT_FAILURE);
    }
    b->pv = b->bgn = NULL;
    b->arg = a;
    b->old = mgr->a;
    mgr->a = b;
}

```

170b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines pushArglist 171a, 175b · uses Arglist 164d, ArgManager 170a, ArgMgr 169e, exit 21b, fprintf 21b, malloc 21b, stderr 21b

```

< scraps.c 170c > + ≡
static char argpop(ArgManager * mgr)
{
    while (mgr->a != NULL)
    {
        ArgMgr * a = mgr->a;
    }
}

```

```

    < Perhaps —return— a character from the current arg 170d >
    < Perhaps start a new arg 171a >
    < Otherwise pop the current arg 171b >
}

return (pop(mgr->m));
}
170c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde,
170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines argpop 174, 175ab · uses ArgManager 170a, ArgMgr 169e, pop 130b

```

We separate individual arguments using spaces.

```

< Perhaps —return— a character from the current arg 170d > ≡
if (a->pv != NULL)
{
    char c = *a->pv++;

    if (c != '\0')
        return c;
    a->pv = NULL;
    return ' ';
}
170d · used in 170c

```

I'm pretty sure that only the first case is ever used. I don't think the others can occur in this context, which makes the whole —ArgManager— thing doubtful.

```

< Perhaps start a new arg 171a > ≡
if (a->arg) {
    Arglist * b = a->arg;

    a->arg = b->next;
    if (b->name == NULL) {
        a->bgn = a->pv = (char *)b->args;
    } else if (b->name == (Name *)1) {
        a->bgn = a->pv = "{Embedded Scrap}";
    } else {
        pushArglist(mgr, b->args);
    }
}
171a · used in 170c · uses Arglist 164d, Name 144b, next 121b, pushArglist 170b

```

```

< Otherwise pop the current arg 171b > ≡
} else {
    mgr->a = a->old;
    free(a);
}
171b · used in 170c

```

```

< scraps.c 171c > + ≡
static char
prev_char(ArgManager * mgr, int n)
{
    char c = '\0';
    ArgMgr * a = mgr->a;
    Manager * m = mgr->m;

```

```

    if (a != NULL) {
        < Get the nth previous character from an argument 171d >
    } else {
        < Get the nth previous character from a scrap 172a >
    }

    return c;
}

```

171c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `prev_char` 179a · uses `ArgManager` 170a, `ArgMgr` 169e, `Manager` 125c

```

< Get the nth previous character from an argument 171d > ≡
if (a->pv && a->pv - n >= a->bgn)
    c = *a->pv;
else if (a->bgn) {
    int j = strlen(a->bgn) + 1;

    if (n >= j)
        c = a->bgn[j - n];
    else
        c = ' ';
}

```

171d · used in 171c · uses `strlen` 21b

When the offset `k` falls within the current slab we index it directly. When `k` is negative we must reach into the previous slab: since `k` is already negative, `SLAB_SIZE + k` yields the correct index (e.g. $k = -1$ gives `chars[1023]`, the last element).

```

< Get the nth previous character from a scrap 172a > ≡
int k = m->index - n - 2;

if (k >= 0)
    c = m->scrap->chars[k];
else if (m->prev)
    c = m->prev->chars[SLAB_SIZE + k];

```

172a · used in 171c · uses `SLAB_SIZE` 121b

3.4.2 Building the Automata

```

< Function prototypes 172b > + ≡
extern void search(void);

```

172b · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `search` 172c

```

< scraps.c 172c > + ≡
static void build_gotos(Name *tree);
static int reject_match(Name *name, char post, ArgManager *reader);

void search(void)
{
    int i;
    for (i=0; i<256; i++)
        root[i] = NULL;
    max_depth = 10;
}

```

```

depths = (Goto_Node **) arena_getmem(max_depth * sizeof(Goto_Node *));
for (i=0; i<max_depth; i++)
    depths[i] = NULL;
build_gotos(user_names);
< Build failure functions 173b >
< Search scraps 174 >
}

172c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde,
170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines search 30, 41c, 172b, 184 · uses arena_getmem 182b, ArgManager 170a,
build_gotos 172d, depths 169c, Goto_Node 169a, max_depth 169c, Name 144b, reject_match 179a, root 169c, user_names 144c

< scraps.c 172d > + ≡
static void build_gotos(Name *tree)
{
    while (tree) {
        < Extend goto graph with tree->spelling 173a >
        build_gotos(tree->rlink);
        tree = tree->llink;
    }
}

172d · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d,
169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines build_gotos 172c · uses Name 144b

< Extend goto graph with tree->spelling 173a > ≡
{
    int depth = 2;
    char *p = tree->spelling;
    char c = *p++;
    Goto_Node *q = root[(unsigned char)c];
    Name_Node * last;
    if (!q) {
        q = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
        root[(unsigned char)c] = q;
        q->moves = NULL;
        q->fail = NULL;
        q->output = NULL;
        q->next = depths[1];
        depths[1] = q;
    }
    while ((c = *p++)) {
        Goto_Node *new_node = goto_lookup(c, q);
        if (!new_node) {
            Move_Node *new_move = (Move_Node *) arena_getmem(sizeof(Move_Node));
            new_node = (Goto_Node *) arena_getmem(sizeof(Goto_Node));
            new_node->moves = NULL;
            new_node->fail = NULL;
            new_node->output = NULL;
            new_move->state = new_node;
            new_move->c = c;
            new_move->next = q->moves;
            q->moves = new_move;
            if (depth == max_depth) {
                int i;
                Goto_Node **new_depths =
                    (Goto_Node **) arena_getmem(2*depth*sizeof(Goto_Node *));
                max_depth = 2 * depth;
                for (i=0; i<depth; i++)

```

```

        new_depths[i] = depths[i];
        depths = new_depths;
        for (i=depth; i<max_depth; i++)
            depths[i] = NULL;
    }
    new_node->next = depths[depth];
    depths[depth] = new_node;
}
q = new_node;
depth++;
}
last = q->output;
q->output = (Name_Node *) arena_getmem(sizeof(Name_Node));
q->output->next = last;
q->output->name = tree;
}

```

173a · used in 172d · uses [arena_getmem 182b](#), [depths 169c](#), [goto_lookup 169d](#), [Goto_Node 169a](#), [max_depth 169c](#), [Move_Node 169b](#), [Name_Node 168d](#), [next 121b](#), [root 169c](#)

⟨ Build failure functions 173b ⟩ ≡

```

{
    int depth;
    for (depth=1; depth<max_depth; depth++) {
        Goto_Node *r = depths[depth];
        while (r) {
            Move_Node *m = r->moves;
            while (m) {
                char a = m->c;
                Goto_Node *s = m->state;
                Goto_Node *state = r->fail;
                while (state && !goto_lookup(a, state))
                    state = state->fail;
                if (state)
                    s->fail = goto_lookup(a, state);
                else
                    s->fail = root[(unsigned char)a];
                if (s->fail) {
                    Name_Node *p = s->fail->output;
                    while (p) {
                        Name_Node *q = (Name_Node *) arena_getmem(sizeof(Name_Node));
                        q->name = p->name;
                        q->next = s->output;
                        s->output = q;
                        p = p->next;
                    }
                }
                m = m->next;
            }
            r = r->next;
        }
    }
}

```

173b · used in 172c · uses [arena_getmem 182b](#), [depths 169c](#), [goto_lookup 169d](#), [Goto_Node 169a](#), [max_depth 169c](#), [Move_Node 169b](#), [Name_Node 168d](#), [next 121b](#), [root 169c](#)

3.4.3 Searching the Scraps

⟨ *Search scraps* 174 ⟩ ≡

```
{
  for (i=1; i<scraps; i++) {
    char c, last = '\0';
    Manager rd;
    ArgManager reader;
    Goto_Node *state = NULL;
    rd.prev = NULL;
    rd.scrap = scrap_array(i).slab;
    rd.index = 0;
    reader.m = &rd;
    reader.a = NULL;
    c = argpop(&reader);
    while (c) {
      while (state && !goto_lookup(c, state))
        state = state->fail;
      if (state)
        state = goto_lookup(c, state);
      else
        state = root[(unsigned char)c];
      ⟨ Skip over at at 175a ⟩
      ⟨ Skip over a scrap use 175b ⟩
      ⟨ Skip over a block comment 49d ⟩
      last = c;
      c = argpop(&reader);
      if (state && state->output) {
        Name_Node *p = state->output;
        do {
          Name *name = p->name;
          if (!reject_match(name, c, &reader) &&
              scrap_array(i).sector == name->sector &&
              (!name->uses || name->uses->scrap != i)) {
            Scrap_Node *new_use =
              (Scrap_Node *) arena_getmem(sizeof(Scrap_Node));
            new_use->scrap = i;
            new_use->next = name->uses;
            name->uses = new_use;
            if (!scrap_is_in(name->defs, i))
              add_uses(&(scrap_array(i).uses), name);
          }
          p = p->next;
        } while (p);
      }
    }
  }
}
```

174 · used in 172c · uses [add_uses](#) 176c, [arena_getmem](#) 182b, [ArgManager](#) 170a, [argpop](#) 170c, [defs](#) 121c, [goto_lookup](#) 169d, [Goto_Node](#) 169a, [Manager](#) 125c, [Name](#) 144b, [Name_Node](#) 168d, [next](#) 121b, [reject_match](#) 179a, [root](#) 169c, [scraps](#) 122a, [scrap_array](#) 122a, [scrap_is_in](#) 176b, [Scrap_Node](#) 144a, [sector](#) 121c, [slab](#) 121c, [uses](#) 121c

⟨ *Skip over at at* 175a ⟩ ≡

```
if (last == nw_char && c == nw_char)
{
  last = '\0';
  c = argpop(&reader);
}
```

175a · used in 174 · uses [argpop](#) 170c, [nw_char](#) 28ab

⟨ *Skip over a scrap use* 175b ⟩ ≡

```
if (last == nw_char && c == '<')
{
    char buf[MAX_NAME_LEN];
    char * p = buf;
    Arglist * args;
    int idx;

    c = argpop(&reader);
    while ((c = argpop(&reader)) != nw_char)
        *p++ = c;
    c = argpop(&reader);
    *p = '\0';
    if (sscanf(buf, "%d", &idx) != 1) {
        fprintf(stderr, "%s: found an internal problem (3)\n", command_name);
        exit(-1);
    }
    args = arglist_lookup(idx);
    pushArglist(&reader, args);
}
```

175b · used in 174 · uses Arglist 164d, arglist_lookup 134d, argpop 170c, command_name 28c, exit 21b, fprintf 21b, MAX_NAME_LEN 23b, nw_char 28ab, pushArglist 170b, stderr 21b

⟨ *Forward declarations for scraps.c* 176a ⟩ + ≡

```
static void add_uses(Uses **root, Name *name);
static int scrap_is_in(Scrap_Node *list, int i);
```

176a · defined at 134cd, 141b, 161b, 176a · used in 122a · uses add_uses 176c, Name 144b, root 169c, scrap_is_in 176b, Scrap_Node 144a, Uses 176d

⟨ scraps.c 176b ⟩ + ≡

```
static int scrap_is_in(Scrap_Node * list, int i)
{
    while (list != NULL) {
        if (list->scrap == i)
            return TRUE;
        list = list->next;
    }
    return FALSE;
}
```

176b · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines scrap_is_in 174, 176a · uses FALSE 23a, next 121b, Scrap_Node 144a, TRUE 23a

⟨ scraps.c 176c ⟩ + ≡

```
static void add_uses(Uses * * root, Name *name)
{
    int cmp;
    Uses *p, **q = root;

    while ((p = *q, p != NULL)
        && (cmp = robs_strcmp(p->defn->spelling, name->spelling)) < 0)
        q = &(p->next);
    if (p == NULL || cmp > 0)
```

```

    {
        Uses *new_use = arena_getmem(sizeof(Uses));
        new_use->next = p;
        new_use->defn = name;
        *q = new_use;
    }
}

```

176c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines add_uses 128b, 129a, 174, 176a · uses arena_getmem 182b, Name 144b, next 121b, robs_strcmp 147c, root 169c, Uses 176d

⟨ *Type declarations* 176d ⟩ + ≡

```

typedef struct uses {
    struct uses *next;
    Name *defn;
} Uses;

```

176d · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines Uses 97ab, 98b, 100, 102b, 122b, 123de, 176acf, 178a · uses Name 144b, next 121b, uses 121c

⟨ *Function prototypes* 176e ⟩ + ≡

```

extern void format_uses_refs(FILE *, int);

```

176e · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses FILE 21b, format_uses_refs 176f

⟨ *scraps.c* 176f ⟩ + ≡

```

void
format_uses_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).uses;
    if (p != NULL)
        ⟨ Write uses references 177a ⟩
}

```

176f · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines format_uses_refs 60ab, 176e · uses FILE 21b, scrap_array 122a, Uses 176d, uses 121c

⟨ *Write uses references* 177a ⟩ ≡

```

{
    char join = ' ';
    fputs("\\WEFTdot \\WEFTtxtIdentsUsed\\nobreak\\", tex_file);
    do {
        ⟨ Write one use reference 177b ⟩
        join = ',';
        p = p->next;
    }while (p != NULL);
}

```

177a · used in 176f · uses fputs 21b, next 121b

⟨ *Write one use reference* 177b ⟩ ≡

```

Name * name = p->defn;
Scrap_Node *defs = name->defs;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c {\\WEFTtint{weftkw}\\verb%c%s%c}\\nobreak\\ ",

```

```

        join, nw_char, name->spelling, nw_char);
if (defs)
{
    do {
        < Write one referenced scrap 177c >
        first = FALSE;
        defs = defs->next;
    }while (defs!= NULL);
}
else
{
    fputs("\\WEFTnotglobal", tex_file);
}

```

177b · used in 177a · uses [defs 121c](#), [FALSE 23a](#), [first 182a](#), [fprintf 21b](#), [fputs 21b](#), [Name 144b](#), [next 121b](#), [nw_char 28ab](#), [page 121c](#), [Scrap_Node 144a](#), [TRUE 23a](#)

```

< Write one referenced scrap 177c > ≡
fputs("\\WEFTlink{weft", tex_file);
write_scrap_ref(tex_file, defs->scrap, -1, &page);
fputs("{}{", tex_file);
write_scrap_ref(tex_file, defs->scrap, first, &page);
fputs("}", tex_file);

```

177c · used in 177b, 178c · uses [defs 121c](#), [first 182a](#), [fputs 21b](#), [page 121c](#), [write_scrap_ref 123b](#)

```

< Function prototypes 177d > + ≡
extern void format_defs_refs(FILE *, int);

```

177d · defined at [41a](#), [53a](#), [68a](#), [70b](#), [79a](#), [81a](#), [113ab](#), [117a](#), [118b](#), [122b](#), [124a](#), [130a](#), [133a](#), [142a](#), [145b](#), [147b](#), [164b](#), [172b](#), [176e](#), [177d](#), [179d](#), [181d](#) · used in [21a](#) · uses [FILE 21b](#), [format_defs_refs 178a](#)

```

< scraps.c 178a > + ≡

```

```

void
format_defs_refs(FILE * tex_file, int scrap)
{
    Uses * p = scrap_array(scrap).defs;
    if (p != NULL)
        < Write defs references 178b >
}

```

178a · defined at [25b](#), [121bc](#), [122a](#), [123abcde](#), [124bcdef](#), [125cdef](#), [129d](#), [130bc](#), [131](#), [132abcde](#), [133d](#), [134b](#), [142b](#), [168d](#), [169abcde](#), [170abc](#), [171c](#), [172cd](#), [176bcf](#), [178ad](#), [179ac](#) · defines [format_defs_refs 60ab](#), [177d](#) · uses [defs 121c](#), [FILE 21b](#), [scrap_array 122a](#), [Uses 176d](#)

```

< Write defs references 178b > ≡
{
    char join = ' ';
    fputs("\\WEFTdot \\WEFTtxtIdentsDefed\\nobreak\\", tex_file);
    do {
        < Write one def reference 178c >
        join = ',';
        p = p->next;
    }while (p != NULL);
}

```

178b · used in 178a · uses [fputs 21b](#), [next 121b](#)

```

< Write one def reference 178c > ≡

```

```

Name * name = p->defn;
Scrap_Node *defs = name->uses;
int first = TRUE, page = -1;
fprintf(tex_file,
        "%c {\\WEFTtint{weftkw}\\verb%c%s%c}\\nobreak\\ ",
        join, nw_char, name->spelling, nw_char);
if (defs == NULL
    || (defs->scrap == scrap && defs->next == NULL)) {
    fputs("\\WEFTtxtIdentsNotUsed", tex_file);
}
else {
    do {
        if (defs->scrap != scrap) {
            < Write one referenced scrap 177c >
            first = FALSE;
        }
        defs = defs->next;
    }while (defs!= NULL);
}

```

178c · used in 178b · uses [defs 121c](#), [FALSE 23a](#), [first 182a](#), [fprintf 21b](#), [fputs 21b](#), [Name 144b](#), [next 121b](#), [nw_char 28ab](#), [page 121c](#), [Scrap_Node 144a](#), [TRUE 23a](#), uses [121c](#)

Rejecting Matches

A problem with simple substring matching is that the string “he” would match longer strings like “she” and “her.” Norman Ramsey suggested examining the characters occurring immediately before and after a match and rejecting the match if it appears to be part of a longer token. Of course, the concept of *token* is language-dependent, so we may be occasionally mistaken. For the present, we’ll consider the mechanism an experiment.

```

< scraps.c 178d > + ≡
#define sym_char(c) (isalnum(c) || (c) == ' _')

static int op_char(char c)
{
    switch (c) {
        case '!': case '#': case '%': case '$': case '^':
        case '&': case '*': case '-': case '+': case '=': case '/':
        case '|': case '~': case '<': case '>':
            return TRUE;
        default:
            return c==nw_char ? TRUE : FALSE;
    }
}

```

178d · defined at [25b](#), [121bc](#), [122a](#), [123abcde](#), [124bedef](#), [125cdef](#), [129d](#), [130bc](#), [131](#), [132abcde](#), [133d](#), [134b](#), [142b](#), [168d](#), [169abcde](#), [170abc](#), [171c](#), [172cd](#), [176bcf](#), [178ad](#), [179ac](#) · defines [op_char 179a](#), [sym_char 179a](#) · uses [FALSE 23a](#), [nw_char 28ab](#), [TRUE 23a](#)

```

< scraps.c 179a > + ≡
static int reject_match(Name *name, char post, ArgManager *reader)
{
    int len = strlen(name->spelling);
    char first = name->spelling[0];
    char last = name->spelling[len - 1];
    char prev = prev_char(reader, len);
    if (sym_char(last) && sym_char(post)) return TRUE;
    if (sym_char(first) && sym_char(prev)) return TRUE;
}

```

```

    if (op_char(last) && op_char(post)) return TRUE;
    if (op_char(first) && op_char(prev)) return TRUE;
    return FALSE; /* Here is 179a-01 */
}

```

179a · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `reject_match` 172c, 174 · uses `ArgManager` 170a, `FALSE` 23a, `first` 182a, `Name` 144b, `op_char` 178d, `prev_char` 171c, `strlen` 21b, `sym_char` 178d, `TRUE` 23a

3.5 Labels

Refer to 179b-01. And another one 179a-01.

```

< Get label from 179b > ≡
char label_name[MAX_NAME_LEN];
char * p = label_name;
while (c = @1, c != nw_char) /* Here is 179b-01 */
    *p++ = c;
*p = '\0';
c = @1;

```

179b · used in 72a, 127a, 139a · uses `MAX_NAME_LEN` 23b, `nw_char` 28ab

```

< scraps.c 179c > + ≡
void
write_label(char label_name[], FILE * file)
< Search for label(< Write the label to file 180a>,< Complain about missing label 180b>) 180g >

```

179c · defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac · defines `write_label` 72a, 139a, 179d · uses `FILE` 21b

```

< Function prototypes 179d > + ≡
void write_label(char label_name[], FILE * file);

```

179d · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `FILE` 21b, `write_label` 179c

```

< Write the label to file 180a > ≡
write_single_scrap_ref(file, lbl->scrap);
fprintf(file, "-%02d", lbl->seq);
180a · used in 179c · uses fprintf 21b, write_single_scrap_ref 123c

```

```

< Complain about missing label 180b > ≡
fprintf(stderr, "Can't find label %s.\n", label_name);
180b · used in 179c · uses fprintf 21b, stderr 21b

```

```

< Save label to label store 180c > ≡
if (label_name[0])
< Search for label(< Complain about duplicate labels 180e>,< Create a new label entry 180d>) 180g >
else
{
    < Complain about empty label 180f >
}

```

180c · used in 127a

```

< Create a new label entry 180d > ≡

```

```

lbl = (label_node *)arena_getmem(sizeof(label_node) + (p - label_name));
lbl->left = lbl->right = NULL;
strcpy(lbl->name, label_name);
lbl->scrap = current_scrap;
lbl->seq = ++lblseq;
*plbl = lbl;

```

180d · used in 180c · uses [arena_getmem 182b](#), [label_node 181b](#)

⟨ *Complain about duplicate labels* 180e ⟩ ≡

```
fprintf(stderr, "Duplicate label %s.\n", label_name);
```

180e · used in 180c · uses [fprintf 21b](#), [stderr 21b](#)

⟨ *Complain about empty label* 180f ⟩ ≡

```
fprintf(stderr, "Empty label.\n");
```

180f · used in 180c · uses [fprintf 21b](#), [stderr 21b](#)

⟨ *Search for label(Found,Notfound)* 180g ⟩ ≡

```

{
    label_node * * plbl = &label_tab;
    for (;;)
    {
        label_node * lbl = *plbl;

        if (lbl)
        {
            int cmp = label_name[0] - lbl->name[0];

            if (cmp == 0)
                cmp = strcmp(label_name + 1, lbl->name + 1);
            if (cmp < 0)
                plbl = &lbl->left;
            else if (cmp > 0)
                plbl = &lbl->right;
            else
            {
                Found
                break;
            }
        }
        else
        {
            Notfound
            break;
        }
    }
}

```

180g · used in 179c, 180c · uses [label_node 181b](#), [label_tab 181a](#)

⟨ *global.c* 181a ⟩ + ≡

```
label_node * label_tab = NULL;
```

181a · defined at 25e, 181a · defines [label_tab 180g](#), 181c · uses [label_node 181b](#)

⟨ *Type declarations* 181b ⟩ + ≡

```

typedef struct l_node
{
    struct l_node * left, * right;

```

```

    int scrap, seq;
    char name[1];
} label_node;

```

181b · defined at 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b · used in 21a · defines `label_node` 180dg, 181ac

⟨ *Global variable declarations* 181c ⟩ + ≡

```
extern label_node * label_tab;
```

181c · defined at 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c · used in 21a · uses `label_node` 181b, `label_tab` 181a

3.6 Memory Management

I manage memory using a simple scheme inspired by Hanson’s idea of *arenas* [2]. Basically, I allocate all the storage required when processing a source file (primarily for names and scraps) using calls to `arena_getmem(n)`, where `n` specifies the number of bytes to be allocated. When the storage is no longer required, the entire arena is freed with a single call to `arena_free()`. Both operations are quite fast.

⟨ *Function prototypes* 181d ⟩ + ≡

```
extern void *arena_getmem(size_t n);
extern void arena_free(void);
extern void arglist_reset(void);

```

181d · defined at 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d · used in 21a · uses `arena_free` 183, `arena_getmem` 182b, `arglist_reset` 134d, `size_t` 21b

⟨ *arena.c* 181e ⟩ + ≡

```
typedef struct chunk {
    struct chunk *next;
    char *limit;
    char *avail;
} Chunk;

```

181e · defined at 25d, 181e, 182ab, 183 · defines `Chunk` 182acd · uses `next` 121b

We define an empty chunk called `first`. The variable `arena` points at the current chunk of memory; it’s initially pointed at `first`. As soon as some storage is required, a “real” chunk of memory will be allocated and attached to `first->next`; storage will be allocated from the new chunk (and later chunks if necessary).

⟨ *arena.c* 182a ⟩ + ≡

```
static Chunk first = { NULL, NULL, NULL };
static Chunk *arena = &first;

```

182a · defined at 25d, 181e, 182ab, 183 · defines `arena` 182bcd, 183, `first` 30, 62e, 97c, 98ad, 100, 101, 104b, 122b, 123b, 177bc, 178c, 179a, 183, 184 · uses `Chunk` 181e

3.6.1 Allocating Memory

The routine `arena_getmem(n)` returns a pointer to (at least) `n` bytes of memory. Note that `n` is rounded up to ensure that returned pointers are always aligned. We align to the nearest 8-byte segment, since that’ll satisfy the more common 2-byte and 4-byte alignment restrictions too.

⟨ *arena.c* 182b ⟩ + ≡

```
void *arena_getmem(size_t n)
{
    char *q;
    char *p = arena->avail;
    n = (n + 7) & ~7;          /* ensuring alignment to 8 bytes */

```

```

q = p + n;
if (q <= arena->limit) {
    arena->avail = q;
    return p;
}
< Find a new chunk of memory 182c >
}

```

182b · defined at 25d, 181e, 182ab, 183 · defines `arena_getmem` 35b, 39a, 45c, 52a, 123a, 125d, 126a, 128b, 129ad, 133bc, 146a, 149a, 164e, 167c, 172c, 173ab, 174, 176c, 180d, 181d · uses `arena` 182a, `size_t` 21b

If the current chunk doesn't have adequate space (at least `n` bytes) we examine the rest of the list of chunks (starting at `arena->next`) looking for a chunk with adequate space. If `n` is very large, we may not find it right away or we may not find a suitable chunk at all.

```

< Find a new chunk of memory 182c > ≡
{
    Chunk *ap = arena;
    Chunk *np = ap->next;
    while (np) {
        char *v = sizeof(Chunk) + (char *) np;
        if (v + n <= np->limit) {
            np->avail = v + n;
            arena = np;
            return v;
        }
        ap = np;
        np = ap->next;
    }
    < Allocate a new chunk of memory 182d >
}

```

182c · used in 182b · uses `arena` 182a, `Chunk` 181e, `next` 121b

If there isn't a suitable chunk of memory on the free list, then we need to allocate a new one.

```

< Allocate a new chunk of memory 182d > ≡
{
    size_t m = n + 10000;
    np = (Chunk *) malloc(m);
    np->limit = m + (char *) np;
    np->avail = n + sizeof(Chunk) + (char *) np;
    np->next = NULL;
    ap->next = np;
    arena = np;
    return sizeof(Chunk) + (char *) np;
}

```

182d · used in 182c · uses `arena` 182a, `Chunk` 181e, `malloc` 21b, `next` 121b, `size_t` 21b

3.6.2 Freeing Memory

To free all the memory in the arena, we need only point `arena` back to the first empty chunk.

```

< arena.c 183 > + ≡
void arena_free(void)
{
    arena = &first;
    arglist_reset();
}

```

183 · defined at 25d, 181e, 182ab, 183 · defines `arena_free` 39b, 181d · uses `arena` 182a, `arglist_reset` 134d, `first` 182a

Chapter 4

Man page

Here is the UNIX man page for weft:

```
<weft.1 184> ≡
.TH WEFT 1 "March 2026"
.SH NAME
weft, a literate programming tool
.SH SYNOPSIS
.B weft
.br
\fBweft\fP [options] [file] ...
.SH DESCRIPTION
.I weft
is a literate programming tool like Knuth's
.I WEB,
only simpler.
A
.I weft
file contains program source code interleaved with documentation.
When
.I weft
is given a
.I weft
file, it writes the program file(s).
By default only tangling is performed. Weaving (documentation
generation) is requested with \fB-w\fP.
.SH COMMAND LINE OPTIONS
.br
\fB-t\fP Suppresses weave output (even if \fB-w\fP is given).
.br
\fB-o\fP Suppresses generation of the output files.
.br
\fB-d\fP List dangling identifier references in indexes.
.br
\fB-c\fP Forces output files to overwrite old files of the same
name without comparing for equality first.
.br
\fB-v\fP The verbose flag. Forces output of progress reports.
.br
\fB-n\fP Forces sequential numbering of scraps (instead of page
numbers).
.br
\fB-s\fP Doesn't print list of scraps making up file at end of
```

each scrap.

`\fB-p path`\fP Prepend path to the filenames for all the output files.

`\fB-V string`\fP Provide the string for replacement of the @v operation. This is intended as a means for including version information in generated output.

`\fB-x`\fP Include cross-references in comments in output files.

`\fB-h options`\fP Turn on hyperlinks using the hyperref package of LaTeX and provide the options to the package.

`\fB-r`\fP Turn on hyperlinks using the hyperref package of LaTeX, with the package options being in the text.

`\fB-I path`\fP Provide a directory to search for included files. This may appear several times.

.br

`\fB-m`\fP Emit a JSON map of all output files, fragments, their definitions, and dependency relationships to stdout. No weave or tangle is performed. Useful for AI tooling and project navigation.

.br

`\fB-e name`\fP Extract the named fragment and all its transitive dependencies as Markdown to stdout. Fragment names support prefix matching. No weave or tangle is performed.

.br

`\fB-R file[:line]`\fP Read section markers from a tangled output file and emit JSON mapping to stdout. If a line number is given, emit the single innermost scrap covering that line. Without a line number, emit all scrap regions. No .weft source files are needed.

.br

`\fB-w [format]`\fP Request weave output. Without a format token, the format is taken from the `\fB@W`\fP directive in the source file. With `\fBmd`\fP, a `\fB.md`\fP file is generated; with `\fBtex`\fP, a `\fB.tex`\fP file is generated. A CLI format overrides `\fB@W`\fP. If `\fB-w`\fP is given without a format and no `\fB@W`\fP exists, weft exits with an error.

.br

`\fB--bodies`\fP Include raw scrap body text in `\fB-m`\fP JSON output. Each definition entry gets a `\fB"body"`\fP field with the JSON-escaped source code of the scrap.

.br

`\fB--prose`\fP Include the documentation paragraph preceding each `\fB@d`\fP/\fB@o\fP directive as a `\fB"prose"`\fP field in `\fB-m`\fP JSON output.

.br

`\fB--diff`\fP Dry-run tangle: show a unified diff of what would change in each output file without writing anything. If a file does not yet exist, all lines are shown with a `\fB+`\fP prefix.

.br

`\fB--lint`\fP Static analysis of the literate source. Reports unused fragments (defined but never referenced), undefined references (used but never defined), and fuzzy-match suggestions for likely typos (Levenshtein distance ≤ 3).

.br

`\fB--callers`\fP When combined with `\fB-e name`\fP, show the upward call chain from `\fB@o`\fP roots down to the named fragment as an indented tree.

.br

`\fB--errors`\fP Read compiler or linter output from stdin and annotate `\fBfile:line`\fP patterns with `\fB.weft`\fP source locations by parsing section markers embedded in the tangled output. No `\fB.weft`\fP source files are needed. Example: `\fBmake 2>&1 | weft --errors`\fP.

.br

`\fB--help`\fP Print a comprehensive help message including all options,

usage examples, and an AI-assisted workflow guide, then exit.

.SH FORMAT OF WEFT FILES

A

.I weft

file contains mostly ordinary

.I LaTeX.

The file is read and copied to output (.tex file) unless a

.I weft

command is encountered. All

.I weft

commands start with an ‘‘at-sign’’ (@).

Files and fragments are defined with the following commands:

.PP

@o \fIfile-name flags scrap\fP where scrap is smaller than one page.

.br

@O \fIfile-name flags scrap\fP where scrap is bigger than one page.

.br

@d \fIfragment-name scrap\fP. Where scrap is smaller than one page.

.br

@D \fIfragment-name scrap\fP. Where scrap is bigger than one page.

.PP

@q \fIfragment-name scrap\fP. Where scrap is smaller than one page.

The scrap is not expanded in the output, allowing you to construct output files which can, perhaps after further processing, be input to

.I weft.

.br

@Q \fIfragment-name scrap\fP. Where scrap is bigger than one page.

Likewise.

.PP

Scraps have specific begin and end

markers;

which begin and end marker you use determines how the scrap will be typeset in the .tex file:

.br

\fB@{\fP...\fB@}\fP for code, typeset with the LaTeX \fBlistings\fP package (load \fB\usepackage{listings}\fP in your preamble).

.br

\fB@[\fP...\fB@]\fP for LaTeX paragraph mode formatting, and

.br

\fB@(\fP...\fB@)\fP for LaTeX math mode formatting.

.br

Any amount of whitespace

(including carriage returns) may appear between a name and the beginning of a scrap.

.PP

Several code/file scraps may have the same name;

.I weft

concatenates their definitions to produce a single scrap.

Code scrap definitions are like macro definitions;

.I weft

extracts a program by expanding one scrap.

The definition of that scrap contains references to other scraps, which are themselves expanded, and so on.

\fIweft\fP’s output is readable; it preserves the indentation of expanded scraps with respect to the scraps in which they appear.

.PP

.SH PER FILE OPTIONS

When defining an output file, the programmer has the option of using flags

to control the output.

.PP

\fB-d\fR option,

.I weft

will emit line number indications at scrap boundaries.

.br

\fB-i\fR option,

.I weft

supresses the indentation of fragments (useful for \fBFortran\fR).

.br

\fB-t\fP option makes \fIweft\fP

copy tabs untouched from input to output.

.br

\fB-c\fIx\fP Include comments in the output file.

\fIx\fP may be \fBc\fP for C-style comments, \fB+\fP for C++ and

\fBp\fP for perl and similar.

.br

\fB-s\fP Suppress section markers and \fB#line\fP directives in the

tangled output for this file. Language auto-detection still sets

the comment style and the \fBlang\fP tag (used for Markdown syntax

highlighting), but no markers appear in the output.

.PP

.SH MINOR COMMANDS

.br

@@ Causes a single ‘‘at-sign’’ to be copied into the output.

.br

@_ Causes the text between it and the next {\tt @_} to be made bold

(for keywords, etc.) in the formatted document

.br

%% Comments out a line so that it doesn’t appear in the output.

.br

@i \fBfilename\fR causes the file named to be included.

.br

@f Creates an index of output files.

.br

@m Creates an index of fragments.

.br

@u Creates an index of user-specified identifiers.

.br

@W \fIformat\fP Declares the weave output format (\fBmd\fP or \fBtex\fP).

Used by \fB-w\fP when no explicit format is given on the command line.

.br

@l \fItag\fP Sets the language for the next @o or @d directive.

The tag is looked up in the language table to determine the comment

style for section markers and the \fBclass\fP attribute for Markdown

syntax highlighting. Place it on the line before the directive it

applies to.

.br

@L \fIname style\fP [+d] Defines a custom language mapping.

\fIname\fP is the tag (e.g. \fBnim\fP, \fBgls\fP),

\fIstyle\fP is one of \fB/\fP, \fB#\fP, \fB--\fP, \fB/*\fP, or \fB<!--\fP.

The optional \fB+d\fP flag enables \fB#line\fP directive generation.

User-defined languages override built-in entries. Must appear in the

document preamble before any @o or @d that depends on it.

.PP

.SH SECTION MARKERS

By default, when a language is detected (via @l, @L, or file extension

of @o), weft wraps each scrap in the tangled output with opening and

closing section markers using the appropriate comment syntax. For example,

C output includes:
.nf

```
/* {1: source.w:10} */  
int main() { return 0; }  
/* {1} */
```

.fi

The opening marker shows the scrap number and source location; the closing marker shows only the scrap number. Nested fragment expansions produce nested markers.

.PP

Languages with \fB#line\fP support (C, C++, Objective-C, etc.) also emit \fB#line\fP directives at scrap boundaries.

.PP

The \fB-s\fP per-file flag suppresses both section markers and \fB#line\fP directives.

.PP

.SH LANGUAGE INFERENCE

When no @l tag precedes an @o directive, weft infers the language from the output file's extension. For example, \fB@o app.js\fP automatically detects JavaScript (// comments, no #line). The inferred language is used for section markers in the tangle and for the \fBclass="language-X"\fP attribute in the Markdown weave.

.PP

Over 250 file extensions are recognised, covering C, C++, Python, JavaScript, Go, Rust, Ruby, Lua, SQL, HTML, and many more. Extensions not in the table produce no markers.

.PP

To mark an identifier for inclusion in the index, it must be mentioned at the end of the scrap it was defined in. The line starts with @| and ends with the \fBend of scrap\fP mark \fB@}\fP.

.PP

.SH ERROR MESSAGES

.PP

.SH BUGS

.PP

.SH AUTHOR

Preston Briggs.

Internet address \fBpreston@cs.rice.edu\fP.

.SH MAINTAINER

Simon Wright

Internet address \fBsimon@pushface.org\fP

.br

Keith Harwood

Internet address \fBKeith.Harwood@vitalmis.com\fP

184 · uses [exit 21b](#), [FILE 21b](#), [first 182a](#), [main 25f](#), [next 121b](#), [page 121c](#), [prose 121c](#), [scraps 122a](#), [search 172c](#)

Chapter 5

Indices

Three sets of indices can be created automatically: an index of file names, an index of fragment names, and an index of user-specified identifiers. An index entry includes the name of the entry, where it was defined, and where it was referenced.

5.1 Files

`arena.c` defined at 25d, 181e, 182ab, 183
`global.c` defined at 25e, 181a
`global.h` defined at 21a
`input.c` defined at 25a, 117de, 118ac, 121a
`latex.c` defined at 24a, 53b, 54, 61b, 65c, 66ab, 67ab, 68bcd, 69, 70a, 74b, 75b, 77ab, 79bd
`main.c` defined at 23c, 25f
`markdown.c` defined at 24b, 81bc, 85b, 87d, 88a, 91ab, 93b, 94ab, 96a, 97a, 98b
`names.c` defined at 25c, 145c, 146ab, 147ac, 148b, 149b, 156, 158, 159a, 162a, 164ce, 165, 168bc
`output.c` defined at 24c, 99, 100, 101, 102ab, 103abc, 104ab, 105ab, 106, 107abc, 108, 109, 111, 112, 113c
`pass1.c` defined at 23d, 41bc
`scraps.c` defined at 25b, 121bc, 122a, 123abcde, 124bcdef, 125cdef, 129d, 130bc, 131, 132abcde, 133d, 134b, 142b, 168d, 169abcde, 170abc, 171c, 172cd, 176bcf, 178ad, 179ac
`weft.1` defined at 184

5.2 Fragments

⟨ Accumulate prose character 42a ⟩ used in 41d
⟨ Accumulate scrap and return scraps++ 126b ⟩ used in 125f
⟨ Add a propagated argument 167b ⟩ used in 166
⟨ Add an at character to the block or break 47b ⟩ used in 47a
⟨ Add an inline scrap argument 167c ⟩ used in 166
⟨ Add any other character to the block 47c ⟩ used in 47a
⟨ Add buff to current arg list 168a ⟩ used in 167ab
⟨ Add letters to scraps with duplicate page numbers 143a ⟩ used in 142b
⟨ Add macro call argument 167e ⟩ used in 166
⟨ Add more indentation char 136a ⟩ used in 135, 137e, 138a
⟨ Add one char to the block buffer 47a ⟩ used in 46b
⟨ Add plain string argument 167a ⟩ used in 166
⟨ Add the source path to the include path list 39a ⟩ used in 38b
⟨ Add user identifier definition 129a ⟩ used in 128a
⟨ Add user identifier use 128b ⟩ used in 128a
⟨ Add scrap to name's definition list 45c ⟩ used in 45ab
⟨ Allocate a new chunk of memory 182d ⟩ used in 182c
⟨ Apply pending language if no explicit flags 160b ⟩ used in 159a

< *Apply pending language to macro* 161a > used in 162b, 164a
 < *Begin or end a block comment* 49e > used in 57c
 < *Begin the cross-reference environment* 64e > used in 60ab
 < *Begin the scrap environment* 62a > used in 60ab
 < *Bold Keyword* 72e > used in 70e
 < *Bold md Keyword* 90b > used in 89b
 < *Build a new arglist* 133c > used in 132e
 < *Build failure functions* 173b > used in 172c
 < *Build fragment definition* 45b > used in 43d
 < *Build output file definition* 45a > used in 43d
 < *Build source_name and tex_name* 38b > used in 38a
 < *Check at-sequence for end-of-scrap* 70e > used in 69
 < *Check for ambiguous prefix* 146c > used in 146b
 < *Check for cosmetic fence line* 82b > used in 82a
 < *Check for end of scrap name* 134a > used in 133d
 < *Check for long options* 29d > used in 29c
 < *Check for macro invocation in scrap* 138a > used in 135
 < *Check for macro parameters* 52a > used in 134a
 < *Check for terminating at-sequence and return name* 162b > used in 162a
 < *Check md at-sequence for end-of-scrap* 89b > used in 88a
 < *Cleanup and install name* 163d > used in 162b, 164a, 166
 < *Close the current sector* 44b > used in 43d, 57c, 82c
 < *Collect a block comment* 46b > used in 43d
 < *Collect include-file name* 120a > used in 119b
 < *Collect language apply in pass1* 42c > used in 43d
 < *Collect user language definition in pass1* 43a > used in 43d
 < *Collect user-specified index entries* 128a > used in 126c
 < *Collect weave format declaration in pass1* 43b > used in 43d
 < *Comment this macro use* 141a > used in 139d, 140ac
 < *Compare the temp file and the old file* 115b > used in 114b
 < *Complain about duplicate labels* 180e > used in 180c
 < *Complain about empty label* 180f > used in 180c
 < *Complain about missing label* 180b > used in 179c
 < *Copy block comment from scrap* 49a > used in 138a
 < *Copy file name into file* 139c > used in 138a
 < *Copy fragment title into file* 140c > used in 138a
 < *Copy label from scrap into file* 139a > used in 138a
 < *Copy label from source into* 72a > used in 57c, 70e, 82c, 89b
 < *Copy macro into file* 139d > used in 138a
 < *Copy version info into file* 71b > used in 70e, 89b, 138a
 < *Copy version info into md file* 83b > used in 82c
 < *Copy version info into tex file* 58b > used in 57c
 < *Copy defs->scrap to file* 135 > used in 134b
 < *Copy source_file into md_file* 82a > used in 81c
 < *Copy source_file into tex_file* 57b > used in 54
 < *Create a new label entry* 180d > used in 180c
 < *Create new name entry* 149a > used in 146b, 148b
 < *Create new scrap, managed by writer* 126a > used in 125f
 < *Emit a quoted scrap argument inline* 74a > used in 74b
 < *End block* 62d > used in 49e, 62a, 64a
 < *Enter the next argument* 163a > used in 162b
 < *Expand macro into file* 58c > used in 57c
 < *Expand md macro into* 84a > used in 82c
 < *Expand md tab into spaces* 89a > used in 88a
 < *Expand tab into spaces* 70c > used in 69, 137e
 < *Extend goto graph with tree->spelling* 173a > used in 172d
 < *Fill in the md scrap* 86a > used in 84bc
 < *Fill in the middle of the scrap environment* 62e > used in 60ab

< *Find a free temporary file 114a* > used in 113e
 < *Find a new chunk of memory 182c* > used in 182b
 < *Finish md scrap 86b* > used in 84bc
 < *Finish the cross-reference environment 64f* > used in 60ab
 < *Finish the scrap environment 64a* > used in 60ab
 < *For all remaining scraps 143c* > used in 143a
 < *Format a file index entry 76a* > used in 75b
 < *Format a md file index entry 93c* > used in 93b
 < *Format a md index entry 94c* > used in 94b
 < *Format a md user index entry 96b* > used in 96a
 < *Format a user index entry 79e* > used in 79d
 < *Format an index entry 78a* > used in 77b
 < *Format macro name 72g* > used in 70e
 < *Format macro parameters 52b* > used in 72g, 74b
 < *Format md macro name 90d* > used in 89b
 < *Forward declarations for scraps.c 134cd, 141b, 161b, 176a* > used in 122a
 < *Function prototypes 41a, 53a, 68a, 70b, 79a, 81a, 113ab, 117a, 118b, 122b, 124a, 130a, 133a, 142a, 145b, 147b, 164b, 172b, 176e, 177d, 179d, 181d* > used in 21a
 < *Get comment delimiters 160a* > used in 159b
 < *Get label from 179b* > used in 72a, 127a, 139a
 < *Get label while collecting scrap 127a* > used in 126c
 < *Get the nth previous character from a scrap 172a* > used in 171c
 < *Get the nth previous character from an argument 171d* > used in 171c
 < *Global variable declarations 27a, 28ac, 44c, 48c, 64b, 117b, 125a, 144c, 181c* > used in 21a
 < *Global variable definitions 27b, 28b, 29a, 44d, 48d, 64c, 117c, 125b, 145a* > used in 25e
 < *Handle an “at” character 119a* > used in 118a
 < *Handle at-sign during scrap accumulation 126c* > used in 126b
 < *Handle macro invocation in scrap 129b* > used in 126c
 < *Handle macro parameter substitution 50c* > used in 138a
 < *Handle optional per-file flags 159b* > used in 159a
 < *Handle reverse map mode 36b* > used in 25f
 < *Handle tab characters on output 137e* > used in 135
 < *Handle the file name in argv[arg] 38a* > used in 36d
 < *Handle EOF 120b* > used in 118a
 < *HTML-escape character c 88b* > used in 88a
 < *If we break the line at this word 49c* > used in 49a
 < *Include a fragment use in comment 141d* > used in 141b
 < *Include an embedded scrap in comment 141c* > used in 141b
 < *Include block comment in a scrap 48f* > used in 126c
 < *Include files 21b* > used in 21a
 < *Indent suppressed 137d* > used in 137b
 < *Insert appropriate indentation 137b* > used in 135, 136b
 < *Insert debugging information if required 136b* > used in 135, 138b
 < *Insert section closing marker 137a* > used in 135
 < *Insert section opening marker 136c* > used in 135
 < *Interpret at-sequence 57c* > used in 57b
 < *Interpret command-line arguments 29bc* > used in 25f
 < *Interpret md at-sequence 82c* > used in 82a
 < *Interpret the argument string s 33* > used in 29c
 < *Italic “whatever” 72f* > used in 70e
 < *Italic md “whatever” 90c* > used in 89b
 < *Limits 23b* > used in 21a
 < *Look for end of scrap name and return 166* > used in 165
 < *Make this argument 163c* > used in 163a
 < *Move a word to the file 49b* > used in 49a
 < *Move the temporary file to the target, if required 114b* > used in 113e
 < *Note a delimiter collision 70d* > used in 69
 < *Open an include file 119b* > used in 119a

< *Operating System Dependencies* 26 > used in 21a, 25e
 < *Otherwise pop the current arg* 171b > used in 170c
 < *Perhaps add an include path* 35b > used in 29c
 < *Perhaps add letters to the page numbers* 143d > used in 143a
 < *Perhaps comment this macro* 140a > used in 50c, 139d
 < *Perhaps get the extract name* 35e > used in 29c
 < *Perhaps get the hyperref options* 35d > used in 29c
 < *Perhaps get the prepend path* 35a > used in 29c
 < *Perhaps get the reverse map argument* 36a > used in 29c
 < *Perhaps get the version info string* 35c > used in 29c
 < *Perhaps put a delayed indent* 140b > used in 49a, 140a
 < *Perhaps —return— a character from the current arg* 170d > used in 170c
 < *Perhaps skip white-space* 48a > used in 47c
 < *Perhaps start a new arg* 171a > used in 170c
 < *Print help message* 30 > used in 29d
 < *Process a file* 39b > used in 38a
 < *Process the remaining arguments (file names)* 36d > used in 25f
 < *Put out the indent* 137c > used in 50c, 137b
 < *Read line in .aux file* 142c > used in 142b
 < *Rename the temporary file to the target* 116 > used in 114b, 115b
 < *Resync the line number after a fragment reference* 138b > used in 138a
 < *Reverse cross-reference lists* 50a > used in 41c
 < *Rob's ordering* 77c > used in 77b, 94b
 < *Save label to label store* 180c > used in 127a
 < *Save macro name* 129c > used in 129b
 < *Save macro parameters* 51 > used in 129b
 < *Save pending prose* 42b > used in 45ab
 < *Scan at-sequence* 43d > used in 41d
 < *Scan the source file, looking for at-sequences* 41d > used in 41c
 < *Search for label(Found,Notfound)* 180g > used in 179c, 180c
 < *Search scraps* 174 > used in 172c
 < *Set locale information* 36c > used in 25f
 < *Set up name, args and next* 133b > used in 132e
 < *Show diff of temp file vs real file* 115a > used in 114b
 < *Show md presence of a block comment* 90a > used in 89b
 < *Show presence of a block comment* 48e > used in 70e
 < *Skip at-L language definition line* 43c > used in 57c, 82c
 < *Skip commented-out code* 72d > used in 70e, 89b, 126c
 < *Skip invisibles on p* 148a > used in 147c
 < *Skip over a block comment* 49d > used in 174
 < *Skip over a scrap use* 175b > used in 174
 < *Skip over an in-text scrap* 45d > used in 43d
 < *Skip over at at* 175a > used in 174
 < *Skip over at-sign or go to skipped* 46a > used in 45d
 < *Skip over index entries* 72c > used in 70e, 89b
 < *Skip to the next nw-char* 48b > used in 46b
 < *Skip until scrap begins, then return name* 164a > used in 162a
 < *Skip whitespace* 46c > used in 46b
 < *Sort key of size n for ordering* 77d > used in 77b, 94b
 < *Start block* 62b > used in 49e, 62a
 < *Step i to the next valid scrap* 143b > used in 143a
 < *Step to next sector* 44a > used in 43d, 57c, 82c
 < *Switch block* 62c > used in 62a
 < *Type declarations* 23a, 50b, 113d, 144ab, 163b, 164d, 167d, 176d, 181b > used in 21a
 < *version 1* > used in 23b
 < *Warn (only once) about needing to rerun after Latex* 124g > used in 123b, 142c
 < *Write abbreviated definition list* 73 > used in 72g, 74b
 < *Write defining scrap numbers* 78b > used in 78a

< *Write defs references 178b* > used in 178a
 < *Write file defs 64g* > used in 60a
 < *Write file's defining scrap numbers 76b* > used in 76a
 < *Write in-text scrap 64d* > used in 57c
 < *Write index of file names 75a* > used in 57c
 < *Write index of macro names 76c* > used in 57c
 < *Write index of user-specified names 79c* > used in 57c
 < *Write LaTeX limbo definitions 55a* > used in 54
 < *Write macro definition 60b* > used in 57c
 < *Write macro defs 65a* > used in 60b
 < *Write macro refs 65b* > used in 60b
 < *Write md defining scrap numbers 95a* > used in 94c
 < *Write md defs references 98c* > used in 98b
 < *Write md file cross-refs 86d* > used in 84b
 < *Write md file defs 86e* > used in 86d
 < *Write md file's defining scrap numbers 93d* > used in 93c
 < *Write md in-text scrap 86c* > used in 82c
 < *Write md index of file names 93a* > used in 82c
 < *Write md index of macro names 93e* > used in 82c
 < *Write md index of user-specified names 95c* > used in 82c
 < *Write md macro cross-refs 87a* > used in 84c
 < *Write md macro definition 84c* > used in 82c
 < *Write md macro defs 87b* > used in 87a
 < *Write md macro refs 87c* > used in 87a
 < *Write md output file definition 84b* > used in 82c
 < *Write md referencing scrap numbers 95b* > used in 94c
 < *Write md uses references 97b* > used in 97a
 < *Write one def reference 178c* > used in 178b
 < *Write one md def reference 98d* > used in 98c
 < *Write one md referenced scrap 98a* > used in 97c, 98d
 < *Write one md use reference 97c* > used in 97b
 < *Write one referenced scrap 177c* > used in 177b, 178c
 < *Write one use reference 177b* > used in 177a
 < *Write out files->spelling 113e* > used in 113c
 < *Write output file definition 60a* > used in 57c
 < *Write referencing scrap numbers 78c* > used in 78a
 < *Write the AtBeginDocument setup 57a* > used in 55a
 < *Write the hyperlink usage macro 56a* > used in 55a
 < *Write the label to file 180a* > used in 179c
 < *Write the macro's name 61a* > used in 60b, 78a
 < *Write the md macro's name 85a* > used in 84c, 94c
 < *Write the modern decoration macros 56b* > used in 55a
 < *Write the scrap formatting macros 55b* > used in 55a
 < *Write uses references 177a* > used in 176f

5.3 Identifiers

Knuth prints his index of identifiers in a two-column format. I could force this automatically by emitting the `\twocolumn` command; but this has the side effect of forcing a new page. Therefore, it seems better to leave it this up to the user.

`add_to_use`: 51, 129b, 129d, 130a, 167e.

`add_uses`: 128b, 129a, 174, 176a, 176c.

`already_warned`: 124g, 125a, 125b, 142b.

`arena`: 182a, 182bcd, 183.

`arena_free`: 39b, 181d, 183.

`arena_getmem`: 35b, 39a, 45c, 52a, 123a, 125d, 126a, 128b, 129ad, 133bc, 146a, 149a, 164e, 167c, 172c, 173ab, 174, 176c, 180d, 181d, 182b.

Arglist: 50c, 53b, 58c, 72g, 74b, 81b, 84a, 90d, 91b, 122b, 129b, 130c, 132de, 133abcd, 134bd, 139d, 140c, 141b, 145b, [164d](#), 164e, 165, 167cd, 168a, 169e, 170b, 171a, 175b.
 arglist_count: [134d](#).
 arglist_lookup: 130c, 133d, [134d](#), 175b.
 arglist_register: 129c, [134d](#).
 arglist_reset: [134d](#), 181d, 183.
 arglist_table: [134d](#).
 ArgManager: [170a](#), 170bc, 171c, 172c, 174, 179a.
 ArgMgr: [169e](#), 170abc, 171c.
 argpop: [170c](#), 174, 175ab.
 ARG_CHR: 58c, 61a, 72g, 74b, 85a, 90d, 91b, 133b, 141ad, 163a, [163b](#), 166, 167b.
 backup: [132c](#), 137b.
 begin_tex_escape: 48e, [68d](#), 70e, 72efg.
 blockBuff: 46b, [48c](#), 48df.
 bodies_flag: [27a](#), 27b, 29d, 100.
 buildArglist: [164e](#), 166, 167c, 168a.
 build_gotos: 172c, [172d](#).
 callers_flag: [27a](#), 27b, 29d, 39b.
 Chunk: [181e](#), 182acd.
 clear_marks: [103a](#), 104ab.
 code_inline: 52b, [68d](#), 69.
 collect_file_name: 45a, 60a, 84b, 145b, [159a](#).
 collect_lang_def: 43a, 145b, [156](#).
 collect_lint_names: [105b](#), 107a.
 collect_macro_name: 45b, 60b, 84c, 145b, [162a](#).
 collect_numbers: 39b, 142a, [142b](#).
 collect_scrap: 45ab, 51, 122b, [125f](#), 167c.
 collect_scrap_name: 58c, 72g, 84a, 90d, 129b, 145b, [165](#), 167e.
 collect_weave_format: 43b, 145b, [158](#).
 command_name: [28c](#), 29ab, 30, 33, 36d, 39b, 43d, 45d, 46a, 54, 58c, 65b, 70d, 72g, 74b, 81c, 87c, 90d, 91b, 104ab, 114a, 116, 119ab, 120a, 121a, 124g, 126bc, 128a, 133d, 134d, 139d, 146c, 156, 158, 159ab, 160a, 162ab, 163d, 164a, 165, 166, 175b.
 comment_ArglistElement: 141a, [141b](#), 141d.
 comment_begin: 49a, 136c, 137a, 140a, [161b](#).
 comment_end: 49a, 136c, 137a, 140a, [161b](#).
 comment_mid: 49a, [161b](#).
 compare: [145c](#), 146b, 147a.
 compare_flag: [27a](#), 27b, 33, 114b.
 copy_scrap: 52b, 53b, 62e, 64d, [69](#).
 current_sector: 39b, 44ab, [44c](#), 44d, 76c, 79c, 93e, 95c, 126a, 128a, 162a, 165.
 dangling_flag: [27a](#), 27b, 33, 79e, 96b.
 defs: 45c, 50c, 58c, 60ab, 64g, 65a, 72g, 73, 74b, 76b, 78b, 79e, 84a, 86e, 87b, 90d, 91b, 93d, 95a, 96b, 97c, 98ad, 100, 102b, 105b, 106, 113e, [121c](#), 122b, 123e, 126a, 129a, 133b, 134b, 135, 136bc, 137a, 139cd, 140ac, 141ac, 144b, 149a, 167d, 168b, 174, 177bc, 178ac.
 delayed_indent: 50c, [134c](#), 135, 137d, 139d, 140ab.
 depths: [169c](#), 172c, 173ab.
 diff_flag: [27a](#), 27b, 29d, 114b.
 double_at: [117d](#), 119a, 121a.
 dump_scrap_json: 100, 124a, [131](#).
 dump_scrap_text: 102b, 124a, [130c](#), 131.
 Embed_Node: 50c, 133b, 141c, [167d](#).
 emit_name_json: [100](#), 101.
 end_line: 100, [121c](#), 124d, 126ac.
 end_tex_escape: 48e, [68d](#), 70e, 72efg.
 EQUAL: [145c](#), 146b, 147a.
 errors_flag: [27a](#), 27b, 29d, 36b.
 exit: [21b](#), 25f, 29d, 36bd, 39b, 45d, 46a, 66b, 104ab, 109, 114a, 119ab, 120a, 121a, 126bc, 128a, 133d, 134d, 139d, 159a, 162ab, 163d, 164a, 165, 166, 170b, 175b, 184.

EXTENSION: [145c](#), [146b](#), [147a](#).
 extract_flag: [27a](#), [27b](#), [33](#), [35e](#).
 extract_fragment: [102b](#), [104b](#).
 extract_name: [27a](#), [27b](#), [35e](#), [39b](#).
 extra_scraps: [62e](#), [64a](#), [64b](#), [64c](#), [74a](#), [86ab](#), [91b](#).
 FALLTHROUGH: [23a](#), [43d](#), [57c](#), [70e](#), [82c](#), [118a](#), [146b](#).
 FALSE: [23a](#), [27ab](#), [33](#), [35abcde](#), [36a](#), [39b](#), [57bc](#), [62d](#), [64d](#), [65c](#), [68d](#), [69](#), [74a](#), [78b](#), [79be](#), [82abc](#), [84a](#), [86c](#), [87d](#), [91b](#), [95a](#), [96b](#), [97c](#), [98d](#), [103a](#), [119a](#), [121a](#), [139d](#), [142b](#), [147a](#), [149ab](#), [156](#), [159b](#), [160b](#), [176b](#), [177b](#), [178cd](#), [179a](#).
 fclose: [21b](#), [54](#), [74a](#), [81c](#), [109](#), [111](#), [113e](#), [114a](#), [115ab](#), [120b](#), [131](#), [142b](#).
 FILE: [21b](#), [30](#), [53b](#), [54](#), [61b](#), [65c](#), [67a](#), [68cd](#), [69](#), [74ab](#), [75b](#), [77b](#), [79d](#), [81bc](#), [85b](#), [87d](#), [88a](#), [91ab](#), [93b](#), [94b](#), [96a](#), [97a](#), [98b](#), [99](#), [100](#), [102b](#), [109](#), [111](#), [113e](#), [115ab](#), [117de](#), [122b](#), [123bc](#), [124a](#), [130c](#), [131](#), [134b](#), [141b](#), [142b](#), [176ef](#), [177d](#), [178a](#), [179cd](#), [184](#).
 file_line: [121c](#), [124c](#), [126ab](#), [135](#).
 file_name: [41ac](#), [53a](#), [54](#), [81ac](#), [121c](#), [124b](#), [126ab](#), [136bc](#).
 file_names: [39b](#), [41c](#), [50a](#), [75a](#), [93a](#), [103c](#), [104b](#), [107a](#), [113b](#), [144c](#), [145a](#), [159a](#).
 find_closest: [105b](#), [106](#).
 find_lang: [149b](#), [160b](#).
 find_name: [102a](#), [104ab](#).
 first: [30](#), [62e](#), [97c](#), [98ad](#), [100](#), [101](#), [104b](#), [122b](#), [123b](#), [177bc](#), [178c](#), [179a](#), [182a](#), [183](#), [184](#).
 fopen: [21b](#), [54](#), [81c](#), [109](#), [111](#), [114a](#), [115ab](#), [119b](#), [121a](#), [142b](#).
 format_defs_refs: [60ab](#), [177d](#), [178a](#).
 format_entry: [53b](#), [76c](#), [77b](#).
 format_file_entry: [53b](#), [75a](#), [75b](#).
 format_user_entry: [53b](#), [79c](#), [79d](#).
 format_uses_refs: [60ab](#), [176e](#), [176f](#).
 fprintf: [21b](#), [33](#), [36cd](#), [39b](#), [41c](#), [43d](#), [45d](#), [46a](#), [48e](#), [52b](#), [54](#), [56a](#), [58c](#), [60a](#), [62ae](#), [64g](#), [65b](#), [66b](#), [70d](#), [72efg](#), [74b](#), [76a](#), [79e](#), [81c](#), [84b](#), [86ae](#), [87c](#), [90bcd](#), [91b](#), [93c](#), [96b](#), [97c](#), [98d](#), [99](#), [100](#), [102b](#), [104ab](#), [106](#), [107a](#), [109](#), [113e](#), [114a](#), [116](#), [119ab](#), [120a](#), [121a](#), [123b](#), [124g](#), [126bc](#), [128a](#), [130c](#), [131](#), [133d](#), [134d](#), [136bc](#), [137a](#), [139cd](#), [141ab](#), [146c](#), [156](#), [158](#), [159ab](#), [160a](#), [162ab](#), [163d](#), [164a](#), [165](#), [166](#), [170b](#), [175b](#), [177b](#), [178c](#), [180abef](#).
 fputs: [21b](#), [49a](#), [50c](#), [52b](#), [55ab](#), [56b](#), [57a](#), [58bc](#), [60ab](#), [61b](#), [62abcde](#), [64aefg](#), [65abc](#), [67a](#), [68cd](#), [69](#), [70e](#), [71b](#), [72g](#), [73](#), [74ab](#), [75a](#), [76abc](#), [78abc](#), [79ce](#), [83b](#), [84bc](#), [85b](#), [86ae](#), [87bcd](#), [88ab](#), [89b](#), [90abd](#), [91ab](#), [93acde](#), [94c](#), [95abc](#), [96b](#), [97bc](#), [98acd](#), [99](#), [100](#), [101](#), [102b](#), [103c](#), [109](#), [112](#), [115a](#), [123b](#), [131](#), [135](#), [139c](#), [140a](#), [177abc](#), [178bc](#).
 getc: [21b](#), [115b](#), [118a](#), [119ab](#), [120ab](#), [121a](#).
 getenv: [21b](#), [36c](#).
 get_scrap_defs: [98b](#), [122b](#), [123e](#).
 get_scrap_uses: [97a](#), [100](#), [102b](#), [122b](#), [123d](#).
 goto_lookup: [169d](#), [173ab](#), [174](#).
 Goto_Node: [169a](#), [169bcd](#), [172c](#), [173ab](#), [174](#).
 GREATER: [145c](#), [146b](#), [147a](#).
 has_sector: [76c](#), [79a](#), [79b](#), [79c](#), [93e](#), [95c](#).
 hyperoptions: [27a](#), [27b](#), [35d](#), [55a](#), [56a](#).
 hyperopt_flag: [27a](#), [27b](#), [33](#), [35d](#).
 hyperref_flag: [27a](#), [27b](#), [33](#), [35d](#), [55a](#).
 incl: [27a](#), [27b](#), [35b](#), [39a](#), [119b](#).
 includepath_flag: [27a](#), [27b](#), [33](#), [35b](#).
 include_depth: [117d](#), [119b](#), [120b](#), [121a](#).
 init_scraps: [41c](#), [122b](#), [123a](#).
 instance: [84a](#), [132e](#), [133ab](#), [139d](#).
 isgraph: [21b](#), [120a](#), [159a](#), [165](#).
 islower: [21b](#), [147c](#).
 isspace: [21b](#), [64a](#), [86b](#), [128a](#), [159ab](#), [162a](#), [164a](#).
 is_file_name: [103b](#), [103c](#).
 json_puts: [99](#), [100](#), [109](#).
 label_node: [180dg](#), [181a](#), [181b](#), [181c](#).
 label_tab: [180g](#), [181a](#), [181c](#).
 LangEntry: [149b](#), [160b](#).
 lang_table: [149b](#).
 lang_to_flags: [149b](#), [160b](#), [161a](#).

latex_code_putc: 53b, [67a](#), 68c, 69, 74a.
 latex_language: 53b, 62e, [67b](#).
 LESS: [145c](#), 146b, 147a.
 letter: 112, [121c](#), 123b, 126a, 143d.
 levenshtein: [105a](#), 105b.
 lint_flag: [27a](#), 27b, 29d, 39b.
 lint_name: [106](#), 107a.
 lint_names: [105b](#).
 lint_name_count: [105b](#), 107a.
 load_entry: [77a](#), 77b.
 lookup: 50c, [132d](#), 133b.
 macro_names: 39b, 41c, 50a, 76c, 93e, 104b, 107a, 113b, [144c](#), 145a, 163d.
 main: [25f](#), 184.
 malloc: [21b](#), 77b, 94b, 170b, 182d.
 Manager: [125c](#), 125def, 130bc, 132c, 133d, 135, 170a, 171c, 174.
 map_flag: [27a](#), 27b, 33, 39b.
 markdown_flag: [27a](#), 27b, 33, 39b.
 MarkerFrame: [107b](#), 107c, 109, 111, 112.
 MAX_ARGLISTS: [134d](#).
 max_depth: [169c](#), 172c, 173ab.
 MAX_INDENT: 84a, [113d](#), 113e, 136a, 137c.
 MAX_LINT_NAMES: [105b](#).
 MAX_NAME_LEN: [23b](#), 128a, 130c, 133d, 159a, 162a, 165, 167a, 175b, 179b.
 md_copy_scrap: 81b, 86ac, [88a](#).
 md_format_defs_refs: 81b, 86d, 87a, [98b](#).
 md_format_entry: 81b, 93e, [94b](#).
 md_format_file_entry: 81b, 93a, [93b](#).
 md_format_user_entry: 81b, 95c, [96a](#).
 md_format_uses_refs: 81b, 86d, 87a, [97a](#).
 md_load_entry: [94a](#), 94b.
 md_print_scrap_numbers: 81b, 86e, 87bc, [87d](#), 93d, 95b.
 md_scrap_type: [88a](#), 89b.
 md_write_arg: 81b, 85a, [85b](#), 89b, 91b.
 md_write_ArglistElement: 81b, 90d, [91b](#).
 md_write_literal: 81b, 90d, [91a](#), 91b.
 Move_Node: [169b](#), 169d, 173ab.
 Name: 30, 45ab, 50c, 53b, 58c, 60ab, 69, 72g, 74b, 75b, 77abd, 79abd, 81b, 84abc, 88a, 90d, 91b, 93b, 94ab, 96a, 97c, 98d, 100, 101, 102ab, 103abc, 104ab, 105b, 106, 107a, 113abc, 124af, 128a, 129bd, 130a, 132bde, 133b, 139d, 141b, [144b](#), 144c, 145ab, 146b, 147a, 148b, 149a, 159a, 162a, 164bcde, 166, 167c, 168bd, 171a, 172cd, 174, 176acd, 177b, 178c, 179a.
 name_add: 128b, 129a, 145b, [148b](#), 159a.
 Name_Node: [168d](#), 169a, 173ab, 174.
 next: 27a, 30, 33, 35b, 39a, 45c, 50c, 58c, 64g, 65abc, 72g, 73, 74b, 76b, 78bc, 79e, 86e, 87bcd, 90d, 91b, 93d, 95ab, 96b, 97bc, 98cd, 100, 102b, 103c, 104a, 117a, 119b, [121b](#), 125d, 128b, 129ad, 130b, 132de, 133bc, 134b, 141ad, 144a, 164de, 167ce, 168acd, 169abd, 171a, 173ab, 174, 176bcd, 177ab, 178bc, 181e, 182cd, 184.
 number_flag: [27a](#), 27b, 33, 39b, 142b.
 num_scrap: 77b, 94b, [122a](#), 122b.
 nw_char: [28a](#), [28b](#), 30, 39b, 41d, 43d, 45d, 46a, 47a, 48bf, 49d, 51, 52ab, 57bc, 60a, 66a, 68c, 69, 70ae, 72ce, 76a, 79e, 82ac, 88a, 89b, 90b, 118a, 119a, 121a, 126bc, 127a, 128a, 129bc, 130c, 133d, 135, 137d, 138a, 139cd, 141abd, 146c, 156, 158, 159a, 162ab, 163a, 164a, 165, 166, 167a, 175ab, 177b, 178cd, 179b.
 op_char: [178d](#), 179a.
 output_flag: [27a](#), 27b, 33, 39b.
 owner: 103c, 104a, [121c](#), 124f, 126a, 132b.
 page: 65c, 78b, 79e, 87d, 95a, 96b, 97c, 98ad, [121c](#), 122b, 123bc, 126a, 142bc, 143bd, 177bc, 178c, 184.
 Parameters: [50b](#), 52a, 84a, 122b, 133d, 134b, 135.
 parse_close_marker: [108](#), 109, 111.
 parse_open_marker: [107c](#), 109, 111.
 pass1: 39b, 41a, [41c](#), 70e, 89b.

pending_lang: 42c, [144c](#), 145a, 160b, 161a.
 pending_prose: [41b](#), 41c, 42b, 45ab.
 pop: 49abd, 52a, [130b](#), 130c, 133d, 134a, 135, 137b, 138a, 139b, 170c.
 pop_scrap_name: [133d](#), 139d.
 PREFIX: [145c](#), 146b, 147a.
 prefix_add: 145b, [146b](#), 163d.
 prev_char: [171c](#), 179a.
 prev_sector: 39b, 44a, [44c](#), 44d.
 print_callers: [103c](#), 104a.
 print_scrap_numbers: 53b, 64g, 65ab, [65c](#), 76b, 78c.
 prose: 27a, 29d, 30, 62e, 100, [121c](#), 124e, 126a, 132a, 184.
 prose_buf: [41b](#), 42ab.
 prose_flag: 27a, 27b, 29d, 41d, 42b, 100.
 prose_len: [41b](#), 41c, 42ab.
 push: 48f, 51, [125d](#), 125e, 126bc, 127a, 129bc.
 pushArglist: [170b](#), 171a, 175b.
 pushes: 51, [125e](#), 127a, 129c.
 putc: [21b](#), 49ab, 57bc, 58c, 67a, 69, 70c, 72g, 74ab, 76ab, 78abc, 79e, 82abc, 88ab, 89ab, 90d, 91b, 93c, 94c, 95a, 123b, 135, 137ce, 138a, 139d, 140abc.
 reject_match: 172c, 174, [179a](#).
 remove: [21b](#), 114b, 115ab.
 reverse: 27a, 30, 168b, [168c](#).
 reverse_lists: 50a, 145b, [168b](#).
 reverse_lookup_in_file: [111](#), 112.
 reverse_map_arg: [27a](#), 27b, 36ab.
 reverse_map_flag: [27a](#), 27b, 33, 36a.
 robs_strcmp: 77c, 102a, 147b, [147c](#), 148b, 149a, 176c.
 root: 103b, [169c](#), 172c, 173ab, 174, 176ac.
 save_string: 35b, 39a, 42b, 119b, 126a, 145b, [146a](#), 146b, 149a, 164c, 168a.
 SCRAP: [122a](#), 123a, 126a.
 ScrapEntry: [121c](#), 122a, 123a, 126a.
 scraps: 24ab, 27a, 30, 52b, 53b, 60ab, 62a, 64ae, 65c, 81b, 84bc, 86bd, 87ad, 100, [122a](#), 123a, 126a, 142bc, 143bc, 174, 184.
 scrap_array: [122a](#), 123bde, 124bcdef, 126abc, 128b, 129a, 130c, 132ab, 135, 136bc, 142bc, 143bd, 174, 176f, 178a.
 SCRAP_BITS: [122a](#).
 scrap_end_line: 100, 124a, [124d](#).
 scrap_file_line: 100, 102b, 106, 124a, [124c](#).
 scrap_file_name: 100, 102b, 106, 124a, [124b](#).
 scrap_flag: [27a](#), 27b, 33, 60a, 86d.
 scrap_is_in: 174, 176a, [176b](#).
 SCRAP_MASK: [122a](#), 126a.
 Scrap_Node: 45c, 50c, 53b, 58c, 65c, 73, 74a, 76b, 78bc, 79e, 81b, 87d, 90d, 91b, 93d, 95ab, 96b, 97c, 98d, 100, 102b, 103c, 104a, 122b, 128b, 129ad, 133b, 134b, [144a](#), 144b, 167cd, 168bc, 174, 176ab, 177b, 178c.
 scrap_owner: 103c, 104a, 124a, [124f](#).
 scrap_prose: 100, 124a, [124e](#).
 SCRAP_SHIFT: [122a](#), 126a.
 SCRAP_SIZE: [122a](#), 123a, 126a.
 scrap_type: [68b](#), 68d, [69](#), 70de, 72g.
 search: 30, 41c, 172b, [172c](#), 184.
 sector: 30, 53ab, 54, 76c, 77b, 78a, 79bcde, 81abc, 93e, 94bc, 95c, 96ab, 100, [121c](#), 126a, 128ab, 129ac, 130c, 139d, 141d, 144b, 145b, 146bc, 147a, 148b, 149a, 162a, 163d, 165, 174.
 setlocale: [21b](#), 36c.
 set_scrap_owner: 45c, 124a, [132b](#).
 set_scrap_prose: 45ab, 124a, [132a](#).
 size_t: [21b](#), 74a, 107c, 112, 181d, 182bd.
 Slab: [121b](#), 121c, 125cd, 126a, 130b.
 slab: 121b, [121c](#), 126a, 130c, 135, 174.
 SLAB_SIZE: [121b](#), 125d, 130b, 132c, 172a.

source_file: [117d](#), [118ac](#), [119ab](#), [120ab](#), [121a](#).
 source_get: [41d](#), [42c](#), [43cd](#), [44ab](#), [45d](#), [46ac](#), [47ab](#), [48ab](#), [51](#), [52b](#), [57bc](#), [58bc](#), [64ad](#), [69](#), [70e](#), [72bcde](#), [75a](#), [76c](#), [79c](#), [82abc](#), [83b](#), [84a](#), [86bc](#), [88a](#), [89b](#), [90b](#), [93ae](#), [95c](#), [117a](#), [118a](#), [119b](#), [120b](#), [126bc](#), [127b](#), [128a](#), [129b](#), [156](#), [158](#), [159ab](#), [160a](#), [162ab](#), [163a](#), [164a](#), [165](#), [166](#), [167a](#).
 source_last: [52b](#), [62e](#), [69](#), [88a](#), [117a](#), [118a](#).
 source_line: [43d](#), [45d](#), [46a](#), [117b](#), [117c](#), [118ac](#), [119ab](#), [120ab](#), [121a](#), [126ac](#), [128a](#), [146c](#), [156](#), [158](#), [159ab](#), [160a](#), [162a](#), [163d](#), [165](#), [166](#).
 source_name: [38ab](#), [39ab](#), [43d](#), [45d](#), [46a](#), [117b](#), [117c](#), [119ab](#), [120ab](#), [121a](#), [126ac](#), [128a](#), [146c](#), [156](#), [158](#), [159ab](#), [160a](#), [162ab](#), [163d](#), [164a](#), [165](#), [166](#).
 source_open: [41c](#), [54](#), [81c](#), [117a](#), [121a](#).
 source_peek: [46c](#), [47b](#), [48a](#), [117a](#), [118a](#), [118c](#), [119ab](#), [120b](#), [121a](#).
 source_ungetc: [47ab](#), [48b](#), [76c](#), [93e](#), [118b](#), [118c](#).
 stack: [109](#), [111](#), [117e](#), [119b](#), [120b](#).
 stderr: [21b](#), [33](#), [36cd](#), [39b](#), [41c](#), [43d](#), [45d](#), [46a](#), [54](#), [58c](#), [64g](#), [65b](#), [66b](#), [70d](#), [72g](#), [74b](#), [81c](#), [86e](#), [87c](#), [90d](#), [91b](#), [104ab](#), [106](#), [107a](#), [109](#), [113e](#), [114a](#), [116](#), [119ab](#), [120a](#), [121a](#), [124g](#), [126bc](#), [128a](#), [133d](#), [134d](#), [139d](#), [146c](#), [156](#), [158](#), [159ab](#), [160a](#), [162ab](#), [163d](#), [164a](#), [165](#), [166](#), [170b](#), [175b](#), [180bef](#).
 strlen: [21b](#), [49a](#), [105a](#), [109](#), [142c](#), [145c](#), [146a](#), [171d](#), [179a](#).
 sym_char: [178d](#), [179a](#).
 tex_flag: [27a](#), [27b](#), [33](#), [39b](#), [41c](#), [139d](#).
 toupper: [21b](#), [147c](#).
 TRUE: [23a](#), [27ab](#), [29d](#), [33](#), [35d](#), [39b](#), [52b](#), [57c](#), [62be](#), [65c](#), [70d](#), [74a](#), [78b](#), [79be](#), [82abc](#), [84a](#), [86ab](#), [87d](#), [91b](#), [95a](#), [96b](#), [97c](#), [98d](#), [102b](#), [103c](#), [119a](#), [123c](#), [124g](#), [139d](#), [147a](#), [149ab](#), [156](#), [159b](#), [176b](#), [177b](#), [178cd](#), [179a](#).
 update_delimit_scrap: [43d](#), [57c](#), [70a](#), [70b](#), [82c](#).
 user_langs: [149b](#), [156](#).
 user_lang_count: [149b](#), [156](#).
 user_lang_names: [149b](#), [156](#).
 user_names: [41c](#), [50a](#), [79c](#), [95c](#), [128b](#), [129a](#), [144c](#), [145a](#), [172c](#).
 Uses: [97ab](#), [98b](#), [100](#), [102b](#), [122b](#), [123de](#), [176ac](#), [176d](#), [176f](#), [178a](#).
 uses: [30](#), [55a](#), [65b](#), [78c](#), [79e](#), [87c](#), [95b](#), [96b](#), [98d](#), [100](#), [102b](#), [103c](#), [104a](#), [106](#), [121c](#), [123d](#), [126a](#), [128b](#), [129d](#), [144b](#), [149ab](#), [168b](#), [174](#), [176df](#), [178c](#).
 verbose_flag: [27a](#), [27b](#), [33](#), [41c](#), [54](#), [81c](#), [113e](#).
 version_info_flag: [27a](#), [27b](#), [33](#), [35c](#).
 version_string: [27a](#), [27b](#), [35c](#), [58b](#), [71b](#), [83b](#).
 weave_flag: [27a](#), [27b](#), [33](#), [39b](#).
 weave_format: [27a](#), [27b](#), [33](#), [39b](#), [158](#).
 weave_format_from_cli: [27a](#), [27b](#), [33](#), [39b](#), [158](#).
 WEFT_VERSION: [23b](#), [27b](#), [29d](#), [30](#).
 write_arg: [53b](#), [58c](#), [61a](#), [61b](#), [70e](#), [74b](#).
 write_ArglistElement: [53b](#), [72g](#), [74b](#).
 write_callers: [39b](#), [104a](#), [113b](#).
 write_error_annotator: [36b](#), [112](#), [113b](#).
 write_extract: [39b](#), [104b](#), [113b](#).
 write_files: [39b](#), [113a](#), [113c](#).
 write_label: [72a](#), [139a](#), [179c](#), [179d](#).
 write_lint: [39b](#), [107a](#), [113b](#).
 write_literal: [53b](#), [68c](#), [72g](#), [74b](#).
 write_map: [39b](#), [101](#), [113b](#).
 write_md: [39b](#), [81a](#), [81c](#).
 write_reverse_map: [36b](#), [109](#), [113b](#).
 write_scraps: [50c](#), [74a](#), [84a](#), [91b](#), [113e](#), [122b](#), [134b](#), [139d](#), [141c](#).
 write_scrap_ref: [65c](#), [78b](#), [79e](#), [87d](#), [95a](#), [96b](#), [98a](#), [122b](#), [123b](#), [123c](#), [177c](#).
 write_single_scrap_ref: [52b](#), [58c](#), [60ab](#), [64e](#), [65b](#), [73](#), [76b](#), [78c](#), [79e](#), [84bc](#), [87c](#), [90d](#), [91b](#), [93d](#), [95b](#), [96b](#), [122b](#), [123c](#), [140ac](#), [180a](#).
 write_tex: [39b](#), [53a](#), [54](#).
 xref_flag: [27a](#), [27b](#), [33](#), [140ac](#).

Bibliography

- [1] A[lfred] V. Aho and M[argaret] J. Corasick, *Efficient string matching: An aid to bibliographic search.*, CACM **18** (1975), no. 6, 333–340. [171](#)
- [2] C[hris] Hanson, *Efficient stack allocation for tail-recursive languages.*, (1990). [184](#)
- [3] Donald E. Knuth, *Literate programming*, The Computer Journal **27** (1984), no. 2, 97–111. [1](#)
- [4] ———, *The T_EXbook*, Computers and Typesetting, vol. 1986aA, Addison-Wesley, Reading, MA, USA, 1986. [1](#)
- [5] ———, *T_EX: The program*, Computers and Typesetting, vol. B, Addison-Wesley, Reading, MA, USA, 1986b1986. [1](#)
- [6] ———, *METAFont: THE PROGRAM*, Computers and Typesetting, vol. D, Addison-Wesley, Reading, MA, USA, 1986d1986. [1](#)
- [7] Leslie Lamport, *L^AT_EXa document preparation system user's guide and reference manual*, Addison-Wesley, Reading, MA, USA, 1985. [2](#)
- [8] Silvio Levy, *WEB adapted to C, another approach*, TUB **8** (1987), no. 1, 12–13. [1](#)
- [9] Norman Ramsey, *Literate-programming tools need not be complex*, Report at `ftp.cs.princeton.edu` in `/reports/1991/351.ps.Z`. Software at `ftp.cs.princeton.edu` in `/pub/noweb.shar.Z` and at `bellcore.com` in `/pub/norman/noweb.shar.Z`. CS-TR-351-91, Department of Computer Science, Princeton University, August 1992, Submitted to *IEEE Software*. [1](#)
- [10] Ross Williams, *Funnelweb user's manual*, `ftp.adelaide.edu.au` in `/pub/compression` and `/pub/funnelweb`, University of Adelaide, Adelaide, South Australia, Australia, 1992. [1](#)